

—— // ZMAX // ——

fusevm

v0.14.5

THE MACHINE

bytecode · the tiered JIT · the native compiler

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	5
Chapter 1 — The Pitch	7
Chapter 2 — The Chunk	11
Chapter 3 — The Value	14
Chapter 4 — Stack, Slots, and Frames	18
Chapter 5 — The Opcode Set: A Map	21
Chapter 6 — Loads, Stack, and the Naming of Storage	24
Chapter 7 — Arithmetic, Strings, and Comparison	27
Chapter 8 — Logic, Bitwise, and Control Flow	29
Chapter 9 — Arrays, Hashes, and the Collection Model	32
Chapter 10 — Higher-Order Blocks and Ranges	35
Chapter 11 — The Math Intrinsic	37
Chapter 12 — Fused Superinstructions	41
Chapter 13 — The Interpreter	43
Chapter 14 — Tiered Execution	46
Chapter 15 — The Host Boundary	48
Chapter 16 — The Shell Inside the Machine	52
Chapter 17 — AWK Inside the Machine	55
Chapter 18 — The Extension Mechanism and the Five Frontends	59
Chapter 19 — Cranelift, the Backend	61
Chapter 20 — The Linear JIT	65
Chapter 21 — The Block JIT	67
Chapter 22 — The Tracing JIT	70
Chapter 23 — The Native-Code Disk Cache	74
Chapter 24 — The AOT Compiler: The Threaded Path	77

Chapter 25 — Native Lowering: Scalars in Registers	80
Chapter 26 — The Basic-Block Compiler	83
Chapter 27 — Locals in Registers	87
Chapter 28 — Kinds: The Type Lattice	89
Chapter 29 — Floats, Math, and Libcalls	92
Chapter 30 — The Inline/Shim Boundary	95
Chapter 31 — Deopt	98
Chapter 32 — Differential Correctness	101
Chapter 33 — How Fast, Really	104
Chapter 34 — From Chunk to Standalone Binary	109
Chapter 35 — Reading a Disassembly	112
Chapter 36 — Loads and Stack, Op by Op	114
Chapter 37 — Variables and Slots, Op by Op	116
Chapter 38 — Arrays and Hashes, Op by Op	118
Chapter 39 — Arithmetic and String, Op by Op	120
Chapter 40 — Comparison, Logical, and Bitwise, Op by Op	122
Chapter 41 — Control Flow, Functions, and I/O, Op by Op	124
Chapter 42 — Collections and Higher-Order, Op by Op	126
Chapter 43 — The Math Intrinsics, Op by Op	127
Chapter 44 — The Fused Superinstructions, Op by Op	129
Chapter 45 — The Shell Ops, Op by Op	131
Chapter 46 — The AWK Ops, Op by Op	133
Chapter 47 — A Counted Loop Through the Linear JIT	135
Chapter 48 — The Same Loop Through the Block JIT	137
Chapter 49 — The Same Loop Through the Tracing JIT	139
Chapter 50 — A Chunk Through the AOT Native Path, Step by Step	141
Chapter 51 — From Chunk to Native Object to Standalone Binary	143
Chapter 52 — The Five Frontends Against the Protocols	145
Chapter 53 — A Real Program, End to End	147
Chapter 54 — The Tier Decision in VM::run, Traced	149

Chapter 55 — Serialization: A Chunk Round-Tripped	150
Chapter 56 — The Differential Fuzzers, Walked	152
Chapter 57 — The VMPool and the Frontend Bridge, Walked	154
Chapter 58 — Errors, Halts, and the AWK Signal Channel, Traced	155
Chapter 59 — The Disk Cache Lifecycle, Walked	157
Chapter 60 — A Catalogue of Worked Idioms	159
Chapter 61 — A Recursive Call, Traced Through the Frame Stack	161
Chapter 62 — The ShellHost Trait, Method by Method	163
Chapter 63 — The AwkHost Trait, Method by Method	165
Chapter 64 — The Builtin ID Tables	167
Chapter 65 — The Benchmark Harnesses, Walked	169
Chapter 66 — The Contract Tests, Walked	171
Chapter 67 — The Capability Matrix and Versioning	172
Chapter 68 — The Coercion Tables in Full	173
Chapter 69 — Cross-Language Value Flow	175
Chapter 70 — The Edge Cases the Tests Pin	177
Chapter 71 — The Op Enum’s Encoding, in Detail	179
Chapter 72 — What the Machine Does Not Do	181
Chapter 73 — The JIT Test Suite, Walked	183
Chapter 74 — Error Reporting and Diagnostics	185
Chapter 75 — The Chunk’s Supporting Tables, in Use	186
Chapter 76 — The op Vector as Data: Layout and Dispatch	188
Chapter 77 — Threaded and Native, the Same Chunk Both Ways	190
Chapter 78 — A Boundary Loop, Spilled and Continued	192
Chapter 79 — Provisional Frontiers	194
Chapter 80 — The Classic Algorithms as Chunks	195
Chapter 81 — Allocation Discipline: Arc, In-Place, and the Pool	197
Chapter 82 — Reading the Source: A Guided Tour	199
Chapter 83 — The Extension Handler Protocol, in Code	201
Chapter 84 — The Single Thread and the Shape of Concurrency	203

Chapter 85 — Lowering Control Flow: How a Front End Builds Loops	205
Chapter 86 — The Numeric Tower and Its Consequences	207
Chapter 87 — The Division of Labor: fusevm’s Analysis and Cranelift’s Allocator	209
Chapter 88 — fusevm Among the Reusable Machines	211
Chapter 89 — The Long Life of a Chunk	213
Chapter 90 — Why One Machine	215
Appendix A — The Value Variants	217
Appendix B — The Opcode Sections	218
Appendix C — The Native AOT Op Set	220
Appendix D — The JIT Tier Reference	221
Appendix E — Glossary	222
Appendix F — The Public API	224
Appendix G — Features, Crate Types, and Dependencies	225
Appendix H — The Source Tree	226
Appendix I — The Disassembly Notation	227

Before You Begin

This is not the reference manual. The reference enumerates every one of fusevm's opcodes — all two hundred and twenty-four of them — every `Value` variant, every public entry point on the `VM`, and you reach for it when you need to know exactly what `Op::SlotIncLtIntJumpBack` does or which methods the host trait demands. This book is the other thing: the one you read in a chair, front to back, to understand not just *what* fusevm does but *why* it is shaped the way it is, and how a stack of bytecode becomes, by degrees, native machine code that runs within a small constant factor of hand-written Rust.

fusevm is a stack-based bytecode virtual machine written in Rust. That sentence is true and almost completely fails to describe it. A fuller description: fusevm is the *single shared engine* underneath an entire family of languages — a shell (zshrs), an AWK (awkrs), a systems scripting language (stryke), a vimscript (vimlrs), and an elisp (elisprs) — none of which is its own interpreter. Each is a front end: a lexer, a parser, and a lowering to fusevm opcodes. In exchange every one of them inherits, for free, a fast interpreter, a three-tier just-in-time compiler, and a closed-world ahead-of-time compiler that turns a whole program into a standalone native binary. The expensive part of a fast language — the machine and the compiler — is written once and shared many ways. That economy is the whole story, and everything in this book is a consequence of it.

The numbers say it plainly. The engine is roughly twenty-four thousand lines of production Rust across eleven source files, shadowed by more than five times its weight in tests; it ships two hundred and twenty-four opcodes, eleven fused superinstructions, twenty-nine first-class shell operations, sixty-one first-class AWK operations, and a three-tier Cranelift JIT — and five separate languages are in production against it today. The version on the tin is `0.14.5`, which is to say: young, moving, and honest about both.

The book is in five movements. The first is the machine as data: the `Chunk`, the `Value`, the stack, the slots and frames — the things an opcode reads and writes. The second is the machine's vocabulary: the opcode set walked group by group, from the loads through the arithmetic and control flow into the math intrinsics and the fused superinstructions that are fusevm's namesake. The third is the machine in motion: the interpreter's dispatch loop and the tier ladder that decides, at runtime, how hard to work on a given piece of code. The fourth is the embedding: how a front end plugs a language into the machine through the host traits and the extension mechanism, and why a shell and an AWK are first-class citizens rather than awkward guests. The fifth — the longest, because it is the deepest — is compilation: the Cranelift backend, the three JIT tiers, the native-code disk cache, and the closed-world AOT compiler that lowers the bytecode to native registers, holds locals in machine registers, falls back to the interpreter exactly where it must, and resumes the interpreter mid-program when a value escapes what registers can hold. A short shelf of reference appendices closes the book: the value variants, the opcode sections, the set of operations the native compiler can lower, the JIT tier reference, and a glossary.

Everything here is grounded in the actual crate. Every opcode name, type, and architectural claim is traceable to a specific source file in the fusevm tree — `op.rs`, `value.rs`, `chunk.rs`, `vm.rs`, `host.rs`, `awk_host.rs`, `awk_builtins.rs`, `shell_builtins.rs`, `jit.rs`, `aot.rs`, `lib.rs`. Where a number or a behavior is provisional — fusevm is young, and its version says so — this book marks it as such rather than rounding up. fusevm is the work of MenkeTechnologies.

A word on how to read it. The five movements build on each other, and a reader new to fusevm will get the most from the book front to back, because the compiler movement assumes the value model, the opcode set, and the interpreter that precede it — the kind lattice in the AOT chapters is the `Value` variants of Chapter 3 seen from the compiler's side, the deopt machinery is the dispatch loop of Chapter 13 resumed mid-program, and the boundary chapters are the host seam of the fourth movement reached from the register world. A reader who already knows bytecode VMs can skim the first movement and start at the opcode set; a reader who cares only about the compilers can start at Cranelift and refer back. But the book is written as one argument —

that one machine, carefully built, can be cheap to target and expensive to leave — and each movement is a step in it, so the whole reads better than the parts.

A word, too, on what is provisional. fusevm is at version [0.14.5](#), which is to say it is moving, and some of what this book describes is a frontier rather than a finished edge. The AOT native path's reach into dynamic-kind heap sources is explicitly future work, flagged as such where it comes up. A few constants in the source carry stale doc comments that disagree with the code — the block-JIT warm-up threshold is the notable one, where a comment says ten and the code says one — and this book follows the code, not the comment, and says so at each such point. Where a benchmark number is quoted it is from the project's own criterion harness on Apple M-series hardware, and where a claim is an architectural intention rather than a measured fact, the prose says “intended” or “future work” rather than asserting it as done. The aim throughout is that a reader could check any sentence against the named source file and find it true, or find it honestly marked as not-yet.

Let's start.

Chapter 1 — The Pitch

Every virtual machine is an argument about where a language’s cost should be paid. The argument fusevm makes has two halves, and they are easier to state than to believe. The first half: a new language should cost a front end, not a runtime — a lexer and a lowering, with the machine and its compilers supplied. The second half: that shared machine should not merely *interpret* the bytecode its front ends emit; it should be able to compile it, ahead of time, to a standalone native executable, and just-in-time at runtime for code that did not exist until the program ran. Both halves are unusual. Taken together they describe a machine that is cheap to target and expensive to leave — and the tension between “small enough that five languages share it” and “powerful enough to compile any of them to native code” is the whole engineering problem.

Start with what fusevm is made of, because the claims that follow are only credible if the foundation is. fusevm is a stack machine. There is an operand stack of boxed values; opcodes pop their inputs from it and push their outputs to it. There are *slots* — indexed local variables in a call frame — reached by number rather than by name, so a local access is an array index, not a hash lookup. There are *globals*, indexed the same way. There is a **Chunk**: a flat vector of opcodes plus a pool of constants, the compiled form of a program. And there is the **VM**: the struct that owns the stack, the frames, the globals, and the cursor (**ip**, the instruction pointer) that walks the opcodes. That is the entire model, and it is deliberately small, because smallness is what lets a shell and an AWK and three other languages all lower onto it.

The compactness is not an accident; it is a design value the project states outright. fusevm ships no parser, no garbage collector, no standard library — only the dispatch loop, the JIT bridge, and the extension hooks. The front ends layer everything else on top. It is, by intent, a VM you can read end to end in an afternoon, and the reason that is possible is that everything language-shaped has been pushed out of the core and into a front end. What remains is the part that is genuinely hard to write well and genuinely worth writing once: the machine that moves values and the compilers that make it fast.

A machine, not a language

fusevm is not tied to any source language, and this is the load-bearing fact. The opcode set — two hundred and twenty-four operations at the time of writing, organized into seventeen sections — is the contract. A front end that can lower its source to those opcodes inherits everything the machine does well, and the machine is engineered to do one thing above all: run fast. It ships *fused superinstructions* — single opcodes that perform the work of common opcode sequences, so the hot path dispatches once where a naive bytecode would dispatch many times. It uses slot-indexed fast paths for locals. And it carries, behind feature flags, two whole compilers: a three-tier Cranelift JIT for code discovered at runtime, and a closed-world Cranelift AOT compiler for whole programs known in advance.

Two of the opcode sections tell you exactly what fusevm was built to host. It has first-class *shell* operations — pipelines, redirections, subshells, command substitution — and first-class *AWK* operations — field splitting, the `$1/$2/$NF` field model, `getline`, `printf`, the math builtins. Those are not synthesized awkwardly on top of a general-purpose core; they are opcodes the machine understands as named variants of its instruction enum. That is fusevm being designed, from the metal up, to host a shell and an AWK as naturally as it hosts addition. When `zshrs` lowers a pipeline or `awkrs` lowers a field reference, it emits a single opcode the VM routes directly to a host — which is why those two notoriously special languages embed as cleanly as they do.

The five front ends do not all use the machine the same way, and the differences are instructive. `stryke` registers roughly four hundred and fifty extended ops; `zshrs` registers about twenty; `awkrs` registers about ninety-five; `elisprs` registers ten; and `vimlrs` takes the other road entirely, routing roughly five hundred and ten

builtin IDs through `CallBuiltin` rather than extension opcodes. None of them collide, because each front end owns its own ID space. One machine, five wildly different vocabularies layered on top of the same two hundred and twenty-four core instructions.

AOT machine, JIT'd eval

Here is the distinction that organizes the last movement of the book. The machine *itself* — its dispatch loop, its opcode handlers, its compilers — is Rust, compiled ahead of time by `cargo` into whatever binary embeds it. Nothing about the engine is built at runtime; there is no toolchain the user installs, no warm-up that compiles the interpreter. But `fusevm` goes further than most embedded machines: it can take a *program* — a `Chunk` — and compile *that* ahead of time too, into a relocatable object file and a standalone native binary, with the chunk embedded as a data blob and a native entry function that drives it. And for code that genuinely cannot be compiled in advance — a line typed at a REPL, a string passed to `eval` — it has the JIT, which lowers that code to native instructions on the spot and lifts its hot loops to register speed.

So `fusevm` spans the whole range: an AOT-native engine, an AOT compiler for whole programs, and a JIT for runtime code. The rest of this book walks that range from the bottom — the data the machine moves — up through the interpreter, and on into the two compilers, ending with the part that took the most engineering to get right: an AOT path that holds a program's locals in machine registers and only touches the boxed world at the edges.

The smallest complete program

It is worth seeing, before anything else, how little stands between a front end and a running program, because the smallness is the argument made concrete. Here is a complete use of `fusevm` as a library, lifted from the crate's own documentation:

```
use fusevm::{Op, ChunkBuilder, VM, VMResult, Value};

let mut b = ChunkBuilder::new();
b.emit(Op::LoadInt(40), 1);
b.emit(Op::LoadInt(2), 1);
b.emit(Op::Add, 1);

let mut vm = VM::new(b.build());
match vm.run() {
    VMResult::Ok(val) => println!("result: {}", val.to_str()),
    VMResult::Error(e) => eprintln!("error: {}", e),
    VMResult::Halted => {}
}
```

There is no parser here, no garbage collector, no runtime to initialize. A front end's entire relationship with the machine is in this fragment: build a `Chunk` by emitting opcodes, hand it to a `VM`, and call `run`, which returns one of three things — a value, an error, or a halt. Everything in this book elaborates one or another part of those four lines. The `ChunkBuilder` is Chapter 2; the `Ops` are the second movement; `VM::new` and `run` are the third; and the `VMResult` is the contract every front-end driver consumes. A new language reaches a working, JIT-able, AOT-compileable runtime the moment it can produce the op sequence in the middle of that example — which is exactly the cost the project set out to charge, and no more.

What the machine refuses to be

It is as instructive to name what `fusevm` is *not* as what it is, because the refusals are where the design's discipline lives. It is not a language: it ships no syntax, no surface, no parser, so there is nothing to argue about regarding how a loop should read. It is not a memory manager: it has no garbage collector, no allocator of its

own; values that own heap data own it through Rust's ordinary `Arc`, `Box`, `Vec`, and `HashMap`, and a front end that wants a cyclic object graph keeps that graph in its own heap behind an `Obj` handle. It is not a standard library: there are no string-formatting builtins, no file-system helpers, no networking, in the core — those are the front end's, registered through the extension mechanism. And it is not a sandbox or a security boundary: the host trait decides what `exec` does, and a host that spawns real processes spawns real processes. Each refusal pushes a concern out to where it belongs and keeps the engine small enough that one person can hold all of it in their head — which, for a machine that five languages depend on, is the property that matters most.

The five languages, named

The abstraction “five front ends” is more concrete and more persuasive when the five are named, because their diversity is the real test of the claim that fusevm is genuinely language-agnostic rather than a shell engine with pretensions. The five are `zshrs`, a Rust-rewritten Z-shell; `awkrs`, an AWK; `stryke`, a Perl-compatible systems scripting language; `vimlrs`, a vimscript; and `elisprs`, an Emacs Lisp. No two of them agree about much. A shell's central act is running processes and wiring pipelines; an AWK's is splitting records into fields and matching patterns; a Perl-descended language's is text munging with sigils and regular expressions; a vimscript's is poking at editor buffers; and a Lisp's is manipulating cons cells with pointer identity and mutable shared structure. These are nearly the five most different things a “scripting language” can mean, and the argument for fusevm is that all five lower to the same two hundred and twenty-four opcodes, the same value model, the same compilers. The diversity is not incidental to the pitch; it is the proof of it, because a machine that could host only languages resembling a shell would not have demonstrated language-agnosticism at all. That a Lisp and an AWK and a shell share one engine — that the day the engine learned to hold float slots in registers, the AWK's accumulators and the Lisp's numeric loops both got faster with no language-specific work — is the whole thesis made flesh.

Where the languages diverge in their use of the machine

The five do not merely target the machine; they target it in revealingly different ways, and the spread is a map of the machine's flexibility. `zshrs` leans hardest on the first-class shell opcodes and registers only about twenty extension ops, because most of what a shell does the core already names. `awkrs` leans on the sixty-one first-class AWK opcodes and a registered `AwkHost`, adding about ninety-five extension ops for the AWK semantics the core leaves host-side. `stryke` is the most ambitious, registering roughly four hundred and fifty extension ops for the largest standard library of the five. `vimlrs` takes the opposite tack from `stryke`, routing about five hundred and ten builtins through the flat `CallBuiltin` dispatch table rather than the extension `match`. And `elisprs` registers just ten extension ops, because its real work is in its own heap of cons cells, reached through `Obj` handles, with fusevm carrying the handles and `elisprs` owning what they point to. Five languages, five distributions of work across the same mechanisms — first-class opcodes, extension handlers, builtin tables, host traits, `Obj` handles — and none of them colliding, because each owns its own ID space. The mechanisms are general enough that a shell can mostly use named opcodes and a Lisp can mostly use opaque handles and both are equally at home.

The economy, stated as a sum

The pitch reduces to an economy, and the economy is worth stating as a sum because it is the whole business case for the architecture. The expensive parts of a fast language are four: a fast interpreter, a JIT, an AOT compiler, and the correctness apparatus that keeps all three honest. Written conventionally, a family of five languages pays for all four five times — five interpreters, five JITs (if any of them can afford one), five AOT stories, five test suites pinning five sets of semantics — and the cost is so high that, in practice, four of the five simply do without the JIT and the AOT compiler, because no small language can justify writing them. fusevm changes the sum: the four expensive parts are written *once*, in the shared engine, and the five languages each pay only for a lexer and a lowering. The marginal cost of the JIT to `awkrs` is zero, because `awkrs` did not

write the JIT; it inherited it by lowering to the opcodes the JIT already compiles. The marginal cost of the AOT compiler to elisprs is zero, for the same reason. So the sum is not five times four but one times four plus five times (lexer + lowering), and the second term is small. This is why a shell and an AWK and a vimsript and an elisp can all have a tracing JIT and an AOT compiler — not because each could afford to write one, but because none had to. The economy is the pitch, and the rest of the book is the demonstration that the shared part is genuinely shareable: that one value model, one opcode set, one host boundary, and one compiler really do serve five languages without forcing them to resemble each other.

Young, and honest about it

A pitch should be honest about its own maturity, and fusevm's is, in two ways worth naming at the outset so the rest of the book can be read in the right spirit. The first is the version: `0.14.5`, a pre-1.0 number that says, by convention, “still moving, interfaces may shift, not everything is finished.” This book takes that seriously — it marks the AOT native path's dynamic-kind sources as future work rather than pretending they are done, it follows the code over stale doc comments where the two disagree, and it says “intended” where a claim is a design goal rather than a measured fact. The second is the scope of what is genuinely finished versus genuinely frontier. The interpreter, the value model, the opcode set, the host boundary, the three JIT tiers, and the threaded AOT path are finished and heavily tested; the native AOT path is finished for the scalar core and the statically-typed boundary and is an active frontier for dynamic-kind heap sources. A reader should hold both facts: the machine is real, in production under five languages, with a test suite over five times the size of its source — and it is also young enough that some of what this book describes is a line still moving. The honesty is not a hedge; it is the same discipline that runs through the source, where the fuzzers report the declines as carefully as the lowerings and the benchmark tables publish the `VMPool` loss beside the wins. A pitch you can trust is one that tells you where it is not yet done, and this is where fusevm is not yet done — which is exactly the information a reader needs to weigh everything that follows.

Chapter 2 – The Chunk

A program, to fusevm, is a `Chunk`. Everything the machine runs — a whole shell script, a single `eval`'d expression, a compiled AWK program, a function body, a command-substitution body — is a `Chunk`, and nothing else is executable. It is worth being precise about what a chunk contains, because the entire system, interpreter and both compilers, is defined against this one structure, declared in `chunk.rs`.

A chunk has nine fields, and they divide cleanly into the ones that execute and the ones that support execution. The executing core is three:

- `ops: Vec<Op>` — the bytecode instructions in execution order. There is no tree here, no nesting; control flow is expressed by opcodes that move the instruction pointer (`Op::Jump`, `Op::JumpIfFalse`, and the rest), and a structured `if/while` in the source language has already been flattened into forward and backward jumps by the time it reaches a chunk.
- `constants: Vec<Value>` — the literal pool: literals, regex patterns, format strings. An opcode that needs a string or a large number does not carry it inline; it carries a `u16` index into this pool (`Op::LoadConst(u16)`) and the value lives there once.
- `names: Vec<String>` — the table of global variable and function names, interned and deduped, so that a global access can be a numeric index (`Op::GetVar(u16)`) rather than a string lookup. The name table exists so the *runtime* never has to do a string comparison to find a variable.

The supporting fields are where the chunk's range shows. `lines: Vec<u32>` is a parallel array, one entry per op, recording the source line each instruction came from — purely for error reporting, never read during execution. `sub_entries: Vec<(u16, usize)>` maps a name-pool index to the op index where a subroutine begins, so a `Call` can resolve a function name to an instruction pointer. `block_ranges: Vec<(usize, usize)>` records `(start_ip, end_ip)` pairs for the bodies of `map/grep/sort/foreach`, which the higher-order opcodes reference by index. `sub_chunks: Vec<Chunk>` holds nested chunks — the bodies of `$(cmd)` command substitutions, `<(cmd)/>(cmd)` process substitutions, trap handlers, and with-redirect blocks — each referenced by the `u16` payload of the op that spawns it (`Op::CmdSubst(u16)`, `Op::ProcessSubIn(u16)`, `Op::TrapSet(u16)`). And `source: String` is the file name, again for error messages.

There is a ninth field that does no work during execution but matters enormously for the compilers: `op_hash: u64`, a hash of the ops and constants, computed once when the chunk is built. The JIT caches compiled native code keyed by this hash, so a chunk that has been compiled before is recognized in $O(1)$ rather than recompiled. It is a small thing that pays for itself the moment the same code runs twice — and the benchmarks measure that lookup at roughly twenty-four nanoseconds, a rounding error against any real computation.

Why flat, why indexed

The two design choices in the chunk — flat opcodes and indexed access — are both in service of the same thing: making the common operation cheap. Flat opcodes mean the dispatch loop is a single `while` over a vector with no recursion and no pointer chasing; the instruction pointer is a `usize`, advancing it is an add, and a jump is an assignment. Indexed access means a local variable read is `frame.slots[i]` — a bounds-checked array index — rather than a hash-map probe keyed by a name. The front end does the work of assigning slot numbers at compile time so the runtime never spends a cycle on a name. These are the choices that make the interpreter fast enough that, for cold code, you may never need the compilers at all.

The hash itself is worth one more sentence, because its precise definition has consequences. `op_hash` is computed over the ops and the constants only — not the lines, not the names, not the source file. Two chunks that differ only in which line a statement appeared on, or what a global happened to be named, produce the

same hash and therefore share compiled native code, which is correct, because neither line numbers nor name strings change what the machine does. The hash also hashes a float constant by its bit pattern, so a chunk holding `0.0` and one holding `-0.0` hash differently even though the two compare equal — exactly the discipline that keeps the JIT cache from ever serving wrong code for a bit-distinct constant. And because `op_hash` carries `#[serde(skip)]`, it is *not* serialized: a chunk written to disk and read back has a zero hash until something recomputes it, which the build path does.

Building a chunk

Front ends do not construct `Chunk` literals by hand; they use `ChunkBuilder`, which accumulates ops, interns constants, registers names, and patches jumps. Its API is small and every method does exactly one obvious thing. `emit(op, line)` pushes an op and its source line, returning the op's index. `add_constant(val)` appends to the constant pool and returns the `u16` index to embed. `add_name(name)` interns a name — returning the existing index if the name has been seen before, so the pool is deduplicated — and is backed by a private `HashMap<String, u16>` that exists only during building. `current_pos()` returns the current op count, which is how a front end computes a jump target. `add_sub_entry`, `add_block_range`, and `add_sub_chunk` populate the supporting tables, the last two returning the `u16` index the referencing op will carry. `set_source` names the file. And `build()` finalizes everything: it walks the ops and constants through a `DefaultHasher`, stores the result in `op_hash`, and hands back the finished `Chunk`.

The patch step is the interesting one, because it is how a single-pass front end emits forward jumps. When a front end emits a forward `if`, it does not yet know where the `else` branch begins, so it emits `Op::JumpIfFalse(0)` with a placeholder target and remembers the op's index from `emit`'s return value; later, when it knows the target, it calls `patch_jump(idx, target)` to fill it in. `patch_jump` is deliberately strict: it matches only the five jump variants (`Jump`, `JumpIfTrue`, `JumpIfFalse`, `JumpIfTrueKeep`, `JumpIfFalseKeep`) and panics on anything else, so a front-end bug that tries to patch a non-jump op fails loudly at build time rather than silently corrupting the bytecode. This back-patching is the standard trick for emitting forward jumps without a second pass, and the strict matcher is the standard insurance against getting it wrong.

A chunk being built, step by step

The abstract description becomes concrete the moment you watch a front end build an `if`. Suppose the source is `if (x) { print 1 }`. A single-pass compiler emits the predicate, then a `JumpIfFalse` whose target it does not yet know, then the body, and finally patches the jump to land just past the body:

```
let mut b = ChunkBuilder::new();
let x = b.add_name("x");
b.emit(Op::GetVar(x), 1);           // push x
let jmp = b.emit(Op::JumpIfFalse(0), 1); // placeholder target - remember `jmp`
b.emit(Op::LoadInt(1), 1);         // body: push 1
b.emit(Op::PrintLn(1), 1);         // print it
let after = b.current_pos();       // where the body ends
b.patch_jump(jmp, after);          // fill in the jump target
let chunk = b.build();             // op_hash computed here
```

Every method in this fragment is the one described above, and the shape — emit a branch with a placeholder, record its index, emit the body, read `current_pos`, patch — is the entire technique by which structured source becomes a flat op vector with no second pass. A `while` loop adds one wrinkle: a *backward* jump at the bottom, whose target is known (the top of the loop, recorded before the body) so it needs no patching, and a forward `JumpIfFalse` at the top, patched the same way as the `if`. Those two jumps — one forward and patched, one backward and immediate — are the skeleton of every loop in every front end, and the backward one is the signal the tracing JIT and the basic-block compiler both read to find loops.

The cost of a chunk, and why sub-chunks are separate

A `Chunk` is not free — it owns several vectors — and the design makes two choices to keep that cost where it belongs. The first is that a chunk is `Clone` and `Serialize/Deserialize`, so a front end can cache a built chunk, persist it, or hand a copy to another thread; the only field that does not survive serialization is `op_hash`, which is cheap to recompute and meaningless across the serialization boundary anyway. The second is that nested execution units — command-substitution bodies, process substitutions, trap handlers — are stored as *whole sub-chunks* in the `sub_chunks` vector rather than inlined into the parent's op stream. This matters because those bodies run under different rules: a `$(cmd)` body's output is captured, a trap handler fires asynchronously, a process substitution feeds a FIFO. Keeping each as its own `Chunk`, referenced by the `u16` payload of the spawning op, means the host can run it on a fresh or pooled VM with its own stack and frames, entirely isolated from the parent's, and the parent's flat op vector stays a clean linear sequence the compilers can analyze without descending into nested control flow. The chunk is flat *because* the genuinely nested things were factored out into sub-chunks.

Chapter 3 – The Value

Everything on fusevm's operand stack, in its slots, and in its globals is a `Value`. It is a Rust enum, declared in `value.rs`, and its shape is the single most consequential design decision in the machine — because, as the compilation chapters will show, the whole difficulty of generating fast native code comes down to the fact that a `Value` is too big and too varied to live in a machine register.

The variants are eleven, and the doc comment on the enum names the goal: “small (1 word tag + 1-2 words payload) so the dispatch loop stays cache-friendly.” They are `Undef` (the absence of a value, and the `#[default]`); `Bool(bool)`; `Int(i64)`; `Float(f64)`; `Str(Arc<String>)` — a reference-counted string, so a clone is a pointer bump rather than a copy, which matters the moment a closure captures a string; `Array(Vec<Value>)`; `Hash(HashMap<String, Value>)`; `Status(i32)` — an exit status, shell-specific but, as the comment says, “universal enough”; `Ref(Box<Value>)` — a reference, for pass-by-reference and nested structures; `NativeFn(u16)` — a handle to a builtin for dispatch; and `Obj(u32)` — an opaque handle into a front end's own object heap.

The `Obj` variant repays a closer look, because it is how the machine carries a front end's private objects without understanding them. An elisp cons cell, a symbol, a vector — elisprs keeps these in its own heap and hands fusevm only a `u32` handle. The comment in `value.rs` is precise about why this variant exists and `Array` could not stand in for it: `Obj` is *identity-comparable* — the handle *is* the identity — which is exactly what elisp's `eq` and `setcar` need and what a structurally-compared `Array` could never provide. fusevm carries the handle through the stack, slots, and globals; the front end's extension handler owns what it points to. This is the seam that lets a Lisp, with its mutable shared structure and pointer identity, run on a machine that knows nothing about cons cells.

A `Value` is therefore a tagged union the size of its largest variant — far larger than a 64-bit register, and not something you can `iadd` directly. The README's size table makes the spread concrete: `Undef` is a tag with no payload, `Int/Float` are eight inline bytes, `Bool` is one, `Status` is four, `Str/Ref` are a pointer, `Array` is three words, and `Hash` is seven. Hold that thought; it is the reason the AOT compiler exists in the shape it does.

The quiet correctness details

Two of `Value`'s trait implementations repay close reading because they encode semantics the whole machine depends on. Equality follows IEEE 754: two `Float` NaNs are *not* equal, exactly as `f64::NAN != f64::NAN`, so a comparison opcode on NaN behaves the way every other language's does. But hashing cannot follow IEEE — a `HashMap` requires that equal keys hash equally and demands total consistency — so the hand-written `Hash` impl hashes a float by its *bit pattern* (`f.to_bits()`), which means two NaNs with identical bits hash the same even though they compare unequal. The two implementations disagree on purpose, each correct for its job, and the test suite pins both: one test asserts `Value::Float(NAN) != Value::Float(NAN)`, another asserts two same-bit NaNs hash equal. The `Hash` impl also mixes in `std::mem::discriminant`, so `Int(0)`, `Bool(false)`, and `Status(0)` — three values whose payloads are all zero — hash to three different values, which a test also pins.

The other detail is `is_truthy`, the function every conditional jump consults. It is mostly unsurprising — `Int(0)` is false, a non-empty `Array` is true, `Undef` is false — but two cases are not. An empty string *and* the string “0” are both false, matching shell and AWK truthiness, so `Value::str("0").is_truthy()` returns `false` where a naive “non-empty string is true” rule would return `true`. And a `Status` is true when its code is zero, because in a shell a zero exit status means success means true — the inverse of how an integer's truthiness

works (`*c == 0` for `Status`, `*n != 0` for `Int`). That single inversion will come back, much later, as one of the trickiest cases the native compiler has to get right, and the only reason it does is that this function is the single source of truth and the compiler is written to match it exactly. The remaining variants are all unconditionally truthy from fusevm's view — `Ref`, `NativeFn`, and `Obj` included; an `Obj` is always non-nil to the core because elisp's `nil` maps to `Undef` and elisp truthiness is decided front-end-side.

Coercions

A dynamically typed machine coerces constantly, and `Value` centralizes it in four methods that the test suite pins as a cross-type matrix, because, as the comment says, “a single-branch refactor can't silently change e.g. `if @arr` semantics from ‘non-empty’ to ‘always true’.”

`to_int` turns any value into an `i64` — a `Float` truncates toward zero (`3.9 → 3`, `-3.9 → -3`), a `Bool` is 0 or 1, a `Status` yields its code, a `Str` parses or falls back to 0, and — the one that surprises people — an `Array` yields its `length`, not its first element, matching Perl and AWK numeric coercion of an array. `to_float` is the analogous widening, with the quirk that only `Bool(true)` produces `1.0` and every other non-numeric drops through to `0.0`. `to_str` formats, and is built on `as_str_cow`, which produces a string view that *borrow*s when it can and allocates only when it must: a `Str` borrows its inner `Arc<String>`, and so do `Undef` (the empty string), `Bool("1" or "", Perl-compatible)`, and the `Hash/Ref` placeholders (`"(hash)"`, `"(ref)"`); an `Int`, `Float`, `Status`, `Array`, `NativeFn`, or `Obj` must allocate, because their text is computed. Borrowing here is not a micro-detail: string comparison, concatenation, hash-key lookup, and I/O all run through `as_str_cow`, and the borrowed path is what keeps those off the allocator.

These coercions are not incidental; they are the semantics of every arithmetic and string opcode, and the compilers reproduce them instruction by instruction. When a later chapter says the native compiler “matches the interpreter bit-for-bit,” it means it emits the machine-code equivalent of these exact coercions — including the saturating float-to-int cast that `to_int` performs and the truncation direction it chooses. One last subtlety the tests pin: `len()` on a `Str` is the *byte* count, not the character count (Rust's `String::len`), so “`é`” has length two; every `length` builtin in every front end inherits that choice, and a refactor that switched it to char count would silently shift the output of every UTF-8 program.

The coercion matrix as a test

The clearest way to see how load-bearing these coercions are is to read the property tests that pin them, because the test names *are* the specification. The suite asserts, among dozens of cases, that an array's `to_int` is its length and not its first element; that a `Status` round-trips its exit code through `to_int` rather than collapsing to 0 or 1; that a float truncates toward zero in both directions (`3.9 to 3`, `-3.9 to -3`); that an empty string, the string `"0"`, an empty array, and an empty hash are all falsy while a single-element array is truthy; and that a `Bool` stringifies Perl-style to `"1"` or `""`. The comment above this block of tests states the stakes plainly: these triples “are the foundation every fusevm-hosted language depends on,” pinned so “a single-branch refactor can't silently change e.g. `if @arr` semantics from ‘non-empty’ to ‘always true’.” A coercion is not a convenience the machine offers; it is a contract five languages were built against, and the test suite treats it that way.

Why the string view borrows

One coercion deserves a second look because it is where the `Arc<String>` choice pays off. `as_str_cow` returns a `Cow<str>` — a clone-on-write string view — and for a `Str` it returns `Cow::Borrowed`, handing back a reference into the inner `Arc<String>` with no allocation at all. Every string comparison (`StrEq` and its siblings), every concatenation, every hash-key lookup, and every I/O write runs through this method, and the borrowed path is what keeps those operations off the allocator on the overwhelmingly common case where the operand is already a string. The allocating cases — an `Int`, a `Float`, a `Status`, an `Array`, an `Obj` —

allocate only because their text genuinely has to be computed; there is no string sitting in memory to borrow. The `Arc` wrapper is what makes the borrow safe to hand out and cheap to clone when it must be cloned: a closure capturing a string bumps a reference count rather than copying bytes, and a value passed between two of the five front ends shares one allocation. The size table from the README makes the trade visible — a `Str` is one pointer on the stack regardless of how long the string is — and `as_str_cow` is the method that turns that representational choice into avoided allocations on the hot path.

The representation table

The eleven variants split cleanly into the inline immediates — small enough to live in the `Value`'s payload word — and the heap-backed kinds that hold a pointer to data elsewhere, and the split is exactly the line the AOT native path cares about. The README's representation table:

Variant	Representation	Payload
<code>Undef</code>	tag only	0 bytes
<code>Bool(bool)</code>	inline	1 byte
<code>Int(i64)</code>	inline	8 bytes
<code>Float(f64)</code>	inline	8 bytes
<code>Status(i32)</code>	inline	4 bytes
<code>NativeFn(u16)</code>	inline	2 bytes
<code>Obj(u32)</code>	inline	4 bytes
<code>Str(Arc<String>)</code>	heap	pointer
<code>Ref(Box<Value>)</code>	heap	pointer
<code>Array(Vec<Value>)</code>	heap, in-place	3 words
<code>Hash(HashMap<String, Value>)</code>	heap, in-place	7 words

The inline kinds — `Undef`, `Bool`, `Int`, `Float`, `Status`, `Obj` — are exactly the kinds the native path can hold in a machine register, because their entire value fits in a word or two; the native `Kind` lattice is, not coincidentally, `Int/Bool/Float/Status/Obj`, the inline kinds minus the two function-handle variants. The heap kinds — `Str`, `Ref`, `Array`, `Hash` — cannot live in a register, which is why a chunk that produces one as a register result is declined to the threaded path, and why the boundary chapter's reload problem is hard exactly for the heap kinds. Reading the representation table is reading, in advance, the dividing line the whole compilation movement is organized around: inline kinds go in registers, heap kinds stay boxed, and the analysis is the apparatus that decides which side of the line each value falls on.

Equality and hashing, disagreeing on purpose

The deliberate disagreement between `Value`'s equality and its hashing is worth one more pass, because it is a textbook case of two trait implementations being individually correct and jointly contradictory, on purpose, and getting it wrong would corrupt every hash a front end builds. `PartialEq` follows IEEE 754: a float compares by value, so two NaNs are unequal and `0.0` equals `-0.0`. That is right for a comparison opcode, because a program that compares two NaNs should see them as different, exactly as every other language does. But a `HashMap` demands an invariant that IEEE equality violates: equal keys must hash equally, and a hash must be total and consistent. If `Value`'s `Hash` followed IEEE, two NaN keys would be unequal (fine) but might hash differently (fine) — yet `0.0` and `-0.0`, which IEEE says are *equal*, would also hash differently if hashed by value, and that would break the map's invariant for any program that used a float as a key. So `Hash` hashes a float by its *bit pattern* (`to_bits`), which makes `0.0` and `-0.0` hash differently — consistent with them being bit-distinct — and makes two same-bit NaNs hash equally. The two impls disagree about NaN (unequal under `PartialEq`, equal-hashing under `Hash` when the bits match) and about signed zero, and each disagreement is the correct choice for that impl's job: comparison wants IEEE value semantics, hashing wants total bit semantics. The test suite pins both — one test asserts NaN is unequal to itself, another asserts two same-bit

NaNs hash equal, a third asserts `Int(0)`, `Bool(false)`, and `Status(0)` hash apart via the discriminant — because the disagreement is intentional and a “fix” that made them agree would break either comparison or hashing. It is a small monument to the principle that “correct” is relative to a job, and one value type can need two different correctnesses.

The `Obj` handle, and how a Lisp runs on a non-Lisp machine

The `Obj(u32)` variant is the most conceptually interesting of the eleven, because it is how a machine that knows nothing about cons cells runs a Lisp, and its design rewards a careful read. The problem `elisprs` faces is that Emacs Lisp is built on mutable, shared, pointer-identity structure: a cons cell can be `setcar`'d to change its head in place, two variables can point at the *same* cons cell so that mutating one is visible through the other, and `eq` tests pointer identity, not structural equality. None of `fusevm`'s other variants can represent this. An `Array` is compared structurally, not by identity, so two `Arrays` with the same contents are `eq` when they should not be; and `fusevm` has no notion of a heap of mutable shared objects with identity. So `Obj` does not try to represent the object at all — it represents a *handle* to it, a `u32` index into `elisprs`'s own heap, where the actual cons cells live. `fusevm` carries the handle through its stack, slots, and globals without ever dereferencing it; `elisprs`'s extension handler owns the heap the handle points into. The handle *is* the identity — two `Objs` are `eq` exactly when their `u32`s are equal, which is pointer identity by another name — so `eq` and `setcar` work because the handle preserves the one property `Array` could not. This is the cleanest possible separation: the front end keeps the part of its semantics the core cannot model (a heap of identity-bearing mutable objects) entirely host-side, and hands the core only an opaque token it can shuffle around the stack. The core provides the stack, the control flow, and the arithmetic; `elisprs` provides the cons cells; and the `Obj` handle is the one-word bridge between them, which is also why the AOT native path can carry an `Obj` in its integer register lane — a handle is just an integer to everyone but the front end that owns the heap.

Constructors, and the `Arc` that makes strings cheap

`Value` ships a small set of constructors — `Value::int`, `Value::float`, `Value::str`, `Value::bool`, `Value::array`, `Value::hash`, `Value::status` — and the one worth a second look is `str`, because its signature encodes the cheap-clone property the rest of the machine depends on. `Value::str` takes anything that converts into a `String` (`impl Into<String>`) and wraps it in an `Arc`, so a string constructed from a `&str` literal and one constructed from an owned `String` produce the same `Value::Str(Arc<String>)`, and a test pins that the two are equal. The `Arc` is the load-bearing choice: it makes cloning a string `Value` a reference-count bump rather than a byte copy, which is why a closure can capture a string, a value can flow between two front ends, and a string can be pushed and popped across the stack many times without ever copying its bytes. A test confirms that cloning a `Value::str` shares the `Arc` — the clone and the original point at the same allocation, verified with `Arc::ptr_eq`. The other constructors are thin wrappers over their variants, present mostly for readable construction in front-end code and tests, but `str` is where a representational decision becomes an API guarantee: every string in the machine is reference-counted, so passing strings around is cheap, and the constructor is the front door through which that guarantee enters. It is a small API that quietly determines the cost of the single most common heap value a scripting language handles.

Chapter 4 – Stack, Slots, and Frames

`fusevm` holds running state in three places, and knowing which is which explains most of the opcode set.

The **operand stack** is a `Vec<Value>` and it is where computation happens. `Op::LoadInt(5)` pushes; `Op::Add` pops two and pushes one; `Op::Pop` discards. The stack is transient — values live on it only between the opcode that produces them and the opcode that consumes them. When a program is running natively (the AOT chapters), the operand stack is the thing that gets lifted into machine registers, and the boxed `Vec<Value>` sits empty.

Slots are the local variables. They live in a *frame*, not on the operand stack, and they are reached by index: `Op::GetSlot(0)` reads the zeroth local of the current frame and pushes it; `Op::SetSlot(0)` pops and stores. A frame's slots are a `Vec<Value>` that grows as needed, and reading an unwritten slot returns `Undef` — a detail the compiler must honor, because an uninitialized read is a real, observable thing, and the difference between `Undef` and a zero-initialized register is the subject of an entire later chapter.

Globals are a flat `Vec<Value>`, indexed by the name-table position, reached by `Op::GetVar` / `Op::SetVar`, with `Op::DeclareVar` introducing a fresh binding in the current scope. They persist for the life of the VM. Arrays and hashes that live in globals are mutated *in place* — `ArrayPush`, `ArrayPop`, `ArrayShift`, `ArraySet`, `HashSet`, `HashDelete` operate directly on the value in the globals vector, with no clone-modify-writeback cycle, and the read-only accessors (`ArrayGet`, `ArrayLen`, `HashGet`, `HashKeys`, `HashValues`) borrow straight from it. For a scripting language that builds up arrays and hashes element by element in a loop, that in-place discipline is the difference between linear and quadratic.

Frames and the call stack

A `Frame` records a `return_ip` (where to resume when the call returns), a `stack_base` (the operand-stack height at entry, so a return can truncate back), and the `slots` vector. `VM::new` seeds a single root frame, so even top-level code has somewhere to put its locals. A call (`Op::Call`, `Op::PushFrame`) pushes a new frame; a return pops it and restores the operand stack to the saved base. This is an ordinary call stack, and the only thing worth flagging is that slots are *per-frame* — a recursive function gets fresh slots each call, which is what recursion requires and what the slot-indexing model gives for free. The frame is also exactly the structure the tracing JIT must reconstruct when a compiled trace deopts inside an inlined callee, which is why `Frame` is part of the public API (`fusevm::Frame`): the compiler builds synthetic ones to hand back to the interpreter.

The VM struct and VMPool

The `VM` ties it together: the `stack`, the `frames`, the `globals`, the `ip`, the chunk being run, and a `last_status` (the shell's `$?` , a `Value::Status`). It also holds the embedding hooks — the shell host, the AWK host, the extension handlers — which the fourth movement covers, and, behind the `jit` feature, the JIT compiler and its caches. `run()` returns a `VMResult: Ok(Value)` with the program's result, `Error(String)` on a runtime fault, or `Halting` when the chunk ran off its end or a control-flow opcode stopped it. That three-way return is the contract every front end driver consumes.

A fresh VM is cheap to make — about a hundred and thirty nanoseconds for a small chunk — but a caller that runs *many* small chunks back to back (a REPL, a batch of `evals`, a per-record AWK rule) can skip even that with `VMPool`, which recycles `VM` instances. `acquire` pops a recycled VM and resets its state via `VM::reset`; `release` returns it; `with` brackets the two around a closure. The project is refreshingly honest about when

this helps: for a tiny three-op chunk the pool is actually *slower* (about a hundred and sixty-three nanoseconds against a hundred and thirty), because `reset` does more bookkeeping — dropping the old chunk, clearing globals, zeroing the deopt buffer — than `VM::new` skips. The pool wins when the chunk shape is non-trivial: a large name pool or globals vector whose capacity is worth preserving, many slots whose `Vec` capacity carries across reuse, an evaluation loop where the per-call allocation genuinely dominates. The API is shipped so the caller can choose, and all five front ends do choose it — they drive `fusevm::VM` through bridge layers that put a `VMPool` on the front end's `Runtime` so slot, stack, and globals capacities recycle across invocations.

The pool's shape in use is small enough to show whole, from the crate's own example:

```
use fusevm::{ChunkBuilder, Op, VMPool, VMResult, Value};

let mut pool = VMPool::new();
for _ in 0..1000 {
    let mut b = ChunkBuilder::new();
    b.emit(Op::LoadInt(40), 1);
    b.emit(Op::LoadInt(2), 1);
    b.emit(Op::Add, 1);
    pool.with(b.build(), |vm| {
        assert!(matches!(vm.run(), VMResult::Ok(Value::Int(42))));
    });
}
```

`pool.with` is the bracketing convenience: it acquires a recycled VM, resets it onto the supplied chunk, runs the closure, and releases the VM back to the pool — the same three-step `acquire/run/release` dance the front-end bridges drive directly when they want finer control. The honest performance note bears repeating in code terms: for this exact three-op chunk the pool is *slower* than a bare `VM::new` per iteration, because `reset` does more bookkeeping than `new` skips. The pool is for the case the example does not show — a thousand iterations over chunks with large name pools, many slots, or a tracing JIT already warmed — where the preserved vector capacities and the avoided re-allocation pay for the reset.

Three storage classes, one mental model

It is worth fixing the three storage classes against the opcodes that reach them, because nearly every opcode in the second movement is “do something, then move the result among these three places.” The operand stack is reached implicitly — almost every op pops its inputs from it and pushes its output to it, and the explicit stack ops (`Pop`, `Dup`, `Swap`, `Rot`) exist only to rearrange it. Slots are reached by the `GetSlot/SetSlot` pair and the slot super-ops, and they are frame-local and register-liftable. Globals are reached by `GetVar/SetVar/DeclareVar`, and they are VM-wide and persistent. A front end's register allocator — the part of it that assigns slot numbers — is choosing, for each of a program's variables, whether it lives in the fast frame-local slot space or the broader global space, and that single choice is the largest lever it has over how fast the resulting bytecode runs, because the compilers can hold a slot in a machine register and a global, mostly, in another machine register, but only if the front end put the variable somewhere the analysis can track. The storage model is small on purpose, and its smallness is what makes “which of three places” a question a compiler can answer.

What the VM owns

The `VM` struct is the machine made concrete, and enumerating what it owns is a useful map of everything the rest of the book elaborates. It owns the execution state — the `stack` (the operand stack), the `frames` (the call stack of `Frames`, each with its `return_ip`, `stack_base`, and `slots`), the `globals` vector, and the `ip` cursor. It owns the program — the `chunk` being run. It owns the shell's exit-status register, `last_status`, a `Value::Status`. It owns the embedding hooks — the optional shell host, the optional AWK host, the extension and wide-extension handlers, the builtin table — each installed through a setter and taken-out-and-restored across calls. And behind the `jit` feature it owns the JIT compiler and its caches, plus the deopt buffer the

tracing JIT and the AOT resume path write into. A `VM` is, in other words, exactly the union of the three storage classes, the program, the status, the embedding seams, and the optional compiler — nothing more — which is why it is cheap to construct (about a hundred and thirty nanoseconds) and cheap to reset. The fields are the chapters: the stack and frames and globals are this movement, the host-and-handler fields are the fourth movement, and the JIT fields are the fifth. Reading the struct definition is reading the table of contents of the machine.

Reset versus new

The `VMPool` recycling story turns on one method, `VM::reset`, and understanding what it does versus what `VM::new` does explains exactly when pooling pays. `VM::new` allocates a fresh VM: new stack, new frames with a seeded root frame, new globals sized to the chunk's name pool, a zeroed deopt buffer. `VM::reset` takes an existing VM and re-points it at a new chunk while *preserving the allocations*: it drops the old chunk, clears the globals and resizes them, truncates the stack and frames back to a root frame, and zeroes the deopt buffer — but the `Vec` capacities of the stack, the frame slots, and the globals survive, so a subsequent run that grows them to the same size pays no reallocation. This is the whole trade: `reset` does *more bookkeeping* than `new` skips (it has to clear and resize rather than start empty), but it *preserves capacity* that `new` would reallocate. For a tiny chunk that never grows its vectors, the bookkeeping dominates and `reset` loses; for a chunk that grows a large globals vector or many frame slots, the preserved capacity dominates and `reset` wins. The pool is the mechanism for amortizing capacity across a sequence of runs, and `reset` is the operation that does the amortizing — which is why the front-end bridges, which run many chunks with non-trivial shapes, all put a `VMPool` on their runtime and let `reset` recycle the vectors that would otherwise be reallocated on every invocation.

Chapter 5 – The Opcode Set: A Map

The opcode set is the machine’s vocabulary and its public contract. There are two hundred and twenty-four opcodes, declared as variants of one `enum Op` in `op.rs`, and the way to understand them is by section, because each section corresponds to a capability a front end can rely on. This chapter is the map; the five that follow walk the territory.

The seventeen sections, with their counts, are: constants and stack (twelve); variables and slots (seven); arrays and hashes (twenty); arithmetic (nine); string (three); comparison (fourteen); logical and bitwise (nine); control flow (five); functions and scope (five); I/O (three); collections (two); higher-order blocks (five); the fused superinstructions (eleven); the single `CallBuiltin` dispatch op (one, fronting a hundred and forty builtin IDs); the shell ops (twenty-nine); the AWK ops (sixty-one); and the two extension ops, `Extended` and `ExtendedWide`. The math intrinsics — the always-float transcendental family and the integer `Gcd/Lcm/Abs` group — live among the arithmetic-adjacent ops and are walked as their own chapter because the compilers treat them specially.

Three facts about the *encoding* are worth fixing before the walk, because they recur. First, operands are sized for their job: a `u16` for pool indices, which caps a chunk at sixty-four thousand names or constants; a `u8` for small inline counts like an argument count or a print arity; and a `usize` for jump targets, which never need to be small. Second, the whole enum is required by a test to fit in twenty-four bytes (`size_of::() <= 24`), because the dispatch loop streams over a `Vec<Op>` and a fat op would thrash the cache — so no variant may carry a large inline payload, which is part of why constants are pooled and referenced by index rather than carried inline. Third, `Op` has both a derived `PartialEq` and a hand-written `Hash` that mixes the discriminant with each variant’s payload (and hashes a float payload by bits), because the chunk’s `op_hash` runs over the op vector and must be stable, total, and collision-resistant across variants that happen to share a payload value — `Jump(7)` and `Call(7, 0)` must not collide, and a test pins that they do not.

The shape of a handler

Every opcode, in every execution tier, ultimately means whatever the interpreter says it means. The single function `VM::exec_op` is the one definition of each opcode’s behavior — a large `match` with one arm per variant — and the AOT threaded path, the deopt resume path, and the test harness all reach it through the same door. This is the discipline that keeps a five-strategy execution stack (interpreter, linear JIT, block JIT, tracing JIT, AOT) correct: there is exactly one place that decides what `Op::Add` does, and every other place either calls it or is differentially tested against it. Keep that fact close; it is the reason the rest of this book can describe four compilers without describing four semantics.

The remaining chapters of this movement walk the sections in roughly the order a program uses them: the data-movement ops that load and store; the arithmetic, string, and comparison ops that compute; the logic, bitwise, and control-flow ops that decide; the collection and higher-order ops that work on aggregates; the math intrinsics; and finally the fused superinstructions that are the machine’s namesake. The shell and AWK sections, large enough and embedding-specific enough to deserve it, are held for the fourth movement alongside the host traits they dispatch through.

An opcode is just a tag and a small payload

A useful intuition for the whole second movement is that an opcode is almost nothing — a tag and, at most, a small inline payload — and that this near-emptiness is the source of its speed. Reading `op.rs` confirms the

spareness: a nullary op like `Add` or `Pop` is a bare discriminant; a slot or name op like `GetSlot(u16)` carries two bytes; the inline-constant loads `LoadInt(i64)` and `LoadFloat(f64)` carry eight; and the largest variants — the three-field loop fusions like `SlotIncltIntJumpBack(u16, i32, usize)` — are sized so the whole enum still fits the twenty-four-byte budget a test enforces. There is no opcode that carries a string, an array, or any heap payload inline; those live in the constant pool and are reached by a `u16` index. The consequence is that the op vector is a dense, cache-friendly array of small fixed-size records, and walking it — the inner act of interpretation — is a sequential scan over memory the prefetcher loves. The fused superinstructions push exactly this logic one step further: by folding a seven-op loop tail into one twelve-or-sixteen-byte record, they replace seven cache-line touches and seven dispatches with one.

The discipline the Hash impl enforces

The hand-written `Hash` impl for `Op` looks like boilerplate and is in fact a correctness mechanism, because it is what makes `op_hash` — and therefore the entire JIT cache — trustworthy. Three properties matter. It mixes in `std::mem::discriminant` before any payload, so two ops that happen to share a payload value but differ in kind never collide: `Jump(7)` and `Call(7, 0)` hash apart, and a test pins it. It hashes a float payload by `to_bits`, so `LoadFloat` of `0.0` and of `-0.0` — equal under `PartialEq` but bit-distinct — hash differently, which is exactly what stops the cache from serving code compiled for one constant when the chunk actually holds the other. And it is *total*: every one of the two hundred and twenty-four variants has an arm, the nullary ones falling into a single catch-all that contributes only the discriminant, so there is no op whose identity the hash ignores. Because `op_hash` runs this impl over the whole op vector at build time, and because the JIT and disk cache key on `op_hash`, a single sloppy arm here would manifest as the cache occasionally running the wrong native code — which is why the impl is hand-written and exhaustively tested rather than derived.

Seventeen sections, and what they reveal

The opcode set is organized into seventeen sections, and the *shape* of that organization — which sections are large, which are small, which exist at all — is a portrait of what fusevm was built to do. The universal computational core is striking in its smallness: arithmetic is nine opcodes, string just three, control flow five, functions and scope five between them. A general-purpose stack machine could be built from these forty-odd opcodes and nothing else. The bulk of the two hundred and twenty-four is elsewhere, and where it is tells the story. The two largest sections are the AWK ops (sixty-one) and the shell ops (twenty-nine), which together are nearly forty percent of the instruction set — a machine that devotes that much of its vocabulary to records, fields, pipelines, and redirections is a machine built to host a shell and an AWK as first-class citizens, not as libraries. The arrays-and-hashtables section (twenty) is large because an aggregate data model is where scripting languages live. And the fused superinstructions (eleven) exist as their own section because performance, specifically the cost of dispatch in hot loops, was a first-order design concern rather than an afterthought. Read the section sizes as a budget, and the budget says: a small universal core, a large investment in the two languages the machine was built around, a serious aggregate-data model, and a dedicated line item for loop speed. The seventeen sections are not an arbitrary taxonomy; they are where the design spent its opcodes, and what a machine spends its opcodes on is what it was built to do.

One handler, five reachers

The discipline that `exec_op` is the single definition of every opcode's meaning is worth stating once more as a numbered fact, because it is the premise the rest of the book leans on hardest. There are five ways code runs in fusevm — the interpreter, the linear JIT, the block JIT, the tracing JIT, and the AOT native path — and there is exactly one definition of what each opcode does. The interpreter calls `exec_op` directly. The AOT threaded path calls it through the `fusevm_aot_exec_op` shim, op by op. The AOT deopt path resumes `VM::run`, which calls `exec_op`. And the three native-code paths — linear, block, AOT-native — do not call `exec_op` but are *differentially tested against* the interpreter that does, so their generated code is verified to agree with `exec_`

`Op`'s semantics on hundreds of thousands of random inputs. So of the five execution strategies, two literally call the one handler and three are mechanically checked against it. This is why a five-strategy execution stack does not fracture into five subtly different languages: there is one source of truth for meaning, reached directly where possible and verified against where not, and every chapter that describes a compiler is, underneath, describing a translation of `exec_op` into faster code that was then proven to still mean what `exec_op` means.

The twenty-four-byte budget, and what it forces

The test that `size_of::<Op>() <= 24` looks like a micro-optimization and is in fact a design constraint that shapes the whole instruction set, so it is worth tracing what it forces. The `Op` vector is a `Vec<Op>` the dispatch loop streams over sequentially, and the cost of that stream is dominated by cache behavior: each cache line holds a fixed number of bytes, so a smaller `Op` means more opcodes per line, fewer line fills, and a hotter loop. Capping `Op` at twenty-four bytes — three machine words — keeps the vector dense. But the cap also *forbids* things, and the forbidden things explain several design choices elsewhere. It forbids carrying a string or an array inline in an opcode, which is why constants are pooled and referenced by a `u16` index rather than embedded — a `LoadConst(u16)` is four bytes where an inline string would be a fat pointer plus length plus capacity. It forbids an opcode with many large fields, which is why the three-field loop fusions use a `u16`, an `i32`, and a `usize` (sized to fit) rather than three `i64`s. And it is enforced by a test precisely because a careless addition — a new opcode variant carrying an inline `String`, say — would silently bloat the enum past the budget and slow every chunk's dispatch, a regression with no visible error. The twenty-four-byte cap is the reason the constant pool exists, the reason names are interned and indexed, and the reason the largest opcodes are carefully sized: a small constraint at the type level that propagates outward into how the whole machine represents its program.

Nop's two jobs

The humblest opcode, `Nop`, does nothing and yet earns its place for two distinct reasons, both worth naming because they show how even a no-op participates in the machine's design. Its first job is *measurement*: a `Nop` is a unit of pure dispatch, so a loop of them measures the interpreter's dispatch overhead with no operation cost mixed in, which is exactly the 1.22-billion-operations-per-second floor the benchmark suite reports. Without an opcode that does provably nothing, that floor could not be measured cleanly, because every other opcode does *something* whose cost would contaminate the reading. Its second job is as a *branch sentinel* — a placeholder the compiler can emit where it needs an instruction to exist (a jump target, a patch site) but has no work to put there yet. A front end's single-pass compiler occasionally needs a stable instruction slot before it knows what will occupy it, and a `Nop` is a safe filler that the dispatch loop runs harmlessly. That a do-nothing opcode has two real jobs is a small illustration of the machine's economy: nothing in the instruction set is decorative, and even the empty operation is there because two different parts of the system — the benchmark harness and the front-end compilers — genuinely need a way to spell “do nothing.” The doc comment on `Nop` names both jobs precisely — “consumed for cycle-counting and as a branch sentinel” — which is the kind of small, honest documentation that runs through the whole source.

Chapter 6 – Loads, Stack, and the Naming of Storage

The first thing any program does is get a value onto the stack, and the **load** ops do exactly that. `LoadInt(i64)` and `LoadFloat(f64)` carry their value inline — small enough to fit the twenty-four-byte budget — and push it. `LoadConst(u16)` pushes a value from the constant pool, which is how anything that will not fit inline (a string, a regex, a format string) reaches the stack. `LoadTrue`, `LoadFalse`, and `LoadUndef` push the three constant singletons without consuming a pool slot, because a boolean and the absence of a value are common enough to deserve their own one-byte opcodes. That is six ops, and between them they are the only way a literal enters a running program.

The **stack** ops shuffle the top without computing anything: `Pop` discards the top; `Dup` duplicates it; `Dup2` duplicates the top *two* (so `a b` becomes `a b a b`, which a compound assignment needs); `Swap` exchanges the top two; and `Rot` rotates the top three (`a b c` → `b c a`). These five are the plumbing of a stack machine — the way a front end arranges operands for the op that will consume them — and they are deliberately minimal, because every stack op the front end can avoid emitting is a dispatch the VM can avoid running. Together with the six loads and the `Nop` (a no-op kept for cycle-counting and as a branch sentinel), these make the twelve-op “constants and stack” section.

Three namespaces, reached three ways

The **variable and slot** section is seven ops, and it is where fusevm’s storage model becomes visible. There are three ways to name storage, and they trade generality for speed. The slowest and most general is the **global**: `GetVar(u16)` and `SetVar(u16)` reach a global by its name-pool index, and `DeclareVar(u16)` introduces a fresh binding. A global access is a vector index, not a hash lookup — the name has already been resolved to a number at compile time — but it reaches into the VM-wide globals vector, which is the broadest scope and the one that persists for the life of the machine.

The faster path is the **slot**: `GetSlot(u16)` and `SetSlot(u16)` reach a local in the current frame by index. The difference from a global is not the indexing — both are numeric — but the locality: a slot lives in the frame, gets fresh storage on every call, and is the storage the JIT and AOT compilers lift into machine registers. When a later chapter says a counted loop runs “entirely in registers,” it means its induction variable was a slot, and `GetSlot/SetSlot` became `use_var/def_var` on a Cranelift `Variable` rather than a load and store against the boxed frame. Slots are the reason the machine is fast, and they are why a front end’s compiler works hard to assign every local a slot number.

Two more ops in this section, `SlotArrayGet(u16)` and `SlotArraySet(u16)`, are a fusion in miniature: they index *into* an array that lives in a slot, taking the index from the stack and reaching the array through the slot in one op rather than loading the array onto the stack first. They exist because indexing a local array in a loop is common enough that saving the intermediate `GetSlot` is worth a dedicated opcode — the same bias toward the hot path that the fused superinstructions chapter makes its whole subject.

A literal’s journey onto the stack

To see the loads and stack ops working together, follow the expression `2 + 3 * 4` from source to bytecode. A front end parses it into a tree honoring precedence, then emits a post-order walk: load `2`, load `3`, load `4`,

multiply, add. As a chunk:

```
b.emit(Op::LoadInt(2), 1); // stack: [2]
b.emit(Op::LoadInt(3), 1); // stack: [2, 3]
b.emit(Op::LoadInt(4), 1); // stack: [2, 3, 4]
b.emit(Op::Mul, 1);        // stack: [2, 12]
b.emit(Op::Add, 1);        // stack: [14]
```

Nothing here names a variable, allocates, or branches; the stack is a scratchpad that fills and drains as the expression evaluates, and at the end exactly one value remains. This is the canonical shape of every expression in every front end, and it is why the stack ops are so few: a well-structured expression needs almost no explicit shuffling, because post-order emission naturally leaves each operand where the next op expects it. The shuffles — [Dup](#), [Swap](#), [Rot](#), [Dup2](#) — earn their place only at the seams: a compound assignment `x += 1` that needs the old value both to add to and to store back, a [Dup2](#) that lets an array index-and-store reuse the array and the index, a [Swap](#) that fixes operand order for a non-commutative op whose source order differs from its stack order. A front end that emits clean post-order bytecode emits very few shuffles, and every shuffle it avoids is a dispatch the VM skips.

Declaration is not assignment

The variable section's third op, [DeclareVar\(u16\)](#), is worth a paragraph because the distinction it draws — declaring a binding versus assigning to one — is a real semantic difference the machine preserves. [SetVar](#) writes a value into an existing global; [DeclareVar](#) introduces a *fresh* binding in the current scope, which matters for a language with lexical scoping where an inner `my $x` should shadow an outer one rather than overwrite it. The core does not implement scope rules itself — those are the front end's, expressed in which name indices it chooses and when it emits [DeclareVar](#) — but it provides the op so the front end can mark the difference in the bytecode, and so the analysis in the compilers can see where a binding comes into existence. The array and hash sections carry the same triple — [GetArray/SetArray/DeclareArray](#), [GetHash/SetHash/DeclareHash](#) — for the same reason: the act of bringing a name into scope is distinct from the act of writing through it, and the opcode set keeps the two distinguishable rather than collapsing them into a single store.

Slots versus globals, in the generated code

The distinction between a slot and a global is invisible in the source — both are just variables — but it is the single most consequential choice a front end's compiler makes per variable, because the two reach the machine through different storage and lower to different native code. A global is an index into the VM-wide [globals](#) vector; a slot is an index into the current frame's [slots](#). In the interpreter both are vector reads, comparably fast. But in the compilers they diverge: a slot becomes a Cranelift [Variable](#), lifted into a machine register for the duration of a loop, never touching memory between writes; a global becomes a register too in the native path, but it is VM-wide state the analysis must be more careful about, and a global aliased by a named array disqualifies a chunk from native lowering entirely. The practical guidance this implies for a front end is sharp: a variable that lives and dies within a function, especially a loop counter or accumulator, should be a slot, because the compilers can hold a slot in a register and a loop over slots is the case the whole machine is tuned for; a variable that must persist across calls or be visible program-wide is a global, with the understanding that it is less aggressively optimized. The front end's choice of slot versus global for each variable is, in effect, a hint to the compiler about whether the variable can live in a register, and getting that choice right is the largest lever a front end has over how fast its bytecode runs.

Why fast paths live in the interpreter

It might seem that the place to hand-tune integer arithmetic is the compiler, where it becomes a single machine instruction, and that the interpreter's fast paths are a lesser concern. The opposite is true, and the reason is the tiered-execution philosophy of the third movement: most code runs interpreted, because most code is cold, and a fast path in the interpreter benefits every chunk on its first run, before any compilation threshold is crossed. The compilers help the hot code that proves itself; the interpreter's fast paths help everything, including the vast majority of code that runs a handful of times and is never compiled at all. This is why `arith_int_fast` and `cmp_int_fast` exist and are hand-tuned: they make the common case — integer arithmetic on two integers — fast in the tier that runs the most code. And because the interpreter's fast path is also the specification the compilers match, tuning it is doubly valuable: it speeds cold code directly and defines the behavior the compiled hot code must reproduce. The interpreter is not a slow fallback the compilers exist to escape; it is the workhorse that runs most of every program, and it is optimized accordingly, with the compilers handling the minority of code hot enough to repay their cost.

Chapter 7 — Arithmetic, Strings, and Comparison

The **arithmetic** section is the dense core of any numeric program, and it is nine ops: `Add`, `Sub`, `Mul`, `Div`, `Mod`, `Pow`, `Negate`, `Inc`, `Dec`. The binary four pop two values and push one; `Negate` is unary; `Inc` and `Dec` adjust the top of stack in place by one. Their semantics are exactly the `Value` coercions of Chapter 3 made operational: integer arithmetic on two integers stays integer and *wraps* (the interpreter's fast path passes `i64::wrapping_add` and friends), and the moment either operand is non-integer both promote to `f64`. That “both `Int`, else float” rule is not a convenience; it is a specification the compilers reproduce instruction by instruction, down to inserting the integer-to-float conversion exactly where the interpreter's `to_float` would have run.

Two arithmetic ops carry behavior worth naming now because it returns later. `Div` yields a `Float` for ordinary operands but `Undef` for division by zero — `Undef`, not a NaN and not a trap — which is a shell-arithmetic semantic, and which is precisely the value a register cannot hold, making `Div` the first opcode the native compiler has to handle with a `deopt` rather than a pure `fdiv`. `Mod` on floats is an `fmod`, computed through a runtime helper because there is no machine modulo instruction for floating point. `Pow` is wrapping-integer for two integer operands and `f64::powf` otherwise, again through a helper, because no CPU has a power instruction either. These three — `Div`, `Mod`, `Pow` — are the arithmetic ops that force the compilers to reach outside the pure-instruction world, and the chapters on libcalls and `deopt` are largely about them.

Strings, and why there are only three

The **string** section is just three ops — `Concat` (the `.` operator), `StringRepeat` (Perl's `x`), and `StringLen` — and the smallness is the point. A front end's string handling is mostly *its own*, expressed through builtins and extension ops, because string semantics differ wildly between a shell and an elisp. What the core provides is the irreducible minimum that every front end needs: gluing two strings, repeating one, and measuring one. `Concat` runs through `as_str_cow`, so concatenating a `Str` with a `Str` borrows both inner buffers rather than reallocating to read them — the borrowed coercion path of Chapter 3 earning its keep on the single most common string operation in any scripting language.

Comparison, split two ways

The **comparison** section is fourteen ops, and it is split down the middle by a distinction every Perl-descended language insists on: *numeric* comparison versus *string* comparison are different operators. The numeric seven are `NumEq`, `NumNe`, `NumLt`, `NumGt`, `NumLe`, `NumGe`, and the spaceship `Spaceship` (`<=>`, returning `-1`, `0`, or `1`); the string seven are `StrEq`, `StrNe`, `StrLt`, `StrGt`, `StrLe`, `StrGe`, and `StrCmp` (the string spaceship `cmp`). The reason fusevm has both, rather than one polymorphic compare, is that `"10" < "9"` is true as strings and false as numbers, and the front end — which knows from the source whether the programmer wrote `<` or `lt` — picks the opcode that means what was written. The numeric comparisons follow IEEE for floats, so a NaN compares unequal to everything including itself, and the native compiler chooses Cranelift's ordered and unordered `fcmp` conditions precisely so that compiled comparisons agree with the interpreter's on NaN — another place the single source of truth is reproduced rather than reinvented.

The fast path is the specification

The arithmetic ops are the clearest case of a principle that governs the whole machine: the interpreter's *fast path* is not merely an optimization but the authoritative definition the compilers must match. `arith_int_fast`, the interpreter helper behind `Add/Sub/Mul/Mod`, peeks the top two stack values, and when both are `Int` it applies `i64::wrapping_add` (or the corresponding wrapping operation) and truncates the stack by one. Two facts in that sentence are load-bearing for everything downstream. First, the arithmetic *wraps* rather than panicking on overflow, so `i64::MAX + 1` is `i64::MIN`, not a crash — and the native compiler emits a wrapping `iadd` to agree, because a checked add would diverge on exactly the inputs a fuzzer loves to find. Second, integer modulo by zero yields zero rather than trapping, which is why the native path's `Mod` guards the `y == 0` case with a `select` that produces zero rather than letting a hardware `srem` fault. When a later chapter says the native arithmetic “matches the interpreter bit-for-bit,” this is the bit-for-bit it means: the same wrapping, the same mod-by-zero, the same truncation direction, reproduced in machine code because the interpreter is the specification and the compiler is its translation.

Why two spaceships

The presence of *both* a numeric `Spaceship` (the `<=>` operator) and a string `StrCmp` (the `cmp` operator) in the comparison section is a small monument to the Perl heritage fusevm hosts, and it is worth understanding why a machine that wants to be small keeps two three-way comparisons. The answer is that the front end knows, from the source, which kind of ordering the programmer asked for, and the two orderings genuinely differ. Sorting `["10", "9", "100"]` numerically yields `["9", "10", "100"]`; sorting them as strings yields `["10", "100", "9"]`, because the character `1` sorts before `9` lexically. A single polymorphic compare would have to *guess* which the programmer meant — usually by sniffing whether the operands look numeric, which is exactly the fragile heuristic that makes shell comparisons surprising. fusevm refuses the guess: `Spaceship` always compares as numbers, `StrCmp` always as strings, and the front end picks the opcode that matches the operator the source used. The same split runs through all fourteen comparison ops, and it is the reason `SortBlock` can be handed a numeric or a string comparator without the machine ever having to infer intent.

Inc, Dec, and the slot super-ops they anticipate

Two of the nine arithmetic opcodes, `Inc` and `Dec`, adjust the top of stack by one, and they sit at an interesting place in the design because they anticipate the slot super-operations the fusion chapter describes. A bare `Inc` operates on the operand stack: it pops the top, adds one, pushes the result. That is useful, but the overwhelmingly common increment is not “increment the top of stack” but “increment a local variable” — a loop counter, an accumulator index — which as bare opcodes is `GetSlot`, `LoadInt(1)`, `Add`, `SetSlot`, four dispatches and stack traffic. The slot super-ops exist precisely to collapse this: `PreIncSlot` increments a slot and pushes the new value, `PreIncSlotVoid` does it pushing nothing, `PostIncSlot` pushes the old value. So `Inc` and `Dec` are the general stack-level primitives, and the slot increment family is their specialization for the common case of incrementing a named local, with the void variants for when the value is not used. Reading them together shows the layering the whole opcode set uses: a general primitive for the general case (`Inc` on the stack), a specialized fused opcode for the common case (`PreIncSlot` on a slot), and a void variant for the common-case-without-a-result (`PreIncSlotVoid`). A front end's peephole emits the most specialized form the situation allows, and the dispatch loop, the interpreter fast paths, and the compilers each have one handler per form — the same one-recognized-pattern-one- opcode economy that the fused superinstructions raise to the level of whole loops.

Chapter 8 – Logic, Bitwise, and Control Flow

The **logical and bitwise** section is nine ops, and it bundles two families that look alike and behave differently. The logical three — `LogNot`, `LogAnd`, `LogOr` — are the *eager* boolean operators: they evaluate both operands and push a boolean. The comment in `op.rs` flags the contrast that makes them eager rather than short-circuiting: short-circuit `&&` and `||` are *not* these ops; they are control-flow jumps (the `JumpIf*Keep` family), because short-circuiting means *not evaluating* the right operand, which is a branch, not an operator. `LogAnd` and `LogOr` exist for the cases where both sides are already on the stack and the front end wants the boolean combination without a branch. The bitwise six — `BitAnd`, `BitOr`, `BitXor`, `BitNot`, `Shl`, `Shr` — are the integer bit operators, and they coerce their operands through `to_int` before operating, which is why a float or a string can be bit-anded at all.

Five ops for all control flow

The **control flow** section is, remarkably, five ops, and they express every loop, conditional, and short-circuit in every front end. `Jump(usize)` is unconditional. `JumpIfTrue(usize)` and `JumpIfFalse(usize)` pop the top of stack, consult `is_truthy`, and branch — these are the workhorses of `if` and `while`. The two “keep” variants, `JumpIfTrueKeep` and `JumpIfFalseKeep`, test the top *without consuming it*, and they are how short-circuit operators are lowered: `a || b` evaluates `a`, and `JumpIfTrueKeep` leaves `a` on the stack and skips `b` when `a` is already true, so the expression’s value is the operand that decided it. That the whole of structured control flow reduces to five jump opcodes is the flat-chunk design paying off: there is no loop opcode, no if opcode, only targets and conditions, and the front end’s job is to compute the targets.

Because the targets are plain `usize` offsets into the op vector, a *backward* jump — a target less than the current instruction pointer — is, by definition, the bottom of a loop. The tracing JIT arms its recorder at exactly these backward branches, the block compiler discovers loop structure by finding them, and the fused superinstructions collapse the common increment-test-and-branch-back idiom that sits at every counted loop’s foot. The whole machine reads loop structure off the direction of these five ops.

Functions and scope

The **functions** section is three ops and the **scope** section two, and together they manage the call stack. `Call(u16, u8)` calls a subroutine by name-pool index with an argument count, resolving the name through the chunk’s `sub_entries` table to find the entry instruction pointer. `Return` returns with no value (it pushes `Undef`), and `ReturnValue` returns using the top of stack as the result — the distinction matters because a function called in void context and one called for its value lower differently. `PushFrame` and `PopFrame` bracket a new frame on the frame stack, giving the called code its own fresh slots and recording where to resume. These five are the entirety of fusevm’s calling convention, and their simplicity is what lets the tracing JIT *inline* a call: resolving `Call` to the callee’s bytecode IP and splicing the callee’s ops into the trace is tractable precisely because a call is a name lookup and a frame push, not a complex protocol.

The **I/O** section rounds out the universal core with three ops: `Print(u8)` and `PrintLn(u8)` emit a given number of values from the stack to stdout (the latter with a trailing newline), and `ReadLine` reads a line from stdin and pushes it as a string. These are deliberately thin — a front end’s real I/O goes through builtins and

the host — but they are universal enough, and useful enough in the compiled native path as *sink* opcodes, that the core names them directly.

A loop in five opcodes

The claim that all control flow reduces to five jump opcodes is best believed by building a loop out of them. Here is `while (i < 10) { i += 1 }`, with `i` in slot zero, lowered the way a front end would before its peephole runs:

```
let top = b.current_pos();           // loop header (backward-jump target)
b.emit(Op::GetSlot(0), 1);           // push i
b.emit(Op::LoadInt(10), 1);          // push 10
b.emit(Op::NumLt, 1);                // push (i < 10)
let exit = b.emit(Op::JumpIfFalse(0), 1); // forward jump out - patch later
b.emit(Op::PreIncSlotVoid(0), 1);     // i += 1, no stack push
b.emit(Op::Jump(top), 1);             // backward jump to header
let after = b.current_pos();
b.patch_jump(exit, after);           // exit lands past the loop
```

Two jumps carry the whole loop: the forward `JumpIfFalse` that leaves when the condition fails, patched once its target is known, and the backward `Jump` to `top` whose target was known from the start. That backward jump is the structural signal the rest of the machine reads — the tracing JIT arms its recorder there, the basic-block analyzer marks `top` as a leader, and a front end's loop peephole recognizes this exact six-op shape and rewrites the `PreIncSlotVoid`-plus-compare- plus-backward-jump tail into a single `SlotIncLtIntJumpBack(0, 10, top)`. The five control-flow opcodes are deliberately primitive precisely so that this recognition is possible: there is no opaque “loop” instruction hiding the structure, only targets and conditions a peephole and an analyzer can both see.

The keep variants and short-circuiting

The two “keep” jumps repay a closer look because they encode short-circuit evaluation, which a naive reading might expect from the logical ops and which the logical ops deliberately do *not* provide. Consider `a || b`. The value of the expression is `a` when `a` is truthy and `b` otherwise, and crucially `b` must not be evaluated at all when `a` is truthy. A front end lowers it by evaluating `a`, then `JumpIfTrueKeep` to the end: the “keep” means `a` stays on the stack as the expression's value, and the jump skips the code that would evaluate `b`. Only when `a` is falsy does control fall through, `Pop` discard the falsy `a`, and `b`'s code run to leave `b` as the value. The eager `LogOr` op, by contrast, evaluates both operands and pushes a plain boolean — useful when both sides are cheap and already computed, wrong when the right side has effects or cost that the short-circuit must avoid. The split is the same discipline as the two spaceships: the front end knows from the source whether the programmer wrote a short-circuiting `||` or a bitwise- style eager combination, and it emits the control-flow form or the operator form accordingly.

Return and ReturnValue, and why both

The functions section draws a distinction that looks redundant and is not: `Return` and `ReturnValue` are two opcodes, one returning nothing (pushing `Undef`) and one returning the top of stack. A reader might ask why a single return op could not always take the top of stack — return `Undef` by pushing it first if there is nothing to return. The answer is about context and efficiency, and it mirrors the void-context fused superinstructions. A function called for its value — `x = f()` — must leave a result; a function called as a statement — `f()`; — leaves nothing, and forcing it to push and then immediately pop an `Undef` is wasted work in the common case. By having two return opcodes, the front end emits `ReturnValue` when the call site wants a value and `Return` when it does not, so a void call returns with no spurious push and a value call returns exactly the computed value. This is the same density instinct as `AddAssignSlotVoid` and `PreIncSlotVoid`: avoid

producing a value the program will not use. The distinction also matters to the tracing JIT's call inlining, which truncates the abstract stack to the frame's entry mark on `Return` and additionally carries the returned value on `ReturnValue` — two different stack effects the inliner must model correctly, and modeling them is simpler when the bytecode already distinguishes them than when it would have to infer void-ness from context.

I/O ops as compilation sinks

The three I/O opcodes — `Print`, `PrintLn`, `ReadLine` — are thin by design, but `Print` and `PrintLn` earn a second look because they play an outsized role in the compiler movement as *sink* opcodes. A sink is an op that consumes scalar values and produces nothing dynamic, and `Print(n)` is the canonical example: it pops `n` values and emits them, leaving the stack shorter and producing no value the surrounding code consumes. This property is what lets a hot numeric loop with an embedded `print` compile to native code: the AOT native path spills the print's operands from registers to the boxed stack, calls the print through the `exec_op` shim, and continues in registers, because the print took exactly what it was given and gave nothing back. A loop that prints its running total every iteration would, without sink handling, force the whole loop to fall back to the threaded path; with it, the arithmetic stays native and only the print briefly touches the boxed world. So the I/O section, despite being three of the smallest opcodes in the set, is where the boundary chapter's "spilling for sinks" begins — `Print`, `PrintLn`, and the status-setting `SetStatus` are the three sinks the native path handles — and the thinness of the I/O ops is exactly what makes them clean sinks: a `Print` that returned a value, or mutated dynamic state the loop read back, would not be a sink and could not be spilled-and-continued so simply.

Chapter 9 – Arrays, Hashes, and the Collection Model

The **arrays and hashes** section is the largest of the universal sections — twenty ops — because an aggregate data model is where a scripting language spends much of its time, and fusevm gives both the ordered list and the associative array first-class opcodes.

The array ops come in two flavors, distinguished by where the array lives. The *named* accessors reach an array variable by name-pool index: `GetArray(u16)`, `SetArray(u16)`, `DeclareArray(u16)` move the whole array, and `ArrayGet`, `ArraySet`, `ArrayPush`, `ArrayPop`, `ArrayShift`, `ArrayLen` operate on the array named by the index, taking their other operands from the stack. The crucial property, stated in the README and load-bearing for performance, is that the mutating ops work *in place* on the value sitting in the globals vector — there is no clone-modify-writeback. `ArrayPush(idx)` pushes the stack top onto the array at `idx` without ever materializing the array on the operand stack. `MakeArray(u16)` is the constructor: it pops `N` values and pushes them as a single `Array`, which is how an array literal `[a, b, c]` is built.

The hash ops mirror the arrays one for one: `GetHash`, `SetHash`, `DeclareHash` for the whole hash; `HashGet`, `HashSet`, `HashDelete`, `HashExists`, `HashKeys`, `HashValues` for the contents; and `MakeHash(u16)` to build one from `N` key-value pairs popped off the stack. `HashGet` takes a key from the stack and pushes the value; `HashExists` pushes a boolean; `HashKeys` and `HashValues` push arrays. The same in-place discipline applies: `HashSet` and `HashDelete` mutate the hash in the globals vector directly, and the read-only accessors borrow from it. Because a `Hash` is a `HashMap<String, Value>`, every key is a string — the front end stringifies whatever the source used as a subscript through `as_str_cow` before it reaches the opcode, which is why an integer and the string of that integer index the same hash slot.

Ranges and the boundary of the core

The **collections** section is two ops, `Range` and `RangeStep`, which build an array from bounds: `Range` pops a `from` and a `to` and pushes the array of integers between them; `RangeStep` adds a step. They are in the core rather than a front end because `1..10` is universal enough — every front end has some range syntax — and because building the array once, in a single op, beats a loop of pushes. They also mark a boundary: beyond simple ranges and the basic aggregate operations, collection processing becomes the higher-order ops of the next chapter, and beyond *those* it becomes front-end builtins. The core provides the spine — make, get, set, push, pop, the range — and lets each front end grow the rest.

In-place mutation, and the quadratic it avoids

The single most consequential property of the array and hash ops is one the README states flatly and a performance-minded reader should dwell on: the mutating accessors operate *in place* on the value living in the globals vector, with no clone-modify-writeback. Picture the alternative. A naive aggregate model treats a value as immutable: to push onto an array, you read the whole array onto the stack, append, and write the whole array back. In a loop that builds an array of `N` elements, that is `N` reads and `N` writes of an ever-growing array — $O(N^2)$ total work and $O(N^2)$ bytes copied — and it is the classic reason naive scripting languages fall off a cliff on large collections. fusevm's `ArrayPush(idx)` instead reaches the `Vec<Value>` sitting in `globals[idx]` and pushes onto it directly; the array never touches the operand stack, never gets cloned, and the loop is

honestly linear. The same holds for `ArrayPop`, `ArrayShift`, `ArraySet`, `HashSet`, and `HashDelete`, and the read-only accessors — `ArrayGet`, `ArrayLen`, `HashGet`, `HashExists`, `HashKeys`, `HashValues` — borrow from the globals vector rather than copying out of it. This is why the `PushIntRangeLoop` fused superinstruction can build an array of a million integers in a tight native loop: there is no per-element copy for it to eliminate, only the per-element dispatch, because the in-place model already eliminated the copy.

Hashes are string-keyed, and what that costs

A `Hash` is a `HashMap<String, Value>`, and the choice of `String` keys — rather than arbitrary `Value` keys — is a deliberate simplification with a visible consequence the front ends rely on. Because every key is a string, the machine stringifies whatever the source used as a subscript before it reaches `HashGet` or `HashSet`, running it through `as_str_cow`. The upshot is that `$h{1}` and `$h{"1"}` index the *same* slot, because the integer `1` and the string `"1"` produce the same key text — exactly the associative-array semantics Perl and AWK programmers expect, where a numeric subscript is silently a string subscript. AWK takes this further with its `SUBSEP` multi-dimensional emulation, joining several subscripts with a separator into one string key, and because the core hash is already string-keyed, the AWK host gets that for free rather than needing a separate multi-key map. The cost is that a key must be stringified to be looked up, which is an allocation when the subscript is not already a string — but the `as_str_cow` borrow keeps it free when it is, and the string-key model is what makes the hash behavior identical across the five front ends that disagree about almost everything else.

The stack effects, tabulated

The array and hash opcodes each have a precise stack effect — what they pop and what they push — and tabulating them is the clearest way to see how the named-variable ops separate the *container* (named by the opcode's `u16` index) from the *operands* (taken from the stack). For the array ops, with `A` the array named by the index:

Op	Pops	Pushes	Effect
<code>ArrayGet(a)</code>	index	value	<code>A[index]</code>
<code>ArraySet(a)</code>	value, index	—	<code>A[index] = value</code>
<code>ArrayPush(a)</code>	value	—	append to <code>A</code> in place
<code>ArrayPop(a)</code>	—	value	remove and return last
<code>ArrayShift(a)</code>	—	value	remove and return first
<code>ArrayLen(a)</code>	—	length	element count of <code>A</code>
<code>MakeArray(n)</code>	<code>n</code> values	array	build an array literal

The hash ops mirror them, with `H` the hash named by the index:

Op	Pops	Pushes	Effect
<code>HashGet(h)</code>	key	value	<code>H[key]</code>
<code>HashSet(h)</code>	value, key	—	<code>H[key] = value</code>
<code>HashDelete(h)</code>	key	deleted value	remove <code>H[key]</code>
<code>HashExists(h)</code>	key	bool	<code>key in H</code>
<code>HashKeys(h)</code>	—	array of keys	the keys of <code>H</code>
<code>HashValues(h)</code>	—	array of values	the values of <code>H</code>
<code>MakeHash(n)</code>	<code>n</code> key/value pairs	hash	build a hash literal

Read down the “pops” column and the in-place discipline is visible: the mutating ops (`ArrayPush`, `ArraySet`, `HashSet`, `HashDelete`) push *nothing*, because they mutate the named container in the globals vector rather

than producing a new one, and the container itself never reaches the operand stack. The only ops that put a whole aggregate on the stack are `MakeArray`, `MakeHash`, and the key/value-list producers, which genuinely construct a new value. This is the table form of the chapter's central claim — that aggregates are mutated in place, not clone-modify-written-back — and it is why a loop of `ArrayPush` is linear: each push pops one value and mutates the container, with no aggregate copied onto or off of the stack.

Chapter 10 — Higher-Order Blocks and Ranges

The **higher-order** section is five ops, and it is where fusevm steps slightly beyond “stack machine primitive” toward “scripting-language operation,” because `map`, `grep`, and `sort` are operations every Perl-descended language has and none expresses comfortably as a bare loop. `MapBlock(u16)`, `GrepBlock(u16)`, `SortBlock(u16)`, and `ForEachBlock(u16)` each take a `u16` that is *not* a name or a constant index but a **block index** — an entry into the chunk’s `block_ranges` table, which records the `(start_ip, end_ip)` of the block body. `SortDefault` is the fifth, a sort with the default string comparison that needs no block at all.

The block-range mechanism is the interesting part, because it is how fusevm represents a *body of code as data* without a closure object. When a front end compiles `map { $_ * 2 } @xs`, it emits the doubling body into the chunk’s op vector like any other code, registers its `(start, end)` range with `add_block_range` to get an index, and emits `MapBlock(index)` over the array. The VM, executing `MapBlock`, runs the op range once per element, with the element installed where the body expects it, and collects the results. The body is not a separate chunk and not a heap-allocated closure; it is a labeled span of the same flat op vector, which is the cheapest possible representation and the one most amenable to the compilers, since the body’s ops are right there in line.

This is also where the higher-order ops show the limits of the universal core. The block runs with whatever topic-variable convention the front end established when it compiled the body; fusevm does not know what `$_` is, only that the body is a span to run per element. The coordination — what the iteration variable is called, how the result is accumulated, whether `grep`’s predicate is truthy by shell rules or by the front end’s own — lives in how the front end lowered the body and which ops it chose. The core provides the iteration machinery; the meaning of the iteration is the front end’s. It is the same division of labor as everywhere else: fusevm supplies the fast, general mechanism, and the language on top supplies the semantics.

A block is a labeled span, not a closure

The deepest thing to understand about the higher-order ops is the representational decision behind the block index, because it is what makes `map` and `grep` compilable rather than opaque. A conventional language compiles `map { ... } @xs` to a *closure* — a heap-allocated object pairing a function pointer with captured environment — and then calls that closure once per element through an indirect call the optimizer usually cannot see through. fusevm does not allocate a closure. The block body is emitted into the *same* flat op vector as the surrounding code, its `(start_ip, end_ip)` recorded in the chunk’s `block_ranges` table, and the higher-order op carries only the `u16` index into that table. When the VM runs `MapBlock(idx)`, it runs the op range `[start, end)` once per element, in line, with no indirect call and no allocation. The body’s ops are right there in the chunk for the compilers to analyze, which is why a numeric `map` body can in principle be lowered as natively as any other loop — there is no closure boundary hiding it.

The trade is that a block, being a labeled span rather than a first-class value, cannot be stored in a variable, returned, or passed around the way a closure can — it exists only at the site that names it. For the operations the higher-order ops serve — `map`, `grep`, `sort`, `foreach` — that is exactly the right trade, because those operations consume their block immediately and never need it to outlive the call. A front end that wants genuine first-class functions builds them on top, with its own representation behind an `Obj` handle or a `sub_entries` reference; the higher-order ops are for the common case where a block is a one-shot transformation, and they make that common case cheap by refusing to allocate for it.

Ranges, and the boundary between core and front end

The two collection opcodes, `Range` and `RangeStep`, sit at an instructive boundary, because they are the last operation the core provides before collection handling becomes either a fused superinstruction or a front-end builtin, and the placement says something about how the core decides what to own. A range is universal — every front end has some `1..10` syntax — and building the array in one opcode beats a loop of pushes, so the core owns it. But the core stops there: it does not own `map` over a range as a single fused operation in general, it does not own list comprehensions, it does not own lazy ranges. Those it leaves to the higher-order block ops, to the fused loops like `PushIntRangeLoop` (which a front end's peephole emits when it recognizes a push-loop over a range), or to front-end builtins. The line is drawn at “operations general enough that every front end needs them and simple enough to express in one opcode,” and `Range` is on the near side of it while a lazy generator is on the far side. This is the same judgment the whole opcode set embodies — the core names the universal, simple primitives and lets the front ends and the peephole compose the rest — and the collection section, being only two opcodes wide, is where that judgment is most visible: even something as common as a range gets exactly one pair of opcodes, and everything richer is built on top rather than baked in.

Chapter 11 — The Math Ininsics

Most of fusevm's arithmetic is the nine-op core of Chapter 7, but there is a whole second tier of numeric operations that exists for a specific reason: a front end whose `sqrt` or `sin` is *always* floating point wants an opcode that the JIT and the AOT compiler can lower to a single machine instruction or a single libcall, with identical results in every tier, rather than a builtin call that defeats compilation. fusevm provides that tier as a family of always-float and integer-intrinsic opcodes, and they are worth a chapter because the compilers treat them as the proving ground for native lowering.

The always-float transcendental family is large: `SqrtFloat`, `SinFloat`, `CosFloat`, `ExpFloat`, `LogFloat`, `Atan2Float`, `TanFloat`, `AsinFloat`, `AcosFloat`, `AtanFloat`, `SinhFloat`, `CoshFloat`, `TanhFloat`, `Log2Float`, `Log10Float`, and `AbsFloat`. Each pops its operand(s), coerces to `f64` in every tier, and pushes a `Float`. The naming convention is deliberate: these are the “-Float” siblings of the AWK math ops (`AwkSin` and the rest), and the difference is that the AWK ops dispatch through the AWK host and would be inert in a non-AWK embedding, whereas the `*Float` ops are *host-independent* — they run in the plain interpreter, lower in the JIT, and stay disk-cacheable, which is why a front end like stryke whose `sin` is unconditionally floating point emits `SinFloat` rather than `AwkSin`.

Instruction, libcall, or both

What makes this family the compilers' proving ground is that it splits cleanly into three lowering strategies, and `op.rs` documents which each op takes. `SqrtFloat` lowers to a native Cranelift `fsqrt` with no helper and no relocation, so its chunk is trivially disk-cacheable; `AbsFloat` is a native `fabs`; the rounding quartet `CeilFloat`, `FloorFloat`, `TruncFloat`, and `RoundFloat` are Cranelift's `ceil`, `floor`, `trunc`, and `nearest` (the last being round-half-to-even, an exact match for the interpreter's `round_ties_even`). These cost a single instruction.

The transcendentals have no machine instruction, so they lower to a **libcall** — a call to a tiny `extern "C"` Rust shim that calls the same `f64` method the interpreter calls, so a finite result matches bit-for-bit. The shims have stable names because the disk cache keys relocations by them: `fusevm_jit_sin_f64`, `fusevm_jit_cos_f64`, `fusevm_jit_exp_f64`, `fusevm_jit_log_f64`, `fusevm_jit_tan_f64`, `fusevm_jit_asin_f64`, `fusevm_jit_acos_f64`, `fusevm_jit_atan_f64`, `fusevm_jit_sinh_f64`, `fusevm_jit_cosh_f64`, `fusevm_jit_tanh_f64`, `fusevm_jit_log2_f64`, `fusevm_jit_log10_f64`, and `fusevm_jit_atan2_f64` for the two-argument case. Calling a leaf math function is not a failure to be native — every native compiler calls `pow` rather than inlining it, because there is no `pow` instruction — and the imports are declared only when the op actually appears in a chunk, so a chunk without transcendentals compiles to byte-identical code with no spurious imports.

The integer intrinsics are the third group. `AbsInt` is a native Cranelift `iabs` (a wrapping integer absolute value). `TruncInt` converts a float to an integer with a saturating cast that matches Rust's `as i64` and Cranelift's `fcvt_to_sint_sat`, NaN-to-zero and all, and returns an integer operand unchanged at full `i64` precision. `GcdInt` and `LcmInt` are the one pair that contains a loop, lowered through the `fusevm_jit_gcd_i64` and `fusevm_jit_lcm_i64` helpers — `gcd` returns 0 when both inputs are 0, `lcm` returns 0 when either is 0 and saturates at `i64::MAX` on overflow. And `TimeInt` returns Unix epoch seconds through `fusevm_jit_time_i64`, the one intrinsic that is not pure: it reads the clock, so it is never const-folded. This family — a handful of native instructions, a handful of libcalls, and a couple of internal loops — is the set the AOT native path lowers first and tests most heavily, because if the compiler can reproduce `gcd` and a saturating `int(x)` bit-for-bit, it can reproduce the arithmetic core.

Two siblings, one for AWK and one for everyone

The `*Float` family exists in deliberate parallel to the AWK math ops, and the reason for the duplication is a clean illustration of how `fusevm` separates host-dependent from host-independent behavior. `Op::AwkSin` and `Op::SinFloat` both compute a sine, and both lower in the JIT to the same `fusevm_jit_sin_f64` helper — but they differ in what happens when no host is present and in what NaN-canonicalization they apply. `AwkSin` dispatches through the AWK host (and, in gawk-faithful mode, canonicalizes a NaN result to gawk's `+nan` and may honor locale or precision settings), so it would be inert or panic in a non-AWK embedding. `SinFloat` is host-independent: it runs in the plain interpreter, lowers cleanly in every JIT tier, carries no host relocation, and therefore stays disk-cacheable. A front end whose `sin` is unconditionally floating point and needs no AWK semantics — `stryke`, for instance — emits `SinFloat` and gets a result that is identical across the interpreter, the linear JIT, the block JIT, and the AOT native path, with no host call anywhere in the chain. The duplication is not waste; it is the machine offering the same arithmetic at two different points on the host-dependence axis, and letting each front end choose the point that matches its semantics.

The intrinsics are the compiler's first benchmark

There is a reason this family is walked as its own chapter rather than folded into the arithmetic section, and it is that the math intrinsics are where the AOT native compiler proves itself before it tackles anything harder. Each intrinsic is a tiny, self-contained semantic with a known correct answer, which makes it perfect fuzzing material: `GcdInt` over a full-range `i64` pair exercises the unsigned-absolute-value path, the `i64::MIN` edge where negation overflows, and an internal native loop all at once; `TruncInt` exercises the saturating float-to-int cast across NaN and the infinities; `AbsInt` exercises the wrapping-negate-and-`select` idiom. The differential fuzzers throw three hundred random pairs at each and check every result against the interpreter, and because the intrinsics have no host dependence and no heap values, a failure is unambiguously a codegen bug rather than an environment artifact. If the compiler can reproduce a saturating `lcm` with `umulhi` overflow detection and a Euclid gcd built from three on-the-fly Cranelift blocks, both bit-for-bit against the interpreter, then the machinery — the kind lattice, the register allocation, the libcall imports, the internal-loop emission — is sound enough to carry the rest of the scalar instruction set. The intrinsics are small on purpose, and their smallness is what makes them the compiler's proving ground.

The lowering table

The math intrinsics are the clearest place to see, in one view, the three lowering strategies the compilers use, so here they are tabulated. The single-instruction group costs one Cranelift op and no helper, so its chunks are trivially disk-cacheable:

Op	Cranelift instruction
<code>SqrtFloat</code>	<code>sqrt</code>
<code>AbsFloat</code>	<code>fabs</code>
<code>CeilFloat</code>	<code>ceil</code>
<code>FloorFloat</code>	<code>floor</code>
<code>TruncFloat</code>	<code>trunc</code>
<code>RoundFloat</code>	<code>nearest</code> (round half to even)
<code>AbsInt</code>	<code>ineg + icmp + select</code> (wrapping abs)
<code>TruncInt</code>	<code>fcvt_to_sint_sat</code> (saturating, NaN to 0)

The libcall group has no machine instruction and calls a small shim that calls the same `f64` method the interpreter does, the thirteen transcendentals sharing one `unary_math` dispatcher keyed by id:

Op(s)	Shim / dispatcher id
<code>SinFloat...Log10Float</code> (13 functions)	<code>fusevm_aot_unary_math(id, x)</code> , ids 0–12
<code>Atan2Float</code>	<code>fusevm_aot_atan2(y, x)</code>
<code>PowFloat</code> , <code>Pow</code> (float case)	<code>fusevm_aot_powf(a, b)</code>
<code>Mod</code> (float case)	<code>fusevm_aot_fmod(a, b)</code>

And the internal-loop group is lowered as on-the-fly Cranelift blocks in the middle of the surrounding function, because the operation is itself an iteration:

Op	Lowering
<code>GcdInt</code>	three-block Euclid loop over <code>urem</code> of magnitudes
<code>LcmInt</code>	gcd then <code>udiv/imul</code> , <code>umulhi</code> overflow check, saturate to <code>i64::MAX</code>

`TimeInt` stands apart as the one impure intrinsic — it reads the system clock through `fusevm_jit_time_i64` and is never const-folded. Read down the three tables and you have the compiler's entire numeric vocabulary: an instruction where one exists, a libcall to the interpreter's own function where one does not, and an inline loop where the operation is a loop — each chosen so the compiled result matches the interpreter bit-for-bit on a finite input.

Purity, and the one impure intrinsic

The phrase “bit-for-bit on a finite input” hides a qualification worth making explicit, because it is the line between an intrinsic the compiler may const-fold and one it may not. Almost all the math intrinsics are *pure*: `sqrt(2.0)` is the same number every time it is called, so a compiler that sees `SqrtFloat` applied to a constant may compute the result at compile time and emit the constant directly, and the tracing JIT's `Cell` lattice does exactly this when both operands of an op are known. But one intrinsic is not pure: `TimeInt`, which returns Unix epoch seconds, depends on the system clock and yields a different value on every call. The source flags it precisely — “pure-ish; depends on system clock; not const-foldable” — and the compilers honor the flag by never folding it, always emitting the actual call to `fusevm_jit_time_i64`. This is the one place the math section breaks its own pattern, and the break is principled: const-folding is sound only for pure functions, and a function that reads the clock is not pure, so it is exempted from the folding the pure intrinsics enjoy. The distinction matters because a compiler that wrongly folded `TimeInt` would compute the time *once*, at compile time, and a program that called it in a loop to measure elapsed time would see the same value every iteration — a subtle, time-dependent miscompile. By marking the one impure intrinsic and folding only the pure ones, the compilers get the optimization where it is sound and decline it where it is not, which is the same lower-what-you-can-prove discipline applied to constant folding: fold what you can prove constant, emit a call for what you cannot.

Why “always float” is a guarantee, not a default

The naming convention of the **Float* family — `SqrtFloat`, `SinFloat`, and the rest — uses the word “always” deliberately, and the always-ness is a guarantee the compilers depend on rather than a mere description of typical behavior. An opcode like `Pow` is polymorphic: for two integer operands its JIT lowering keeps an integer result, and only mixed or float operands produce a float. That polymorphism is fine for the interpreter, but it means the result type of `Pow` is not known from the opcode alone — it depends on the operands — which

complicates a compiler that wants to type the result statically. `PowFloat`, by contrast, coerces *both* operands to `f64` in every tier and always produces a `Float`, so its result type is known from the opcode alone, with no operand analysis required. This is why a front end whose `**` is unconditionally floating point — strykelang, for instance — emits `PowFloat` rather than `Pow`: the always-float guarantee makes the chunk disk-cacheable and the result kind statically known, where the polymorphic `Pow` would force the compiler to track operand types to know whether the result is an integer or a float. The same logic runs through the whole `*Float` family: `SqrtFloat` always produces a float, `AbsFloat` always produces a float, and so a chunk full of them has a result kind the analysis can pin down without inspecting operands. The “always” is the property that lets these opcodes have a single, operand-independent lowering — a `fsqrt`, a libcall, a constant result kind — and a front end chooses the always-float opcode precisely when its language’s semantics guarantee floating point, trading the polymorphism it does not need for the static-typeability it does. It is the opcode set offering two versions of an operation at two points on the polymorphism axis, and letting each front end pick the one its semantics make true — the same host-dependence choice the AWK-versus-`*Float` duplication offers, here on the axis of result-type polymorphism rather than host dependence.

Chapter 12 – Fused Superinstructions

The fused superinstructions are fusevm's namesake and the clearest expression of its bias toward the hot path. There are eleven of them, and each one collapses a common multi-opcode idiom into a single opcode that the dispatch loop touches once where a naive bytecode would touch it several times. The principle is stated in the source itself: “the compiler detects hot loop patterns and emits these instead of multi-op sequences,” and “each fused op eliminates N–1 dispatch cycles, stack pushes, and branch mispredictions from the hot path.”

Start with the simplest. A counted loop's bottom — `i += 1; if i < n goto top` — is, naively, a load, a load-one, an add, a store, a load, a compare, and a branch: seven ops, seven dispatches, several stack pushes. fusevm has `SlotIncltIntJumpBack(slot, limit, target)`: one opcode that increments the slot, compares the new value against an integer limit, and branches back to `target` if it is still under. The dispatch loop touches it once. Below it sit the pieces it is built from: `PreIncSlot(slot)` increments a slot and pushes the new value; `SlotLtIntJumpIfFalse(slot, limit, target)` is the fused compare-and-branch that tests a slot against a constant and jumps; and `SlotIncltIntJumpBack` is precisely their composition for the loop backedge.

The loop-body fusions

Three superinstructions go further and fuse an *entire loop body*, not just its backedge. `AccumSumLoop(sum, i, limit)` is the whole counted sum `while i < limit { sum += i; i += 1 }` in one opcode — the README measures the million-iteration version of this at a hundred and forty-two nanoseconds against thirty-one milliseconds for the same loop spelled in unfused ops, which is the single most dramatic number in the project and a direct illustration of what dispatch costs. `ConcatConstLoop(const, s, i, limit)` is the string-append loop `while i < limit { s .= CONST; i += 1 }`, fusing a `LoadConst`, an append, and the backedge; the README clocks a ten-thousand-iteration string build through it at under twelve microseconds. `PushIntRangeLoop(arr, i, limit)` is the array-build loop `while i < limit { push arr, i; i += 1 }`. Each of these is a recognizable shape a front end's loop-lowering peephole can detect and replace wholesale.

The remaining superinstructions are the void-context and post/pre variants that let a front end avoid pushing a value it will not use. `AddAssignSlotVoid(a, b)` is `a += b` with no stack push, for a compound assignment in statement position. `PreIncSlotVoid(slot)` is `++slot` discarded. `PreDecSlot`, `PostIncSlot`, and `PostDecSlot` are the increment/decrement forms that differ only in whether they push the old or the new value — `$slot++` must push the *old* value and `++$slot` the *new*, a distinction every C-descended language insists on and which the front end encodes by choosing the right opcode.

Why fusion is a system property, not a trick

The reason the section exists is that the cost of dispatch is real and the cheapest dispatch is the one you do not do, but the deeper reason it *works* is that fusion is honored by every tier below the bytecode. A front end's loop peephole recognizes the idiom and emits the fused opcode; the interpreter has a single hand-tuned handler for it; the block JIT register-allocates `AccumSumLoop` with Cranelift block parameters so the inner loop touches no memory; and the AOT native path lowers it to a register loop. One recognized pattern, one opcode, and every layer of the machine has exactly one place to make it fast. There is a sharp edge worth flagging, and the README names it: the integer-only fused ops — `PreIncSlot`, `AccumSumLoop`, `SlotIncltIntJumpBack` — *bail to the interpreter* rather than miscompute when the block JIT finds the slot holds a float, because their fast

handlers assume `i64`. The fusion is an optimization that knows its own preconditions and declines when they do not hold, which is the same conservatism the entire compilation half of the book runs on.

What the peephole sees

A fused superinstruction is only as useful as a front end's ability to recognize the idiom it replaces, so it is worth being concrete about what that recognition looks like, using `AccumSumLoop` as the example. The unfused form of `while i < n { sum += i; i += 1 }` is a recognizable seven-or-eight-op sequence: a loop header, a `GetSlot/LoadInt/NumLt/JumpIfFalse` test, a `GetSlot/GetSlot/Add/SetSlot` body for the accumulation, a `PreIncSlotVoid` for the increment, and a backward `Jump`. A front end's loop peephole walks the emitted bytecode looking for exactly this shape — the same slot used as an induction variable in the test and the increment, a second slot accumulating the first, an integer limit — and when it matches, it replaces the whole sequence with one `AccumSumLoop(sum, i, limit)`. The README tabulates the replacement precisely: `AccumSumLoop` stands in for `GetSlot + GetSlot + Add + SetSlot + PreInc + NumLt + JumpIfFalse`. The recognition is a pattern match over a window of ops, and it runs once at compile time; what it buys is paid back on every iteration, because the million-iteration loop now dispatches once per iteration instead of seven or eight times.

Fusion as a contract across tiers

The eleven fused ops are not merely interpreter optimizations; they are a contract the whole tier stack honors, and tracing how `AccumSumLoop` is treated at each tier shows what “one opcode, one place to make it fast” means in practice. The interpreter has a single hand-tuned handler that runs the entire counted sum in a Rust loop with no per-iteration dispatch. The block JIT register-allocates it with Cranelift block parameters, so the inner loop holds `sum` and `i` in machine registers and touches no memory — the README notes that “all mutable state flows through the slots pointer” and `AccumSumLoop` “is register-allocated with block parameters.” The AOT native path lowers it to a register loop the same way. And every tier that cannot guarantee the op's integer precondition declines gracefully: the README is explicit that the integer-only fused ops “bail to the interpreter on a `Float` slot rather than miscompute it.” So one recognized pattern yields one opcode, and that one opcode has exactly one fast handler at each tier plus one well-defined bail condition — which is the entire economy of the fusion design. A front end teaches its peephole the pattern once, and four execution strategies inherit the speedup, each with a single place to maintain and a single condition under which it safely falls back.

The eleven, grouped by what they fuse

The eleven fused superinstructions are easier to hold in mind grouped by what they collapse. Three fuse an *entire loop body*: `AccumSumLoop(sum, i, limit)` is a counted integer sum, `ConcatConstLoop(const, s, i, limit)` is a string-append loop, and `PushIntRangeLoop(arr, i, limit)` is an array-build loop — each replacing a whole header-test-body-increment-backedge sequence with one dispatch. Two fuse the *loop control* without the body: `SlotLtIntJumpIfFalse(slot, limit, target)` is a compare-slot-to-constant-and-branch, and `SlotIncLtIntJumpBack(slot, limit, target)` is the increment-compare-branch-back tail of a counted loop. Six fuse a *single read-modify-write* on a slot, differing in whether they push a value and whether it is the old or new one: `PreIncSlot` and `PreDecSlot` push the new value, `PostIncSlot` and `PostDecSlot` push the old, `PreIncSlotVoid` pushes nothing, and `AddAssignSlotVoid(a, b)` is a void compound add-assign. The grouping reveals the logic of the set: the most common loop shapes (counted sum, string build, array build) get whole-body fusions; the most common loop control (increment-and-test-back) gets a control fusion; and the most common per-statement slot mutations (the increments and compound assignments that pepper any imperative program) get void and value-keeping fusions so a front end never has to emit a stack push for a value it will discard. Eleven opcodes, three granularities — whole body, loop control, single mutation — and between them they cover the idioms a numeric or string loop repeats millions of times, which is exactly where dispatch cost compounds and exactly where the machine refuses to pay it.

Chapter 13 – The Interpreter

The interpreter is `VM::run`, and at its heart is a loop you could have guessed: read the instruction at the cursor, advance the cursor, execute. The cursor is `self.ip`, a `usize`; the loop runs while `ip` is in range and a `halted` flag is unset; and execution is a large `match` on the opcode — `exec_op` — with one arm per operation. Each arm does the obvious thing: `Op::Add` pops two values and pushes their sum, `Op::Jump(t)` sets `self.ip = t`, `Op::GetSlot(s)` pushes the slot's value. There is no cleverness in the loop itself; the cleverness is in keeping each arm short and the dispatch tight, and the raw dispatch overhead — a million `Nops` — is measured at about eight hundred and nineteen microseconds, or 1.22 billion operations per second, which is the floor the rest of the machine builds on.

The fast-path helpers

A few opcodes are hot enough to deserve hand-tuned handlers, and the interpreter has them. `arith_int_fast` is the engine of `Add/Sub/Mul/Mod`: it *peeks* the top two stack slots without popping, and if both are `Int`, applies the integer operation directly and truncates the stack by one; only if one is not an `Int` does it fall back to the general float-promoting path. The integer operations are wrapping — the helper passes `i64::wrapping_add` and its siblings — and integer modulo by zero yields zero rather than trapping. The point is to make the overwhelmingly common case — integer arithmetic on two integers — avoid even the coercion machinery, the peek-and-truncate saving the push-and-pop traffic of a naive handler. `cmp_int_fast` does the same for the numeric comparisons.

These helpers are more than an optimization; they are the precise specification the compilers must match. When the AOT chapter says integer `Add` is wrapping, it is because `arith_int_fast` passes `i64::wrapping_add` and the native code emits a wrapping `iadd` to agree. When the native path's integer modulo guards the `y==0` and `y==-1` cases with a `select`, it is reproducing exactly what `arith_int_fast`'s fallback does. The fast paths are written once, in the interpreter, and every other tier is a translation of them into another form.

One definition of meaning

`exec_op` does not merely mutate the VM in place; it is structured to return a next-instruction signal, and — importantly — the *same* `exec_op` is what the AOT threaded path calls, op by op, through a shim. There is exactly one implementation of what each opcode *means*, and every tier reaches it: the interpreter calls it directly, the AOT threaded path calls it through `fusevm_aot_exec_op`, and the deopt resume path re-enters `run` itself, which loops from `self.ip` as usual. This single-source-of-truth discipline is the load fact of the whole project. A five-tier execution stack — interpreter, linear JIT, block JIT, tracing JIT, AOT native — stays correct not because five implementations were each made correct, but because four of them are *measured against* the one that is correct by definition, and the threaded AOT path literally calls it. The differential-correctness chapter is the proof; this is the premise.

The loop, in the small

The dispatch loop is small enough to hold in the mind whole, and holding it there is the best preparation for the compiler chapters, because each of them replaces a piece of it. In essence:

```

while self.ip < ops.len() && !self.halted {
  let ip = self.ip;
  self.ip += 1;           // advance the cursor first
  match self.exec_op(ip) { // run one opcode's semantics
    ExecFlow::Cont => {}   // keep going
    ExecFlow::Ret(v) => return VMResult::Ok(v),
  }
}

```

Read the instruction at the cursor, advance the cursor, execute, and honor what the execution says to do next. The cursor advances *before* the op runs, so a jump opcode that sets `self.ip` overrides the advance and control lands wherever the jump said — which is how the five control-flow ops express every loop and branch without the loop itself knowing anything about control flow. Each tier in the last movement is, precisely, a way of not running this loop: the linear and block JITs replace the whole loop with a native function, the AOT threaded path replaces the loop with native control flow that still calls `exec_op` per op, and the AOT native path replaces both the loop *and* the per-op calls with register-resident machine code. But every one of them is measured against this loop, because this loop is what the bytecode *means*.

Halting, returning, and the three results

The loop's two exit conditions — the instruction pointer running off the end of the op vector, and the `halted` flag being set — together with the value a `Ret` carries, produce the three-way `VMResult` that every front-end driver consumes. `VMResult::Ok(Value)` is the ordinary case: the program returned a value, whether through `ReturnValue` at the top level or by falling off the end with a value on the stack. `VMResult::Halted` is the case where execution stopped without a return value — a bare end-of-chunk, or a control-flow opcode like `AwkSignal` that halts the chunk to let the front-end driver take over. `VMResult::Error(String)` is a runtime fault, the channel through which a fatal AWK division-by-zero or a host error surfaces. The discipline that keeps this clean is that `AwkSignal` does *not* add a fourth `VMResult` variant: it halts and stashes its code in VM state, read through `VM::awk_signal()` after `run` returns, so a front end that never emits it sees a `Halted` byte-identical to what it always saw. The result type is small and stable on purpose, because five front-end drivers branch on it, and a new variant would be a breaking change to all five at once.

Peek, do not pop

The fast-path helpers deserve a closer mechanical look, because the trick that makes them fast is the same trick the native compiler reproduces. A naive `Add` handler pops two values off the stack, matches on their types, computes, and pushes the result — three stack operations and a match. `arith_int_fast` does less: it *peeks* the top two values without removing them, and in the common case where both are `Int`, it writes the wrapping sum into the second-from-top slot and truncates the stack length by one, leaving the result where the lower operand was. No pop, no push, just a peek, a compute, and a length decrement. The match on types still happens, but only to confirm the integer fast path applies; if either operand is not an `Int`, the helper falls through to the general path that coerces and promotes to float. The savings — avoiding two stack pushes and a pop per arithmetic op — are small per operation and enormous in aggregate, because arithmetic is the most common thing a numeric loop does. And the helper's exact behavior, peek-and-truncate with wrapping integer semantics and a float-promoting fallback, is the specification the AOT native path implements in registers: the native `iadd` is wrapping because `arith_int_fast` is, and the native promotion inserts the int-to-float conversion exactly where the helper's fallback would. The interpreter's optimization and the compiler's correctness are the same code, written once and translated.

ExecFlow, and the one bit of state the loop carries

The dispatch loop carries almost no state between iterations — that is the point of a flat op vector — but it does carry one signal out of each opcode, an `ExecFlow`, and the smallness of that signal is worth noting because it is what keeps the loop a loop. An opcode's handler returns `ExecFlow::Cont` to mean “keep going” or `ExecFlow::Ret(value)` to mean “stop and return this value.” The vast majority of opcodes return `Cont`; only the return ops and the few that terminate a chunk return `Ret`. Crucially, control-flow opcodes like `Jump` do *not* signal through `ExecFlow` — they mutate `self.ip` directly and return `Cont`, so the loop simply reads the new instruction pointer on its next iteration. This keeps the signal binary: every opcode either continues or returns, and “where to continue” is communicated through the shared `ip`, not through the return value. The result is a dispatch loop whose per-iteration logic is “run the op, and if it said `Ret`, stop” — about as little branching as a dispatch loop can have. The AOT threaded path preserves this exact contract: its shim returns the next instruction index, the native code branches on whether that index is negative (the terminate sentinel) or not, and the structure mirrors the interpreter's `Cont`-or-`Ret` precisely, because the threaded path is the interpreter's control flow turned into native branches with the per-op semantics left untouched.

Why the loop advances the cursor first

A detail of the dispatch loop that looks like a stylistic choice is in fact load-bearing, and naming it explains how the five control-flow opcodes work without the loop knowing anything about control flow. The loop reads the current instruction pointer, *advances it*, and only then runs the opcode. Advancing first means the cursor already points at the *next* instruction by the time the opcode runs — which is exactly right for the common case of a non-branching op, where execution should fall through to the following instruction. A branching opcode, then, does not need to compute “the next instruction”; it simply *overwrites* `self.ip` with its target, and because the loop already advanced past the branch, the overwrite is the only thing that determines where control goes. `Jump(t)` sets `self.ip = t`; `JumpIfFalse(t)` sets it to `t` or leaves the already-advanced value, depending on the predicate. The loop itself contains no branch logic at all — it advances and runs, advances and runs — and all of control flow lives in the handful of opcodes that choose to overwrite the cursor. This is why the chunk can be flat and the loop can be trivial: the order of “advance, then run” turns every opcode into a fall-through by default and every jump into a single assignment, with no special case in the loop for branching versus non-branching ops. It is a one-line ordering decision that is the hinge of the whole flat-bytecode design.

The match is the machine

The `exec_op` function is a single large `match` on the opcode, and it is worth appreciating that this unfashionable structure is, deliberately, the whole semantic core of the machine. A modern instinct might reach for a dispatch table of function pointers, one per opcode, or a trait-object per instruction — and `fusevm` uses neither, because a flat `match` is both faster and more maintainable for this job. Faster, because the Rust compiler turns a `match` over a `#[repr]`'d enum into a jump table the CPU's branch predictor handles well, with no indirect-call overhead and no function-call boundary per opcode; the arms are inlined into one function, so a hot opcode's handler shares registers and instruction cache with the dispatch. More maintainable, because the entire meaning of the machine is in one place, readable top to bottom: every opcode's behavior is an arm in one function, and adding an opcode means adding an arm, with the compiler's exhaustiveness check ensuring no opcode is forgotten. That exhaustiveness is itself a correctness mechanism — a new `Op` variant that lacks an `exec_op` arm fails to compile, so the single-source-of-truth cannot accidentally develop a hole. The `match` is also why the threaded AOT path can reuse the semantics so cleanly: `fusevm_aot_exec_op` is a thin shim that calls into the same `match`, so the compiled program's opcodes mean exactly what the interpreted program's do, arm for arm. A big `match` looks primitive next to a vtable or a computed-goto threading scheme, but for a machine that wants one readable definition of meaning, fast dispatch, and compiler-checked completeness, it is the right primitive — and it is the literal center of the system, the function every execution tier ultimately defers to or is tested against.

Chapter 14 — Tiered Execution

fusevm does not have one execution strategy; it has a ladder of them, and it climbs the ladder as code proves itself hot. The principle is that you should spend compilation effort in proportion to how often code runs, and the ladder lets the same machine be lazy about cold code and aggressive about hot code.

The bottom rung is the **interpreter** of the last chapter: it runs everything, cold or hot, and for code that executes once it is the right and only choice — compiling it would cost more than running it. Above it sit three Cranelift JIT tiers, enabled by the `jit` feature, and the AOT compiler, which is not a runtime tier at all but a compile-time one. The three JIT tiers are the **linear JIT** (straight-line expression chunks, compiled on first call), the **block JIT** (whole-chunk control-flow graphs — loops, branches, fused backedges — compiled after a warm-up), and the **tracing JIT** (the hot path through a loop in code the block JIT cannot take, recorded and type-specialized). Each gets its own chapter in the last movement; this one is about how the machine decides which to use.

The thresholds, and a discrepancy worth naming

The tier-up decisions are governed by warm-up thresholds, and the honest numbers are these. The block threshold defaults to **1** — a block-eligible chunk is compiled to native on its first invocation — and the trace threshold defaults to **50** backward branches through a loop header before that loop is recorded. Both are tunable two ways: per process through the environment variables `FUSEVM_JIT_BLOCK_THRESHOLD` and `FUSEVM_JIT_TRACE_THRESHOLD` (read once per thread when the JIT is first touched), and per thread programmatically through `TraceJitConfig` and `JitCompiler::set_config`. Setting the block threshold to **0** is the most eager posture — every block-eligible chunk compiles on its first run — and combined with the disk cache it gives AOT-like speed without an explicit AOT step.

A note for the careful reader: some doc comments in the tree say the block threshold is **10**, but the actual `TraceJitConfig::default()` constant is **1**, and this book follows the code rather than the stale comment. The same care applies to a handful of other constants the source pins — `MAX_SIDE_EXITS` is **50**, `MAX_INLINE_RECURSION` is **4**, `MAX_TRACE_CHAIN` is **4**, and a recorded trace is capped at **256** ops — all of which recur in the tracing-JIT chapter.

One execution path, not two

The architectural choice that makes the ladder safe is that climbing it changes the *speed* of code without ever changing its *meaning*. There is a subtlety in *how* the tiers are reached that is worth stating now, because it shapes the compiler chapters. The tracing JIT is the tier wired directly into `VM::run`'s dispatch loop: with `enable_tracing_jit()` set, the loop consults the JIT at backward branches, recording and then dispatching native traces, and the block JIT short-circuits ahead of it when a chunk is fully block-eligible — so a block-eligible chunk never even arms the trace recorder, and the two tiers cover disjoint cases at runtime. The linear and block tiers, by contrast, are typically invoked by the *front end* over a chunk or a region it has identified, through the `JitCompiler` API directly (`try_run_linear`, `try_run_block`). The interpreter and the JIT are one execution path with several compiled fast lanes, not two parallel machines — which is exactly why a value can flow from interpreted code into a native trace and back out on a deopt without translation, the property the whole last movement depends on.

Spend effort in proportion to heat

The principle underneath the ladder is worth stating as a principle, because it is the answer to a question every JIT designer faces: when do you compile? Compile too eagerly and you spend codegen time on cold code that runs once and is never seen again — the warm-up tax that makes some runtimes painful for short scripts. Compile too lazily and the hot loop that dominates the program’s runtime never reaches native speed. A single threshold cannot be right for both, because “cold” and “hot” are not one decision but a spectrum, so fusevm uses several. The interpreter handles everything at zero compile cost, which is correct for code that runs once. The block tier, with its default threshold of one, compiles an eligible whole chunk on its first invocation — aggressive, because a chunk worth being block-eligible is usually worth compiling. The tracing tier waits fifty backward branches through a loop header before recording, because a trace is expensive to record and specialize and you want to be sure the loop is genuinely hot before paying for it. The thresholds encode a belief about the shape of real workloads, and they are tunable precisely because that belief is approximate: a shell re-running the same rc scripts wants the most eager posture, and `FUSEVM_JIT_BLOCK_THRESHOLD=0` gives it; a one-shot script wants the laziest, and the defaults lean that way.

The disk cache changes the calculus

There is a second-order effect the ladder gains from the disk cache that is worth making explicit, because it blurs the line between JIT and AOT. Normally a warm-up threshold is a bet that a chunk will run *enough times in this process* to repay its compile cost. But with `jit-disk-cache` enabled, the compiled native code persists across process restarts, so the bet changes: a chunk need only run enough times across *all* invocations, ever, to repay the one compile. For a workload that launches the VM repeatedly — a shell, a test runner, a batch job — this means the most eager threshold becomes nearly free: compile every block-eligible chunk on its first run, pay the codegen once, and reload the native code from `~/.cache/fusevm-jit` on every subsequent launch. The warm-up decides *when* a tier engages within a process; the disk cache decides whether the resulting code survives *between* processes; and together they let a JIT-based system approach AOT-like launch-to-native latency without anyone ever invoking the AOT compiler explicitly. The tier ladder and the disk cache are two halves of one answer to “spend effort in proportion to heat” — one measures heat within a run, the other remembers the effort across runs.

Chapter 15 – The Host Boundary

A virtual machine that only computes is useless; it has to *do* things — read files, run processes, talk to the editor that embeds it. fusevm draws a clean line between the machine, which is pure and language-agnostic, and the **host**, which is the embedding environment’s implementation of the effectful operations. The line is a trait, declared in `host.rs`, and the design choice that makes it work is that *every method has a default*.

The `ShellHost` trait is how the shell opcodes reach the outside world. When the VM executes `Op::Exec`, it does not itself `fork/exec` — it calls the host’s `exec(args)`, which decides what running a command means in this embedding. A standalone zshrs binary supplies a host that spawns real processes; an editor embedding might supply one that runs commands in a managed pty; a test might supply one that records the command and returns a canned result. The opcode is the same; the effect is the host’s. And because the trait’s `exec` has a *default* implementation — `std::process::Command` with inherited stdio, returning the exit code or 127 on failure — even a front end that overrides nothing gets working process execution. The companion `exec_bg` does the same with detached, nulled stdio, returning a pid.

Defaults that keep stack discipline

The deeper reason every method has a default is that it lets a *non-shell* front end run shell-containing bytecode without crashing. The trait’s methods divide into the ones with meaningful defaults and the ones that are deliberately inert. `glob` defaults to the system `glob` crate; `word_split` defaults to splitting on whitespace (collapsing runs, matching POSIX IFS); `str_match` defaults to exact equality. But `tilde_expand`, `brace_expand`, `expand_param`, `cmd_subst`, the pipeline methods, the redirect methods, the trap methods — these default to identity, empty, or no-op. The `DefaultHost` struct is nothing but `impl ShellHost for DefaultHost {}` — it accepts every default — and it exists precisely so that “tests and non-shell front ends that still want shell ops to be stack-discipline-correct” have something to install. A chunk that happens to contain `Op::CmdSubst` running under `DefaultHost` produces an empty string rather than a panic, and the operand stack ends exactly as balanced as if a real host had run. The pure part stays pure; the effectful part is optional.

Sub-execution through the chunk

One mechanism in the host boundary is worth singling out because it ties back to the chunk’s structure. Command substitution, process substitution, trap handlers, and with-redirect blocks all need to run a *body* of code and capture its result, and that body is delivered to the host as a `&Chunk` reference — taken from the parent chunk’s `sub_chunks` table, the field introduced in Chapter 2. The front end builds the body with `add_sub_chunk`, gets back a `u16` index, and emits `Op::CmdSubst(index)`; the VM, executing that op, hands the host `&parent.sub_chunks[index]`. The host runs it — on a fresh VM, a pooled VM, whatever it likes — and reports the captured stdout or exit status back. The machine never decides what “run a subshell” means; it only locates the body and calls the host. This is the same division of labor as the extension mechanism, and it is what lets the *pure* AOT compiler reason about a program: everything the host does is behind a call it does not have to model, so moving slots into registers never has to account for what a pipeline does to the file system.

A host, in the small

The trait's surface is wide, but a real host implements only what it needs, because every method has a default. The crate's documentation shows the shape:

```
use fusevm::{ShellHost, VM, Chunk, Value};

struct MyHost;
impl ShellHost for MyHost {
    fn glob(&mut self, pattern: &str, _recursive: bool) -> Vec<String> { /* ... */ vec![] }
    fn tilde_expand(&mut self, s: &str) -> String { /* ... */ s.into() }
    fn cmd_subst(&mut self, sub: &Chunk) -> String { /* run sub, capture stdout */
        String::new() }
    // ... every other method inherits the trait default
}

let mut vm = VM::new(chunk);
vm.set_shell_host(Box::new(MyHost));
```

Three methods overridden, two dozen inherited. The host installs through `set_shell_host` as a `Box<dyn ShellHost>`, and the VM owns it for the run. This is the seam at which an embedding decides what its shell ops *mean*: an interactive shell overrides `exec` to spawn into its job table and `glob` to honor its options; a build tool might override `exec` to run inside a sandbox; a test overrides nothing and runs against the defaults. The trait is `Send`, so a host can move across threads with the VM, and the comment notes the practical reason every method has a default beyond convenience — “so the trait can grow without breaking existing front ends.” A new shell op can be added to the core, with a default host method, and every existing front end keeps compiling; only the front ends that want the new behavior override it.

The borrow dance

There is a mechanical subtlety in how the VM calls into the host that recurs throughout the embedding layer and is worth naming, because it explains an otherwise odd-looking pattern. The host needs, while it runs, to reach back into the VM — to read the operand stack for a command's arguments, to consult globals, to push a result. But the VM owns the host, and Rust will not let the VM lend the host a mutable reference to itself while the host is borrowed from it. The resolution is a take-out-and-restore dance: the VM moves the host *out* of itself (leaving a placeholder), calls the host method passing a mutable reference to the now-host-less VM, and moves the host back in afterward. The same dance governs the extension handler and the AWK host. It is invisible from the front end's side — a host method simply receives whatever it needs — but it is why the host is stored in a way that permits being temporarily removed, and why a host method cannot itself trigger a re-entrant call that would try to take the already-taken host. The pattern is the borrow checker's tax for letting effectful code reach back into the pure machine, paid once in the dispatch plumbing so the host author never has to think about it.

The full surface, and what its defaults say

It is worth cataloguing the `ShellHost` trait's surface, because the *defaults* a method chooses are a quiet specification of how much of a shell the core understands on its own. The methods divide into expansion (`glob`, `tilde_expand`, `brace_expand`, `word_split`, `expand_param`, `array_index`), sub-execution (`cmd_subst`, `process_sub_in`, `process_sub_out`), redirection (`redirect`, `heredoc`, `herestring`, `with_redirects_begin/end`), pipelines (`pipeline_begin`, `pipeline_stage`, `pipeline_end`), scope (`subshell_begin`, `subshell_end`), traps (`trap_set`, `trap_check`), execution (`exec`, `exec_bg`, `call_function`), and matching (`str_match`, `regex_match`). For each, the default tells you whether the core has a sensible language-agnostic answer or whether the behavior is irreducibly the front end's. `glob` defaults to the system `glob` crate

and `word_split` to whitespace splitting, because those have reasonable language-agnostic meanings; `exec` defaults to `std::process::Command` with inherited stdio, returning the exit code or `127` on a spawn failure, because running a process is the same on any Unix. But `tilde_expand` returns its input unchanged, `expand_param` returns the empty string, `cmd_subst` returns nothing, and `regex_match` returns false — because tilde rules, parameter-expansion modifiers, command-substitution capture, and regex dialect are all genuinely the front end's to define, and a wrong default would be worse than an inert one. The trait is a map of the shell-core boundary drawn in default implementations: where the default does real work, the core has an opinion; where it is inert, the front end must speak.

Stack discipline without a shell

The reason `DefaultHost` exists at all — a struct that is literally `impl ShellHost for DefaultHost {}`, accepting every default — is a subtle one that pays off in the compiler chapters, and it is worth making explicit. A chunk emitted by a shell front end may contain shell opcodes, and if such a chunk is ever run by something that is *not* a shell — a test exercising arithmetic, a non-shell front end that happens to share a code path, a differential fuzzer — those shell opcodes must not crash and must not corrupt the operand stack. The inert defaults guarantee exactly this: a `CmdSubst` under `DefaultHost` pops what it should pop and pushes an empty string, a `Pipeline` runs through its begin/stage/ end returning a zero status, a `Redirect` is a no-op. The operand stack ends as balanced as if a real shell had run, even though nothing happened. This stack-discipline-correctness is what lets the interpreter and both compilers be exercised against a host that does nothing, so a test of integer arithmetic never accidentally spawns a process and a fuzzer generating random opcodes never trips over a shell op it did not mean to test. The pure part of the machine stays runnable in isolation precisely because the impure ops degrade to stack-balanced no-ops rather than to panics — which, when the AOT compiler later starts moving state into registers, is exactly the property that lets it reason about what must be written back and what never leaves the machine.

Sub-execution, and the isolation it buys

The sub-chunk mechanism is the host boundary's most structural feature, and it is worth following one sub-execution through to see what the isolation buys. A command substitution `$(echo hi)` compiles its body — the `echo hi` — into a separate `Chunk`, stored in the parent chunk's `sub_chunks` table and referenced by the `u16` payload of `Op::CmdSubst(idx)`. When the VM runs that op, it does not run the body itself; it hands the host `&parent.sub_chunks[idx]`, and the host runs it — typically on a fresh or pooled VM with its own stack, frames, and globals — captures the stdout, and returns it as a string the parent pushes. The isolation is total: the sub-chunk's execution cannot disturb the parent's operand stack or frames, because it runs on a different VM, and the only thing that crosses back is the captured string. This is what makes a subshell a subshell — a child environment whose variable changes do not leak to the parent — and it is delivered not by a special “subshell mode” in the VM but by the simple fact that a sub-chunk is a whole `Chunk` the host runs on a separate machine. The same mechanism serves process substitutions (the host returns a FIFO path instead of captured output), trap handlers (the host runs the chunk when a signal fires), and with-redirect blocks. One representation — a nested `Chunk` referenced by index — and one delivery mechanism — a `&Chunk` handed to the host — cover every form of nested execution a shell needs, with isolation falling out of the separateness rather than being engineered on top of it.

The boundary the compiler will lean on

There is a forward connection worth drawing explicitly between the host boundary and the AOT compiler, because the host trait is what makes the native path's central simplification possible. The native path holds slots and stack in registers and tracks their types, and that reasoning is sound only because everything effectful is behind a host call it does not have to model. When the analysis meets `Op::Exec` or `Op::CmdSubst`, it does not attempt to lower the process spawn or the sub-chunk capture — it cannot, those are the host's to define

— so it either runs the op through the threaded `exec_op` shim or declines the chunk. The clean separation between the pure machine and the effectful host is exactly what lets the compiler draw a line between the analyzable scalar core of a program and the host-dispatched edges, and lower the former while leaving the latter to the interpreter. The host boundary, introduced here as the way a shell reaches the outside world, turns out to be a precondition for the compiler: a machine whose effects were woven into its value model could not be partially compiled, because there would be no clean line to draw, but a machine whose effects are all behind traits can lower its pure core aggressively and fall back at the seams.

Send, and what it permits

A small detail of the host traits' declaration has large consequences: both `ShellHost` and `AwkHost` are `Send`. In Rust, `Send` means a value can be moved to another thread, and requiring it of the hosts is a deliberate design decision rather than an incidental bound. It permits a VM, host and all, to be moved across thread boundaries — which a front end that runs scripts on a worker pool, or evaluates many chunks in parallel, needs. It is also why the host is owned as a `Box<dyn ShellHost>` rather than borrowed: the VM owns its host outright, so moving the VM moves the host with it, and the `Send` bound guarantees that move is sound. The take-out-and-restore dance that lets a host method reach back into the VM works within a single thread's execution, but the `Send` bound is what lets the whole VM-plus-host bundle migrate between threads when it is *not* mid- call. For a machine whose front ends include a shell (which may background jobs) and a scripting language (which may fan work across cores), the ability to move a configured VM to another thread is not a luxury, and the one-word `Send` bound on the host traits is what makes it type-safe. It is the kind of small constraint that, placed at the boundary, keeps an entire category of concurrency bug from being expressible: a host that captured a non-`Send` resource would fail to compile against the bound, surfacing the problem at the trait boundary rather than as a data race at runtime.

A trait, not a build flag, and why it matters

Making the host a trait rather than a compile-time choice — a build flag that bakes one implementation into the binary — is a decision with consequences that reach all the way to the compiler, and it is worth closing the chapter on it. A build flag would let exactly one host exist per compiled fusevm: a `zshrs` binary with the real process-spawning host baked in, a test binary with a stub host baked in, and never the two in one process. The trait instead lets one compiled fusevm serve many hosts, installed at runtime through `set_shell_host`, so the same engine binary can spawn real processes for one VM and record canned results for another, in the same process, at the same time. This is what makes the machine *testable*: the interpreter and both compilers can be exercised against a `DefaultHost` that does nothing, so a test of arithmetic never spawns a process and a fuzzer generating random opcodes never trips over a real pipeline. And testability is not a side benefit here; it is a precondition for the differential correctness the whole compiler movement rests on. The fuzzers that compile hundreds of thousands of random chunks and check them against the interpreter can only do so because those chunks run under an inert host that never reaches outside the machine — otherwise a random chunk containing a shell op would try to execute a random command. The trait boundary keeps the pure part of the machine runnable in isolation, which is exactly what lets the pure part be fuzzed, which is exactly what lets the compilers be trusted. So the choice of a trait over a build flag is not merely about flexibility of embedding; it is the choice that makes the machine verifiable, and a machine you cannot run in isolation is a machine whose compilers you cannot differentially test. The host boundary is a trait because one machine must serve many embeddings — and because one of those embeddings is the test harness that proves the machine correct.

Chapter 16 — The Shell Inside the Machine

fusevm hosts a shell as a native capability, and the twenty-nine-op shell section of the opcode set is the evidence. A shell is mostly *plumbing* — connecting the output of one process to the input of another, redirecting streams to files, substituting the output of a command into a string, running a group of commands in a subshell — and fusevm gives each of those a dedicated opcode rather than asking the front end to build it from primitives. The section is “first-class because process control is universal enough that multiple front ends need it (shell, scripting, build tools),” as the source puts it.

`Op::Exec(u8)` runs a command, popping its arguments from the stack; `Op::ExecBg(u8)` does the same without waiting. `Op::PipelineBegin(u8)`, `Op::PipelineStage`, and `Op::PipelineEnd` bracket an N-stage pipeline so the VM, through the host, can wire the stages together and collect the final status. `Op::Redirect(u8, u8)` attaches a file to a descriptor, the second byte selecting among a documented family of redirect operators. `Op::CmdSubst(u16)` captures a sub-chunk’s stdout as a value; `Op::SubshellBegin/Op::SubshellEnd` isolate a group; `Op::HereDoc(u16)` and `Op::HereString` supply stdin; and the process-substitution pair `Op::ProcessSubIn/Op::ProcessSubOut` give a path to a FIFO that reads or writes a sub-chunk’s streams. The remaining ops — `Glob`, `GlobRecursive`, `TestFile`, `WordSplit`, `BraceExpand`, `TildeExpand`, `ExpandParam`, `CallFunction`, `StrMatch`, `RegexMatch`, the `WithRedirects` pair, `TrapSet/TrapCheck`, and the status pair `SetStatus/GetStatus` — cover the rest of a shell’s surface. The `last_status` field on the VM is the `$?` these ops update, and it is a `Value::Status`, with the truthy-when-zero semantics of Chapter 3.

The little encodings

Three of the shell opcodes carry a `u8` that is itself a small enumeration, and `op.rs` documents each as a named constant module — a detail that shows how much real shell behavior is folded into single opcodes. `TestFile(u8)` encodes the twelve file-test predicates of `[[ -f ]]` and friends as constants: `IS_FILE` is 0, `IS_DIR` 1, `IS_READABLE` 2, on through `IS_SYMLINK` 7, `IS_SOCKET` 8, `IS_FIFO` 9, and the block and character device tests at 10 and 11. `Redirect(fd, op)` encodes nine redirect operators: `WRITE (>)`, `APPEND (>>)`, `READ (<)`, `READ_WRITE (<>)`, `CLOBBER (>|)`, the fd-duplicating `DUP_READ/DUP_WRITE (<&/>&)`, and the combined-stream `WRITE_BOTH/ APPEND_BOTH (&>/&>>)`. And `ExpandParam(u8)` encodes the eighteen parameter-expansion modifiers — the whole `${var:-default}` family — from `DEFAULT` and `ASSIGN` through the prefix/suffix strippers (`STRIP_SHORT`, `STRIP_LONG`, `RSTRIP_SHORT`, `RSTRIP_LONG`), the substitution forms (`SUBST_FIRST`, `SUBST_ALL`), the case modifiers (`UPPER`, `LOWER`, `UPPER_FIRST`, `LOWER_FIRST`), and the indirection, keys, and slice forms. A test pins all eighteen modifier constants as unique. That so much of a shell’s grammar collapses into three opcodes-with-enumerations is the section’s economy: the front end parses `${x##*/}` and emits one `ExpandParam(STRIP_LONG)`, and the host does the work.

Why it belongs in the machine

The reason this belongs in the machine rather than in `zshrs` is the reason the whole project exists: putting it in the machine means it is compiled by the same JIT and AOT as everything else, and it means a *different* front end could emit the same pipeline opcodes if it wanted shell-out behavior. The shell is not special-cased; it is just the front end that uses these opcodes most. And because the opcodes dispatch through the `ShellHost`

trait, the *semantics* of a pipeline or a glob remain the front end's to define — the machine names the operation and locates its operands, the host decides what running it does, and the two stay cleanly separated.

Why a pipeline is three opcodes and not one

The pipeline trio — `PipelineBegin(n)`, `PipelineStage`, `PipelineEnd` — looks like it could be one opcode carrying the whole pipeline, and the reason it is three is worth understanding, because it reveals how the machine keeps stateful, multi-step shell constructs inside the stack discipline. A pipeline `a | b | c` is not a single value-producing operation; it is a setup, a sequence of stage wirings, and a wait. `PipelineBegin(3)` tells the host a three-stage pipeline is starting, so it can allocate pipes; each `PipelineStage` between commands wires the previous stage's output to the next stage's input; and `PipelineEnd` waits for the whole thing and pushes the final command's exit status. Splitting the construct into three opcodes lets the *commands themselves* — the `Exec` ops for `a`, `b`, `c` — sit between the brackets as ordinary opcodes, each able to carry its own redirections and argument expansions, rather than being swallowed into one monolithic pipeline op that would have to re-encode all of that inline. The bracket-and-fill shape is the same one the `WithRedirectsBegin/WithRedirectsEnd` pair uses for scoped redirections and the subshell pair uses for isolation: a “begin,” some ordinary ops, an “end,” with the host maintaining the cross-op state. It is how a stack machine, whose ops are supposed to be self-contained, expresses a construct that is irreducibly stateful across several commands.

The status that is a value

The `$?` mechanism is a small but instructive corner, because it shows a shell concept becoming a first-class `Value`. `Op::SetStatus` and `Op::GetStatus` write and read the VM's `last_status` field, and that field is a `Value::Status(i32)` — not a `Value::Int`. The distinction matters precisely because of the truthiness inversion of Chapter 3: `if cmd; then ...` is true when `cmd` succeeded, which is exit code zero, the opposite of how an integer's truthiness works. By making `last_status` a `Status` rather than an `Int`, the machine ensures that every consumer of `$?` — a conditional, a comparison, a stringification — applies the shell-correct semantics automatically, because `Value::is_truthy` already special-cases `Status` to be true at zero. The cost is a distinct variant the compilers must track as its own `Kind`, and the payoff is that no front end has to remember to invert the status when testing it: the value carries its own truthiness rule. The fact that the native AOT path has to emit `icmp eq, 0` for a `Status` where it emits `icmp ne, 0` for an `Int` is the price of this convenience, paid once in the compiler and verified by a three-hundred-program fuzzer, so that every `if $?` in every compiled shell program is correct without the front end thinking about it.

A hundred and forty builtins behind one opcode

The shell section's twenty-nine first-class opcodes are the universal plumbing, but a real shell has hundreds of builtins, and those reach the machine through a different door: `Op::CallBuiltin(id, argc)`, a single opcode fronting a hundred and forty builtin IDs defined in `shell_builtins.rs`. The IDs are organized into the families a shell user would recognize. There are the core utilities (`cd`, `pwd`, `echo`, `print`, `printf`, `export`, `unset`, `source`, `exit`, `return`, `true`, `false`, `test`, the colon and dot); the declaration builtins (`local`, `declare`, `typeset`, `readonly`, `integer`, `float`); input (`read`, `mapfile`); loop control (`break`, `continue`, `shift`, `eval`, `exec`); aliases (`alias`, `unalias`); option control (`set`, `setopt`, `unsetopt`, `shopt`, `emulate`, `getopts`); history (`fc`, `r`); function management (`autoload`, `functions`, `unfunction`); the directory stack (`pushd`, `popd`, `dirs`); command resolution (`type`, `whence`, `where`, `which`, `hash`, `rehash`, `unhash`); completion (`compgen`, `complete`, `compgopt`, `compadd`, `compdef`, `compinit`); and the zsh-specific machinery (`zstyle`, `zmodload`, `bindkey`, `zle`, `vared`, `zcompile`, `zformat`, `zparseopts`, `zregexparse`). The IDs are assigned in deliberately spaced numeric bands — navigation in the hundred-and-tens, completion in the hundred-and-thirties, zsh internals at one-forty and up — leaving gaps so new builtins slot into the right family without renumbering.

The design point is that a builtin is *not* a special opcode; it is a registered function reached through one generic dispatch op. A front end calls `register_builtin(id, closure)` to install each one, and the closure does whatever the builtin means, reaching the VM's stack and globals through the same borrow dance the host uses. This is why vimlrs can route its five hundred and ten builtins through `CallBuiltin` rather than the extension mechanism: the builtin table is just an indexed set of closures, and a language with a large flat standard library is well served by treating each library function as a builtin ID. The shell's hundred and forty and vimlrs's five hundred and ten are the same mechanism at different scales, and neither adds an opcode to the core — the one `CallBuiltin` op fronts them all.

Chapter 17 – AWK Inside the Machine

If the shell section shows fusevm was built to host a shell, the AWK section — larger, at sixty-one opcodes — shows it was built to host an AWK. AWK is a language about *records and fields*: a line of input split into `$1`, `$2`, up to `$NF`, with a rich library of string and math builtins and an associative-array data model. fusevm encodes all of it, and the design comment in `op.rs` is precise about why the AWK ops are named variants rather than opaque extension numbers: they are “a coherent universal vocabulary,” so they are defined as typed, self-documenting variants for visible IR, even though they still *dispatch* through a host.

The field model is opcodes: `Op::AwkFieldGet` reads `$i` (index popped from the stack), `Op::AwkFieldSet` assigns it (rebuilding `$0` and `NF`), `Op::AwkNf` pushes the field count, and `Op::AwkSetRecord` replaces the whole record and resplits. Special variables flow through `Op::AwkSpecialGet(u16)` and `Op::AwkSpecialSet(u16)`, indexed by name-pool entry (`FS`, `OFS`, `ORS`, `RS`, `NR`, `FNR`, `SUBSEP`, `RSTART`, `RLENGTH`, `FILENAME`, `CONVFMT`, `OFMT`). Output is `Op::AwkPrint(u8)` (join with `OFS`, end with `ORS`), `Op::AwkPrintf(u8)`, and `Op::AwkSprintf(u8)`. Input is the `Op::AwkGetLine(u8)` family, the byte encoding the source kind. And the string builtins are each their own op: `AwkLength`, `AwkSubstr`, `AwkIndex`, `AwkSplit`, `AwkSub`, `AwkGsub`, `AwkMatch`, `AwkToLower`, `AwkToUpper`, `AwkGensub`, alongside the associative-array ops `AwkArrayGet/Set/Exists/Delete/Clear/Len`.

Why a dedicated host, and what runs without one

Most AWK opcodes dispatch through an `AwkHost` (with a `DefaultAwkHost`), the AWK analogue of the shell host, and the reason is stated carefully in `awk_host.rs`: AWK values carry POSIX **numeric-string duality** — a field that looks numeric compares numerically but prints as its original text — plus `CONVFMT/OFMT` formatting and the tight coupling of `$0`, `$n`, and `NF`. The core `Value` cannot represent numeric-string duality, so the authoritative AWK runtime state stays host-side, and the VM is the control-flow-and-stack engine that calls back into it. As with the shell host, every method has a default, so an AWK opcode in a non-AWK embedding is inert but stack-balanced.

But twenty-nine AWK builtins are the exception: they execute natively *even with no host registered*, because they are pure on `fusevm::Value` or depend only on VM-owned state. The math group — `AwkInt`, `AwkSqrt`, `AwkSin`, `AwkCos`, `AwkExp`, `AwkLog`, `AwkAtan2`, plus `intdiv/intdiv0/mkbool` — needs no runtime. The string group — `substr`, `index`, `tolower`, `toupper`, scalar `length(s)` — is pure. The gawk character builtins `ord` and `chr`, the bitwise group (`and`, `or`, `xor`, `compl`, `lshift`, `rshift`), `strtonum`, and the time builtins (`sysptime`, `strftime`, `mktime`, the last two `chrono`-backed) all run host-free. Even `rand` and `srand` work without a host, against a VM-owned PRNG seed — a glibc-style LCG initialized to 1, deterministic — because the seed is execution-intrinsic state that resets with the VM. The result is that an expression like `substr("hello", 2, 3)` compiles and runs to `"ell"` with no `set_awk_host` call at all, while `length($0)` or `length(arr)`, which read field or array state, still require the host.

The arithmetic that traps, and the signal that has no value

Two corners of the AWK section show how carefully it tracks POSIX semantics. First, AWK division is not shell division. `Op::AwkDiv` and `Op::AwkMod` are float divide and modulo that raise a *fatal* runtime error on a zero divisor — the exact gawk messages, `"division by zero attempted"` and `"division by zero`

attempted in `\%'“`— where the `shell-arithmeticOp::Div/Op::Mod` quietly yield `Undef/0`. Because they conditionally trap, `AwkDiv/AwkMod` are interpreter-only; the block-JIT-eligible variants `AwkDivJit/AwkModJit` carry byte-identical semantics through a guarded early-exit that the compilation chapters describe. The same pattern produces JIT-eligible `AwkLshiftJit`, `AwkRshiftJit`, `AwkComplJit`, `AwkSqrtJit`, and `AwkLogJit`, plus the block-JIT field read `AwkGetFieldNum(u16)` that reaches the active record through a host-installed `extern"C"` hook.

Second, AWK control flow has no value. `next`, `nextfile`, and `exit` are statements, not expressions, so they cannot be a `Value` on the stack. `Op::AwkSignal(u8)` carries them as a VM-state side effect: it halts the current chunk and stashes a code (`NEXT`, `NEXTFILE`, or `EXIT`) that the front-end driver reads through `VM::awk_signal()` after `run()` returns, to drive its own record/file/exit flow. `zshrs` and `stryke` never emit it, so `awk_signal()` stays `None` for them and the `Halted` result is byte-identical to before — the channel is a side effect, not a new `VMResult` variant. The point, throughout, is one machine: AWK's field model, its numeric-string duality, and its statement-level control flow are a set of opcodes, a host trait, and a side channel — not a separate interpreter bolted on.

A host-free AWK builtin, end to end

The twenty-nine host-free AWK builtins are easy to undervalue until you see one run with no host at all, so here is `substr` doing exactly that, from the crate's documentation:

```
use fusevm::{VM, ChunkBuilder, Op, Value};

let mut b = ChunkBuilder::new();
let s = b.add_constant(Value::str("hello"));
b.emit(Op::LoadConst(s), 1);
b.emit(Op::LoadInt(2), 1);
b.emit(Op::LoadInt(3), 1);
b.emit(Op::AwkSubstr(3), 1);           // substr("hello", 2, 3)
let mut vm = VM::new(b.build());      // note: no set_awk_host
// vm.run() -> Value::str("ell")
```

No `set_awk_host`, no `AwkHost` implementation, and the program produces `"ell"`, because `AwkSubstr` is pure on `fusevm::Value` — it needs no field state, no special variables, nothing the host owns. The same is true of the math group, the character builtins `ord` and `chr`, the bitwise group, `strtonum`, and the time builtins; even `rand` and `srand` work host-free against a VM-owned PRNG seed. The line that needs the host is `length($0)` or `length(arr)`, because those read field or array state that lives host-side. This split — pure builtins native, stateful builtins host-dispatched — is what lets a non-AWK embedding run a chunk that happens to call `substr` or `sqrt` without registering anything, while still routing the genuinely AWK-shaped operations to the front end that owns AWK's runtime model.

The reserved range, and two layers of extension

The AWK ops are the clearest case of a subtle design move: two independent layers of extensibility coexisting without a central registry, by partitioning the ID space. Under the hood every `Op::Awk*` variant is dispatched as an `Op::ExtendedWide(id, payload)` whose `id` falls in a *reserved range* — `AWK_OP_BASE` is `40_000`, `AWK_OP_END` is `41_000` — and the VM routes any wide-extension op whose ID lands in that thousand-wide window to the registered `AwkHost`, while everything below it goes to the generic extension handler a front end installs with `set_extension_wide_handler`. The reserved range was “chosen high to leave the low `Extended/ExtendedWide` id space free for generic front end handlers,” as the source comment says. The consequence is two non-colliding extension mechanisms: a front end like `stryke` uses the low IDs for its four hundred and fifty extended ops, and `awkrs` uses the reserved high range for its AWK semantics, and neither has to coordinate with the other because the range partition decides ownership. The typed `Op::Awk*` variants

are a readable, self-documenting alternative spelling of the same reserved-range IDs — they map one-to-one onto the numbers and reach the identical host method — so a front end can emit AWK behavior as a named variant for visible IR or as a raw wide-extension number, and both arrive at the same place. Two layers of extensibility, one ID space, no central authority: that is how one machine hosts both a general scripting language and an AWK without either knowing the other exists.

The AwkHost surface, and the lvalue problem

The `AwkHost` trait's method surface is broad — field access (`field_get`, `field_set`, `nf`, `set_record`), special variables (`special_get`, `special_set`), output (`print`, `printf`, `sprintf`), input (`getline`), and the string and math builtins (`length`, `substr`, `index`, `split`, `sub`, `gsub`, `match_re`, `gensub`, `tolower`, `toupper`, and the rest) — and one method signature in it captures why AWK needs a dedicated host rather than a set of pure opcodes. The substitution methods `sub` and `gsub` take a `target_ref: &AwkLvalue` rather than a value, because `sub(/re/, "x", $2)` must *write back* into field two of the current record, and `sub(/re/, "x", arr[k])` must write back into an array element — these are assignable locations, lvalues, not values, and a pure stack-machine opcode that received only the target's current value could read it but never update it in place. `AwkLvalue` names the location — a field index, an array element, a special variable — so the host can perform the substitution and store the result where AWK semantics require. This is the concrete shape of the numeric-string-duality argument made earlier: AWK's field-and-record model is full of assignable, stateful, format-sensitive locations that `fusevm::Value` cannot represent, so the authoritative state stays host-side and the trait's signatures — `AwkLvalue` targets, `Option`-wrapped `getline` operands, `CONVFMT`-aware formatting — are the interface to it.

Why the typed variants exist at all

There is a reasonable question lurking under the AWK section: if every AWK op dispatches through the host as a reserved-range `ExtendedWide`, why give them sixty-one `typed Op: :Awk*` variants instead of just emitting the magic numbers? The answer is visibility and type safety, and it is the same instinct that makes the whole opcode set an enum rather than a vector of integers. A chunk full of `ExtendedWide(40037, 3)` is opaque — you cannot read it, a debugger cannot label it, and a typo in the constant is a silent miscompile. A chunk full of `AwkGsub(3)` is self-documenting: the variant names the operation, the payload's meaning is in the doc comment, and the compiler's exhaustiveness checking ensures every AWK op is handled in the `Hash` impl and the dispatch. The typed variants map one-to-one onto the reserved-range IDs and reach the identical host method through the same `dispatch_awk` path, so there is no behavioral difference — but there is an enormous difference in how readable the IR is, how safely a front end can emit it, and how confidently the maintainers can refactor it. The reserved-range encoding is the *mechanism*; the typed variants are the *ergonomics*; and offering both lets awkrs emit readable, type-checked AWK bytecode while preserving the escape hatch of raw wide-extension numbers for anything the typed set does not yet name. It is the machine refusing to make self-documentation cost performance: the typed variant and the raw number compile to the same dispatch, so a front end never has to choose between readable IR and fast IR.

getline, and the shape of stateful input

The `getline` family is a small case study in why AWK's I/O needed a host method rather than a core opcode, and it repays a look because its signature encodes the irreducible statefulness. AWK's `getline` has several forms — plain `getline` reads the next record from the main input, `getline var` reads into a named variable, `getline < file` reads from a file, `cmd | getline` reads from a command's output — and the host method `getline(source, operand, var_name)` carries all of them: `source` is a code selecting the form, `operand` is the file path or command string when the form needs one, and `var_name` is the target variable for the `var` forms. It returns a status — `1` for a record read, `0` for end of input, `-1` for an error — exactly AWK's `getline` return convention. None of this can be a pure opcode, because reading the next record advances input state,

updates `NR` and `FNR`, may open a file or spawn a command, and resplits into fields — all state the AWK host owns. The opcode `Op::AwkGetline(u8)` carries only the source-kind byte; the operand comes off the stack and the heavy lifting happens host-side. This is the numeric-string- duality argument in its most concrete form: AWK's input is a stateful stream feeding a field model, and the only honest place to keep that state is the front end that defines what a record and a field are, with the core providing the opcode that names the operation and routes it to the host.

The numeric-string duality, in one concrete case

The single most important thing about AWK that the core `Value` cannot represent is worth pinning with a concrete case, because it is the whole justification for the AWK host's existence. In AWK, a field that *looks* numeric carries a dual nature: it compares numerically but prints as its original text. Read a line `010 020` into an AWK program and `$1` is, simultaneously, the number ten (so `$1 < $2` is a numeric comparison and `$1 + 0` is ten) and the string `"010"` (so printing `$1` emits `010`, with the leading zero). A `Value::Int(10)` loses the text; a `Value::Str("010")` loses the cheap numeric comparison; and no single `Value` variant can be both at once. gawk solves this with a “strnum” type that carries both representations and a flag for whether the string looked numeric on input; fusevm's core has no such variant, so the authoritative strnum state stays host-side, in awkrs, and the AWK opcodes route through the `AwkHost` to reach it. This is why the AWK section, despite being sixty-one typed opcodes, is *dispatched* rather than *executed* by the core: the opcodes name the operations and carry the field indices and arg counts, but the values they operate on live in the host's runtime where their dual nature can be preserved. The `CONVFMT` and `OFMT` formatting variables, the `SUBSEP` array keying, the `$0/$n/NF` coupling — all of it is the same story, AWK state too rich for the core `Value`, kept host-side and reached through a trait. The core provides the control flow, the stack, and the opcode names; awkrs provides the one thing the core deliberately cannot, a value model with numeric-string duality.

A statement with no value, carried as state

`AwkSignal` is the one AWK opcode that has no place on the operand stack, and the way it is handled is a small masterclass in adding a capability to a machine without disturbing the machine's contract. AWK has three control-flow statements — `next` (skip to the next input record), `nextfile` (skip to the next input file), and `exit` (end the program) — and none of them is an expression: they do not produce a value, they redirect the per-record processing loop that the awkrs driver runs around the compiled chunk. A naive design might add a fourth `VMResult` variant, `Signal(code)`, so `run` could report the statement. But that would be a breaking change to all five front-end drivers, four of which have no notion of AWK signals and would have to grow a new match arm for a result they can never receive. fusevm instead carries the signal as `VM state`: `Op::AwkSignal(code)` halts the current chunk and stashes the code (`NEXT`, `NEXTFILE`, or `EXIT`) in a field the awkrs driver reads through `VM::awk_signal()` after `run` returns `Halted`. To a non-AWK front end, nothing changed — `run` returns `Halted` exactly as it always did, `awk_signal()` is `None`, and there is no new variant to handle. To awkrs, the channel is exactly what it needs: run the per-record chunk, check `awk_signal()`, and drive its record/file/exit loop accordingly. This is the side-channel pattern done right: a new capability that one front end needs is added as optional VM state read through a new method, not as a change to the shared result type, so the four front ends that do not need it are byte-identical and the one that does gets a clean interface. It is the same instinct as the host traits' all-defaults discipline — grow the machine without breaking the front ends that do not use the growth — applied to a control-flow signal that, by its nature, could not be a value.

Chapter 18 – The Extension Mechanism and the Five Frontends

The shell and AWK sections are first-class because process control and record processing are universal enough to earn named opcodes. Everything else a front end needs that the core does not name flows through the **extension mechanism**, and it is the seam that makes one machine serve five very different languages.

There are two extension opcodes and one builtin-dispatch opcode. `Op::Extended(u16, u8)` carries an extension op ID and a small inline operand; `Op::ExtendedWide(u16, usize)` carries a wide payload for jump targets or large indices; and `Op::CallBuiltin(u16, u8)` calls a registered builtin by ID with an argument count. A front end installs a handler — `fn(&mut VM, u16, u8)` — with `set_extension_handler`, and the VM routes `Extended` ops to it. The handler is taken *out* of the VM for the duration of the call, a borrow-checker dance that lets it reach back into the VM's stack and globals while it runs, and restored afterward. Against `Op::Obj` handles, this is how a front end keeps its own objects on fusevm's stack without fusevm understanding them: elisp's cons-cell operations, vimscript's buffer pokes, anything genuinely language-specific.

Five frontends, five strategies

The five live front ends use these mechanisms in strikingly different proportions, and the spread is the best illustration of the design's flexibility. `stryke` registers roughly four hundred and fifty extended ops — it is the most ambitious language and uses the extension space heavily. `zshrs` registers about twenty, because most of what a shell does is already first-class shell opcodes. `awkrs` registers about ninety-five, layering its full AWK semantics on top of the sixty-one first-class AWK ops. `elisprs` registers just ten, lowering Lisp to fusevm and keeping cons cells in its own heap behind `Obj` handles. And `vimlrs` takes the other road entirely: it routes roughly five hundred and ten builtin IDs through `CallBuiltin` rather than extension ops, treating its standard library as a flat dispatch table. None of these collide, because each front end owns its own ID space — there is no central registry, only the handler a given VM was configured with.

The AWK ops illustrate a refinement of the same idea. They are emitted, under the hood, as `Op::ExtendedWide(id, payload)` with `id` in a reserved range — the constant `AWK_OP_BASE` is `40_000` and `AWK_OP_END` is `41_000` — chosen high so the low extension-ID space stays free for generic front-end handlers. The VM routes any `ExtendedWide` op whose ID falls in that range to the registered `AwkHost`, and the typed `Op::Awk*` variants are a self-documenting alternative that maps one-to-one onto the same IDs and the same dispatch path. So a front end can emit AWK behavior either as a readable named variant or as a raw wide-extension number, and both reach the identical host method. The reserved-range trick is how two layers of extensibility — the generic handler and the AWK host — coexist without a central authority deciding who owns which number.

The adoption pattern

All five front ends drive `fusevm::VM` through a bridge layer rather than emitting ops directly inline, and the common shape of that bridge is worth recording because it is where the architecture meets real programs. The pattern has four parts: a front-end-side eligibility analysis deciding which subroutines or rule bodies can be lowered to fusevm ops at all; an op-vector-to-`Chunk` translation cached behind a `OnceCell` or `HashMap` so

the build runs once per source region rather than once per call; a `VMPool` on the front end's `Runtime` so the slot, stack, and globals vectors recycle across invocations; and a narrow writeback driven by a precomputed `Vec<u16>` of `Op::SetSlot` targets, so only the slots a region actually mutated get copied back into the front end's own runtime. The on-disk JIT cache, keyed by op-hash, handles compiled-code persistence; these per-front-end caches handle the chunk-build and runtime-setup costs the disk cache cannot touch. It is the same separation one more time: the machine supplies the fast, general primitives, and each language supplies the policy for when and how to reach them.

Installing a handler

The extension handler is a closure, and installing it is a single call. The crate's documentation shows the pattern bare:

```
let mut vm = VM::new(chunk);
vm.set_extension_handler(Box::new(|vm, id, arg| {
    match id {
        0 => { /* language-specific op 0: read vm.stack, push a result */ }
        1 => { /* language-specific op 1 */ }
        _ => {}
    }
}));
```

The handler receives a mutable reference to the VM — reached through the same take-out-and-restore dance the host uses — plus the extension op's ID and inline argument, and it does whatever the front end's op means: pops operands from the VM's stack, manipulates the front end's own objects behind `Obj` handles, pushes a result. A front end's whole non-core vocabulary lives in this `match`. The builtin analogue, `register_builtin(id, closure)`, installs a function reachable through `Op::CallBuiltin(id, argc)`, and it is the route `vimlrs` takes for its roughly five hundred and ten builtins — a flat dispatch table rather than a large `match` on extension IDs. Both mechanisms are just function pointers the VM holds and calls; neither requires the core to know anything about what the function does, which is exactly what lets the same compiled engine serve five front ends whose vocabularies share not a single ID.

Why the seam holds the compiler together

It is worth closing the embedding movement by connecting it forward to the compilation movement, because the host-and-extension seam is what makes the AOT compiler's central simplification legal. The native AOT path reasons about a program by holding its slots and stack in machine registers and tracking their types — and that reasoning is only sound because everything *effectful and untyped* is behind a call the compiler does not have to model. When the native path meets an `Exec`, a `CmdSubst`, or an arbitrary `Extended` op, it does not try to lower it; it either spills to the boxed world and calls through the `exec_op` shim, or it declines the whole chunk to the threaded path. The host boundary is what makes “decline” a complete answer: because the effectful operations are all reached through traits and handlers rather than woven into the value model, the compiler can draw a clean line between the scalar, analyzable core of a program and the host-dispatched edges, and lower the former while leaving the latter to the interpreter. The pure machine is compilable *because* the impure parts were pushed behind a seam, and the entire last movement is, in a sense, an exploration of what becomes possible once that seam exists.

Chapter 19 – Cranelift, the Backend

All of fusevm's native code — the three JIT tiers and the ahead-of-time compiler — is emitted through **Cranelift**, version 0.130, so before any of them makes sense it is worth saying what Cranelift is and why fusevm uses it rather than LLVM.

Cranelift is a code generator: you hand it an intermediate representation (CLIF), it optimizes lightly and emits machine code for the host architecture. It is, in category, the same kind of thing as LLVM — a compiler backend — but with different priorities. LLVM optimizes aggressively and compiles slowly; Cranelift optimizes modestly and compiles *fast*, and it is written in Rust with no C++ toolchain dependency. For a JIT, fast compilation is the entire game: the code is being compiled while the program waits, so a backend that takes milliseconds where LLVM takes seconds is not a luxury but a requirement. For fusevm's AOT path, Cranelift's purity-of-Rust matters too — the compiler is just another crate, embeddable anywhere fusevm is, with no external toolchain to install. The whole engine pulls in only twelve direct dependencies, and the Cranelift crates are most of them, all gated behind the `jit` and `aot` feature flags so a front end that wants only the interpreter pays for none of it.

The crates fusevm uses

fusevm reaches Cranelift through several crates, each declared optional in `Cargo.toml`. `cranelift-frontend` provides the `FunctionBuilder`, which constructs SSA form as you emit instructions. `cranelift-codegen` provides the IR and the target ISA. `cranelift-native` detects the host architecture. `cranelift-jit` provides the in-memory module — compile a function and get back a callable pointer — which the `jit` feature uses and which the AOT path also uses to validate its codegen end to end. And `cranelift-object` provides the `ObjectModule` that the `aot` feature uses to emit a relocatable `.o`. The two compilers share one lowering vocabulary — `iadd`, `fdiv`, `icmp`, `fcmp`, `brif`, `select`, the `call` to an imported shim — and differ mainly in what they do with the finished function: the JIT finalizes it to executable memory and calls it; the AOT path writes it into an object file.

The two facilities worth meeting early

Two Cranelift facilities recur throughout the compilation chapters and are worth meeting now. The first is the frontend `Variable`. Rather than constructing SSA ϕ -nodes by hand at every control-flow join — which is what makes naive native code generation painful — you declare a `Variable`, write it with `def_var` and read it with `use_var`, and Cranelift inserts the ϕ -nodes for you when you seal the blocks. This is how every fusevm compiler threads the operand stack and the locals through loops without manual block-argument bookkeeping. A counted loop with a back-edge compiles to a native loop with the loop variables in registers, and the compiler never constructs a single ϕ -node by hand. It is a small API that does an enormous amount of work.

The second is the generic `Module` trait. The AOT compiler's central codegen function, `build_entry`, is written as `build_entry<M: Module>(module, chunk)`, generic over Cranelift's `Module` trait — so the very same code generates a function for the in-memory JIT module (used to validate the compiler by running a freshly compiled chunk on a fresh VM) and for the on-disk object module (used to ship a binary). The only difference is the target ISA's `is_pic` flag: the in-process JIT builds position-dependent code, the object path position-independent code as a relocatable object must. One codegen, two backends, no second implementation to drift — the same single-source-of-truth discipline that governs the opcode semantics, applied to the compiler.

Why not LLVM, stated plainly

The choice of Cranelift over LLVM is the kind of decision that deserves its reasons spelled out, because LLVM is the obvious default and fusevm declined it deliberately. LLVM is the better *optimizer*: given seconds and a willingness to recompile, it will out-optimize Cranelift on almost any code. But fusevm's compilers are not run in a build step a user waits out once; they run *while the program runs*, in the JIT case literally while the interpreter is paused waiting for the native code, and in the AOT case as an embeddable library with no external toolchain. For the JIT, an optimizer that takes seconds is disqualifying no matter how good its output, because the warm-up would dwarf the speedup on all but the longest-running loops — the disk cache's measured numbers make this concrete, a cold block compile at about a hundred and fifty microseconds where LLVM would be orders of magnitude slower. For the AOT path, LLVM would drag a C++ toolchain into a project whose entire identity is “a VM you can `cargo add`,” breaking the single-binary, no-external-runtime promise that makes the five front ends deployable as one artifact. Cranelift trades peak optimization for fast, dependency-free, pure-Rust compilation, and that is exactly the trade a JIT-and-embeddable-AOT system needs. The 1.8-times-slower-than-03 figure the linear JIT posts is the visible cost of that trade, and for code compiled in microseconds it is a bargain.

SSA without the pain

The `Variable` abstraction deserves one more pass, because it is doing more work than its small API suggests, and understanding it makes every subsequent compiler chapter legible. Generating correct native code for a loop traditionally means constructing SSA form by hand: at every point where control flow merges — the top of a loop, the join after an `if` — you must insert a ϕ -node that selects the right incoming value depending on which predecessor block control came from, and getting the ϕ -nodes right at a loop's back-edge, where a variable's value depends on the previous iteration, is the part that makes hand-written codegen error-prone. Cranelift's frontend lets you skip all of it: you declare a `Variable` for each logical value — each operand-stack position, each slot, each global — write it with `def_var` wherever the bytecode stores, read it with `use_var` wherever the bytecode loads, and when you have emitted every block and call `seal_all_blocks`, Cranelift reconstructs the SSA form and inserts every ϕ -node, including the back-edge ones, from the def/use record you left. The compiler's code for a loop ends up reading as linearly as the bytecode did, because the hard part — the ϕ -nodes — is inferred rather than written. Every fusevm compiler leans on this, which is why the basic-block compiler, the locals-in-registers work, and the kind lattice can all be described as “thread the value through a `Variable`” rather than “construct the joins by hand.”

CLIF, and what “lowering” actually means

It helps to be concrete about what fusevm hands Cranelift, because “emit native code” glosses a step that is the heart of every compiler chapter. Cranelift's input is CLIF, Cranelift IR — a typed, SSA, three-address intermediate representation, the same *category* of thing as LLVM IR but simpler. Each CLIF instruction names an operation (`iadd`, `fmul`, `icmp`, `brif`, `call`, `load`, `store`), the typed values it consumes, and the typed value it produces. When a fusevm compiler “lowers” an opcode, it appends CLIF instructions through the `FunctionBuilder`: lowering `Op::Add` on two integer operands appends an `iadd` whose inputs are the two SSA values currently representing the top of the abstract stack and whose output becomes the new top; lowering `Op::JumpIfFalse` appends a `brif`. Cranelift then takes the finished CLIF function, runs its modest optimization passes — some constant folding, some dead-code elimination, register allocation, instruction selection — and emits machine code for the host ISA. The compiler chapters are, in this light, all variations on one activity: walk the chunk's ops, maintain an abstract model of the stack and slots as SSA values, and append the CLIF instructions that compute what each op computes. The differences between the tiers are differences in *how much* of the chunk they can model this way — a straight line, a whole CFG, a recorded trace — and in what they do at the edges they cannot model, but the inner act is always “opcode in, CLIF out.”

The function signature is the contract

Every native function a fusevm compiler emits has a signature, and the signature is the contract between the compiled code and the VM around it — worth fixing because each tier's signature tells you exactly how much VM it needs. The AOT entry is `extern "C" fn(*mut VM) -> i64`: it takes the whole VM as an opaque pointer and returns a status word (always zero; the real result is left in the VM). The block JIT's compiled function is thinner, taking essentially a `*mut i64` slots pointer, because a block-compiled chunk's entire mutable contact with the world is its slots. The linear JIT's is thinner still, taking its inputs and returning a `Value` directly, because a straight-line expression has no slots to mutate and no VM state to touch beyond reading its arguments. Reading the signatures top to bottom is reading the tiers' ambitions: the linear JIT promises only to compute an expression, so it needs only arguments and a return; the block JIT promises to run a loop over locals, so it needs the slots pointer; the AOT path promises to run an arbitrary chunk including the boxed boundary, so it needs the whole VM. The thinness of the lower tiers' signatures is exactly why their compiled code is position-independent enough to cache to disk: a function that depends on nothing but a pointer argument can be mapped into any process and called.

The FunctionBuilder lifecycle

Every fusevm compiler drives Cranelift through the same lifecycle, and naming its phases makes the compiler chapters' "emit a block, thread a `Variable`, seal" shorthand precise. A compilation begins by creating a `FunctionBuilder` over a fresh function with a declared signature. The compiler then creates blocks — one per leader in the native path, one per opcode in the threaded path — and *switches to* a block to append instructions into it. Within a block it declares and uses `Variables` (`def_var` to write, `use_var` to read), appends arithmetic and memory and branch instructions, and ends the block with a terminator (`jump`, `brif`, `return`). Crucially, a block must be *sealed* before its SSA is finalized, and a block can be sealed only once all its predecessors are known — which for a loop header means after the back-edge has been emitted, since the back-edge is one of the header's predecessors. This is why the compilers emit all blocks and all their branches first and call `seal_all_blocks` at the very end: sealing early would finalize a loop header's ϕ -nodes before the back-edge that feeds them existed. After sealing, the builder is finalized, Cranelift runs its passes, and the function is either JIT-finalized to executable memory (the in-process path) or written into the object module (the AOT path). The lifecycle — create, switch, define/use, terminate, seal-all, finalize — is identical across the linear, block, and AOT compilers; what differs is only how many blocks they create and how they decide the abstract stack and slot state at each. When a later chapter says "the compiler threads the operand stack through `Variables` and seals at the end," this lifecycle is the machinery underneath that sentence, and the seal-at-the-end discipline is what makes the back-edge ϕ -nodes come out right.

Modest optimizer, deliberate choice

It is worth being precise about what "Cranelift optimizes modestly" means, because the modesty is a feature for fusevm's use and a reader should understand which optimizations it does and does not get. Cranelift performs the optimizations that are cheap and locally profitable: constant folding, dead-code elimination, some redundancy elimination, instruction selection tuned to the target ISA, and register allocation. What it does not do — and what LLVM at -O3 would — is the expensive global analyses: aggressive loop transformations, scalar evolution that recognizes an arithmetic series and replaces a loop with a closed-form expression, deep inlining heuristics, polyhedral scheduling. For fusevm this division is exactly right. The cheap optimizations are the ones that matter for the code the compilers emit — a tight numeric loop benefits enormously from good register allocation and instruction selection, which Cranelift does well — while the expensive ones are precisely the ones a JIT cannot afford and an embeddable AOT compiler does not need, because the fusevm front ends and the fused superinstructions already do the high-level work (loop fusion is a peephole on the bytecode, not a job left to the backend). There is even a benchmarking benefit to the modesty: because Cranelift does not do scalar evolution, a compiled counted loop genuinely *loops* rather than being folded to a formula, so a benchmark of a loop measures a loop — which is part of why the slot-based inputs and the honest baselines

produce trustworthy numbers. The modest optimizer is not a limitation fusevm tolerates; it is the optimizer fusevm wants, fast enough to run while the program waits and thorough enough on the local code that matters, with the global transformations left to the front end where they belong.

One backend, two finalizations

It is worth fixing the one structural difference between how the JIT and the AOT path *use* Cranelift, because everything else about them is shared. Both build a CLIF function through a `FunctionBuilder`; both lower opcodes to the same `iadd/fdiv/brief/call` vocabulary; both seal at the end. They diverge only at *finalization* — what they do with the finished function. The JIT path, through `cranelift-jit`, finalizes the function to executable memory in the current process and hands back a callable function pointer, which the code then transmutes to the right signature and calls. The object path, through `cranelift-object`, writes the function into an `ObjectModule` that is serialized to a relocatable `.o` on disk, to be linked later. The same `build_entry`, generic over the `Module` trait, feeds both: it does not know or care whether the `Module` it is appending to is a JIT module or an object module, because the `Module` trait abstracts exactly the operations codegen needs — declare a function, define it, declare an import. So the divergence is two implementations of one trait, chosen by the caller, with the codegen above blind to the choice. This is why the AOT path can validate itself by JIT-compiling in process and trust that the result matches the object it would ship: the function is the same up to the moment of finalization, and finalization is the one thing the generic codegen does not touch. One backend, one codegen, two finalizations — and the trait boundary between codegen and finalization is what keeps the two from drifting.

Chapter 20 – The Linear JIT

The first and simplest JIT tier is the **linear JIT**, and it does exactly one thing well: it compiles a straight-line expression chunk — no loops, no branches — to a native function that takes the chunk's inputs and returns a `Value`. It is the tier you reach for when a front end has an expression hot enough to be worth compiling but simple enough that there is nothing to analyze. `JitCompiler::new` builds it; `is_linear_eligible(&chunk)` decides whether a chunk qualifies; and `try_run_linear(&chunk, &args)` compiles on first call, caches, and runs, returning `Some(Value)` — an `Int` or a `Float` — or `None` if the chunk was not eligible.

The eligible op set is the scalar arithmetic core, and the README tabulates it exactly: the load ops (`LoadInt`, `LoadFloat`, `LoadConst` for int or float, `LoadTrue`, `LoadFalse`); the nine arithmetic ops; the seven numeric comparisons plus `Spaceship`; the bitwise six; `LogNot`; the stack shuffles `Pop`, `Dup`, `Swap`, `Rot`; and the register-resident slot ops `GetSlot`, `SetSlot`, `PreIncSlot`, `PreIncSlotVoid`, `AddAssignSlotVoid`. Anything outside that set makes the chunk ineligible and the call returns `None`, leaving the interpreter to run it.

Specification by translation

The linear JIT's virtue is that it has no speculation: its IR matches the bytecode exactly. When either operand of an arithmetic op is a float, both promote to `f64` and Cranelift emits the `fcvt_from_sint` conversion plus the float instruction; when both are integers it emits the integer instruction. The two operations that have no single machine instruction — `Pow` (wrapping integer for two integers, `f64::powf` otherwise) and `Mod` (float `fmod`) — become calls to small runtime helpers, exactly as the interpreter calls the same Rust methods. This is the pattern every fusevm tier follows, introduced here in its simplest form: a native instruction where one exists, a libcall where one does not, and the libcall calls the identical Rust function the interpreter does, so a finite result matches bit-for-bit.

The numbers justify the tier's existence. On slot-based inputs — chosen so the compiler cannot constant-fold the answer away — the linear JIT runs a hundred iterations of mixed arithmetic in seventy-five nanoseconds against the interpreter's 2.2 microseconds (twenty-nine times faster), two hundred iterations of bitwise work in a hundred and thirty nanoseconds against 6.6 microseconds (fifty-one times faster), and the float variant thirteen times faster. Against hand-written Rust compiled at `-O3`, the linear JIT is consistently about 1.8 times slower — which, for a dynamically typed bytecode compiled in microseconds, is the gap closing about as far as a method-at-a-time JIT can close it. The cache lookup that decides whether a chunk was already compiled is $O(1)$ against the precomputed `op_hash`, about twenty-four nanoseconds of overhead — a rounding error against any real computation.

Eligibility as a contract

`is_linear_eligible` is more than a gate; it is a contract that tells a front end exactly which expressions are worth handing to this tier, and its eligible set is the most restrictive of any tier precisely because the linear JIT does the least analysis. The set, from the README, is the scalar arithmetic spine and nothing else: the numeric loads, the nine arithmetic ops, the numeric comparisons and `Spaceship`, the bitwise six, `LogNot`, the four stack shuffles, and the register-resident slot ops `GetSlot/SetSlot/PreIncSlot/PreIncSlotVoid/AddAssignSlotVoid`. There is no control flow in that list — no `Jump`, no `JumpIfFalse` — because the linear JIT compiles a *straight line* and nothing else; the moment a chunk branches, eligibility fails and the chunk is

the block JIT's problem, not the linear JIT's. There are no heap ops, no strings, no arrays, because those cannot live in a register. The restriction is what keeps the tier *free of speculation*: with only straight-line scalar arithmetic, the IR can match the bytecode operation for operation, with no guard, no deopt, no assumption that could be wrong. The linear JIT is the one tier in the system that never speculates, and the price of that certainty is that it accepts only the chunks where certainty is possible.

The libcall pattern, introduced once

The two operations the linear JIT cannot express as a single instruction — `Pow` and `Mod` — introduce a pattern that every later tier reuses, so it is worth fixing here in its simplest setting. There is no `pow` machine instruction and no floating-point `mod` instruction, so the compiler emits a *call* to a small runtime helper: a `powf` shim that calls `f64::powf`, an `fmod` shim that calls Rust's `%` on two `f64`s. The helper is the same Rust code the interpreter runs, so a finite result is bit-for-bit identical between interpreted and compiled execution — which is the whole point, because a `pow` that rounded differently in compiled code would be a miscompile a fuzzer would catch immediately. This is not a concession or a gap; it is how native code has always handled leaf math functions, and a compiled expression that calls `powf` is no less native than one that calls the C library's `pow`. The later tiers extend the pattern — the AOT path's thirteen transcendentals share one `unary_math` dispatcher, the disk cache keys libcall relocations by stable helper IDs so cached code can re-resolve them — but the principle is the one the linear JIT establishes: a native instruction where one exists, a libcall to the interpreter's own function where one does not, and identical results either way.

The cache key is the op-hash

Every JIT tier caches its compiled output, and every tier keys that cache on the chunk's `op_hash`, so it is worth understanding what that key includes and excludes, because the inclusions and exclusions are exactly the conditions under which compiled code may be reused. The `op_hash` is a `DefaultHasher` over the chunk's ops and constants — and nothing else. Not the line numbers, not the name strings, not the source file. The consequence is that two chunks differing only in which line a statement came from, or what a global happened to be named, hash the same and share compiled code, which is correct because neither line numbers nor name strings change what the machine computes. But two chunks differing in a single opcode, or in a single constant — even a float constant that differs only in being `-0.0` versus `0.0`, because the `Hash` impl uses the bit pattern — hash differently and never share code, which is also correct, because they compute different things. The block JIT adds the slot-kind vector to its key, so float-specialized code is never served for an integer slot; the tracing JIT adds the record-anchor IP and a content hash over the recorded path, so two divergent traces through one chunk never collide. The keying is thus precisely as fine as it must be and no finer: it ignores what does not affect computation (lines, names) and distinguishes everything that does (ops, constants, slot kinds, recorded paths). A cache key that was coarser would occasionally serve wrong code; one that was finer would miss reuse opportunities; the op-hash plus the per-tier extensions is tuned to be exactly right, which is what lets the cache be $O(1)$ and trustworthy at once.

Chapter 21 – The Block JIT

The linear JIT compiles a straight line. Real programs branch and loop, and the **block JIT** is the tier that compiles a *whole chunk* — its entire control-flow graph — to native code, and it is the tier that gives the biggest single jump in speed for code that qualifies. `is_block_eligible(&chunk)` decides; the chunk is compiled to one native function after the block warm-up threshold (default one invocation); and the result is keyed by the chunk's `op_hash` so a chunk seen before is not recompiled.

“Block-eligible” means the chunk fits inside what the block compiler can lower end to end: a body of arithmetic, comparisons, locals, fused superinstructions, and structured control flow with no opcode that escapes its model. For such a chunk, the block JIT compiles the whole thing to one native function and runs it with the locals lifted into machine values and zero per-opcode dispatch. The README's numbers are stark: a million-iteration unfused sum loop drops from thirty milliseconds interpreted to three hundred and fifteen microseconds compiled, ninety-five times faster; a hundred-by-hundred nested loop drops from three hundred and forty microseconds to nine and a half, thirty-six times. The mechanism behind the speed is that all mutable state flows through a single slots pointer — a `*mut i64` handed to the native function — and `AccumSumLoop` is register-allocated with Cranelift block parameters, so the inner loop touches no memory at all.

Slots are bits, and the kind goes in the cache key

The block JIT's representational trick is how it handles a slot that holds a float. Slots are promoted to Cranelift `i64 Variables` holding raw bits, and when a slot's kind is `Float`, the `i64` is the `f64` bit pattern: `GetSlot` bitcasts `I64 → F64` (and converts an integer operand with `fcvt_from_sint` before float arithmetic), `SetSlot` bitcasts `F64 → I64`. The kind of each slot is passed in through `try_run_block_kinded` (and its eager sibling), and — this is the crucial correctness move — the *kind vector is folded into the native-code cache key*, both the in-memory key and the on-disk `*.blk.fjit` blob name, so float-specialized code is never reused for an integer slot or vice versa. The plain `try_run_block` treats every slot as `Int`, unchanged for integer consumers. This `SlotKind::Float` support is exactly what lets awkrs block-JIT-compile its `f64` AWK numeric chunks and persist them. The integer-only fused superinstructions guard themselves: faced with a `Float` slot they bail to the interpreter rather than miscompute, because their fast handlers assume `i64`.

Trapping division without a trap

The block JIT also has to reproduce one of AWK's trickiest semantics in native code: a division by zero that must raise a fatal error, even though floating-point `fdiv` and `fmod` do not trap — they yield `inf` or `NaN` and keep going. The block-JIT-eligible ops `AwkDivJit` and `AwkModJit` handle this with a *guarded early-exit*. The codegen pops the divisor, emits `fcmp eq divisor, 0.0`, and branches: the trap block calls a `fusevm_jit_awk_div_trap(code)` libcall — code 1 for div, 2 for mod — into a thread-local channel and returns a sentinel, while the continuation block computes the `fdiv` or the `fmod` libcall. After the compiled block returns, the VM's block-dispatch path calls `take_awk_div_trap()` and, if a code was set, raises the same fatal error the interpreter raises — *before* writing slots back, so the partial native computation never becomes visible. Because the trap libcall is not a registered host-helper ID, these chunks skip on-disk persistence and never touch the shared cache schema, so a front end that emits only `Op::Div/Op::Mod` — zshrs, stryke — gets byte-identical native code with no awk machinery in it at all. The gawk bitwise builtins `AwkAnd`, `AwkOr`, `AwkXor` lower natively too, each operand saturating-cast to `i64` with `fcvt_to_sint_sat` (NaN to zero, infinities to the `i64` bounds) and folded with `band/bor/bxor` — pure integer arithmetic, no libcall, no host.

The slots pointer is the whole interface

The block JIT's runtime interface is startlingly thin, and the thinness is where its speed comes from. A block-compiled chunk is a native function whose entire contact with the VM is a single pointer — a `*mut i64` to the slots array — handed in at the call. All mutable state the loop touches flows through that pointer: the function loads a slot's bits from `slots[i]`, holds them in a machine register for as long as the analysis allows, and stores back through the pointer when it must. For a loop whose induction and accumulator are register-allocated with Cranelift block parameters — `AccumSumLoop` is the canonical case — the inner loop makes *no* memory accesses at all between entry and exit; the slots pointer is read once at the top and written once at the bottom, and the iterations in between are pure register arithmetic. This is why the block JIT posts its largest speedups on tight numeric loops: a million-iteration sum drops from thirty milliseconds to three hundred and fifteen microseconds not because each iteration got a little faster but because the iterations stopped touching memory and stopped dispatching entirely. The narrow interface — one pointer, no VM struct layout, no callbacks on the hot path — is also what makes the compiled function position-independent enough to cache to disk and reload, since it depends on nothing but its argument.

Bits in an integer lane

The representational decision that lets a single native function handle both integer and float slots is the bit-reinterpretation trick, and it is worth slowing down on because it recurs in the AOT native path's dual-variable scheme. Every slot becomes a Cranelift `i64 Variable`, and when the slot's kind is `Float`, that `i64` is not an integer at all — it is the *bit pattern* of an `f64`. A `GetSlot` on a float slot loads the `i64` bits and *bitcasts* them `I64 → F64` before doing float arithmetic; a `SetSlot` *bitcasts* `F64 → I64` before storing. The arithmetic in between is genuine `fadd/fmul`, with integer operands converted in with `fcvt_from_sint`; only the *storage* is reinterpreted bits. The correctness catch is that the same native code cannot serve a slot that is integer in one run and float in another, because the bitcasts would be wrong, so the *kind vector is folded into the cache key* — the in-memory key and the on-disk `*.blk.fjit` blob name both encode which slots are float — and float-specialized code is never reused for an integer slot. This is the mechanism, named `SlotKind::Float`, that lets awkrs block-compile its `f64` numeric chunks and persist them; and it is the same insight the AOT native path generalizes into two parallel `Variables` per stack position, one `i64` lane and one `f64` lane, with the kind saying which is live. The block JIT carries floats by reinterpreting an integer slot; the AOT native path carries them by keeping a second register lane; both descend from the same recognition that a slot's storage and a slot's type are separable.

What block-eligible excludes

`is_block_eligible` is the gate that decides whether a chunk can be block-compiled whole, and the shape of what it *excludes* is the clearest statement of the block JIT's domain. The tier handles arithmetic, comparisons, bitwise and logical ops, slots and globals, the fused superinstructions, and structured control flow — the same scalar-plus-control-flow core the AOT native path handles, which is no accident, since the two are siblings targeting the same kind of chunk. What it excludes is anything that escapes that model: extension ops (a front end's language-specific behavior), host builtins (the `CallBuiltin` dispatch), the shell and most AWK ops (which dispatch through hosts), and polymorphic types the tier cannot assume away. A chunk with any of these is not block-eligible, and `VM::run` routes it to the interpreter with the tracing recorder armed instead. This exclusion is what makes the block and tracing tiers partition the work cleanly: the block JIT takes the chunks it can compile *entirely*, and the tracing JIT takes the hot loops inside the chunks it cannot. The eligibility check is also cached on the VM, so the decision is a field read rather than a re-scan on every invocation — the README names the cached flag `block_eligible_cached` — because the eligibility of a given chunk does not change between runs, only whether it has crossed the warm-up threshold yet. A predicate that is computed once, cached, and partitions the runtime tiers is a small thing that does a lot of work: it is the line between “compile the whole thing” and “trace the hot loop,” drawn per chunk and remembered.

Siblings, not rivals: block JIT and AOT native

The block JIT and the AOT native path are best understood as siblings — they target the same kind of chunk and lower it the same way, differing only in *when* the compilation happens — and seeing the kinship clarifies both. Both take a whole chunk whose every opcode they can model, hold its locals in machine registers, compile its control flow to native control flow, and run it with no per-opcode dispatch. Both share the insight that the numeric, control-flow-heavy core of a script can leave the boxed world entirely. The difference is timing: the block JIT compiles at runtime, after a warm-up threshold proves the chunk worth it, keyed by op-hash so a chunk seen before is not recompiled; the AOT native path compiles before the program runs, as part of producing a standalone binary. The block JIT can therefore specialize on runtime information the AOT path lacks — it knows a slot's kind from the `SlotKind` vector the front end passes at call time — while the AOT path must prove everything statically from the chunk alone. But the lowering vocabulary is shared, which is why the two are described in such parallel terms throughout this book: the kind lattice, the register-resident slots, the libcalls, the boundary, the deopt all appear in both, because both are the same idea of “leave the boxed world for the scalar core” reached from two different moments. The day the block JIT learned to hold float slots in registers and the day the AOT native path learned its `Float` kind were the same conceptual advance arriving at two tiers, and a reader who understands one understands most of the other. They are not competing strategies but one strategy at two times — runtime for the block JIT, ahead-of-time for the AOT path — and the tracing JIT is the third member of the family, reaching the same register-resident native execution by recording a path rather than compiling a chunk.

Chapter 22 – The Tracing JIT

The tracing JIT is fusevm’s answer to code that did not exist until runtime and does not fit the block JIT’s model: an `eval`’d expression, a REPL line, a hot loop in a freshly loaded script that contains an op the block tier declines. It is also the only tier wired directly into `VM::run`’s dispatch loop, reached through `enable_tracing_jit()`. You cannot compile such code ahead of time because it was not there ahead of time, and you cannot compile the whole chunk because parts of it escape the model — so fusevm watches it run, and when a loop proves hot, it compiles the *path* through that loop.

The mechanism is a *recorder* armed at backward branches. A backward jump is, by definition, the bottom of a loop; when one is taken enough times to cross the trace threshold — fifty by default — the recorder begins capturing, on the next iteration through the header, the straight-line sequence of opcodes the program actually executes. That linear trace, with the branches that were taken folded into the recorded path, is far easier to compile than arbitrary control flow. The recorded trace is capped at 256 ops; beyond that the recorder gives up and lets the interpreter run the loop. A compiled trace runs the loop body in native code until the loop’s exit condition becomes false, then returns control to the interpreter at the right instruction.

Specialization, guards, and deopt

The trace compiler’s real work is *specialization*. The bytecode is dynamically typed — `Add` might add two integers or two floats — but a *specific* trace, observed at runtime, almost always sees specific types, and the compiler emits unboxed native arithmetic against the types it saw. Specialization is an assumption, and assumptions need checking, so the trace carries a **slot-type entry guard**: a cheap runtime check, at the top of the trace, that the slot types flowing in still match the ones it was compiled under. If a slot that always held an integer suddenly holds a float, the guard refuses the trace and the interpreter runs the iteration; after five such guard mismatches the trace is blacklisted and never retried. Internal branches inside the trace carry their own `brif` guards that the runtime direction must match the recorded direction, and a mismatch transfers to a per-branch side-exit that spills state and hands the un-recorded direction’s instruction pointer back to the interpreter.

Nine phases of getting harder

What makes the tracing JIT substantial is that “record a loop and compile it” is only the first of nine documented phases, each lifting a restriction the previous one could not. Phase 1 traces loop bodies with integer slots and no calls. Phase 2 adds **cross-call inlining**: `Op::Call` resolves to the callee’s bytecode IP at record time and the callee body inlines into the trace, each inlined frame getting its own slot-variable scope. Phase 3 adds **caller-frame branches** — loops with `if/else` bodies become traceable through the per-branch side-exit machinery. Phase 4 adds **callee-frame branches with frame materialization on deopt**: when a side-exit fires inside an inlined callee, the trace populates a `DeoptInfo` out-parameter that the VM uses to build synthetic `Frames` on `vm.frames`, each with its `return_ip` pointing back to the post-`Call` IP and its slots copied from the trace’s Cranelift `Variables`, so the interpreter resumes mid-callee with a correctly shaped call stack — bounded at four inlined frames and sixteen slots per frame.

Phase 5 adds **value-stack reconstruction on deopt**: a branch can fire while the trace’s abstract stack still holds intermediate values, which are written into `DeoptInfo.stack_buf` (capacity thirty-two) and pushed onto `vm.stack` so the interpreter resumes with the right stack state; phase 5b tags each entry’s kind so a float is bit-cast through `f64::from_bits` and materialized as `Value::Float`, which is what unlocks short-circuit `&&||`

patterns. Phase 6 adds a **side-exit counter and auto-blacklist**: a trace tracks `side_exit_count` distinct from its entry-guard `deopt_count`, and after `MAX_SIDE_EXITS` (fifty) misses it is blacklisted, defending against the pathological case where every iteration pays trace-plus-deopt-plus-interpret. Phase 7 adds **persistent trace metadata** — a serde-serializable `TraceMetadata` (chunk hash, anchor IP, op sequence, recorded IPs, slot-kind snapshot) that `trace_export` extracts and `trace_import` re-installs after verifying the chunk hash still matches, with the file format left deliberately caller-owned. Phase 8 relaxes the no-recursion rule to a depth cap, `MAX_INLINE_RECURSION` of four levels, so tail-recursive helpers trace. And phase 9 adds **side-trace stitching**: when a side-exit fires repeatedly at one IP, the recorder rearms there and records a *side trace* from the exit forward to the loop's backward branch, splitting its anchor into a record-anchor (the cache key) and a close-anchor (the loop header), with chains bounded by `MAX_TRACE_CHAIN` of four hops. The steady-state payoff, with traces primed: a hundred-thousand-iteration counter loop runs in twenty-seven microseconds against the interpreter's 2.35 milliseconds — eighty-seven times faster — and a loop with an internal branch a hundred and forty-eight times faster, both within one percent of the block JIT's floor, the remaining gap being only VM construction and slot copy-in overhead.

Why trace a path instead of compile a method

The tracing JIT answers a different question than the block JIT, and the difference is worth stating because it explains why fusevm has both. The block JIT is a *method-at-a-time* compiler: it takes a whole chunk whose every op it can lower and compiles all of it. That works when the chunk is uniformly eligible, but real hot loops are often embedded in code that is not — a numeric loop inside a function that also does string formatting and host calls, a tight inner loop in a chunk that elsewhere reads arrays of unknown type. A method-at-a-time compiler must reject the whole chunk if any part of it escapes the model, and then the hot loop never gets compiled. The tracing JIT escapes this by compiling not the method but the *path* — the actual straight-line sequence of ops a particular hot loop executes, recorded at runtime, with the branches that were taken folded into the recorded line and the ones that were not left as guards. A trace can thread through code the block JIT would reject, because it follows only what actually ran, and it specializes to the types it actually observed rather than the types the bytecode might in principle hold. The two tiers cover disjoint cases by construction: a block-eligible chunk short-circuits to the block JIT and never arms the recorder, and a chunk the block JIT declines is exactly where the recorder, armed at the loop's backward branch, earns its keep.

Recording is the hard part, compiling is the easy part

It is a counterintuitive fact about tracing JITs, and fusevm's nine phases make it vivid, that the difficulty is overwhelmingly in *recording and resuming* rather than in code generation. Compiling a recorded straight-line trace to native code is nearly as simple as compiling a linear chunk — it is a line of typed operations with guards at the branch points. What is hard is everything around that line. Recording must follow `Op::Call` into a callee and inline its body (phase 2), must capture which direction each internal branch took so it can guard them (phase 3), and must do this across inlined frames and bounded recursion (phases 4 and 8). And *resuming* — the deopt path when a guard fails — must reconstruct a VM state the interpreter can take over: synthetic `Frames` with correct return IPs and copied slots when the exit is inside an inlined callee (phase 4), the operand-stack values that were mid-computation when the branch fired, tagged by kind so a float is rebuilt as `Value::Float` and not `Value::Int` (phase 5 and 5b). The phases that look like the “real” compiler work — emitting the native loop — are the early, easy ones; the later phases are all about the seams where a trace meets reality and has to hand control back without losing a value or corrupting a frame. The blacklist counters (phase 6), the metadata export/import (phase 7), and the side-trace stitching (phase 9) are all defenses and extensions around that same seam. This is why a tracing JIT is a large, careful piece of engineering even when its code generator is modest: the value is in recording the right path and resuming safely when the path is wrong, and fusevm builds that up one restriction-lifting phase at a time, each guarded by its own constants — the fifty-miss blacklist, the four-deep recursion cap, the four-hop chain bound, the two-hundred-fifty-six-op trace limit.

The Cell lattice, where specialization lives

The tracing JIT's type specialization runs on an abstract-value lattice the source calls `Cell`, and naming its four variants makes the whole specialization concrete. A `Cell` is `Const(i64)` — a statically known integer; `ConstF(f64)` — a statically known float; `Dyn` — an integer of unknown value; or `DynF` — a float of unknown value. As the recorder's trace is compiled, the compiler runs an abstract interpretation over the operand stack in these terms, and the lattice does two jobs at once. First, it tracks *type*: a small helper `either_float` decides when a binary op promotes, and a `JitTy` projection maps each `Cell` to `Int` or `Float` so codegen knows which machine type to emit — `Const` and `Dyn` are `Int`, `ConstF` and `DynF` are `Float`. Second, it tracks *value* when it can: an op over two `Consts` folds to a `Const` of the result (`int_op` applied to the two known integers), two `ConstFs` fold to a `ConstF`, and a comparison over known operands folds to a known `Const` boolean. When either operand is dynamic the result degrades to `Dyn` or `DynF` and the compiler emits a real instruction; when both are constant it folds and emits nothing. So the same lattice that tells the compiler whether a stack position is integer or float also constant-folds the parts of the trace whose values were fixed when it was recorded — two analyses for the price of one abstract interpretation.

Specialization is an assumption the guards defend

The deep reason the `Cell` lattice needs guards is that it specializes on what the trace *observed*, not on what the bytecode *guarantees*, and the gap between the two is exactly what the guards close. The bytecode for `x + y` is polymorphic — it adds integers or floats depending on what `x` and `y` hold at runtime — but a recorded trace saw specific types on the iteration it captured, and the compiler emits unboxed `iadd` or `fadd` against those types, projecting each operand's `Cell` to a `JitTy`. That is a bet: the bet that the next iteration's `x` and `y` have the same types as the recorded one's. The slot-type entry guard at the top of the trace is the bet's hedge — a cheap check that the incoming slot types still match the `slot_kinds_at_anchor` snapshot the trace was specialized under — and the per-branch `brif` guards inside the trace hedge the bet that control flow follows the recorded direction. When a guard fails, the trace deopts: it abandons the specialized native code and hands control to the interpreter, which is correct for *any* types because it never specialized. After enough guard failures — five entry-guard mismatches, or fifty side-exits — the trace is blacklisted, on the theory that a trace whose assumptions keep failing is a trace whose recorded path does not match reality, and continuing to run it would pay trace-plus-deopt-plus-interpret on every iteration. The `Cell` lattice is the optimism; the guards and the blacklist are the safety; and together they let the tracing JIT run native code on the assumption that runtime types are stable while never being wrong when they are not. This is the same optimistic-native-with-safe-fallback pattern the AOT deopt reuses — the tracing JIT is simply where fusevm first learned it.

The phases as a capability ladder

The nine phases are not nine unrelated features; they are a single capability climbing one restriction at a time, and reading them as a ladder clarifies why each is the natural next step from the last. Phase 1 traces the simplest possible loop — integer slots, no calls, no internal branches — which is enough to prove the recording-and-compiling machinery works but not enough to trace real code. Real code calls functions, so phase 2 inlines calls. Real loops branch, so phase 3 adds caller-frame branches with side-exits, and phase 4 extends branches into the inlined callees phase 2 introduced — which forces phase 4 to solve frame materialization, because a side-exit inside a callee must hand the interpreter a correctly shaped call stack. Once branches can fire anywhere, they can fire while the abstract stack holds intermediate values, so phase 5 reconstructs the value stack on deopt, and phase 5b tags those values by kind so floats rebuild correctly. Phases 6 through 9 are then about making the whole thing robust under repetition: phase 6 blacklists traces whose guards keep failing, phase 7 persists trace metadata across processes, phase 8 lifts the no-recursion rule to a depth cap, and phase 9 stitches side traces onto hot deopt sites so a repeatedly-failing branch gets its own compiled path rather than falling back forever. Each phase removes the restriction that the previous phase's success made visible: you cannot want frame materialization until you allow callee branches, and you cannot allow

callee branches until you inline calls. The ladder is the order in which the restrictions *had* to fall, and it is why a tracing JIT is built incrementally rather than all at once.

Where block and trace divide the work

It bears repeating, because it is the architectural keystone of the runtime tiers, that the block JIT and the tracing JIT never compete for the same chunk — they partition the work by a clean rule, and the rule is checked before either runs. When `enable_tracing_jit()` is set, `VM::run` first asks whether the chunk is fully block-eligible; if it is, the block JIT compiles and runs it with zero VM-side overhead through the slots pointer, and the trace recorder is never even armed. Only if the chunk is *not* block-eligible — because it contains an extension op, a host builtin, a polymorphic type the block tier cannot assume away — does control reach the interpreter with the recorder armed at backward branches, and there the tracing JIT earns its keep by accelerating the hot loop inside code the block tier could not take whole. So the two tiers cover disjoint cases: the block JIT owns the chunks it can compile entirely, the tracing JIT owns the hot loops in the chunks it cannot. This is why there is no question of which tier “wins” a given chunk — the eligibility check decides, once, before any compilation, and the loser never starts. The partition is what lets the system run all three Cranelift tiers without them interfering: linear for straight-line expressions a front end hands over, block for whole eligible chunks, tracing for hot loops in everything else, each with a domain the others do not enter.

Persisting a trace across processes

Phase 7’s `TraceMetadata` is the tracing JIT’s answer to a problem the disk cache solves differently for the other tiers, and contrasting the two is instructive. The disk cache persists compiled *native code*; `TraceMetadata` persists the *recipe* for a trace — not the machine code, but the recorded op sequence, the anchor IP, the fall-through IP, the recorded branch directions, and the slot-kind snapshot the trace was specialized under. It is a `serde`-serializable struct, and `JitCompiler::trace_export` extracts it from a compiled-trace cache entry while `trace_import` re-installs it on a fresh `JitCompiler` — but only after verifying that the chunk’s op-hash still matches, so a trace recorded against one version of a chunk is never replayed against a different one. The deliberate choice here is that fusevm ships *no file format* for this: persistence is caller-owned, so a front end can serialize `TraceMetadata` as JSON, bincode, into a SQLite row, or anywhere else `serde` reaches. Why persist the recipe rather than the code? Because a recorded trace is a high-level artifact that can be re-compiled cheaply once its recipe is known, and the recipe is small and portable in a way native code is not — it carries no relocations, no architecture dependence, no W^X concerns. The disk cache and `TraceMetadata` are two persistence strategies for two different things: the cache remembers the expensive codegen output for all three tiers, while `TraceMetadata` remembers the even-more-expensive *recording* — the runtime observation of which path was hot and what types it saw — so that a fresh process need not re-observe the program to re-discover its hot traces. One remembers the compilation; the other remembers the profiling.

Chapter 23 — The Native-Code Disk Cache

The three JIT tiers all compile native code at runtime, and compilation, however fast Cranelift is, is not free. For a workload that re-launches the VM repeatedly — a shell re-running its rc scripts, a test suite run over and over, a batch job invoked a thousand times — the same chunks get recompiled on every launch. The `jit-disk-cache` feature eliminates that by persisting the compiled *native code* to disk, so a chunk compiled once is reloaded rather than recompiled across process restarts. It covers all three tiers, is on by default once the feature is enabled, and writes to `~/.cache/fusevm-jit` — overridable with the `FUSEVM_JIT_CACHE_DIR` environment variable or `JitCompiler::set_jit_cache_dir`, and disableable at runtime with `FUSEVM_JIT_CACHE_DIR=off`.

The cache's correctness rests on careful keying. Files are tier-tagged — `.lin.`, `.blk.`, `.trc.` — and keyed by the chunk's op-hash, with the tracing tier additionally keying on the record-anchor IP and verifying a content hash over the recorded ops, IPs, slot types, and constants, so two divergent recorded paths through the same chunk can never collide. Each blob stores the native code plus a small relocation table that is re-patched on load, and loading mmaps the code with the platform's W^X handling — `pthread_jit_write_protect_np` plus instruction-cache invalidation on Apple Silicon, `mprotect` elsewhere. Writes publish through a unique temporary file and an atomic rename, so the cache is safe under many concurrent processes racing to populate it.

Conservative, transparent, and bounded

Three properties make the cache safe to leave on by default. It is **conservative**: the loader refuses any chunk whose code carries a relocation other than a known host-helper call, falling back to the in-memory JIT rather than risk mis-patching an untested target — an untested case degrades to “no caching,” never to a miscompile. This is why the `AwkDivJit/AwkModJit` chunks, whose trap libcall is not a registered host-helper ID, simply skip persistence. It is **behavior-transparent**: the cache eliminates only Cranelift codegen time, leaving tier selection, warm-up thresholds, and results identical to an uncached run, so a cached and an uncached process compute the same answers by the same path. And it is **bounded**: blobs are small — roughly a hundred bytes for a linear chunk, a few kilobytes for a block or trace — but an edited script produces new op-hashes and the old blobs linger, so the cache has a total-size cap (default 256 MiB) enforced by oldest-first eviction down to eighty percent of the cap, applied opportunistically as new blobs are written so most writes pay no scan cost. The cap, a forced prune, the current size, and a full clear are all exposed through `JitCompiler` methods and the `FUSEVM_JIT_CACHE_MAX_BYTES` environment variable. The measured payoff is direct: a cached block load is about thirty-five microseconds against a hundred and fifty-two for cold codegen, the codegen cost paid once and then amortized to a file read.

W^X, and the cost of executable memory

Loading native code from disk is not as simple as reading bytes, because modern operating systems enforce *write-xor-execute*: a page of memory may be writable or executable but not both at once, a defense against code-injection attacks. The disk cache stores a blob of machine code plus a small relocation table, and to run it the loader must map the code into memory, patch its relocations (which requires writing), and then make it executable (which requires the page no longer be writable). On most platforms this is an `mprotect`

flip from writable to executable after patching. On Apple Silicon, where the JIT-hardening is stricter, it is the `pthread_jit_write_protect_np` toggle paired with an explicit instruction-cache invalidation, because the CPU may have cached the old (un-patched) bytes and must be told to refetch. The `libc` dependency that the `jit-disk-cache` feature pulls in exists precisely for these low-level mapping and protection calls. The relocations themselves are what make a cached blob portable across process restarts: a libcall to `fusevm_jit_sin_f64` cannot store an absolute address, because the helper lands at a different address in each process, so the blob records “call helper id `H_SIN_F64` here” and the loader re-resolves the id to the current process’s helper address as it patches. This is also the seam where the loader’s conservatism lives — a relocation it does not recognize as a known host-helper id means an untested target, and the loader declines the whole blob rather than risk patching it wrong.

Eviction without a scan

The cache’s size management is a small study in keeping a maintenance task off the hot path. The problem is real: blobs are keyed by op-hash, an edited script produces new hashes, and the old blobs are never overwritten — they simply linger, so an actively developed codebase would grow the cache without bound. The naive fix, a periodic sweep that stats every file and deletes the oldest, would impose a scan cost on someone, and the someone would be whichever process happened to trigger it. `fusevm` instead caps the total size (default 256 MiB) and enforces the cap *opportunistically*: as a new blob is written, if the cache has grown past the cap, an oldest-first eviction runs down to eighty percent of the cap, and otherwise nothing happens. Most writes, with the cache comfortably under the cap, pay no scan cost at all; only the writes that push it over the line pay for a prune, and the prune frees enough headroom that the next many writes are free again. The mtime-based oldest-first policy approximates least-recently-used cheaply, and the eighty-percent low-water mark means a steady-state workload settles into occasional small prunes rather than thrashing at the cap. The cap, a current-size query, a forced prune, and a full clear are all exposed programmatically and through `FUSEVM_JIT_CACHE_MAX_BYTES`, so a deployment that knows its working set can size the cache to never evict, and a developer who wants a clean slate can nuke it with `rm -rf ~/.cache/fusevm-jit`. The whole design is shaped by the same instinct as the rest of the machine: put the cost where it belongs, and keep it off the path that runs most.

Transparent means you can ignore it

The most important property of the disk cache, for a user, is the one easiest to take for granted: it is *behavior-transparent*, meaning a process with the cache enabled and a process without it compute the same answers by the same tier selection, differing only in how long the first run takes. This is a strong guarantee, and it is what lets the cache be on by default without anyone having to reason about it. The cache eliminates Cranelift codegen time and nothing else: it does not change which tier compiles a chunk, does not change the warm-up thresholds, does not change any result. A chunk that the block JIT would compile on its first invocation still does so; the cache merely means that on the *second process* the native code is loaded from disk instead of regenerated. Because the cache stores the native code that codegen would have produced — keyed by the op-hash that uniquely identifies the ops and constants, plus the slot kinds and, for traces, the recorded path — loading from it is indistinguishable from compiling, except faster. The conservative loader is what preserves this: anything it is not certain it can load correctly, it declines, falling back to in-memory compilation, so the worst case is “no speedup,” never “wrong code.” A cache you can leave on without thinking about it is a cache that has earned transparency by refusing to ever change behavior, and `fusevm`’s earns it by storing exactly the codegen output, keying it exactly, and declining anything it cannot load with certainty. The user gets faster restarts and an identical program; the only thing the cache spends is disk, bounded by the cap.

All three tiers, one cache

A subtlety easy to miss is that the disk cache covers *all three* JIT tiers — linear, block, and tracing — not just one, and the tier-tagging in the filenames (`.lin.`, `.blk.`, `.trc.`) is what makes this safe. Each tier produces a different kind of native code with different keying needs: the linear tier's code is a small expression evaluator keyed by op-hash alone; the block tier's is a whole-CFG function keyed by op-hash plus the slot-kind vector; the tracing tier's is a specialized loop body keyed by op-hash plus the record-anchor IP plus a content hash over the recorded path. By tagging each blob with its tier, the cache keeps the three keyspaces disjoint, so a linear blob is never mistaken for a block blob for the same chunk, and a chunk that is linear-eligible and also appears as a trace anchor gets two separate cached blobs that never collide. This is why the cache can be described as behavior-transparent across the whole tier ladder rather than for one tier: every tier's codegen output is persistable and re-loadable under its own tag and key, so the entire JIT-warm-up cost — not just the block tier's — is eliminated across process restarts. The tracing tier's extra content-hash verification is the strictest, because a recorded path can diverge between runs in a way an op-hash alone would not catch, so the trace blob additionally verifies that the recorded ops, IPs, slot types, and constants match before it is trusted. Three tiers, three keying schemes, one tagged cache: the persistence is as complete as the conservatism allows, covering every tier that compiles native code while never serving one tier's blob for another's request.

Chapter 24 – The AOT Compiler: The Threaded Path

The ahead-of-time compiler — `aot.rs`, behind the `aot` feature — takes a whole chunk and lowers it to native code *before* the program runs, producing a relocatable object file and, ultimately, a standalone binary. It does this through one dispatcher, `build_entry<M: Module>(module, chunk)`, which makes a single decision: it runs `analyze_native(chunk)`, and if the analysis returns a plan, it takes the **native** path (`build_entry_native`); if the analysis returns `None`, it takes the **threaded** path (`build_entry_threaded`). The threaded path is the foundation — the strategy that always works — so it comes first.

The idea of the threaded path is to replace the interpreter's dispatch *loop* with native control flow, while keeping the opcode *semantics* in the interpreter. The compiler emits one Cranelift block per opcode, plus an entry block, a dispatch block, and a return block. The entry block stashes the `*mut VM` pointer into a frontend `Variable` and jumps to the dispatch block with the instruction pointer zero. The dispatch block takes the current IP as a block parameter and uses a Cranelift `Switch` — a jump table, a `br_table` in machine terms — to branch to the op block for that IP, with the return block as the default. Each op block does exactly one thing: it calls the shim `fusevm_aot_exec_op(vm, i)`, which runs that opcode's semantics through the very same `exec_op` the interpreter uses and returns the next instruction index; the block then tests whether that index is negative (the terminate sentinel) and either branches to the return block or back to the dispatch block with the new IP. In pseudocode, the shape the module header itself draws:

```
entry:      jump dispatch(0)
dispatch(ip): switch ip -> [block_0, ..., block_{n-1}] (default -> ret)
block_i:    next = fusevm_aot_exec_op(vm, i); if next < 0 -> ret else dispatch(next)
ret:       fusevm_aot_finish(vm); return 0
```

What this buys, and what it doesn't

The threaded path's virtue is that it is *total*: because every opcode's meaning is still `exec_op`, it can compile *any* chunk, including ones full of strings, arrays, pipelines, and calls — the whole opcode set, with no exceptions. Routing every transition through the dispatch block, rather than letting blocks fall through to each other, is what keeps data-dependent targets correct without the native code ever having to know the VM struct's layout: `Op::Jump`, the `JumpIf*` family, and intra-chunk `Op::Call/Op::Return` all simply return a next index that the dispatch switch resolves. Its limitation is equally clear: the work each opcode does is unchanged, run through a function call into the interpreter, so the speedup over interpreting is modest — you have removed the decode-and-dispatch overhead of the loop, not the cost of the operations. It is the AOT compiler's safety net: the strategy that always works, against which the fast strategy can fall back. And because `build_entry` is generic over the Cranelift `Module`, the very same codegen serves both the in-memory JIT module — used by `run_chunk_native` to validate the compiler by compiling and running a chunk in process — and the on-disk object module that ships a binary. One code path, two backends.

The shims, and the FFI boundary they cross

The threaded path leans on a roster of `#[no_mangle] extern "C"` shim functions, and the roster is worth seeing as a whole because it maps the entire boundary between native code and the boxed VM. The

three structural shims are `fusevm_aot_exec_op(vm, ip)`, which runs one opcode's semantics through the interpreter and returns the next index; `fusevm_aot_finish(vm)`, which computes the tail result when no op stored one; and `fusevm_aot_resume(vm, ip)`, the deopt entry that hands the rest of the run back to `VM::run`. Around them sit a couple dozen more — `set_int_result`, `set_float_result`, `box`, `unbox`, `store_slot_int`, `store_global_float`, `pop_float`, `push_status`, `powf`, `fmod`, `unary_math`, `atan2`, and the rest — each a narrow C-ABI function the generated code can call without knowing the VM's Rust struct layout. The `*mut VM` that every shim takes is an opaque handle; the native code never dereferences it, only passes it along, which is why the module can carry an `#![allow(improper_ctypes)]` and treat the VM as a black box across the FFI line. This is the discipline that lets one generic `build_entry` serve both the in-process JIT and the shipped object: the native code's only knowledge of the VM is the set of `extern "C"` functions it may call, and those functions are linked in identically whether the code runs in this process or in a separately-linked binary.

Total, by construction

It is worth dwelling on *why* the threaded path can compile literally any chunk, because the totality is what makes it a safe fallback rather than a second source of bugs. The native path's correctness is hard-won — it depends on type analysis, definite assignment, and a kind lattice, any of which could in principle be wrong for some chunk. The threaded path has none of that risk, because it does not reinterpret any opcode's meaning: every op still runs through `exec_op`, the exact function the interpreter uses, so a threaded-compiled chunk computes precisely what an interpreted one does, op for op. All the threaded path changes is *how control flows between ops* — a native `Switch` dispatch instead of a `match` in a loop — and that change is purely mechanical and uniform across every opcode. There is no opcode the threaded path handles specially, no analysis it can get wrong, no type it can mispredict. This is why `build_entry` can fall back to it with complete confidence: when `analyze_native` returns `None` because a chunk has a string op, a host call, or a type it cannot pin down, routing that chunk to the threaded path is not a degradation in correctness, only in speed. The native path is the optimization; the threaded path is the guarantee; and a compiler that always has a total, provably-correct fallback can afford to make its optimization as aggressive as it likes, because the cost of the optimization declining is merely the cost of being a little slower, never the cost of being wrong.

The blob is a serialized chunk

The form the chunk takes inside an AOT object is a `bincode` serialization, and the choice of `bincode` over the JSON that `serde` could equally produce is a small but telling one. `bincode` is a compact binary `serde` format: a `Chunk` serialized with it is a tight byte sequence — the op vector, the constant pool, the name table — with no field names, no whitespace, no textual representation of numbers, just the bytes. It is exported as the data symbol `fusevm_aot_chunk_blob`, with its length as a little-endian `u64` in `fusevm_aot_chunk_len`, and at startup `fusevm_aot_run_embedded` deserializes it back into a live `Chunk` with `bincode::deserialize`. The one field that does not survive is `op_hash`, which carries `#[serde(skip)]` and is recomputed if needed — harmless, because the embedded binary is not consulting the JIT cache, it is running the embedded chunk directly. Using `bincode` keeps the embedded blob small (binary, not text) and the round-trip fast (no parsing, just a byte-for-byte reconstruction of the `Vecs`), which matters because every launch of the AOT binary pays the deserialization cost once at startup. That the same `serde` derives serve both the human-readable JSON path (used in tests to round-trip chunks and values) and the compact `bincode` path (used to embed a chunk in a binary) is one more instance of the single-definition discipline: one set of derive macros, two wire formats, chosen per use.

run_chunk_native, the compiler's own test

There is a function in the AOT module whose entire purpose is to keep the compiler honest, and it is worth naming because it embodies the differential discipline at the compiler's own level. `run_chunk_native`

compiles a chunk *in process* through the in-memory JIT module, finalizes it to executable memory, and runs it on a fresh VM — and the test harness then compares its result against the interpreter's. This is how the compiler validates itself end to end without ever writing an object file: the same `build_entry` codegen that would be written into a shipped `.o` is instead JIT-compiled and run immediately, so a codegen bug surfaces in a unit test rather than in a linked binary on a developer's machine. Because `build_entry` is generic over the Cranelift `Module`, the function it compiles for `run_chunk_native` is byte-for-byte the function it would write into an object — the only difference is the target ISA's position-independence flag — so validating the in-process path validates the shipped path. Every differential fuzzer in the AOT module routes through `run_chunk_native`: it generates a random chunk, runs it interpreted and runs it through `run_chunk_native`, and asserts the two agree. The compiler tests itself by being its own JIT, and the JIT-validates-the-AOT-codegen arrangement is why the AOT path can be trusted without a separate “did the linked binary produce the right answer” integration test for every case — though one such test does exist, shelling out to `nm` to confirm the exported symbols, to keep the object-emission path honest too.

The Switch is the threaded path's only cleverness

The threaded path is mostly mechanical, but its one genuinely clever piece is the dispatch `Switch`, and understanding it explains why the threaded path can compile any chunk including ones with data-dependent jumps. A naive translation of the dispatch loop would emit native code that falls through from each opcode's block to the next, but that breaks the moment an opcode is a jump: `Op::Jump(t)` must send control to block `t`, and `Op::Call` and `Op::Return` send control to data-dependent targets that are not known until runtime. The threaded path solves this by routing every transition through a central dispatch block that takes the next instruction pointer as a parameter and uses a Cranelift `Switch` — a jump table, compiled to a `br_table` — to branch to the right opcode block. Each opcode block, after running its `exec_op` shim, does not fall through; it returns the next index to the dispatch block, which switches on it. So a `Jump` is just an `exec_op` that returns the jump target, a `Call` an `exec_op` that returns the callee's entry, a `Return` an `exec_op` that returns the caller's resume point — all handled by the same switch, because the shim computes the target and the native code only branches on it. This is why the threaded path needs no special case per control-flow opcode and never reads the VM struct's layout: the *semantics* of where to go next live in `exec_op`, and the native code's only job is to switch on the index `exec_op` returns. The `Switch` is the price of generality — every transition pays a jump-table lookup rather than a direct branch — and it is exactly that price that buys totality, the ability to compile any chunk including the ones whose control flow is data-dependent.

Chapter 25 — Native Lowering: Scalars in Registers

The fast AOT strategy — `build_entry_native`, chosen when `analyze_native` returns a plan — does what the threaded path does not: it lowers opcodes to *real native instructions*, with values held in machine registers, no per-opcode call into the interpreter at all. The reason this is hard, and the reason it is a separate strategy that only some chunks qualify for, comes straight back to Chapter 3: a `Value` is an eleven-variant tagged union far too big for a register. To emit a native `iadd`, the compiler must first know that the two operands are integers and get their raw `i64`s into registers — which means it must know their *types*, statically, at compile time.

So the native path begins with the analysis, `analyze_native`, which performs an abstract interpretation over the operand stack and decides whether — and how — the chunk can be lowered. For the simplest case, a straight-line integer computation, it is almost trivial: simulate the operand stack at compile time as a stack of typed SSA values, emit `iconst` for `LoadInt`, `iadd/isub/imul` for the arithmetic, and at the end box the single result back into the VM through a small shim. The arithmetic is wrapping, matching `arith_int_fast's i64::wrapping_add`, so the native result equals the interpreter's bit-for-bit. The example program that ships with the crate is exactly this shape — `LoadInt(6); LoadInt(7); Mul`, leaving forty-two — and the AOT pipeline compiles it to an object whose linked binary exits with code forty-two.

Lower what you can prove; fall back on the rest

The governing principle of the native path is stated once and obeyed everywhere: *lower what you can prove, fall back on the rest*. `analyze_native` is conservative to a fault, and it returns `None` — routing the chunk to the threaded path — on any of a long list of conditions: a slot or global index at or beyond the `NATIVE_SLOT_LIMIT` of 4096; a stack underflow; an inconsistent abstract-stack kind at a control-flow join; a slot used with two different kinds; a final result that is not numeric or a heap-object handle; a `GetSlot` or `GetVar` that could read a slot not provably assigned on every path; a named array or hash whose name overlaps a register-cached global; or — a nice touch — *every* reachable op being a deopt point, in which case the threaded path is strictly better and the native path bows out. A chunk is never lowered incorrectly; it is either lowered correctly or not lowered at all. That conservatism is what makes the aggressive parts safe to add, one opcode family at a time, and it is why the native path could grow from “straight-line integer arithmetic” to “nearly the entire scalar instruction set” without ever risking a miscompile — each addition either provably matched the interpreter or fell back. The analysis even returns a trivially-empty plan for an empty chunk, so the degenerate case is handled before the worklist starts.

The whole AOT pipeline, in one example

The crate ships a deliberately tiny end-to-end demonstration, and walking it makes the abstract “compile a chunk to a binary” concrete. The example program builds the chunk for `6 * 7`, emits an object, and the linked binary exits with code forty-two:

```

use fusevm::aot;
use fusevm::chunk::ChunkBuilder;
use fusevm::op::Op;

let mut b = ChunkBuilder::new();
b.emit(Op::LoadInt(6), 1);
b.emit(Op::LoadInt(7), 1);
b.emit(Op::Mul, 1);
let chunk = b.build();
aot::compile_object(&chunk, std::path::Path::new("/tmp/x.o")).expect("emit object");

```

Three opcodes go in; a relocatable object comes out. `analyze_native` looks at `LoadInt`, `LoadInt`, `Mul` and sees a straight-line integer computation with no branches, no slots, no heap, ending in a numeric result — so it returns a plan, and the native path lowers the chunk to a function that `iconsts` six, `iconsts` seven, `imuls` them, and boxes forty-two back into the VM. The object exports the entry function, the serialized chunk, and its length; a front-end runtime links against it, and the binary's exit code is the result. This is the entire AOT promise in its smallest form: a chunk of bytecode becomes a native object becomes a standalone executable, with no interpreter dispatch loop at run time. Everything in the rest of this movement is about widening the class of chunks for which `analyze_native` returns a plan rather than `None` — from this trivial multiply up through loops, float accumulators, status arithmetic, and the boundary to the boxed world.

Conservatism as a feature, not a limitation

It is tempting to read the long list of conditions under which `analyze_native` returns `None` as a list of limitations, but the better reading is that the list is the compiler's safety mechanism, and a generous list of “decline” conditions is what lets the “lower” conditions be aggressive. Consider the alternative: a compiler that tried to lower everything would have to be correct about everything, and the first chunk whose types it mispredicted would silently compute a wrong answer — the worst failure a compiler can have, because it produces no crash and no error, just a wrong number. By returning `None` the instant it cannot *prove* a chunk safe to lower — a slot used with two kinds, a read that might hit an unassigned slot, a result that is not numeric, a global aliased by a named array — the analysis converts every uncertainty into a fallback to the always-correct threaded path. The conservatism is asymmetric on purpose: a false “decline” costs only speed, while a false “lower” would cost correctness, so the analysis is tuned to decline whenever it is unsure. This is what made it possible to grow the native path one opcode family at a time, from integer arithmetic to floats to status to the heap-op boundary, without ever introducing a miscompile — each new family either provably matched the interpreter and was admitted, or did not and was declined, and the differential fuzzers verified both the admissions and the declines. A compiler with a trustworthy fallback can be brave; a compiler without one must be timid; fusevm's native path is brave because the threaded path has its back.

Boxing the result back

A native function runs in registers, but it must hand its result back to the VM as a boxed `Value`, and the small set of result-boxing shims is where the register world rejoins the boxed one. When a native chunk finishes with an integer on top of its abstract stack, it calls `fusevm_aot_set_int_result(vm, n)`, which wraps the raw `i64` in a `Value::Int` and installs it as the VM's result; a float result calls `set_float_result`, a status result `set_status_result`, an `Obj` handle `set_obj_result`. These are the mirror image of the reload shims at the boundary: where `pop_float` pulls a boxed value into a register, `set_float_result` pushes a register value back into the boxed world. The analysis enforces that a native chunk's final result is numeric or an `Obj` handle precisely so that one of these shims always applies — a chunk whose top-of-stack at the end is something else (a bare boolean, say) is declined, because there is no clean way to box it as the program's result. This is why the analysis checks the *end* state's kind as carefully as it checks the joins: the result-boxing shim that runs at the end has to know which kind it is boxing, and the analysis guarantees it by refusing any chunk whose result kind it cannot pin down. The register world is entered when the function is called and left

when it boxes its result, and both transitions go through a tiny, kind-specific shim that is the only place the two worlds touch on the success path.

Why one block per leader

The decision to emit exactly one Cranelift block per basic-block leader, rather than one per opcode or one per chunk, is the structural choice that makes the native path a real compiler, and it is worth understanding why that granularity is right. One block per chunk would mean no control flow — fine for a straight line, useless for a loop. One block per opcode, the threaded path's choice, means every opcode is a branch target, which is correct but pessimal: it forces the operand stack through memory or block arguments at every step, because Cranelift cannot keep a value in a register across a block boundary without a `Variable` or a block parameter, and one-block-per-op has a boundary at every op. One block per *leader* — per genuine branch target and fall-through-after-branch — is the sweet spot: within a basic block, which is a straight run of opcodes with no branches in or out except at its ends, the compiler can keep the whole operand stack in SSA values that flow directly from one instruction to the next, no memory traffic, no block arguments; only at the block *boundaries*, the joins, does it need the `Variable` machinery, and there Cranelift's sealing builds the ϕ -nodes. So the abstract interpretation's first job — discovering the leaders — is what determines how coarse the native code's blocks are, and the coarseness is exactly right: as few blocks as control flow allows, so as much arithmetic as possible flows through registers within a block, with the `Variable` joins paid only where the program genuinely branches.

Chapter 26 — The Basic-Block Compiler

Straight-line arithmetic is a warm-up. Real programs branch and loop, and the native path earns its keep by compiling control flow to native control flow — a genuine basic-block compiler, not a dispatch loop.

`analyze_native`'s abstract interpretation is a worklist over the chunk. Its state, at each instruction pointer it has reached, is a pair: the vector of abstract *kinds* on the operand stack, and the set of slots and globals definitely assigned on every path to that point. It discovers the basic-block leaders — every jump target and every fall-through after a branch — by inserting both the target and the following instruction into a `leaders` set whenever it sees a branch op. It propagates the abstract state forward, and at every edge into a join it merges: operand-stack kinds must *match exactly* (well-formed structured bytecode always satisfies this, and a chunk that does not is simply declined), while the definite-assignment sets are *intersected*, with re-convergence so that a set which shrinks at a join re-queues its successors. The output is a `NativePlan`: where the leaders are, the entry kinds for each, the maximum operand-stack depth, the per-slot kinds, the deopt points, and the definitely-assigned set at each of them.

The Variable trick, again

Then `build_entry_native` emits one Cranelift block per leader and — here is where Chapter 19's `Variables` pay off — threads the operand stack through frontend `Variables` rather than block arguments. The compiler never constructs a single ϕ -node by hand. A `Jump` is a Cranelift `jump`, a `JumpIfFalse` is a `brif`, and when the compiler seals the blocks at the end, Cranelift builds all the joins — including, crucially, the ϕ -nodes at a loop's back-edge — from the def/use record. A counted loop with a back-edge compiles to a native loop with the loop variables in registers, and the compiler's code for it reads as straight as the bytecode did. Comparisons become `icmp` or `fcmp`; conditional jumps become `brif`. The result is a function that runs the loop entirely in registers, and a differential fuzz test — four hundred random arithmetic-and-branch programs, the native result checked against the interpreter — keeps it honest. That fuzzer, like all of them, seeds a `SplitMix64` generator with a fixed constant so a failure reproduces as a single seed rather than a heisenbug.

The abstract stack, position by position

The heart of `analyze_native` is an abstract interpretation that simulates the operand stack at compile time, and it is worth tracing what “abstract” means here, because the precision of this simulation is exactly the compiler's knowledge. The concrete operand stack, at runtime, holds `Values`; the abstract stack, at compile time, holds `Kinds` — `Int`, `Bool`, `Float`, `Status`, or `Obj` — one per position. When the analysis sees `Add`, it pops two kinds, checks both are numeric (declining the chunk if not), and pushes `Float` if either promotes or `Int` if both are integer — mirroring exactly what the interpreter's `arith_int_fast` would decide at runtime, but decided once, statically. When it sees a comparison it pushes `Bool`; when it sees `Dup2` or `Swap` or `Rot` it permutes the top kinds the way the runtime op would permute the top values. The state the analysis carries at each instruction pointer is this kind-vector together with the set of slots and globals definitely assigned, and the whole worklist is a fixpoint over those two pieces of state plus the per-slot kind inference. The reason this works — the reason a compile-time simulation can know the runtime types — is that well-structured bytecode has *consistent* types at every join: the two arms of an `if` leave the stack the same shape, a loop's back-edge restores the shape its header expects. The analysis checks that consistency (kinds must match exactly at

every join, or the chunk is declined) and, having checked it, can trust that the kind it computed for a position is the kind that will actually be there at runtime.

Leaders, joins, and the shape of a plan

The output of the analysis is a `NativePlan`, and its fields are a precise inventory of what `build_entry_native` needs to emit native control flow. The `leaders` set marks where basic blocks begin — every jump target and every instruction after a branch — discovered by inserting both the target and the fall-through whenever the worklist meets a branch op. The `entry_kinds` map records the abstract stack shape at each leader, so codegen knows how many operand-stack `Variables` are live entering each block. `max_depth` bounds how many stack `Variables` to allocate in total. `var_kinds` records the inferred kind of each slot and global, so codegen can type each one's register `i64` or `f64`. `deopt_points` and `inits_at` record where deopts may fire and which slots are definitely assigned there, so the deopt code knows what to write back. The merge rules that build this plan are the two from the previous chapter applied at every join — kinds intersected-to-equal (a mismatch declines), assignment sets intersected with re-convergence (a shrink re-queues successors) — and they run until the worklist reaches a fixpoint. What `build_entry_native` does with the plan is almost mechanical by comparison: one Cranelift block per leader, the operand stack and slots threaded through `Variables`, jumps and branches translated op for op, and `seal_all_blocks` at the end to let Cranelift build the ϕ -nodes. The hard thinking is all in the analysis; the codegen is the analysis made manifest.

The two-piece state, and why both are needed

The analysis carries two pieces of state at every program point — the abstract stack kinds and the definite-assignment set — and it is worth understanding why neither suffices alone, because together they are exactly the knowledge codegen needs and separately they are not. The kind vector answers “what type is each operand-stack position?”, which codegen needs to choose between the integer and float register lanes and to know which boxing shim a result needs. The assignment set answers “which slots and globals are provably written here?”, which codegen needs to avoid reading an uninitialized register as a value and to know which slots to write back at a deopt. A compiler with only the kind vector would emit type-correct code that might read an unassigned slot's zero where the interpreter would read `Undef` — type-safe but value-wrong. A compiler with only the assignment set would know which slots are live but not whether each holds an integer or a float — value-safe but type-blind. The native path needs both because a register has both a type and a definedness, and a wrong answer on either is a miscompile: the wrong type puts the bits in the wrong lane, the wrong definedness substitutes a zero for an `Undef`. The worklist computes both simultaneously, to a joint fixpoint, because they interact — a slot's kind is only known once you reach its assignment, and its assignment is only definite once every path agrees — and the `NativePlan` records both because codegen consults both at every op.

Fixpoint, and why it terminates

The abstract interpretation is a fixpoint computation, and a reader who has met dataflow analysis will want to know that it terminates, because a worklist that re-queues blocks on every change could in principle loop forever. It terminates because both pieces of state move monotonically toward a limit. The definite-assignment sets only ever *shrink* at a join — an intersection can only remove elements — and a set of slot indices can shrink only finitely many times before it is empty, so the re-convergence that re-queues successors when a set shrinks can fire only a bounded number of times. The kind vector does not shrink but is required to *match exactly* at joins, so a kind either agrees (no change, no re-queue) or conflicts (the chunk is declined, the analysis stops). The per-slot kind inference inserts a kind on first sight and thereafter requires equality, so it changes at most once per slot. Each of the three moving parts — assignment sets, stack kinds, slot kinds — can change only finitely often, so the worklist drains and the analysis halts, either with a complete

`NativePlan` or with a `None` the instant any join is inconsistent. This is the ordinary argument that a monotone dataflow analysis over a finite lattice terminates, and it applies here because the lattice — sets of bounded slot indices, a five-element kind domain — is finite and the transfer functions are monotone. The compiler’s analysis is not a heuristic that might run away; it is a dataflow fixpoint with a termination guarantee, which is part of why it can be trusted to either produce a sound plan or cleanly decline.

A counted loop, all the way down

The clearest way to see the basic-block compiler at work is to take the counted loop from the control-flow chapter — `while (i < 10) { i += 1 }`, with `i` in slot zero — and follow it from bytecode to the native shape it becomes. The bytecode, after the front end’s peephole has fused the tail, is roughly: a header at instruction `h`, a `SlotLtIntJumpIfFalse(0, 10, exit)` testing the slot against ten, a `PreIncSlotVoid(0)`, and a backward `Jump(h)`. `analyze_native` walks this and produces a plan: the leaders are `h` (the header, a jump target) and `exit` (the fall-through past the loop); slot zero’s kind is `Int`; the operand stack is empty at both leaders; and slot zero is in the definite-assignment set if it was written before the loop. Codegen then emits two Cranelift blocks and threads slot zero through a single `i64 Variable`. The result reads, in pseudo-CLIF, like this:

```
header:
    v0 = use_var(slot0)           ; load i from its register
    v1 = icmp slt v0, 10          ; i < 10 ?
    brif v1, body, exit           ; branch on the comparison
body:
    v2 = use_var(slot0)           ; load i
    v3 = iadd v2, 1                ; i + 1 (wrapping)
    def_var(slot0, v3)             ; store i back to its register
    jump header                   ; back-edge
exit:
    ; box slot0 and return
```

There is no memory access in the loop — slot zero is a register the whole time — and there is no dispatch, because the entire loop is native control flow. The `def_var/use_var` pair on slot zero is what becomes, after `seal_all_blocks`, the ϕ -node at the `header` block that merges the loop-entry value of `i` with the back-edge value, and the compiler wrote not a single ϕ -node by hand to get it. This is the whole basic-block compiler in one example: leaders become blocks, the slot becomes a `Variable`, the comparison becomes `icmp`, the conditional jump becomes `brif`, the back-edge becomes a `jump`, and Cranelift’s sealing builds the join. The native loop runs `i` in a register from start to finish, which is the headline property, and a four-hundred-program fuzzer over random arithmetic-and-branch chunks confirms that loops of exactly this shape compute what the interpreter computes.

What the back-edge ϕ -node actually is

The worked loop produces one ϕ -node — at the `header` block, merging the loop-entry value of `i` with the back-edge value — and it is worth dwelling on that node, because it is the single place where the `Variable` abstraction does its most valuable work and the place a hand-written compiler is most likely to get wrong. SSA form requires that every value have exactly one definition, but a loop variable has two: the value it enters the loop with (zero, say) and the value the back-edge feeds it (the incremented `i`). A ϕ -node reconciles these — it is a pseudo-instruction at the block where the two definitions meet that says “this value is the loop-entry definition if we arrived from outside the loop, or the back-edge definition if we arrived from the bottom.” Constructing it by hand means knowing, at the moment you emit the header, both predecessors and both incoming values — but the back-edge’s value does not exist yet when you first emit the header, because the loop body has not been compiled. This is the chicken-and-egg that makes hand-written loop codegen painful, and it is exactly what `def_var/use_var` plus deferred sealing resolves: the compiler writes `def_var(slot0, ...)` at the loop-entry and again at the back-edge, reads with `use_var` in the header, and

when it calls `seal_all_blocks` — after both definitions exist — Cranelift looks at the header's now-complete predecessor list and synthesizes the ϕ -node from the two `def_vars`. The compiler never names the ϕ -node, never specifies its incoming values, never reasons about predecessors; it just records where the variable is written and read, and the ϕ falls out of sealing. The one node in the worked loop's pseudo-CLIF that is *not* written by the compiler — the ϕ at `header` — is the node the `Variable` abstraction exists to write for it, and it is the reason every fusevm compiler can describe a loop as “thread the slot through a `Variable`” rather than as a ϕ -construction problem.

Chapter 27 — Locals in Registers

A loop whose body is native is only as fast as its access to its loop variables, and if every `GetSlot/SetSlot` calls back into the VM's boxed frame, the loop is not really native. So the native path holds **slots in registers** too.

Each slot becomes its own SSA `Variable`, typed `f64` if the slot's inferred kind is `Float` and `i64` otherwise; `GetSlot` is a `use_var`, `SetSlot` is a `def_var`, and a counted loop's induction variable lives in a machine register for the loop's whole life, never touching the boxed `Vec<Value>`. Globals get the identical treatment through a parallel set of `Variables`, tagged with a `GLOBAL_TAG` of `0x1_0000` so a global index never collides with a slot index in the unified definite-assignment set. The payoff is that a fully-native loop runs with *no* per-opcode dispatch and *no* per-opcode calls into the runtime — the headline property the whole effort was chasing.

Definite assignment, the correctness catch

There is a trap here, and it is the kind that produces a silent wrong answer rather than a crash. Reading a slot that was never written yields `Undef` in the interpreter — but a freshly initialized register holds `0`, and `Undef` and `0` diverge the moment they are used (`Undef` in arithmetic float-promotes; `0` does not). So the analysis carries the **definite-assignment** dataflow already described: at every program point it knows the set of slots and globals provably written on *every* path, and it declines to lower any chunk whose `GetSlot` or `GetVar` could read an unassigned location. The entry block does zero-initialize every slot and global register, but — the comment in the source stresses this — that zero is never actually *read*; it exists only to keep the `Variable` defined on every edge so Cranelift's SSA construction is happy. The tests pin both directions: a chunk that reads an unset slot falls back, a chunk that sets a slot on only one branch of an `if` falls back, and a chunk that sets it on *both* branches lowers.

The read-modify-write super-ops are the deliberate exception. `PreIncSlot`, `PreDecSlot`, `PostIncSlot`, `PostDecSlot`, `PreIncSlotVoid`, `AddAssignSlotVoid`, `AccumSumLoop`, and the `Slot...JumpBack` family are *exempt* from the definite-assignment requirement, because they coerce their slot through `to_int`, and `to_int(Undef)` is `0` — which matches the zero-initialized register exactly. A test confirms `PreIncSlotVoid` on an unassigned slot coerces to zero and becomes one, natively, in agreement with the interpreter. The exemption is not a loophole; it is the analysis being precise enough to know exactly which reads care about the `Undef`-versus-zero distinction and which provably do not.

The zero that is never read

There is a subtlety in how the native path initializes its slot and global registers that is easy to misread as a bug and is in fact the linchpin of the whole correctness argument. The entry block of a lowered function *does* zero-initialize every slot and global `Variable` — it has to, because Cranelift's SSA construction requires every `Variable` to be defined on every path that reaches a use, and a slot read on the first iteration of a loop, before any write, would otherwise have no definition. So the register starts at zero. But — and the source comment stresses this — that zero is *never actually read* as a value, because definite assignment has already proven that every `GetSlot` is preceded on every path by a `SetSlot`. The zero exists only to keep the variable defined for the SSA machinery; the analysis guarantees a real write always shadows it before any read observes it. This is why the two mechanisms — zero-init for SSA, definite-assignment for correctness — are not redundant: the zero-init makes the code *compile*, and the definite-assignment makes it *correct*, and the

second is what ensures the first never matters. The one place the zero could leak into observable behavior is a deopt, where an unassigned register might get written back to the VM as a `0` where the interpreter expects `Undef` — and that is exactly why `emit_deopt` writes back only the slots in the definite-assignment set and skips the rest, leaving them `Undef`. The zero is a scaffolding value, present for the compiler’s convenience and proven irrelevant to the program’s meaning.

Obj slots and the -1 sentinel

The `Obj` kind earns a small but telling special case in the register initialization, and it shows how carefully the boxed and unboxed worlds are kept consistent. An `Obj` register holds an `i64` handle into the VM’s value arena, and the natural zero-init would set it to `0` — but `0` is a *valid* arena handle, so a `SetSlot` overwriting a zero-initialized `Obj` slot might try to free the object at handle zero, corrupting a live value. The native path avoids this by zero-initializing `Obj` slots to `-1`, an “empty handle” sentinel that the free logic recognizes as “nothing to free,” so the first `SetSlot` into an `Obj` slot frees nothing and simply installs the new handle. This is the kind of detail that only surfaces once a compiler tries to hold reference-counted heap handles in registers across a loop, and it is handled the same way every other native-path hazard is: precisely, in one place, and verified by a differential fuzzer — the two-hundred-program `Obj`-slot string-accumulator test that builds strings across loops of varying length and checks the compiled result against the interpreter. The `-1` sentinel is the `Obj` kind’s version of the `Undef`-versus-zero distinction: a register’s default must be a value the rest of the machinery treats as “absent,” and for a heap handle that default is not zero but a reserved sentinel.

Chapter 28 — Kinds: The Type Lattice

Once the native path holds values in typed registers, it needs a notion of *type*, and that notion is the **Kind** lattice. It has five variants: **Int**, **Bool**, **Float**, **Status**, and **Obj**. Every value on the compile-time operand stack, and every slot and global, carries a **Kind**, and the compiler's correctness rests on tracking it precisely.

The carrier representation is the heart of it. **Int**, **Bool**, and **Status** are carried at runtime as **i64s**; **Float** as an **f64**; and **Obj** — the boxed-heap handle, an index into the VM's value arena — rides the integer lane like an integer. So each operand-stack position is backed by *two* parallel **Variables**: an **ivars** entry typed **I64** and an **fvars** entry typed **F64**, and the position's **Kind** says which one is live. Because the analysis guarantees the kinds are consistent at every join, the right **Variable** is always the one that was written, and the SSA construction at joins — loop back-edges included — picks it up automatically when the blocks are sealed. The lattice carries three small predicates that the codegen consults constantly: **is_numeric** is true for **Int**, **Float**, and **Status**; **is_intlike** for **Int** and **Bool** (a boolean's 0/1 equals its **to_int**); and **promotes_float** for everything that is not **Int**. Arithmetic *promotes* exactly as the interpreter does: if either operand is non-integer the result is **Float**, and the compiler inserts the integer-to-float conversion on the integer operand exactly where the interpreter's **to_float** would run.

Float slots and the kind fixpoint

Slots and globals are not int-only. A slot can hold a float — the canonical AWK accumulator, **sum = 0.0**; **sum += x** — and the compiler supports it by inferring a single chunk-wide kind per slot during the analysis. A **SetSlot** records the stored value's kind through **set_var_kind**, a **GetSlot** reads it back, and a slot used with two different kinds makes **set_var_kind** return false, which disqualifies the chunk. Because kinds can be set later in the worklist than they are first read, the analysis is a *fixpoint* — over slot kinds, stack kinds, and the assignment sets all at once — and it re-converges until nothing changes. Definite assignment guarantees a slot is written before it is read, and the worklist reaches the writing instruction before the reading one, so the kind is always known when needed, loops included, because the pre-loop initializer pins it. Codegen then types each slot's register **i64** or **f64** accordingly, and a test confirms a float accumulator and an integer loop counter can coexist in the same lowered loop.

Status, the quiet hard case

Status is the kind that earns the lattice its complexity, and it is the truthiness inversion of Chapter 3 come due. A **Status** holds an exit code as an **i64**; it float-promotes in arithmetic like any non-integer; it boxes back as a **Value::Status**. But it is *truthy* when its code is *zero*, the opposite of an integer — so the compiler's truthiness helper has an explicit **Status** arm. Where an **Int** truthiness test emits **icmp ne, 0** and a **Float** emits an unordered **fcmp ne, 0.0** (so a NaN is *truthy*, matching the interpreter), the **Status** arm emits **icmp eq, 0**. Get that one branch wrong and every **if \$?** in a compiled shell program inverts. The defense is the same as everywhere else in the native path: a differential fuzz that runs **\$?** through arithmetic, comparison, direct truthiness branches, and as a bare result — three hundred random programs with small codes including zero — against the interpreter, so the inversion is checked, not merely intended. The **Obj** kind, the fifth variant, is what lets the native path lower heap-valued accumulators too: a string built up across a loop in a slot rides as an **Obj** handle, boxed and unboxed at the boundary, and a two-hundred-program fuzz over **Obj**-slot string-accumulator loops keeps that honest as well.

A float accumulator, lowered

The lattice is easiest to believe by watching it carry a mixed-type loop, the canonical case being an AWK-style accumulator: `sum = 0.0; for (i = 0; i < n; i++) sum += i`. Here two slots have two different kinds and the loop has to keep both straight. `sum` is set from a float literal, so `set_var_kind` records its kind as `Float` and its register is typed `f64`. `i` is set from an integer, so its kind is `Int` and its register `i64`. Inside the loop, `sum += i` reads `i` (an `i64`), reads `sum` (an `f64`), and adds them — and because one operand promotes, the analysis pushes `Float` and codegen inserts an `fcvt_from_sint` to widen `i` before the `fadd`, then stores the `f64` result back to `sum`'s float register. The increment `i++` stays entirely in `i`'s integer register. At no point does either slot touch the boxed `Vec<Value>`; the whole loop runs with `sum` in an `f64` register and `i` in an `i64` register, and a differential fuzzer confirms the compiled sum matches the interpreter's. This is precisely the loop the README cites as the reason `SlotKind::Float` exists in the block JIT and the reason the kind lattice carries a `Float` variant in the AOT path: a float accumulator beside an integer counter is the most common shape in numeric scripting, and lowering it natively is what made every AWK accumulator in every embedding faster the day the kind inference learned to handle it.

When the kinds will not agree

The flip side of kind inference is the chunk it must refuse, and the refusal is as important as the inference because it is what keeps the lowering sound. Suppose a slot is assigned a float on one path and an integer on another — `if (c) x = 1.0 else x = 2` — and then read. The slot cannot be both an `f64` register and an `i64` register; its single chunk-wide kind is a contradiction, `set_var_kind` returns false on the second assignment, and `analyze_native` returns `None`, sending the whole chunk to the threaded path where the boxed `Value` can hold either. A test, `native_mixed_kind_slot_falls_back`, pins exactly this. The same refusal covers a slot used as `Int` in one place and `Obj` in another, or a stack join where the two arms of an `if` leave different kinds on top. The analysis does not try to *unify* conflicting kinds — there is no “either int or float” register — because a register has one machine type and that is the point of using one. Instead it recognizes the conflict and declines, trading a little speed on an unusual chunk for never having to represent an ambiguous register. This is the kind lattice's half of the lower-what-you-can-prove discipline: it proves a single kind per slot when one exists, and when none does, it hands the chunk to the path that does not need kinds at all.

Two lanes, one position

The dual-`Variable` scheme that backs each operand-stack position repays a concrete walk, because it is the mechanism that lets a single block of native code carry both integers and floats without a union type. Each position on the abstract operand stack is backed by two Cranelift `Variables` in parallel: an `ivars` entry typed `i64` and an `fvars` entry typed `f64`. At any program point the position's `Kind` says which of the two is live — `Int`, `Bool`, `Status`, and `Obj` live in the `i64` lane, `Float` in the `f64` lane. When a `LoadInt` pushes, codegen writes the integer into the position's `ivars` entry and records the kind `Int`; when an arithmetic op promotes to float, codegen converts the integer operand with `fcvt_from_sint` into the `fvars` lane and records `Float`. Because the analysis guarantees the kind is consistent at every join — both arms of an `if` leave the same lane live on a given position — the SSA construction at the join picks up the right `Variable` automatically, and the unused lane simply has no live definition at that point. The two-lane scheme is why the native path never needs a tagged-union register or a runtime type check within a block: the *type* is a compile-time fact carried in the `Kind`, and the *value* is in whichever of the two typed lanes the kind names. It is the register-level version of the `Value` enum's tag-and-payload, split into a compile-time tag (the `Kind`) and two type-specific payloads (the two `Variables`), so that what is a runtime discriminant in the boxed world becomes a compile-time fact in the native world — which is exactly the move that turns a dynamically typed value into statically typed machine code.

Why Bool and Status share the integer lane

It might seem cleaner to give each of the five kinds its own representation, but `Bool` and `Status` deliberately ride the integer lane alongside `Int`, and the sharing is a correctness convenience worth understanding. A `Bool` is carried as an `i64` that is 0 or 1, because a boolean's `to_int` is exactly 0 or 1 and every arithmetic use of a boolean treats it as that integer — so putting it in the integer lane means a `Bool` flowing into an `Add` needs no conversion, it is already the integer it coerces to. A `Status` is carried as an `i64` holding the exit code, because a status arithmetic promotes like any non-integer and coerces to its code; the only place it differs from an `Int` is truthiness, where it is true at zero rather than non-zero, and that difference is one branch in the truthiness helper, not a different representation. So three of the five kinds — `Int`, `Bool`, `Status` — share the integer lane because their runtime carrier is genuinely an `i64`, and the kind distinction matters only at the few points where their *semantics* diverge: a `Status`'s inverted truthiness, a `Bool`'s boxing back as `Value::Bool` rather than `Value::Int`. The `is_intlike` predicate (`Int` or `Bool`) and the `is_numeric` predicate (`Int`, `Float`, `Status`) are how codegen asks “can I treat this as a plain integer here?” without caring about the kind's identity. Sharing the lane keeps the common operations — arithmetic, comparison — uniform across the three integer-carried kinds, and isolates the divergence to the handful of places it actually occurs.

Promotion, decided once

The `promotes_float` predicate — true for every kind that is not `Int` — is the compile-time embodiment of the interpreter's “both `Int`, else float” arithmetic rule, and tracing how it is used shows the kind lattice doing the interpreter's runtime type-dispatch at compile time. When the interpreter executes an `Add`, it inspects the two values at runtime: if both are `Int` it does integer arithmetic, otherwise it promotes both to `f64`. The native path cannot inspect values at runtime — they are in registers, their types fixed at compile time — so it must decide the same thing *statically*, and it does, by asking `promotes_float` of each operand's `Kind`. If either operand's kind promotes (anything but `Int` — a `Float`, a `Status`, a `Bool` used arithmetically), the result is `Float` and codegen inserts the `fcvt_from_sint` conversion on the integer operand exactly where the interpreter's `to_float` would have run; if both are `Int`, it emits a wrapping `iadd`. The decision the interpreter makes per-execution, the compiler makes per-instruction, once, using the kinds the analysis proved — and because the analysis is sound, the static decision always matches the dynamic one. This is the essential move that turns dynamic typing into static code: the interpreter dispatches on runtime types, the compiler dispatches on compile-time kinds, and the kind lattice is the bridge that makes the second a faithful translation of the first. Get `promotes_float` wrong for even one kind and the compiled arithmetic would promote where the interpreter does not, or fail to promote where it does, producing a result that is an integer where it should be a float or vice versa — which is exactly the discrepancy the float fuzzer, with its mixed int-and-float operands, exists to catch.

Chapter 29 — Floats, Math, and Libcalls

With the kind lattice in place, the native path lowers the whole numeric core, and this chapter catalogs exactly which operation becomes which machine instruction or libcall — because the catalog *is* the correctness argument: every entry either is a single instruction that provably matches the interpreter, or a libcall into the same Rust function the interpreter calls.

Float arithmetic is `fadd/fsub/fmul/fneg`, with int-to-float promotion on mixed operands. Comparisons are `fcmp` (and `icmp` for integers), with the ordered/unordered conditions chosen so NaN behaves exactly as the interpreter's `==` and `<` do. The rounding and absolute-value ops are single instructions: `AbsFloat` is `fabs`, `SqrtFloat` is `sqrt`, `CeilFloat/FloorFloat/TruncFloat` are `ceil/floor/trunc`, and `RoundFloat` is Cranelift's `nearest`, round-half-to-even, an exact match for the interpreter's `round_ties_even`. Integer `Mod` is a `srem` guarded by `select` for the `y==0` and `y==-1` cases (matching the interpreter's mod-by-zero-is-zero and avoiding the `i64::MIN % -1` overflow); float `Mod` is the `fusevm_aot_fmod` libcall.

The operations with no single instruction become libcalls into the shim roster, and the shims call the same `f64` methods the interpreter does, so a finite result matches bit-for-bit. `Pow` and `PowFloat` call `fusevm_aot_powf`. The thirteen transcendentals share *one* dispatcher, `fusevm_aot_unary_math(id, x)`, keyed by a small integer — `Sin` is 0, `Cos` 1, `Tan` 2, the inverse trig 3 through 5, the hyperbolics 6 through 8, `Exp` 9, natural log 10, `Log2` 11, `Log10` 12 — so thirteen functions cost one import, declared only when one of them appears in the chunk. `Atan2Float` has its own two-argument shim, `fusevm_aot_atan2`. This is not cheating: every native compiler in existence calls `pow` rather than inlining it, because there is no `pow` instruction, and a native compiler that calls a leaf math function is still native code.

The integers Cranelift makes you build

Three integer intrinsics have no single instruction and are built inline from several. `AbsInt` is a wrapping integer absolute value: `ineg` the value, compare it to zero with `icmp slt`, and `select` between the negation and the original, so that `abs(i64::MIN)` wraps to `i64::MIN` exactly as the interpreter's wrapping abs does. `TruncInt` is a saturating float-to-int conversion, `fcvt_to_sint_sat`, which matches Rust's `as i64` cast NaN-to-zero and all, and passes an integer operand through unchanged. And `GcdInt` and `LcmInt` — the one integer pair that contains a loop — are lowered as *internal native loops*: `GcdInt` emits three on-the-fly Cranelift blocks for a Euclid loop over the operand magnitudes, and `LcmInt` runs the same gcd then divides, multiplies for the low bits, and uses `umulhi` to detect overflow, saturating to `i64::MAX` when the high half is non-zero and returning zero if either operand is zero. A three-hundred-case differential fuzz over full-range `i64` pairs covers the unsigned-abs path, the `i64::MIN` edge, and the saturation, against the interpreter's `fusevm_jit_gcd_i64/fusevm_jit_lcm_i64` helpers.

Thirteen functions, one import

The `unary_math` dispatcher repays a close look because it is a small, elegant solution to a real codegen problem: how to give native code access to thirteen transcendental functions without bloating every chunk that uses one with thirteen imports. The naive approach declares a separate libcall import for `sin`, `cos`, `tan`, and the ten others, so a chunk that calls `sin` carries an import it uses and twelve it does not, or — worse — the compiler conditionally declares only the ones a chunk uses and the import table varies chunk to chunk, complicating the disk cache. `fusevm` instead routes all thirteen through a single shim, `fusevm_aot_unary_math(id, x)`, where `id` selects the function: `Sin` is 0, `Cos` 1, `Tan` 2, the inverse trig 3 through 5, the

hyperbolics 6 through 8, `Exp` 9, natural log 10, `Log2` 11, `Log10` 12. A chunk that calls any transcendental declares exactly one import — the dispatcher — and passes the right id at each call site. This keeps the import table uniform (one entry, or none) so a chunk's compiled form is stable and cacheable, and it keeps the relocation simple for the disk cache to re-resolve on load. The dispatcher itself is a `match` on the id calling the corresponding `f64` method, so a finite result is bit-for-bit identical to the interpreter's, which calls the same methods. `AwkLogJit` even reuses the natural-log id ten rather than declaring its own, so the AWK log and the host-independent `LogFloat` share one code path through the dispatcher. Thirteen functions, one import, one relocation, one cache-stable import table: the dispatcher is what makes a transcendental as cheap to compile as an addition.

Math that is a loop, lowered inline

Two of the integer intrinsics — `GcdInt` and `LcmInt` — are special because they are the only “intrinsics” whose definition contains a loop, and the way the native path handles them shows the compiler emitting control flow *mid-instruction*. A gcd cannot be a single instruction or a single libcall-free expression; it is Euclid's algorithm, an iteration `while y != 0 { (x, y) = (y, x % y) }`. The native path lowers `GcdInt` by emitting three on-the-fly Cranelift blocks in the middle of the surrounding function — a loop header testing `y != 0`, a body computing the `urem` of the magnitudes and swapping, and an exit — so the gcd loop becomes a native loop nested inside whatever native code surrounds it, with no call out to a helper. `LcmInt` builds on it: it runs the same gcd, divides one operand by it, multiplies for the low bits, and uses `umulhi` — the high half of a 64-by-64 multiply — to detect overflow, saturating to `i64::MAX` when the high half is non-zero and returning zero if either operand was zero. That this can be lowered inline, rather than punted to a libcall, is a small flex: the native path is willing to emit loops and overflow-checked multiplies in the middle of a function when doing so keeps the operation native, and the three-hundred-pair full-range fuzzer confirms the inline gcd and the saturating lcm match the interpreter across the `i64::MIN` edge and the overflow boundary. The intrinsics that are loops are lowered as loops, not as calls, because the native path would rather emit three extra blocks than leave native execution.

NaN, and the ordered/unordered choice

Floating-point comparison hides a subtlety that the native path has to get exactly right, and it is worth surfacing because it is the kind of detail a careless compiler gets wrong and a fuzzer immediately catches. IEEE floating point has a value, NaN, that is *unordered* with respect to every other value, including itself: `NaN == NaN` is false, `NaN < 1.0` is false, `NaN > 1.0` is false. The interpreter's comparisons follow this — `Value::Float(NAN)` is unequal to everything — and the truthiness rule follows from it too: a NaN is truthy, because `NaN != 0.0` is true. When the native path lowers a float comparison or a float truthiness test, it must choose between Cranelift's *ordered* and *unordered* `fcmp` conditions, which differ precisely in how they treat NaN: an ordered condition is false when either operand is NaN, an unordered condition is true. The native truthiness helper emits an *unordered* `fcmp ne, 0.0`, so a NaN compares not-equal-to-zero and is therefore truthy, matching the interpreter; an ordered choice would make NaN falsy and invert every `if (x)` where `x` could be NaN. The comparison opcodes choose ordered or unordered per operator so that `==`, `<`, and the rest agree with the interpreter on NaN inputs. This is one branch in the codegen, easy to get backwards, and it is exactly the kind of thing the four-hundred-program float fuzzer is built to catch — it generates comparisons over quarter-valued floats and checks each against the interpreter, so a flipped ordered/unordered condition would surface as a failing seed rather than as a wrong answer in production.

The heap engine that is not future work

It would be easy to read the boundary chapter's “dynamic-kind sources remain future work” as meaning the native path cannot touch heap values at all, and that is worth correcting, because a general box/unbox engine already lowers a surprising amount of heap-valued code. The key distinction is between a heap op whose

result kind is statically known and one whose result kind is dynamic. A string `Concat` always produces a `Str`; a `Range` always produces an `Array`; many array, hash, and expansion ops produce a value whose kind the analysis can pin down even though the value lives on the heap. For these, the native path stages the operation through the shim — boxing the register operands onto `vm.stack`, calling `exec_op`, and unboxing the result back into a register as an `Obj` handle or whatever statically-known kind it produces — so the op runs against the boxed world but the native code carries on with the result in a register. This is the same box/unbox machinery the sinks and the statically-typed sources use, generalized: it works whenever the *kind* of the produced value is known, regardless of whether the *value* lives on the heap. What it cannot yet do is the op whose result kind is genuinely unknown until runtime — a general array read that might return an integer or a string — because there is no single register lane to unbox an unknown kind into. So the native path's reach into the heap is wider than “scalars only” and narrower than “everything”: it is “everything whose result kind is statically determined,” which the box/unbox engine handles today, with the dynamic-kind remainder waiting on the guarded deopt.

Integer modulo, and the two edges a select guards

Integer `Mod` is the one piece of integer arithmetic the native path cannot lower as a bare machine instruction, and the reason is two hardware edges that a `select` is used to guard, which is worth spelling out because it is a precise example of the compiler reproducing the interpreter's exact behavior down to its corner cases. A hardware signed-remainder instruction, `srem`, faults on two inputs: a zero divisor (division by zero), and `i64::MIN % -1` (where the true quotient overflows `i64`). The interpreter's `arith_int_fast` does not fault on either — its integer modulo by zero yields zero, and it uses wrapping arithmetic that handles the `MIN % -1` case — so the native code must match that non-faulting behavior rather than emit a bare `srem` that would trap. It does so by guarding with `select`: it computes the `srem` on a safe path and selects a zero result when the divisor is zero, and it handles the `-1` divisor so the `MIN % -1` overflow cannot occur. The result is a native integer modulo that produces exactly what the interpreter produces on every input, including the two edges where a naive `srem` would crash the program. This is the kind of detail that separates a compiler that mostly works from one that is differentially correct: the bitwise fuzzer deliberately draws its operands from the small range that exercises modulo by zero and by negative one, precisely because those are the edges where a careless lowering would diverge, and the `select` guards are what make the native modulo match the interpreter's wrapping, non-faulting semantics on exactly those inputs. An instruction that faults where the interpreter does not is not a faster version of the interpreter; it is a different program, and the `select` is what keeps `Mod` the same program at native speed.

Chapter 30 – The Inline/Shim Boundary

Up to here the native path has been all-or-nothing: a chunk is either entirely scalar and lowered, or it contains an opcode outside the model and falls back to the threaded path wholesale. Real programs are rarely entirely scalar — a hot numeric loop that prints a result, an AWK loop that reads `$1` — so the native path grows a **boundary**: a way to run the scalar parts in registers and reach back into the boxed world for the opcodes that need it.

The boundary has two halves, and one is far easier than the other. The easy half is **spilling**, for *sink* opcodes — ones that consume scalar values and produce nothing dynamic. `Print`, `PrintLn`, and `SetStatus` are sinks: the native code spills the top few register values onto the boxed `vm.stack`, boxed per kind (`Int` → `Value::Int`, `Float` → `Value::Float`, and so on, bottom-most first so the interpreter sees them in order), calls the opcode through the `exec_op` shim, and continues. The boxed stack is empty before and after — the opcode pops exactly what was pushed — so no value has to be reloaded into a register and no type guard is needed. `SetStatus` is the silent one-argument case. This alone lets a hot numeric loop with an embedded `print` run fully native, which wholesale fallback never could, and a test confirms a loop-with-print lowers.

The reload half

The harder half is **reloading**: a *source* opcode that produces a value the native code then consumes. The general case is genuinely hard, because the value an arbitrary opcode pushes has no static kind — it could be a string — and a string cannot live in a typed register. But some sources have a *statically known* result kind, and those reload with no guard at all. `AwkGetFieldNum` always pushes a `Float`, reloaded into the float register through `fusevm_aot_pop_float`; `GetStatus` always pushes a `Status`, reloaded into the integer register through `fusevm_aot_pop_int`. For those the native code runs the opcode through the shim, reloads the pushed value into the matching register, and carries on. There is, in fact, more than the two statically-typed sources here: a general box/unbox engine also stages many heap ops — string concat, array and hash operations, ranges, expansions, the AWK string builtins — through the shim, pushing their statically-known result kinds (often `Obj` handles) back into registers. What remains genuinely future work is the *dynamic*-kind source — an arbitrary array read, `ReadLine`, a call — whose result kind cannot be known at compile time, because it forces the question a JIT answers with profiling and an AOT compiler can only answer with a guess: *which* kind to assume. Those still fall back, and the next chapter is the mechanism that will eventually unblock them.

Sinks are easy because the stack balances

The reason spilling for sink opcodes is the easy half of the boundary comes down to a single property: a sink pops exactly what it pushes onto the boxed stack, so the boxed stack is empty before the call and empty after, and nothing has to be reloaded. Take `Print(n)`. The native code holds its `n` operands in registers; to print them it spills them onto `vm.stack`, boxing each per its kind — an `Int` register becomes a `Value::Int`, an `f64` register a `Value::Float`, a `Status` register a `Value::Status` — in bottom-most-first order so the interpreter sees them as the bytecode intended. Then it calls `Print` through the `exec_op` shim, which pops all `n` and emits them. After the call, `vm.stack` is empty again, the native registers are untouched, and execution continues with no value crossing back from the boxed world. Because nothing comes back, there is no type

to guess and no guard to emit — the spill is a one-way push, and a sink is precisely an opcode whose only effect on the value stack is to consume. This is what lets a hot numeric loop with an embedded `print` run fully native: the loop's arithmetic stays in registers, and only at the `print` does the native code briefly touch the boxed stack, spill, call, and return to registers — a few instructions per print, against the alternative of declining the entire loop to the threaded path because it contained one I/O op. A test, `native_loop_with_print_lowers`, confirms the loop lowers rather than bailing.

Reloads are hard because the kind is unknown

The reload half is hard for one reason, and naming it precisely also names the path forward. When native code consumes a value an opcode *produced*, it must put that value into a typed register — an `i64` lane or an `f64` lane — and to do that it must know the value's kind. For a sink there was no produced value, so no kind to know; for a source there is, and a string cannot go in either lane. The native path handles the sources whose kind is *statically known*: `AwkGetFieldNum` always pushes a `Float`, so it reloads through `fusevm_aot_pop_float` into the float register with no guard; `GetStatus` always pushes a `Status`, reloaded through `fusevm_aot_pop_int`. A general box/unbox engine extends this to many heap ops whose result kind is known — string concat, array and hash operations, ranges, expansions, the AWK string builtins — staging them through the shim and pushing their statically-known result kinds (often `Obj` handles) back into registers. What remains genuinely open is the source whose kind is *dynamic*: a general array read might yield an integer, a float, or a string, and the compiler cannot know which without running the program. A JIT answers this with profiling — it watches and specializes to the kind it saw, guarding against the others. An AOT compiler has no runtime to watch, so it can only guess, and the honest engineering choice was to leave the dynamic-kind sources falling back rather than ship a guess. The unblock, when it comes, is a runtime type-guard-then-deopt: assume a kind, check it at runtime, and bail to the interpreter through the existing deopt machinery if the guess was wrong — which is exactly why the next chapter's deopt is described as the mechanism that will eventually open the boundary the rest of the way.

The boundary as a map of what is left

It is worth pausing on the boundary's current frontier, because the line between what lowers and what falls back is the most honest summary of where the AOT native path stands. On the lowered side: all scalar arithmetic, comparison, bitwise, and logic; the math intrinsics; control flow; slots and globals of every scalar kind plus `Obj`; the fused superinstructions; the JIT-eligible AWK scalar ops; sinks like `Print`; statically-typed sources like `AwkGetFieldNum` and `GetStatus`; and, through the box/unbox engine, many heap ops whose result kind is statically known. On the fallback side, two categories remain. The first is the truly dynamic-kind source — a general array or hash read, `ReadLine`, an arbitrary call — whose result could be any type. The second is anything the analysis declines for soundness: a mixed-kind slot, an unassigned read, a non-scalar result, a global aliased by a named array. The first category is *technical* future work — it needs the runtime type-guard-then-deopt the next chapter's machinery makes possible. The second is *permanent* by design — those chunks are correctly handled by the threaded path, and lowering them would mean either an unsound register or a representation the native model deliberately excludes. Reading the boundary this way clarifies what “grow the native path” means: it means converting dynamic-kind sources from fallback to guarded-lowering, one source family at a time, each guarded by a deopt, while the soundness declines stay declines forever. The frontier is not a limitation to be eliminated but a line that moves in one place and is fixed in another.

One boundary mechanism, many ops

The box/unbox machinery the boundary uses is worth recognizing as a single general engine rather than a set of per-op special cases, because that generality is what let the native path's reach into the heap grow without growing its risk. At its core the engine does three things: it *boxes* a register value onto the boxed `vm.stack` (an `i64` lane value becomes a `Value::Int` or a `Value::Status` per its kind, an `f64` lane value

a `Value::Float`, an `Obj` handle unboxed to its arena value), it *calls* an opcode through the `exec_op` shim, and it *unboxes* the result back into a register if there is one. Every member of the boundary is a configuration of these three steps. A sink boxes its operands, calls, and unboxes nothing. A statically-typed source boxes nothing (it has no operands the native code holds), calls, and unboxes one value of a known kind. A heap op with a known result kind boxes its operands, calls, and unboxes an `Obj` handle. The deopt is the same engine run to completion: it boxes *all* live state — slots, globals, the whole operand stack — and hands off to the interpreter without unboxing anything back, because native execution is over. So the sinks, the statically-typed sources, the heap-op staging, and the deopt are not four mechanisms but four uses of one box/call/unbox engine, parameterized by which steps run and which kinds they handle. This is why adding a new boundary case — a new statically-typed source, a new staged heap op — is cheap: it is a new configuration of an existing engine, not new machinery, and it inherits the engine's correctness. The dynamic-kind sources that remain future work are exactly the case the engine cannot yet configure, because the unbox step needs a kind it does not have until runtime — and supplying that kind through a guard is the one extension the engine still needs.

Chapter 31 — Deopt

Some opcodes cannot be lowered to a register result even in principle, because their result is not a scalar the register model can hold — and the canonical example is the humble `Div`. The interpreter's division yields a `Float` for ordinary operands but `Undef` for division by zero, and `Undef` is not a number. The native path handles this — and reuses the mechanism for any future dynamic-source guard — with `deopt`: a one-way exit from native code back to the interpreter, mid-program.

A deopt has to leave the VM in a state the interpreter can resume from, and that is most of the work. `emit_deopt` writes back, in a precise order. First the slots — but *only* the ones in the definite-assignment set, boxed per kind through `store_slot_int/store_slot_float/store_slot_obj`; an unassigned slot is *skipped*, left `Undef`, because writing its register's zero would make the resumed interpreter see `0` where it expects `Undef`, which is exactly the bug definite-assignment exists to prevent. Then the globals, the same way, keyed with the `GLOBAL_TAG`. Then the live operand stack, spilled bottom-most first into `vm.stack` in interpreter order, an `Obj` handle unboxed back to its real value on the way. Finally it sets the instruction pointer with an `iconst` and calls `fusevm_aot_resume(vm, ip)`, which runs `VM::run()` from that instruction pointer — which works without modification, because `run` already loops from `self.ip` and only tries the block JIT at `ip == 0` — and returns. Native code never re-enters after a deopt; the interpreter owns the remainder.

Div, and why this is the unlock

`Div` uses deopt for its rare case only. Notably, the compiler does not even pop the operands before the guard: it emits `fcmp eq divisor, 0.0` and a `brif`, and only the zero-divisor branch takes the deopt, re-executing the division in the interpreter at the original IP so it produces the exact `Undef`. The common path is a native `fddiv`. A differential fuzz with a divisor of $(n \% 9) - 4$ — so roughly one zero divisor in nine — confirms both paths match the interpreter across three hundred programs, and a dedicated test sets a slot to a known value natively, forces a divide-by-zero, and checks the resumed interpreter reads the slot back correctly, proving the state reconstruction works. The analysis also guards a subtler hazard: a slot assigned on only *some* paths to a deopt point would be ambiguous at resume, so `analyze_native` requires that the “may-assigned” set equal the “must-assigned” set at every deopt point (with `Div`'s own deopt exempt, since its successors are native and definite-assignment-checked). The same `fusevm_aot_set_error` early-return branch backs the trapping AWK ops — `AwkDivJit/AwkModJit` with their zero-divisor error codes, the bitwise `AwkLshiftJit/AwkRshiftJit/AwkComplJit` with their negative-operand codes — so a single error-and-resume machinery serves both the deopt-for-`Undef` case and the trap-for-fatal case. Deopt is the general machinery; `Div` is its first user; and everything left undone in the native path — the dynamic-kind sources — now needs only a type guard in front of a deopt that already works.

Resume works because run already loops from ip

The most elegant thing about deopt is how little new machinery resuming requires, and it is worth appreciating because it is a payoff of a design choice made chapters earlier. When `emit_deopt` has written the VM's state back, it calls `fusevm_aot_resume(vm, ip)`, which sets the instruction pointer and calls `VM::run()`. And `run` simply works from there — no special “resume mode,” no replay, no reconstruction beyond the state writeback — because `run`'s dispatch loop already begins from `self.ip` and runs forward to the end. The interpreter does not know or care that the first however-many instructions were executed natively; it picks up at the deopt IP with a correctly reconstructed stack, slots, and globals, and runs the remainder exactly as if it had

been interpreting all along. The one guard the design needed was that `run` only tries the block JIT at `ip == 0`, so resuming mid-program at a non-zero IP does not accidentally re-trigger compilation of the chunk it is in the middle of. That single condition is the whole accommodation the interpreter makes for deopt; everything else falls out of the fact that the dispatch loop was always a loop from `self.ip`. A mechanism that lets compiled native code hand control back to the interpreter at an arbitrary point, mid-program, with no replay and almost no special-casing, is the kind of capability that usually costs a great deal of plumbing — and here it costs a state writeback and a function call, because the interpreter was built loop-from-ip from the start.

One machinery, two users

The deopt path and the AWK trap path look like two different features and are in fact one mechanism with two triggers, which is worth seeing because it shows how a general capability pays for itself across cases that seemed unrelated. `Div`'s zero-divisor case needs to leave native execution and let the interpreter produce the `Undef` a register cannot hold — a *deopt*, resuming the interpreter to finish the program. The AWK ops `AwkDivJit/AwkModJit` need to leave native execution and let the VM raise a fatal error on a zero divisor — a *trap*, ending the run with an error. Both are the same shape: a runtime check (`fcmp eq divisor, 0.0`), a `brif` to an early-exit block, and a return-to-the-VM that the dispatch path interprets. The deopt block calls `fusevm_aot_resume`; the trap block calls `fusevm_aot_set_error` with a code the VM maps to the gawk message. The bitwise AWK JIT ops reuse the trap branch with their own codes — negative operands to `lshift/rshift/compl` — and `AwkSqrtJit/AwkLogJit` use a softer variant that calls a warn libcall and continues with NaN rather than exiting. So one early-exit-and-return-to-VM machinery serves the deopt-for-`Undef` case, the trap-for-fatal case, and the warn-and-continue case, distinguished only by what the exit block calls. Building the general machinery for `Div` is what made the AWK trapping ops cheap to add, and it is also what will make the dynamic-kind sources cheap to add later: the hard part — leaving native execution cleanly with the VM in a coherent state — is done once and reused wherever native code needs to hand control back.

The writeback, in order

The state reconstruction `emit_deopt` performs has a definite order, and the order is not arbitrary — it is the order that leaves the VM looking exactly as the interpreter would have left it at the deopt instruction pointer. The sequence is: first the slots, then the globals, then the operand stack, then set the instruction pointer and resume. The slots are written back only if they are in the definite-assignment set, each boxed by its kind — `store_slot_int`, `store_slot_float`, or `store_slot_obj` — and an unassigned slot is *skipped entirely*, left as the `Undef` the frame already holds, because writing its register's zero would substitute a `0` for an `Undef`. The globals follow, the same way, keyed with the `GLOBAL_TAG` bit so a global index does not collide with a slot index. Then the live operand stack is spilled, bottom-most first so the interpreter sees it in the right order, each value boxed by kind, with an `Obj` handle unboxed back to its real arena value on the way down. Only after all of that does the code set the instruction pointer with an `iconst` and call `fusevm_aot_resume`, which hands the rest of the run to `VM::run`. The order matters because the interpreter, on resume, expects a fully materialized VM state at that IP: the slots and globals it will read must hold the values the native code computed, and the operand stack must hold whatever was mid-computation when the deopt fired. Get the order or the kind-boxing wrong and the resumed interpreter computes from a corrupt state — which is precisely why a dedicated test sets a slot natively, forces a divide-by-zero deopt, and checks the resumed interpreter reads the slot back correctly. The writeback is the bridge from the register world to the boxed world, and it is built to leave no trace that the program was ever running natively.

Why operands are not popped before the Div guard

A detail of the `Div` lowering that looks like an oversight is in fact a correctness requirement, and it illustrates how carefully the deopt and the native fast path have to agree. When the native path compiles a `Div`, it does

not pop the dividend and divisor off the abstract stack before emitting the zero-divisor guard. The reason is that if the guard fires and the code deopts, the interpreter must re-execute the `Div` at the original instruction pointer — and to do that, the interpreter needs the operands still on the boxed operand stack, in the positions the bytecode expects. If the native code had popped them before the guard, they would be gone from the abstract stack, and the deopt's operand-stack spill would reconstruct a stack missing the two values the re-executed `Div` needs. So the guard runs with the operands still logically on the abstract stack: on the common nonzero path the code then consumes them with an `fdiv`, and on the rare zero path it deopts with them intact, and the spill writes them back so the interpreter's `Div` finds them. This is the kind of interaction that makes a deopt-bearing compiler subtle — the fast path and the bail path must agree about exactly what state exists at the branch — and it is handled here by the simple discipline of not destroying state until past the point where a deopt might need it. The differential fuzzer with roughly one zero divisor in nine is what proves the two paths agree across three hundred programs, and the dedicated slot-reconstruction test is what proves the spilled state is what the interpreter expects.

May versus must, at a deopt point

There is a refinement of the definite-assignment analysis specific to deopt points that is worth surfacing, because it closes a soundness gap that the basic definite-assignment check does not. The ordinary check ensures a `GetSlot` never reads an unassigned slot. But a deopt point has a stronger requirement: when a deopt fires, `emit_deopt` writes back the slots in the definite-assignment set, and the resumed interpreter must find each slot holding either the value the native code computed or the `Undef` it would have held — never a register's stale zero. Consider a slot assigned on *some* but not all paths to a deopt point. The “must-assigned” set (intersection at joins) excludes it, so `emit_deopt` would skip writing it back, leaving it `Undef` — but on the paths where it was assigned, its true value is in a register the writeback just skipped, and the interpreter would resume seeing `Undef` where the program had a value. To close this, `analyze_native` computes a second dataflow, the “may-assigned” set (union at joins), and requires that at every deopt point the may-assigned set *equals* the must-assigned set — meaning every slot assigned on any path is assigned on every path, so there are no conditionally-assigned slots whose value would be lost. A chunk with a conditionally-assigned slot live at a deopt point is declined. `Div`'s own deopt is exempt from this stricter check, because its successors are native and definite-assignment-checked in the ordinary way, so its writeback is sound without the may-equals-must condition. This is the kind of second-order soundness concern that only arises once a compiler can bail mid-program, and it is handled the same way every other native-path hazard is: a dataflow that proves the safe case and a decline for the unsafe one, with a dedicated fuzzer — the four-hundred-program conditional-assign-before-deopt test — checking that the declines are correct.

Chapter 32 – Differential Correctness

A compiler that is sometimes wrong is worse than no compiler, and fusevm's native path is verified the only way a compiler of a dynamically typed machine can be: **differentially**, against the interpreter that is the specification. The project's test-to-production ratio makes the seriousness concrete — roughly twenty-four thousand lines of production Rust shadowed by more than a hundred and twenty thousand lines of tests, over five times its own weight — and the differential fuzzers are where that weight lands hardest.

The discipline is simple to state. The interpreter is correct by definition — it is the single source of truth for what every opcode means. So every test of the native path runs the same chunk two ways, interpreted (`VM::run`) and compiled (`run_chunk_native`), through a harness that compares the `VMResult` exactly, `Ok` against `Ok`, `Halted` against `Halted`, `Error` against `Error`. For fixed cases this catches obvious mistakes; the real defense is the *fuzzers*. Each seeds a deterministic SplitMix64 generator with a fixed, per-test constant — so a failure reproduces as a single seed rather than a one-in-a-thousand ghost — and builds hundreds of random chunks per category, checking each compiled result against the interpreter.

The categories, with their counts, are a map of the native path's risk surface: four hundred balanced integer-arithmetic-and-branch programs; four hundred more for the integer bitwise and shift ops, with operands deliberately in the $-8..8$ range to exercise modulo by zero and by negative one; four hundred float programs with int/float promotion (asserting that more than fifty actually lowered, so the test cannot pass by trivially falling back); three hundred for `$?` through every consumer; three hundred for division with about one zero divisor in nine; three hundred for gcd and lcm over full-range integers; three hundred for the AWK bitwise ops across the shift-mask boundary; two hundred for `Obj`-slot string accumulators. A miscompiled wrapping add, an inverted truthiness, a botched saturation: each shows up as a single failing seed, and the seed makes it reproducible.

The negative space

The fuzzers also encode the *negative* space, and this is the part that is easy to overlook and load-bearing to keep. A chunk that the analysis should *decline* — a mixed-kind slot, an unassigned-slot read, a bare boolean result, a slot assigned on only one branch, a global aliased by a named array — is asserted to fall back, and *then* asserted to still match the interpreter on the threaded path. There is even a fuzzer that builds an integer expression with a trailing string concat to force a partial deopt and asserts that more than a hundred of its four hundred programs actually deopted, so “the deopt path is exercised” is itself a checked property, not a hope. So the tests verify not only that lowered code is correct but that *un-lowerable* code is correctly recognized as such, and that the fallback it routes to is correct too. The whole edifice — every chapter of native lowering — rests on this one practice, and it is the reason the native path could grow as aggressively as it did without accumulating risk.

Why differential, and not unit tests

It would be reasonable to ask why a compiler this careful does not simply rely on a large suite of hand-written unit tests, each checking a specific opcode's lowered output, and the answer reveals something about compiling a dynamically typed machine. A unit test for native `Add` would assert that some specific chunk produces some specific value — but the space of chunks the native path accepts is enormous and combinatorial, because an `Add` can follow any sequence of loads, slots, branches, and promotions, and the bug that matters is usually not “`Add` is wrong” but “`Add` is wrong *after this particular sequence* that left the abstract stack in a state the analysis mishandled.” Hand-written tests probe points; the bugs live in the spaces between the points. A differential

fuzzer against the interpreter probes the spaces: it generates *random* sequences — random balanced RPN arithmetic, random branch structure, random slot assignments before a deopt — and for each one it has an oracle, the interpreter, that is correct by construction. It needs no hand-specified expected value, because the expected value *is whatever the interpreter computes*, which is the definition of correct. This is the only scalable way to verify a compiler of a dynamically typed machine: you cannot enumerate the chunks, so you sample them randomly and check each against the specification, and you make the sampling deterministic so a failure is a reproducible seed rather than a ghost. The unit tests that do exist pin specific known-hard cases — the `Status` truthiness inversion, the `i64::MIN` gcd edge, the divide-by-zero slot reconstruction — but the fuzzers are the backbone, because they test the combinations no human would think to enumerate.

The negative tests are the subtle ones

Among the fuzzers, the negative-space tests deserve emphasis because they guard a failure mode that is invisible to the positive tests: a chunk that *should* fall back but does not. Suppose a future change to `analyze_native` made it too eager — admitting a chunk with a mixed-kind slot, say, that it should have declined. The positive fuzzers might not catch it, because the over-eagerly-lowered chunk might happen to compute the right answer on the random inputs tried. The negative tests catch it directly: they construct a chunk that the analysis is *required* to decline — a mixed-kind slot, an unassigned-slot read, a bare boolean result, a global aliased by a named array — and assert first that `native_lowerable` returns false (the chunk took the threaded path) and then that the result still matches the interpreter. The first assertion is the one that matters for this failure mode: it pins the *boundary* of what the native path accepts, not just the correctness of what it lowers. There is even a fuzzer that builds an integer expression with a trailing string concat specifically to force partial deopts, and asserts that more than a hundred of its four hundred programs *actually deopted* — turning “the deopt path gets exercised” into a checked property rather than an assumption. Together the positive and negative fuzzers pin both halves of the lower-what-you-can-prove discipline: the positive tests verify that what is lowered is correct, and the negative tests verify that what is not lowerable is correctly recognized and correctly handled by the fallback. A compiler is only as trustworthy as its knowledge of its own limits, and the negative tests are how the native path’s limits are kept honest.

Determinism, and why SplitMix64

The fuzzers all draw their randomness from the same source, an inline SplitMix64 generator seeded with a fixed per-test constant, and the choice is deliberate enough to deserve a word. SplitMix64 is a tiny, fast, well-distributed pseudo-random generator: a single `u64` state advanced by adding a fixed odd constant (`0x9E37_79B9_7F4A_7C15`, the golden-ratio increment) and then run through two multiply-xor-shift mixing rounds. It needs no external crate, no allocation, and no setup — a few lines inline in each test — which matters because a test dependency that pulled in a real RNG would be heavier than the test it serves. But the deeper reason is *determinism*. Each fuzzer seeds its generator with a distinct fixed constant, so the sequence of random chunks it produces is identical on every run, on every machine. When a fuzzer fails, it fails on the same chunk every time, and the failing chunk is reconstructible from the seed and the iteration count — there is no “it failed once on CI and I cannot reproduce it” ghost. A differential fuzzer that was nondeterministic would be nearly useless, because a failure you cannot reproduce is a failure you cannot fix; the fixed seed is what turns a random test into a reproducible one. The distinct-seed-per-category discipline also means the categories explore different regions of the chunk space without overlapping by accident, so the four hundred arithmetic programs and the four hundred bitwise programs are genuinely four hundred each rather than the same programs twice.

The harness is three steps

The differential harness at the center of every fuzzer is small enough to state in full, and its smallness is the point: `assert_native_matches_interp` runs a chunk both ways and compares the results exactly. It interprets the chunk with `VM::run`, compiles and runs it with `run_chunk_native`, and asserts the two

`VMResults` agree — `Ok` against `Ok` with equal values, `Halted` against `Halted`, `Error` against `Error`. There is no expected-value table, no golden file, no hand-specified answer, because the interpreter *is* the expected value: whatever it computes is correct by definition, and the test passes when the compiler agrees with it. This is what lets a single small harness verify hundreds of thousands of random chunks across all the categories — it does not need to know what any chunk should produce, only that the two ways of running it produce the same thing. The harness is the operational form of the single-source-of-truth principle: the interpreter is the specification, the compiler is an implementation of it, and the test is “do they agree.” Everything else in the differential-correctness story — the categories, the counts, the negative space, the deterministic seeds — is scaffolding around this one comparison, which is the whole verification strategy distilled to an equality check between a slow correct answer and a fast one.

Chapter 33 — How Fast, Really

The point of all this machinery is speed, and the claims should be measured rather than asserted. fusevm's benchmarks run through criterion on Apple M-series hardware, and they are worth reading in groups, because each group isolates a different part of the machine.

The first group is the interpreter and its fused superinstructions, and it shows what dispatch costs. A million-iteration sum loop spelled in *unfused* ops runs in thirty-one milliseconds; the *same* loop as a single `AccumSumLoop` runs in a hundred and forty-two nanoseconds — a factor of more than two hundred thousand, which is not a typo but a direct measure of what removing a quarter-million-times-a dispatch does. An iterative Fibonacci of thirty-five is 2.7 microseconds; a ten-thousand-iteration string build through `ConcatConstLoop` is under twelve microseconds; and the raw dispatch floor — a million `Nops` — is eight hundred and nineteen microseconds, 1.22 billion operations per second. The fused superinstructions are not a micro-optimization; they are the difference between a loop that interprets and a loop that does not.

The second group is the linear JIT against the interpreter and against hand-written Rust, on slot-based inputs chosen so nothing constant-folds: twenty-nine times faster than the interpreter on mixed arithmetic, fifty-one times on bitwise, thirteen on float, and consistently about 1.8 times slower than LLVM `-O3` — the gap a method-at-a-time JIT pays for compiling in microseconds. The third group is the block JIT on real control flow: the unfused million-iteration loop at three hundred and fifteen microseconds (ninety-five times faster than interpreted), the nested loop thirty-six times faster. And the fourth is the tracing-JIT VM at steady state, where a hundred-thousand-iteration counter loop is eighty-seven times faster than the interpreter and a branchy loop a hundred and forty-eight times, both within one percent of the block JIT's direct floor — the remaining gap being only the constant hundred-and-fifty-to-two-hundred-nanosecond VM construction and slot copy per call, not the native execution, which is identical.

Reading the numbers honestly

Two disciplines keep these figures honest. The first is the choice of slot-based inputs throughout the JIT comparisons: a benchmark whose answer the compiler can fold to a constant measures the folder, not the loop, so the inputs are arranged so the compiler genuinely has to iterate. The second is the apples-to-apples framing of the JIT-versus-interpreter rows — they are cached-JIT against interpret, the real “run a hot chunk” workload, with the $O(1)$ op-hash cache lookup (about twenty-four nanoseconds) folded in. The project is also candid about where it does *not* win: the `VMPool` section openly measures the pool as *slower* than `VM::new` for tiny chunks, because `reset` does more bookkeeping than `new` skips, and recommends the pool only for non-trivial chunk shapes. A benchmark suite that flatters the system it measures is worse than none; this one is built to report the losses alongside the wins.

The classic-algorithm numbers, read closely

The classic-algorithm benchmarks are worth reading individually, because each one isolates a different cost and the spread tells you where the machine spends its time. The iterative Fibonacci of thirty-five at 2.7 microseconds is essentially a measure of the fused-loop fast path, since the inner loop is a counted accumulation. The recursive Fibonacci of twenty at 1.28 milliseconds across twenty-one thousand calls — about sixty nanoseconds per call — is a measure of *call overhead* specifically: pushing a frame, binding arguments, returning a value, the parts of the machine the fused loops bypass entirely. Ackermann of three and four, ten thousand calls in seven hundred and seventy-four microseconds, stresses the same call machinery

harder with deeper recursion. The two sum-loop numbers are the dramatic pair: the same million-iteration sum is thirty-one milliseconds spelled in unfused ops and a hundred and forty-two nanoseconds as a single `AccumSumLoop`, and the gap between them is the entire argument for fused superinstructions reduced to two measurements. The raw dispatch floor — a million `Nops` at eight hundred and nineteen microseconds, 1.22 billion ops per second — is the denominator everything else is measured against: it is how fast the interpreter can do *nothing*, and it tells you that for any real op the cost is dominated by the op's work, not the dispatch, which is exactly why removing dispatch (fusion, then compilation) is where the speedups live.

What the JIT-versus-native gap actually measures

The recurring “about 1.8 times slower than native Rust” figure deserves one careful paragraph, because it is easy to misread as a shortcoming and is better read as a ceiling. When the linear JIT runs mixed slot arithmetic in seventy-five nanoseconds against hand-written Rust's forty-two, the 1.8 ratio is not the cost of the bytecode being dynamically typed — the JIT has already resolved all the types and is emitting the same `iadds` and `fmu1s` the Rust compiler would. It is the cost of Cranelift being a fast, modest optimizer rather than LLVM at `-O3`: Cranelift does less instruction scheduling, less aggressive register allocation, fewer peephole rewrites, because it is compiling in microseconds. The gap is, in other words, the visible price of the Cranelift-over-LLVM choice, and 1.8 times is a remarkably small price for a compiler that runs while the program waits. The tracing-JIT numbers make the same point from the other direction: at a hundred thousand iterations the tracing-JIT VM is within one percent of the block JIT's direct floor, and the entire remaining gap is the constant hundred-and-fifty-to-two-hundred-nanosecond cost of constructing the VM and copying slots in and out per `run` call — not the native execution, which is byte-for-byte identical. Read together, the benchmark groups say something precise: dispatch is the interpreter's cost and fusion removes it; type resolution is the dynamic language's cost and compilation removes it; and what remains is a small, honest constant factor over hand-written native code, paid for compiling in microseconds instead of seconds.

The tables, for reference

For the reader who wants the figures in one place rather than in prose, here are the four benchmark groups as the criterion harness reports them on Apple M-series hardware. The classic-algorithm group, exercising the interpreter and its fused superinstructions:

Benchmark	Time	Ops/sec
<code>fib_iterative(35)</code>	2.7 μ s	374k
<code>fib_recursive(20)</code> — 21,891 calls	1.28 ms	783
<code>ackermann(3,4)</code> — 10,547 calls	774 μ s	1.3k
<code>sum(1..1M)</code> fused <code>AccumSumLoop</code>	142 ns	7.0M
<code>sum(1..1M)</code> unfused loop ops	31.0 ms	32
<code>nested_loop(100×100)</code>	352 μ s	2.8k
<code>dispatch_nop_1M</code> — raw dispatch	819 μ s	1.22 Gops/sec
<code>string_build(10k)</code> via <code>ConcatConstLoop</code>	11.9 μ s	84k

The linear-JIT group, on slot-based inputs that defeat constant folding, against the interpreter and against hand-written Rust:

Workload	Interpreter	JIT (cached)	Native Rust	JIT vs interp	JIT vs native
<code>slot_mixed × 100</code>	2.2 μ s	75 ns	42 ns	29× faster	1.8× slower

Workload	Interpreter	JIT (cached)	Native Rust	JIT vs interp	JIT vs native
<code>slot_bitwise × 200</code>	6.6 μs	130 ns	74 ns	51× faster	1.8× slower
<code>slot_float × 200</code>	3.1 μs	246 ns	137 ns	13× faster	1.8× slower

The block-JIT group, on real control flow:

Benchmark	Interpreter	Block JIT	Speedup
<code>sum(1..1M)</code> unfused loop	30.0 ms	315 μs	95×
<code>nested_loop(100×100)</code>	340 μs	9.5 μs	36×

And the tracing-JIT group, at steady state with traces primed, measuring the whole `VM::run` path against the interpreter and against the block JIT's direct floor:

Benchmark	Iterations	Interpreter	Block JIT (direct)	Tracing-JIT VM	vs Interp	vs Block
<code>counter_loop</code>	1,000	24.0 μs	309 ns	474 ns	51×	1.53× slower
<code>counter_loop</code>	10,000	236.1 μs	2.69 μs	2.79 μs	84×	1.04× slower
<code>counter_loop</code>	100,000	2,354 μs	26.71 μs	26.95 μs	87×	1.01× slower
<code>loop_with_branch</code>	1,000	40.2 μs	300 ns	474 ns	85×	1.58× slower
<code>loop_with_branch</code>	10,000	410.3 μs	2.68 μs	2.83 μs	145×	1.06× slower
<code>loop_with_branch</code>	100,000	3,942 μs	26.46 μs	26.64 μs	148×	1.01× slower

The trend across the tracing rows is the one worth carrying away: as the iteration count grows, the per-call VM-construction overhead amortizes, and the tracing-JIT VM converges to within one percent of the block JIT's floor — because at a hundred thousand iterations the native loop body dominates, and the native loop body is identical between the two. The small constant gap at a thousand iterations is the VM construction and slot copy, not the execution.

The VMPool numbers, and the honesty in them

One more table belongs here because it is the project's clearest act of benchmarking honesty — the `VMPool` measurements, which report a *loss*:

Pattern	Time/call
<code>VM::new(chunk)</code> per call	130 ns
<code>pool.acquire(chunk)</code> per call	163 ns

On a three-op chunk, the pool is slower than constructing a fresh VM each time, because `reset` does more bookkeeping — dropping the old chunk, clearing globals, zeroing the deopt buffer — than `VM::new` skips. The README states the conclusion plainly: the pool helps for “multi-chunk evaluation loops with non-trivial chunk shapes,” where large name pools, many slots, or a warmed tracing JIT make the preserved capacities worth the reset, and for “uniform tight loops, pure `VM::new` is fine.” A benchmark table that shows the system’s own feature losing on the simple case is the strongest possible evidence that the other tables are not cherry-picked: a project willing to publish where its pooling does not pay is a project reporting measurements rather than marketing them.

Tracking improvements over time

A benchmark is a snapshot; the harder discipline is keeping the numbers from quietly regressing as the machine changes, and fusevm uses criterion’s baseline mechanism for exactly that. The workflow is to save a named baseline before a change — `cargo bench --bench vm_bench -- --save-baseline before` — make the change, and then compare against it — `cargo bench --bench vm_bench -- --baseline before` — so a regression shows up as a percentage delta against the saved numbers rather than having to be eyeballed against memory. criterion’s statistical machinery (mean over many runs, with warm-ups and outlier detection) is what makes these comparisons trustworthy rather than noise: a one-percent change on a microsecond benchmark is within measurement error, and criterion reports the confidence interval so a real regression is distinguishable from jitter. The benches themselves are organized by what they isolate — `vm_bench` for interpreter and fused-op throughput, `jit_vs_interp` and `jit_crossover` for the JIT tiers, `jit_trace` for tracing, `jit_disk_cache` for the cache, and `aot_native` for the AOT path — each gated behind the feature it needs, so a contributor can measure the part they changed without building the parts they did not. This is the benchmarking equivalent of the differential fuzzers: just as the fuzzers keep correctness from regressing by comparing every change against the interpreter, the saved baselines keep performance from regressing by comparing every change against the last known-good numbers. A machine whose whole pitch is speed needs both guards — one for “is it still correct,” one for “is it still fast” — and the criterion baselines are the second.

What the dispatch floor really measures

The single most informative number in the whole benchmark suite is also the least glamorous: the raw dispatch floor, a million `Nops` in eight hundred and nineteen microseconds, 1.22 billion operations per second. It is worth dwelling on because it is the denominator that gives every other number meaning. A `Nop` does nothing — it is in the instruction set precisely as a cycle-counter and a branch sentinel — so a loop of a million of them measures *only* the cost of the dispatch mechanism: fetch the op, advance the cursor, match on it, find it is a `Nop`, continue. That cost, roughly eight hundred picoseconds per op, is the irreducible per-opcode overhead of the interpreter, and it sets the scale for everything. It tells you that an opcode doing real work — an `Add`, a `GetSlot` — costs its dispatch (about eight hundred picoseconds) plus its work, and that for cheap ops the dispatch is a large fraction of the total. This is *why* fusion and compilation produce the speedups they do: a fused superinstruction removes six or seven dispatches from a loop tail, collapsing six or seven eight-hundred-picosecond charges into one; a compiled loop removes the dispatch entirely, leaving only the work. Read the unfused-versus-fused sum loop — thirty-one milliseconds versus a hundred and forty-two nanoseconds — against the dispatch floor and the arithmetic checks out: the unfused loop pays millions of dispatches, the fused one pays one per iteration, and the floor is the per-dispatch price that turns “millions” into thirty-one milliseconds. The dispatch floor is not a brag; it is the unit in which every other speedup is

denominated, and a reader who internalizes it can read the rest of the benchmark suite as arithmetic rather than as a list of claims.

Apples to apples, deliberately

The JIT-versus-interpreter comparisons make a methodological choice that is worth defending explicitly, because it is the difference between a benchmark that informs and one that flatters. The comparison is *cached-JIT against interpret* — the compiled-code path with its native function already in the cache, against the interpreter running the same chunk — and the JIT rows fold in the $O(1)$ op-hash cache lookup (about twenty-four nanoseconds) that decides whether the chunk was already compiled. This is the honest “run a hot chunk” workload: a chunk that has run before is recognized and its native code reused, which is exactly what happens in a real program where a loop runs many times or a function is called repeatedly. The alternative framings would each mislead. Comparing *cold-compile-plus-run* against *interpret* would charge the JIT a one-time codegen cost on every measurement, understating its steady-state speed for code that runs more than once — which is most code worth compiling. Comparing the JIT’s native execution alone, with the cache lookup excluded, would overstate it by hiding the dispatch-to-the-cache cost a real call pays. The chosen framing — cached lookup plus native execution, against interpretation — is the one that matches how the JIT is actually used, and the slot-based inputs ensure the compiler cannot cheat by folding the answer to a constant. The result is a set of numbers that say what they appear to say: this is how much faster a hot chunk runs compiled than interpreted, lookup included, on inputs the compiler genuinely had to compute. A benchmark’s framing is a claim about what it measures, and this suite’s framing claims to measure the real hot-chunk workload — which is why its losses, like the [VMPool](#) table, are reported alongside its wins rather than framed away.

The constant factor, and where it goes

The most-quoted result — the JIT and the AOT native path landing within a small constant factor of hand-written Rust — deserves a closing decomposition, because understanding *where* that factor goes is understanding what a fast bytecode VM can and cannot achieve. The factor splits into three parts. The first is the optimizer gap: Cranelift is about 1.8 times off LLVM -03 on real computation, the visible price of compiling in microseconds, and this part is irreducible without a slower compiler the JIT could not afford. The second is the call overhead: each `VM: :run` invocation pays a constant hundred-and-fifty-to-two-hundred nanoseconds to construct the VM and copy slots in and out, which is why the tracing-JIT VM is within one percent of the block JIT’s direct floor at a hundred thousand iterations (the constant amortizes) but 1.5 times off at a thousand (the constant dominates). The third is, increasingly, nothing — the native loop body itself executes at genuine register speed, byte-for-byte identical to what the block JIT produces directly, because once the slots are in registers and the dispatch is gone, a compiled fusevm loop *is* the same machine instructions hand-written Rust would run. Read the three parts together and the headline becomes precise: a hot fusevm loop runs at register speed, with a one-time per-call constant for VM setup and a steady ~1.8x optimizer gap, and the “constant factor over native Rust” is the sum of those two overheads applied to a loop body that is already native. The factor is not the cost of being a bytecode VM — the bytecode is gone by the time the loop runs — but the cost of a fast compiler and a per-call boundary, both of which a system that compiles while the program waits chooses deliberately, and both of which shrink to irrelevance as the loop runs longer. That is the honest shape of the speed claim: not “as fast as native” without qualification, but “native loop body, plus a fast-compiler gap, plus a per-call constant” — which, at the iteration counts where compilation pays for itself at all, rounds to native.

Chapter 34 — From Chunk to Standalone Binary

The native and threaded paths produce a Cranelift function; the last step turns that into something you can ship. `aot::compile_object(&chunk, path)` emits a relocatable object file with three exported pieces. The first is the native entry function, exported as the symbol `fusevm_aot_entry` — an `extern "C" fn(*mut VM) -> i64` built by `build_entry`. The second is the chunk itself, serialized with `bincode` and exported as the data symbol `fusevm_aot_chunk_blob`. The third is the blob's length, exported as `fusevm_aot_chunk_len`, a little-endian `u64`. A relocatable object with the compiled program as code, the chunk as data, and the length to find it: that is the entire artifact.

A front end's AOT `main` links against this object, and at runtime the entry point `fusevm_aot_run_embedded` ties it together: it deserializes the embedded chunk blob, constructs a `VM`, calls the front end's `fusevm_aot_register_builtins` (the one symbol the front end must supply, so its builtins are present), runs the native `fusevm_aot_entry`, and maps the `VMResult` to a process exit code — an `Ok(Int n)` exits with `n`, an `Ok(Status s)` with `s`, an `Error` prints to `stderr` and exits one. The result is a self-contained native executable: no interpreter process, no JIT warm-up, no toolchain on the target. The chunk travels inside the binary as data; the entry function is the compiled program. (On macOS the link needs `-framework CoreFoundation`, the one platform footnote.)

Two ISAs, one codegen

The compiler builds the Cranelift target ISA twice with different settings, and the difference is exactly one flag. The in-memory JIT path — `run_chunk_native`, used to validate the compiler end to end by compiling a fresh chunk, running it on a fresh `VM`, and checking the result — uses a `host_isa` with `is_pic` false, position-dependent code finalized to executable memory and called through a transmuted function pointer. The object path uses an `object_isa` with `is_pic` true, as a relocatable `.o` must. But the `codegen` — `build_entry`, generic over the Cranelift `Module` — is identical for both. The same function that the test harness JIT-compiled and runs in memory is the one written into the shipped object. There is no second implementation to drift, which is the single-source-of-truth discipline that governs the opcode semantics, applied one final time to the compiler itself, and verified by a test that shells out to `nm -g` to confirm all three symbols are exported from the emitted object.

The chunk travels as data

The single most distinctive thing about a fusevm AOT binary is the form the program takes inside it, and it is worth stating plainly because it inverts the usual picture of a compiler. In a conventional ahead-of-time compiler, the source program *becomes* the machine code; there is nothing left of the original but the instructions it lowered to. In a fusevm AOT binary, the chunk is still *present*, serialized with `bincode` and exported as the data symbol `fusevm_aot_chunk_blob`, alongside the native entry function that drives it. At startup `fusevm_aot_run_embedded` deserializes the blob back into a `Chunk`, builds a `VM`, and the native entry runs against that reconstructed `VM`. Why carry the chunk at all, when the entry function already encodes the program's behavior? Because the native entry, in the threaded path and at the boundary, still calls back into `exec_op`, which needs the chunk's ops, constants, and names to interpret the opcodes it runs — the very same single-source-of-truth that keeps the compiled and interpreted semantics from forking means the runtime still

needs the chunk on hand. The result is a binary that is half compiled program and half embedded data, and the duality is not waste: it is what lets the native path lower the scalar core to instructions while leaving the heap and host ops to run through the interpreter against the embedded chunk, all inside one standalone executable. The chunk travels as data because the machine that runs it is still, at the edges, the same interpreter — and shipping the chunk is what lets a single binary be both the compiler's output and the interpreter's input.

What the front end must supply

The AOT binary is not quite self-contained on its own; it needs one symbol from the front end, and the shape of that requirement is the final illustration of the machine's division of labor. The emitted object exports the entry, the chunk blob, and the length, and it *imports* the runtime shims (which the fusevm static library provides) plus exactly one front-end symbol: `fusevm_aot_register_builtins`. This is the function that installs the front end's builtins and extension handlers onto the VM before the native entry runs, and it is the front end's sole obligation to the AOT pipeline. Everything else — the value model, the opcode semantics, the compiler, the runtime — comes from fusevm; the front end contributes its lexer and lowering at build time (producing the chunk) and its builtins at link time (through this one symbol). The static-library crate type in `Cargo.toml`, `staticlib`, exists precisely so the AOT object can link against `libfusevm.a` and pull in the runtime, and the `lib` crate type beside it keeps the normal rlib for front ends that depend on fusevm as an ordinary crate. So a standalone fusevm program is three things linked together: the compiled chunk-driver from the AOT object, the fusevm runtime from the static library, and the front end's builtins from one registration function — and the seam between them is exactly the seam this whole book has been describing, the line between the language-agnostic machine and the language that targets it, drawn one last time at the linker.

Position-independence, the one real difference

The claim that the in-process JIT and the shipped object share identical codegen is true, and the single exception — the position-independence flag — is worth understanding because it is the only place the two backends genuinely differ. Native code can be written two ways with respect to addresses: *position-dependent*, where the code assumes it lives at a fixed address and bakes absolute addresses into its instructions, and *position-independent* (PIC), where the code computes addresses relative to itself and can be loaded anywhere. The in-process JIT builds with `is_pic` false: the code is compiled, finalized to a known address in this process's memory, and called immediately, so position-dependence is fine and slightly faster. The object path builds with `is_pic` true, because a relocatable `.o` will be placed at an address the linker chooses, possibly loaded into a position-independent executable, so its code must not assume where it lives. That one flag, set on the target ISA, is the whole difference between `host_isa` and `object_isa`; the `build_entry` codegen above it is byte-for-byte the same, because the difference is in how Cranelift *emits* the addresses, not in what the compiler *asks for*. This is why validating the in-process path validates the shipped path: the only thing the object path does that the JIT path does not is emit relative addresses instead of absolute ones, a Cranelift-internal concern that the fusevm codegen never touches. One generic function, one set of CLIF instructions, two address modes — and the address mode is the only thing that knows whether the result will run here or ship in a binary.

The exit code is the result, boxed

The last detail of the standalone binary is how a `Chunk`'s result becomes a process exit code, because it closes the loop from bytecode to the shell that ran the binary. `fusevm_aot_run_embedded` runs the native entry, gets back a `VMResult`, and maps it: an `Ok(Value::Int(n))` exits with `n` truncated to the process exit range, an `Ok(Value::Status(s))` exits with `s` (the natural mapping, since a `Status` is an exit code), any other `Ok` exits with zero, and an `Error` prints its message to stderr and exits with one. This mapping is small but it is where the value model meets the operating system, and the choice to honor `Value::Status` directly is the same truthiness-aware decision that runs through the whole machine: a shell program whose last command failed leaves a `Status` with a non-zero code, and the binary exits with that code, so a fusevm-compiled shell

script sets `$?` in the calling shell exactly as the script intended. The example binary the crate ships makes this concrete — its chunk computes `6 * 7` and the linked binary exits forty-two — and the mapping is what makes that exit code meaningful rather than incidental. From a `ChunkBuilder` emitting three opcodes, through `analyze_native` and `build_entry` and `compile_object`, to a linked binary that exits with the computed value: the whole AOT pipeline is, in the end, a way of turning a chunk's result into a process's exit code, and the `VMResult`-to-exit-code mapping is its final translation.

Chapter 35 – Reading a Disassembly

The five movements so far have described the machine; the chapters that follow walk it, opcode by opcode and program by program, in the disassembled bytecode itself. This is the reference movement — the part of a VM companion that a reader returns to with a specific opcode in mind — and it is deliberately concrete: every opcode in the next eleven chapters is shown with its exact stack effect and, where it clarifies, a small real bytecode listing; every compiler tier in the four after that is shown lowering one specific loop; and the frontends are shown driving the protocols against real op sequences. Nothing here is new machinery — it is the machinery of the first five movements, seen from the ground, in the form a front end actually emits and the VM actually runs.

To read what follows you need the notation, so this chapter fixes it. A chunk is, recall, a flat `Vec<Op>` plus a constant pool, a name pool, and the supporting tables of Chapter 2. A *disassembly* is that op vector printed one instruction per line, with the instruction's index, its opcode, its inline operands, and — the part that makes a disassembly readable — a running comment showing the operand stack after the instruction executes. The convention this book uses is:

idx	op	operand	; stack after
0	LoadInt	40	; [40]
1	LoadInt	2	; [40, 2]
2	Add		; [42]

The leftmost column is the instruction pointer — the `usize` the VM's `ip` walks — so a `Jump(2)` means “set `ip` to 2,” landing on the instruction labelled 2. The operand column holds the inline payload: an `i64` for `LoadInt`, a `u16` pool index for `LoadConst`, a `usize` target for `Jump`. The stack comment is the single most useful thing in a listing, because a stack machine's whole behavior is the evolution of that stack, and reading the comments down the page is reading the program execute.

From source to builder to ops

A disassembly is the *output* of a front end; the *input* is source, and the bridge is the `ChunkBuilder` of Chapter 2. It is worth seeing the three forms side by side once, because the rest of the movement moves freely between them. The source expression `40 + 2`, the builder calls that lower it, and the disassembly they produce:

```
// builder calls
let mut b = ChunkBuilder::new();
b.emit(Op::LoadInt(40), 1); // -> index 0
b.emit(Op::LoadInt(2), 1); // -> index 1
b.emit(Op::Add, 1);        // -> index 2
let chunk = b.build();
```

```
// disassembly of chunk.ops
0 LoadInt 40 ; [40]
1 LoadInt 2 ; [40, 2]
2 Add ; [42]
```

Each `emit` appends one op and returns its index; the indices in the disassembly are exactly those return values; and the 1 second argument to `emit` is the source line, recorded in the chunk's parallel `lines` array for error reporting and never shown in a disassembly because it does not affect execution. When a later chapter

says “the front end emits this sequence,” it means a series of `emit` calls producing exactly the op vector the disassembly shows.

Pools, names, and how a listing references them

Two of a chunk’s tables show up in disassemblies as indices, and the listings in this movement annotate them. The constant pool holds anything too large to carry inline — a string, a regex, a big float — and `LoadConst(i)` pushes `constants[i]`. A disassembly shows both the index and, in a comment, the pooled value:

```
0 LoadConst 0 ; push constants[0] = "hello" -> ["hello"]
```

The name pool holds variable and function names, interned and deduplicated, and the variable, slot, and call opcodes carry a `u16` index into it. `GetVar(3)` reads the global named `names[3]`; a listing annotates the name:

```
0 GetVar 3 ; read global names[3] = "count" -> [<count>]
```

The reason the runtime never sees a name string — only an index — is the whole point of the name pool, made in Chapter 2: a variable access is a vector index, not a hash lookup, because the front end resolved the name to a number at compile time. In a disassembly the index is the truth and the name is a courtesy annotation; the VM runs the index.

Reading a backward jump

One last piece of notation, because it recurs in every loop in the movement: a *backward* jump, whose target index is less than the jumping instruction’s own index. Backward jumps are loop bottoms, and a disassembly makes the loop visible as an arrow back up the page:

```
2 GetSlot      0 ; [i]
3 LoadInt     10 ; [i, 10]
4 NumLt       ; [i<10]
5 JumpIfFalse  9 ; []      (forward exit, patched)
6 PreIncSlotVoid 0 ; []      i += 1
7 Jump        2 ; []      —> back to index 2
8 Nop         ;          (unreachable filler)
9 ...         ;          loop exit lands here
```

The `Jump 2` at index 7 sends control back to index 2, the loop header, and the `JumpIfFalse 9` at index 5 leaves the loop when the condition fails. That backward `Jump 2` is the structural signal the whole machine reads — the tracing JIT arms its recorder there, the basic-block analyzer marks index 2 a leader, and a front end’s peephole recognizes the shape and fuses it. Every loop disassembly in this movement has exactly this skeleton: a header, a test with a patched forward exit, a body, and a backward jump. With the notation fixed, the walk can begin.

Chapter 36 – Loads and Stack, Op by Op

The constants-and-stack section is twelve opcodes — the `Nop`, the six loads, and the five stack shuffles — and it is the right place to start a per-opcode walk because these are the opcodes with the simplest stack effects and the ones every other opcode's example depends on. Each entry below gives the opcode, its inline operand if any, its stack effect written as *before* → *after*, and the semantics; the ones whose behavior is not obvious carry a small listing.

Nop – the no-op

`Nop` takes no operand and has stack effect `... → ...` — it touches nothing. Its two jobs, from the source comment, are cycle-counting (a loop of `Nops` measures pure dispatch, the 1.22-Gops/sec floor) and serving as a branch sentinel, a stable instruction slot a single-pass compiler can emit before it knows what will occupy it. In a disassembly a `Nop` is most often the unreachable filler after a backward jump, as in the loop skeleton of the previous chapter.

The six loads

`LoadInt(i64)` has effect `... → ..., n`: it pushes its inline integer. The operand is the full `i64`, carried inline because it fits the twenty-four-byte opcode budget.

`LoadFloat(f64)` is the analogue for floats, `... → ..., f`, pushing its inline `f64`.

`LoadConst(u16)` has effect `... → ..., constants[i]`: it pushes the pooled constant at the index. This is how a string, a regex, or any value too large to carry inline reaches the stack.

`LoadTrue`, `LoadFalse`, and `LoadUndef` take no operand and push `Value::Bool(true)`, `Value::Bool(false)`, and `Value::Undef` respectively — the three constant singletons given their own one-byte opcodes because a boolean and the absence of a value are common enough to deserve them. A program that needs a literal `true` emits `LoadTrue`, not a `LoadConst` into a pool slot.

A listing that uses four of the six:

```
0 LoadInt    7      ; [7]
1 LoadFloat  2.5    ; [7, 2.5]
2 LoadConst  0      ; push constants[0] = "x"  -> [7, 2.5, "x"]
3 LoadUndef           ; [7, 2.5, "x", undef]
```

The five shuffles

`Pop` has effect `..., a → ...`: it discards the top, used when a statement's result is unused.

`Dup` has effect `..., a → ..., a, a`: it duplicates the top, the classic need being a value used twice, as in `x = y = 0` where the zero is duplicated so both assignments get it.

Dup2 has effect ..., a, b → ..., a, b, a, b: it duplicates the top *two*, which a compound indexed assignment needs — `a[i] += 1` wants the array-and-index pair preserved for the store and consumed for the load.

Swap has effect ..., a, b → ..., b, a: it exchanges the top two, fixing operand order for a non-commutative op whose source order does not match its stack order.

Rot has effect ..., a, b, c → ..., b, c, a: it rotates the top three, the rarest shuffle, for the occasional three-operand rearrangement.

A listing showing why **Swap** exists — subtraction is not commutative, so if the front end happened to push the operands in the wrong order, a **Swap** fixes it before **Sub**:

```
0 LoadInt 3      ; [3]
1 LoadInt 10     ; [3, 10]
2 Swap          ; [10, 3]
3 Sub           ; [7]      computes 10 - 3, not 3 - 10
```

The deliberate sparseness of this set — no “pick the third element,” no “roll four” — is the judgment, made in the loads-and-stack chapter, that clean post-order emission makes deep stack manipulation unnecessary: a front end that finds itself wanting one has usually mis-ordered its tree walk, and the fix belongs there, not in a new opcode.

Chapter 37 – Variables and Slots, Op by Op

The variable-and-slot section is seven opcodes, and walking them one by one makes concrete the three-namespace storage model of Chapter 4 — globals reached by name index, slots reached by frame index, and the slot-array fast paths. Every opcode here carries a `u16`, and the `u16` means different things in the two namespaces: a name-pool index for the global ops, a frame-slot index for the slot ops.

The global ops

`GetVar(u16)` has effect `... → ..., globals[name]`: it reads the global whose name is at the given name-pool index and pushes it. The name-pool index is resolved to a global slot once, at first use; thereafter the access is a vector index.

`SetVar(u16)` has effect `..., v → ...`: it pops the top and stores it into the named global. A listing of `count = 5`, with `count` at name index 0:

```
0 LoadInt 5 ; [5]
1 SetVar 0 ; []      globals["count"] = 5
```

`DeclareVar(u16)` has effect `... → ...` on the stack but introduces a *fresh* binding in the current scope — the declaration-versus-assignment distinction of Chapter 6. A front end with lexical scoping emits `DeclareVar` where an inner `my $x` should shadow an outer one rather than overwrite it; the core does not implement the scope rule, it provides the opcode so the front end can mark where a binding comes into existence.

The slot ops

`GetSlot(u16)` has effect `... → ..., slots[i]`: it reads the *i*-th local of the current frame and pushes it. Reading an unwritten slot yields `Undef`, the detail the compilers must honor through definite assignment.

`SetSlot(u16)` has effect `..., v → ...`: it pops the top and stores it into slot *i*. A counted loop's `i = 0` initialization, with `i` in slot 0:

```
0 LoadInt 0 ; [0]
1 SetSlot 0 ; []      slots[0] = 0
```

The difference between a slot and a global is not the indexing — both are numeric — but the locality: a slot lives in the frame, gets fresh storage on every call, and is the storage the JIT and AOT compilers lift into machine registers. When a compiler chapter says a loop “runs entirely in registers,” it means its induction variable was a slot, and `GetSlot/SetSlot` became `use_var/def_var` on a Cranelift `Variable`.

The slot-array fast paths

`SlotArrayGet(u16)` has effect `..., index → ..., value`: it takes an index off the stack, reaches the array living in slot *i*, and pushes the indexed element — indexing *into* a slot-resident array without first loading the array onto the stack.

`SlotArraySet(u16)` has effect ..., `value`, `index` → ...: it stores a value at an index into the array in slot `i`, again without materializing the array on the stack.

These two are a fusion in miniature, the same bias toward the hot path the fused superinstructions raise to whole loops. Indexing a local array in a loop is common enough that saving the intermediate `GetSlot` is worth a dedicated opcode. A listing of `a[i] = v` where `a` is in slot 2, `i` in slot 0, `v` in slot 1:

```
0  GetSlot      1  ; [v]          push v
1  GetSlot      0  ; [v, i]       push i
2  SlotArraySet 2  ; []          a[i] = v, a lives in slot 2
```

Compared to the unfused form — load the array, load the index, load the value, `ArraySet` — the slot-array op saves the explicit array load every iteration, and the analysis in the compilers can reason about a slot-resident array more easily than one that flows through the stack. The variable-and-slot section is small, seven opcodes, but it is where fusevm's storage model becomes operational, and the choice a front end makes here — slot versus global for each variable — is the largest lever it has over how fast the resulting bytecode runs.

Chapter 38 – Arrays and Hashes, Op by Op

The arrays-and-hashes section is twenty opcodes, the largest of the universal sections, and walking it one by one makes concrete the in-place mutation discipline that keeps aggregate operations linear. The opcodes split into the *named* accessors — which carry a `u16` name-pool index identifying the aggregate variable — and the constructors `MakeArray` and `MakeHash`. Throughout, `A` and `H` denote the array and hash named by the opcode's index.

The array movers and accessors

`GetArray(u16)`, `SetArray(u16)`, and `DeclareArray(u16)` move and declare a whole array variable, mirroring `GetVar/SetVar/DeclareVar` for the array namespace: `GetArray` pushes the array value, `SetArray` pops and stores it, `DeclareArray` introduces a fresh array binding.

The element accessors carry the in-place property that matters. `ArrayGet(a)` has effect `..., index → ..., value`: pop an index, push `A[index]`. `ArraySet(a)` has effect `..., value, index → ...`: pop both and store, mutating `A` in the globals vector directly. `ArrayPush(a)` has effect `..., value → ...`: append to `A` in place, pushing nothing. `ArrayPop(a)` has effect `... → ..., value`: remove and return the last element. `ArrayShift(a)` has effect `... → ..., value`: remove and return the first. `ArrayLen(a)` has effect `... → ..., length`: push the element count.

The crucial reading is the “pushes nothing” on `ArrayPush` and `ArraySet`: they mutate the named array where it lives and never put the whole aggregate on the stack, which is why a loop of pushes is honestly linear. A listing building an array by pushing in a loop, with the array at name index 0 and the counter in slot 0:

```
0 GetSlot    0    ; [i]           push i
1 ArrayPush  0    ; []           A.push(i), A is names[0]
2 PreIncSlotVoid 0 ; []           i += 1
3 ...                (loop control)
```

There is no array on the stack at any point — only the value being pushed — so the million-element version copies nothing per iteration, which is exactly what lets the `PushIntRangeLoop` fused superinstruction lower it to a tight native loop.

`MakeArray(u16)` is the constructor, effect `..., v0, v1, ..., v{n-1} → ..., [v0..v{n-1}]`: pop `n` values and push them as one `Array`. This is how an array literal `[a, b, c]` is built — push the three elements, then `MakeArray(3)`:

```
0 LoadInt   1    ; [1]
1 LoadInt   2    ; [1, 2]
2 LoadInt   3    ; [1, 2, 3]
3 MakeArray  3    ; [[1, 2, 3]]
```

The hash movers and accessors

The hash opcodes mirror the array ones. `GetHash(u16)`, `SetHash(u16)`, `DeclareHash(u16)` move and declare a whole hash. `HashGet(h)` has effect `..., key → ..., value`: pop a key, push `H[key]`. `HashSet(h)` has effect `..., value, key → ...`: pop both and store in place. `HashDelete(h)` has effect `..., key → ..., deleted`: remove `H[key]` and push what was removed. `HashExists(h)` has effect `..., key → ..., bool`: push whether the key is present. `HashKeys(h)` has effect `... → ..., array` and `HashValues(h)` has effect `... → ..., array`: push the keys or values as an array.

`MakeHash(u16)` is the constructor, effect `..., k0, v0, ..., k{n-1}, v{n-1} → ..., {...}`: pop `n` key-value pairs and push a hash. Because a `Hash` is a `HashMap<String, Value>`, every key is a string — the front end stringifies the subscript through `as_str_cow` before it reaches the opcode, which is why `$h{1}` and `$h{"1"}` index the same slot. A listing setting `h["k"] = 9` with `h` at name index 0:

```
0 LoadInt    9    ; [9]           push value
1 LoadConst  0    ; [9, "k"]       push key constants[0] = "k"
2 HashSet    0    ; []           H["k"] = 9
```

The whole section is twenty opcodes and one discipline: the named aggregate is identified by the opcode's index and never travels on the stack for a mutation, so the read-only accessors borrow from the globals vector and the mutating ones write to it in place — the difference, as Chapter 9 put it, between a linear and a quadratic aggregate model.

Chapter 39 – Arithmetic and String, Op by Op

The arithmetic section is nine opcodes and the string section three, and walking them makes operational the “both `Int`, else float” promotion rule and the borrowed-string-view discipline. Every binary arithmetic op has effect `..., a, b → ..., result`; the unary ones `..., a → ..., result`.

The binary four and their fast path

`Add`, `Sub`, `Mul` share the integer fast path `arith_int_fast`: each peeks the top two values, and when both are `Int`, applies the wrapping integer operation and truncates the stack by one; otherwise both promote to `f64`. The wrapping is the specification — `i64::MAX` plus one is `i64::MIN`, not a panic — and the compilers emit wrapping `iadd/isub/imul` to match.

`Div` has effect `..., a, b → ..., result` but a result that is not always a number: it yields a `Float` for ordinary operands and `Undef` for division by zero, the shell-arithmetic semantic that makes `Div` the first opcode the AOT native path must guard with a deopt.

`Mod` yields the remainder, integer-by-zero giving zero (not a trap) and the float case an `fmod` through a runtime helper, because no machine has a floating modulo instruction.

`Pow` has effect `..., base, exp → ..., result`: wrapping integer for two integer operands, `f64::powf` otherwise, again through a helper.

A listing of `(2 + 3) * 4` showing the post-order shape:

```
0 LoadInt 2 ; [2]
1 LoadInt 3 ; [2, 3]
2 Add      ; [5]
3 LoadInt 4 ; [5, 4]
4 Mul      ; [20]
```

The unary and in-place trio

`Negate` has effect `..., a → ..., -a`: unary minus. `Inc` has effect `..., a → ..., a+1` and `Dec` `..., a → ..., a-1`: they adjust the top of stack by one. As Chapter 8 noted, `Inc` and `Dec` are the general stack-level primitives, and the slot increment super-ops (`PreIncSlot` and its family) are their specialization for the common case of incrementing a named local — a front end’s peephole emits the most specialized form the situation allows.

The three string ops

`Concat` has effect `..., a, b → ..., ab`: it glues two values as strings, running both through `as_str_cow` so two `Str` operands borrow their inner buffers rather than reallocating to read them — the borrowed coercion path earning its keep on the most common string operation.

`StringRepeat` has effect `..., s, n → ..., s*n`: Perl’s `x`, repeating a string `n` times.

`StringLen` has effect `..., s → ..., length`: the byte length of the top-of-stack string, byte count not character count, the choice every `length` builtin inherits.

A listing of `"ab" . "cd"`:

```
0  LoadConst 0      ; ["ab"]      constants[0] = "ab"
1  LoadConst 1      ; ["ab", "cd"] constants[1] = "cd"
2  Concat      ; ["abcd"]
```

The string section is only three opcodes because a front end's real string handling is its own, expressed through builtins and extension ops; the core provides the irreducible minimum — glue, repeat, measure — that every front end needs, and `Concat`'s borrowed coercion is what keeps the common case off the allocator.

Chapter 40 – Comparison, Logical, and Bitwise, Op by Op

This walk covers the comparison section (fourteen opcodes, split numeric and string) and the logical-and-bitwise section (nine). Every comparison and binary bitwise op has effect `..., a, b → ..., result`; the unary `LogNot` and `BitNot` have effect `..., a → ..., result`.

The numeric comparisons

`NumEq`, `NumNe`, `NumLt`, `NumGt`, `NumLe`, `NumGe` each compare their two operands numerically and push a boolean, following IEEE for floats so a NaN compares unequal to everything including itself. `cmp_int_fast` gives the all-integer case a hand-tuned path, the same peek-and-decide trick as `arith_int_fast`.

`Spaceship` has effect `..., a, b → ..., n` where `n` is `-1`, `0`, or `1`: the numeric three-way compare `<=>`, the comparator a numeric sort uses.

A listing of `i < 10`, the test at a counted loop's head, with `i` in slot 0:

```
0  GetSlot  0      ; [i]
1  LoadInt 10     ; [i, 10]
2  NumLt    ; [i<10]    pushes Bool
```

The string comparisons

`StrEq`, `StrNe`, `StrLt`, `StrGt`, `StrLe`, `StrGe` are the string analogues, comparing their operands as strings through `as_str_cow`. `StrCmp` has effect `..., a, b → ..., n`: the string three-way compare `cmp`.

The reason the core carries *both* a numeric and a string family — fourteen comparison opcodes rather than seven polymorphic ones — is the distinction Chapter 7 drew: `"10" < "9"` is true as strings and false as numbers, and the front end, knowing from the source whether the programmer wrote `<` or `lt`, emits the opcode that means what was written. The machine never guesses which ordering was meant, which is why `SortBlock` can take a numeric or a string comparator with no inference.

The logical ops

`LogNot` has effect `..., a → ..., !a`: boolean negation, consulting `is_truthy`. `LogAnd` and `LogOr` have effect `..., a, b → ..., result` and are *eager* — they evaluate both operands and push a boolean. As Chapter 8 stressed, these are not the short-circuit operators: `&&` and `||` that must avoid evaluating the right operand are control-flow jumps, the `JumpIf*Keep` family. `LogAnd` and `LogOr` exist for when both sides are already on the stack and the front end wants the boolean combination without a branch.

The bitwise ops

`BitAnd`, `BitOr`, `BitXor` have effect `..., a, b → ..., result`, coercing both operands through `to_int` before operating, which is why a float or a string can be bit-anded at all. `BitNot` has effect `..., a → ..., ~a`. `Shl` and `Shr` have effect `..., a, n → ..., result`: left and right shift. A listing of `x & 0xff` with `x` in slot 0:

```
0  GetSlot  0      ; [x]
1  LoadInt 255     ; [x, 255]
2  BitAnd   ; [x & 255]
```

These twenty-three opcodes — fourteen comparisons, nine logical-and-bitwise — together with the arithmetic and string sections walked in the previous chapter are the scalar computational core, and they are exactly the set the linear JIT lowers (minus the string ops, which are heap-valued): a front end that keeps its hot expressions inside these opcodes keeps them inside the tier that compiles them to a native function with no speculation.

Chapter 41 – Control Flow, Functions, and I/O, Op by Op

The control-flow section is five opcodes, the functions-and-scope sections five between them, and the I/O section three — thirteen opcodes that, walked one by one, show how every loop, conditional, call, and print in every front end is expressed. The control-flow opcodes carry a `usize` target; the function opcodes a name index and an argument count.

The five control-flow jumps

`Jump(usize)` has stack effect $\dots \rightarrow \dots$ and sets `ip` to the target unconditionally.

`JumpIfTrue(usize)` has effect $\dots, p \rightarrow \dots$: pop the predicate, and if `is_truthy(p)`, jump.

`JumpIfFalse(usize)` has effect $\dots, p \rightarrow \dots$: pop the predicate, and if not truthy, jump. This is the workhorse of `if` and `while` — the test at a loop head, the branch around an `if` body. A listing of `if (x) { ... }`, with `x` in slot 0 and the body at indices 2–3:

```
0  GetSlot      0      ; [x]
1  JumpIfFalse  4      ; []      skip body if x is falsy (target patched)
2  ...body...
3  ...body...
4  ...after...    ; <- JumpIfFalse lands here
```

`JumpIfTrueKeep(usize)` has effect $\dots, p \rightarrow \dots, p$: it tests the top *without consuming it* and jumps if truthy.

`JumpIfFalseKeep(usize)` is the same testing for falsy. These are the short-circuit operators. A listing of `a || b`, with `a` in slot 0 and `b` in slot 1, leaving the deciding operand as the value:

```
0  GetSlot      0      ; [a]
1  JumpIfTrueKeep 4      ; [a]      if a truthy, keep a and skip b
2  Pop          ; []      a was falsy; discard it
3  GetSlot      1      ; [b]      value becomes b
4  ...          ; [a or b] the surviving value is the result
```

The “keep” is the whole mechanism: `a` stays on the stack as the expression’s value when it already decided the result, and the jump skips the code that would evaluate `b`.

Functions and scope

`Call(u16, u8)` has effect $\dots, \text{arg0}, \dots, \text{arg}\{n-1\} \rightarrow \dots, \text{result}$: it calls the subroutine named by the name-pool index with the argument count, resolving the name through the chunk’s `sub_entries` table to find the entry instruction pointer. `Return` has effect $\dots \rightarrow \dots, \text{undef}$: it returns with no value, pushing `Undef`. `ReturnValue` has effect $\dots, v \rightarrow \dots, v$ across the call boundary: it returns using the top of stack as the result. The two return opcodes are the void-versus-value distinction of Chapter 9 — a function called as a statement emits `Return`, one called for its value `ReturnValue`, so a void call returns with no spurious push.

`PushFrame` and `PopFrame` have stack effect $\dots \rightarrow \dots$ but manage the frame stack: `PushFrame` brackets a new frame, giving called code its own fresh slots and recording where to resume; `PopFrame` pops it and restores

the operand stack to the saved base. These five — `Call`, `Return`, `ReturnValue`, `PushFrame`, `PopFrame` — are the entirety of the calling convention, and their simplicity is what lets the tracing JIT inline a call: resolving `Call` to the callee's bytecode IP and splicing its ops into the trace is tractable because a call is a name lookup and a frame push, not a complex protocol.

I/O

`Print(u8)` has effect `..., v0, ..., v{n-1} → ...`: pop `n` values and emit them to stdout. `PrintLn(u8)` is the same with a trailing newline. `ReadLine` has effect `... → ..., line`: read a line from stdin and push it as a string. These are deliberately thin — a front end's real I/O goes through builtins and the host — but `Print` and `PrintLn` are exactly the *sink* opcodes the AOT native path spills for: they consume scalars and produce nothing, so a hot numeric loop with an embedded `print` runs native, the arithmetic in registers and only the print briefly touching the boxed stack. A listing of `print x, y` with `x, y` in slots 0 and 1:

```
0 GetSlot 0 ; [x]
1 GetSlot 1 ; [x, y]
2 Print 2 ; []          emit x and y, pop both
```

Chapter 42 – Collections and Higher-Order, Op by Op

The collections section is two opcodes and the higher-order section five, and walking them shows the boundary where the universal core hands collection processing to the block mechanism and then to the front ends.

Ranges

`Range` has effect `..., from, to → ..., [from..to]`: pop two bounds and push the array of integers between them. `RangeStep` has effect `..., from, to, step → ..., [from, from+step, ...]`: the same with a stride. They are in the core because `1..10` is universal — every front end has range syntax — and building the array in one opcode beats a loop of pushes. A listing of `1..5`:

```
0 LoadInt 1 ; [1]
1 LoadInt 5 ; [1, 5]
2 Range    ; [[1, 2, 3, 4, 5]]
```

The higher-order blocks

`MapBlock(u16)`, `GrepBlock(u16)`, `SortBlock(u16)`, and `ForEachBlock(u16)` each carry a *block index* — not a name or constant index but an entry into the chunk’s `block_ranges` table, which records the `(start_ip, end_ip)` of the block body. `MapBlock` runs the body once per element of a slot-array and collects the results; `GrepBlock` keeps the elements whose body is truthy; `SortBlock` orders by the body’s comparison; `ForEachBlock` runs the body for effect. `SortDefault` is the fifth, a sort with the default string comparison that needs no block.

The block-range mechanism is the representational heart, walked in Chapter 10: the body is emitted into the *same* flat op vector as the surrounding code, its `(start, end)` registered with `add_block_range` to get an index, and the higher-order op carries only that index. There is no closure object and no indirect call — the body’s ops are in line, which is why a numeric map body can in principle be lowered as natively as any other loop. A schematic disassembly of `map { _ * 2 } @xs`, with the body at indices 5–7 registered as block 0 and the doubling body referring to the per-element topic the front end installed:

```
0 GetArray 0 ; [@xs]      push the array to map over
1 MapBlock 0 ; [@result]  run block 0 per element, collect
...
5 GetSlot   <topic> ; body: push the element
6 LoadInt   2      ; body: push 2
7 Mul       ; body: element * 2 -> result for this element
```

The coordination — what the topic variable is, how results accumulate, whether `grep`’s predicate is truthy by shell rules or the front end’s own — lives in how the front end lowered the body and which ops it chose. The core provides the iteration machinery; the meaning of the iteration is the front end’s. Beyond these seven opcodes, richer collection processing becomes front-end builtins, which is the boundary Chapter 10 drew: the core names the universal, simple primitives — make, get, set, push, pop, range, map, grep, sort — and lets each front end grow the rest.

Chapter 43 – The Math Ininsics, Op by Op

The math intrinsics are the always-float transcendental family, the rounding quartet, and the integer intrinsics — the opcodes a front end emits when its `sqrt` or `sin` is unconditionally floating point and it wants a single, operand-independent lowering the compilers can specialize. Walking them one by one is also walking the three lowering strategies, because each intrinsic is labelled in `op.rs` with the instruction or libcall it becomes.

The single-instruction floats

Each of these has effect `..., x → ..., result` and lowers to one Cranelift instruction. `SqrtFloat` is `sqrt`. `AbsFloat` is `fabs`. `CeilFloat`, `FloorFloat`, `TruncFloat` are `ceil`, `floor`, `trunc`. `RoundFloat` is `nearest`, round-half-to-even, an exact match for the interpreter's `round_ties_even`. These cost a single instruction and carry no relocation, so their chunks are trivially disk-cacheable. A listing of `sqrt(x)` with `x` in slot 0:

```
0 GetSlot    0    ; [x]
1 SqrtFloat   ; [sqrt(x)]  native fsqrt
```

The transcendental libcalls

These have effect `..., x → ..., result` (or `..., y, x → ..., result` for the two-argument case) and lower to a libcall, because no machine instruction computes them. `SinFloat`, `CosFloat`, `ExpFloat`, `LogFloat`, `TanFloat`, `AsinFloat`, `AcosFloat`, `AtanFloat`, `SinhFloat`, `CoshFloat`, `TanhFloat`, `Log2Float`, `Log10Float` — thirteen functions — share one dispatcher shim, `fusevm_aot_unary_math(id, x)`, keyed by a small id (Sin 0, Cos 1, Tan 2, the inverse trig 3–5, the hyperbolics 6–8, Exp 9, ln 10, Log2 11, Log10 12), so thirteen functions cost one import declared only when one appears in the chunk. `Atan2Float` has effect `..., y, x → ..., result` and its own two-argument shim `fusevm_aot_atan2`. Each shim calls the same `f64` method the interpreter calls, so a finite result matches bit-for-bit. The named JIT helpers (`fusevm_jit_sin_f64` and the rest) carry stable ids the disk cache keys relocations by.

The integer intrinsics

`AbsInt` has effect `..., n → ..., |n|`: a wrapping integer absolute value built from `ineg`, an `icmp`, and a `select`, so `abs(i64::MIN)` wraps to `i64::MIN` exactly as the interpreter does. `TruncInt` has effect `..., x → ..., int(x)`: a saturating float-to-int conversion matching Rust's `as i64`, NaN-to-zero and all, returning an integer operand unchanged at full precision. `GcdInt` and `LcmInt` have effect `..., a, b → ..., result`: the one pair containing a loop, lowered as internal native loops — `GcdInt` as three on-the-fly Cranelift blocks for a Euclid loop, `LcmInt` with `umulhi` overflow detection saturating to `i64::MAX`. `TimeInt` has effect `... → ..., seconds`: Unix epoch seconds, the one impure intrinsic, never const-folded because it reads the clock.

A listing of `gcd(a, b)` with `a`, `b` in slots 0 and 1:

```

0  GetSlot  0      ; [a]
1  GetSlot  1      ; [a, b]
2  GcdInt   ; [gcd(a,b)]  internal native Euclid loop

```

Two siblings deserve a final note, because they recur: `PowFloat` and the `*Float` family are the host-independent, always-float counterparts of the AWK math ops (`AwkSqrt`, `AwkSin`, and the rest of Chapter 46). `AwkSin` dispatches through the AWK host and would be inert without one; `SinFloat` runs in the plain interpreter, lowers in every JIT tier, and stays disk-cacheable. A front end whose `sin` is unconditionally floating point — `strykelang` — emits `SinFloat`, getting a result identical across the interpreter, the JIT, and the AOT native path. The always-float guarantee is what makes the result kind known from the opcode alone, which is what makes the chunk statically typeable, which is what makes it lowerable.

Chapter 44 – The Fused Superinstructions, Op by Op

The eleven fused superinstructions are fusevm's namesake, and walking them one by one — with the unfused sequence each replaces shown as a disassembly beside it — is the clearest way to see what fusion buys. Each entry gives the opcode, its operands, what it replaces, and a listing.

The read-modify-write slot ops

`PreIncSlot(u16)` has effect `... → ...`, `new`: increment slot `i`, push the new value. It replaces `GetSlot`, `LoadInt(1)`, `Add`, `SetSlot` plus a `GetSlot` to leave the value — five ops:

```
unfused:                                fused:
0  GetSlot  0  ; [i]                    0  PreIncSlot 0  ; [i+1]
1  LoadInt 1  ; [i, 1]
2  Add      ; [i+1]
3  SetSlot  0  ; []
4  GetSlot  0  ; [i+1]
```

`PreIncSlotVoid(u16)` has effect `... → ...`: increment slot `i`, push nothing — the statement-form `++i` whose value is discarded. `PreDecSlot(u16)` has effect `... → ...`, `new`: `--i` pushing the new value. `PostIncSlot(u16)` has effect `... → ...`, `old`: `i++` pushing the `old` value. `PostDecSlot(u16)` has effect `... → ...`, `old`: `i--` pushing the old value. The pre/post and value/void distinctions are exactly the ones every C-descended language insists on, encoded by choosing the right opcode so a front end never pushes a value it will not use.

`AddAssignSlotVoid(u16, u16)` has effect `... → ...`: `a += b` for two slots, no stack push, replacing `GetSlot a`, `GetSlot b`, `Add`, `SetSlot a`.

The loop-control fusions

`SlotLtIntJumpIfFalse(u16, i32, usize)` has effect `... → ...`: compare slot `i` to the integer limit and jump to the target if the comparison is false — a fused compare-and-branch replacing `GetSlot`, `LoadInt(limit)`, `NumLt`, `JumpIfFalse`.

`SlotIncLtIntJumpBack(u16, i32, usize)` has effect `... → ...`: increment slot `i`, compare the new value to the limit, and branch back to the target if still under — the entire counted-loop backedge in one dispatch, replacing `PreIncSlotVoid` plus `SlotLtIntJumpIfFalse`:

```
unfused loop tail:                    fused:
6  PreIncSlotVoid 0  ; []              6  SlotIncLtIntJumpBack 0, 10, 2 ; []
7  GetSlot        0  ; [i]                                     (i++, if
~  i<10 goto 2)
8  LoadInt       10 ; [i, 10]
9  NumLt          ; [i<10]
10 JumpIfTrue     2  ; []    back to 2
```

The whole-body fusions

`AccumSumLoop(u16, u16, i32)` has effect `... → ...`: the entire counted sum `while i < limit { sum += i; i += 1 }` in one opcode, the three operands being the sum slot, the index slot, and the limit. It replaces a header test, a four-op accumulation body, an increment, and a backedge — the README tabulates it as standing in for `GetSlot + GetSlot + Add + SetSlot + PreInc + NumLt + JumpIfFalse`, and the benchmark of the million-iteration version is a hundred and forty-two nanoseconds against thirty-one milliseconds for the unfused form.

`ConcatConstLoop(u16, u16, u16, i32)` has effect `... → ...`: the string-append loop `while i < limit { s .= CONST; i += 1 }`, the operands being the constant index, the string slot, the index slot, and the limit.

`PushIntRangeLoop(u16, u16, i32)` has effect `... → ...`: the array-build loop `while i < limit { push arr, i; i += 1 }`, the operands being the array name index, the index slot, and the limit.

Each whole-body fusion is a recognizable shape a front end's loop peephole detects and replaces wholesale, and each tier below has one fast handler: the interpreter runs it in a Rust loop, the block JIT register-allocates it with Cranelift block parameters so the inner loop touches no memory, the AOT native path lowers it to a register loop. The integer-only fusions bail to the interpreter rather than miscompute when a slot is a float — the fusion knows its own precondition and declines when it does not hold.

Chapter 45 – The Shell Ops, Op by Op

The shell section is twenty-nine opcodes — the largest embedding-specific section after AWK — and walking it makes concrete how a shell's plumbing becomes single opcodes dispatched through the `ShellHost` trait. Most of these ops carry their real semantics in the host; the opcode names the operation, carries its small inline encoding, and routes operands.

Execution and pipelines

`Exec(u8)` has effect `..., arg0, ..., arg{n-1} → ..., status`: pop `n` argument values, run the command through `host.exec`, push the exit status. `ExecBg(u8)` is the same without waiting, through `host.exec_bg`, pushing a pid. `PipelineBegin(u8)` tells the host an `n`-stage pipeline is starting; `PipelineStage` wires the next stage; `PipelineEnd` waits and pushes the final status. The trio is the bracket-and-fill shape Chapter 16 described, letting the commands themselves sit between the brackets as ordinary `Exec` ops. A schematic of a | b:

```
0 PipelineBegin 2 ; []      host: set up a 2-stage pipeline
1 ...exec a...   ;         stage 1
2 PipelineStage ; []      wire stage 1 -> stage 2
3 ...exec b...   ;         stage 2
4 PipelineEnd   ; [status] wait, push last status
```

Redirection, substitution, subshells

`Redirect(u8, u8)` attaches a file to a descriptor, the first byte the source fd and the second the operator (WRITE 0 for `>`, APPEND 1 for `>>`, READ 2 for `<`, and the rest of the nine from `op::redirect_op`). `HereDoc(u16)` supplies a heredoc body from the constant pool; `HereString` supplies a here-string from the stack. `CmdSubst(u16)` runs the sub-chunk at the given index and captures its stdout as a value — the `sub_chunks` mechanism of Chapter 15. `SubshellBegin` and `SubshellEnd` isolate a group. `ProcessSubIn(u16)` and `ProcessSubOut(u16)` spawn a sub-chunk and push a FIFO path for `<(cmd)` and `>(cmd)`. `WithRedirectsBegin(u8)` and `WithRedirectsEnd` bracket a scoped redirection applied to a compound command.

Expansion and word handling

`Glob` has effect `..., pattern → ..., [matches]`: expand a glob through `host.glob`. `GlobRecursive` is the `**` variant. `ExpandParam(u8)` performs a parameter expansion, the byte selecting one of the eighteen modifiers from `op::param_mod` (DEFAULT 0 for `${v:-x}`, STRIP_LONG 6 for `${v##p}`, UPPER 11 for `${v^^}`, and the rest). `WordSplit` splits a string by IFS; `BraceExpand` expands `{a,b}` and `{1..10}`; `TildeExpand` expands `~` and `~user`. A listing of `${name:-default}` with `name` at name index 0 and the default on the stack:

```
0 LoadConst    1 ; ["default"]  the fallback
1 ExpandParam   0 ; [value]      modifier DEFAULT, var names[0]
```


Tests, status, traps, matching

`TestFile(u8)` has effect ..., `path` → ..., `bool`: a file test, the byte selecting one of twelve predicates from `op::file_test` (`IS_FILE` 0, `IS_DIR` 1, ... `IS_CHAR_DEV` 11). `SetStatus` has effect ..., `code` → ... and `GetStatus` effect ... → ..., `status`: write and read the VM's `last_status`, which is a `Value::Status` so its truthiness inverts (true at zero). `TrapSet(u16)` installs a trap handler sub-chunk for a signal; `TrapCheck` processes pending traps. `CallFunction(u16, u8)` calls a user-defined shell function by name with `n` args, falling through to `host.call_function` then `host.exec` if not found. `StrMatch` has effect ..., `string`, `pattern` → ..., `bool`: a glob-pattern match for `[[ x = pat ]]` and `case`. `RegexMatch` has effect ..., `string`, `regex` → ..., `bool`: the `=~` operator. A listing of `[[ -f path ]]`:

```
0 LoadConst 0 ; ["path"] the filename
1 TestFile   0 ; [bool]   IS_FILE test
```

The twenty-nine shell opcodes are first-class because process control is universal enough that multiple front ends need it, and because being in the machine means they are compiled by the same JIT and AOT as everything else — though their *semantics*, being effectful, remain the host's to define. The opcode names the operation and locates its operands; the host decides what running it does.

Chapter 46 – The AWK Ops, Op by Op

The AWK section is sixty-one opcodes, the largest in the machine, and walking it group by group shows how AWK's record-and-field model, its rich builtin library, and its associative arrays become typed opcodes dispatched through the `AwkHost` — with the twenty-nine host-free builtins flagged as they appear. AWK values carry POSIX numeric-string duality the core `Value` cannot represent, so most of these route to the host; the host-free ones are pure on `Value` or depend only on VM-owned state.

Fields and records

`AwkFieldGet` has effect `..., i → ..., field`: read field `$i` (the index popped from the stack). `AwkFieldSet` has effect `..., value, index → ...`: assign `$i`, rebuilding `$0` and `NF`. `AwkNf` has effect `... → ..., count`: push `NF`. `AwkSetRecord` has effect `..., value → ...`: replace the whole record `$0` and resplit. All four require the host, because the record and field state live there. A listing of `$1` — push the field index, read the field:

```
0 LoadInt      1 ; [1]
1 AwkFieldGet   ; [$1]  host reads field 1 of the current record
```

Special variables and output

`AwkSpecialGet(u16)` and `AwkSpecialSet(u16)` read and write a special variable by name-pool index — `FS`, `OFS`, `ORS`, `RS`, `NR`, `FNR`, `SUBSEP`, `RSTART`, `RLENGTH`, `FILENAME`, `CONVFMT`, `OFMT`. `AwkPrint(u8)` has effect `..., v0, ..., v{n-1} → ...`: join the `n` args with `OFS`, terminate with `ORS`, emit. `AwkPrintf(u8)` formats and emits (last popped is the format). `AwkSprintf(u8)` has effect `..., args... → ..., string`: like `printf` but pushes the formatted string. These route to the host because `OFS/ORS/CONVFMT` formatting is host state.

Input

`AwkGetline(u8)` has effect `... → ..., status`: the `getline` family, the byte encoding the source kind (plain, into-a-variable, from-a-file, from-a-command), pushing the `getline` status 1, 0, or -1. It routes to the host because reading a record advances input state and updates `NR/FNR`.

The string builtins

`AwkLength(u8)` has effect `..., [x] → ..., n`: `length` or `length(x)`, zero args meaning `length($0)`. `AwkSubstr(u8)` has effect `..., s, m, [n] → ..., sub`: a 1-based substring with POSIX clamping. `AwkIndex` has effect `..., s, t → ..., pos`: the 1-based position of `t` in `s` or 0. `AwkSplit(u8)` has effect `..., s, [fs] → ..., count`: split into an array, push the field count. `AwkSub(u8)` and `AwkGsub(u8)` have effect `..., args... → ..., count`: substitute the first / every match, writing back through an `AwkLvalue` target, pushing the substitution count. `AwkMatch` has effect `..., s, re → ..., RSTART`: set `RSTART/RLENGTH`, push `RSTART`. `AwkToLower` and `AwkToUpper` have effect `..., s → ..., result`. `AwkGensub(u8)` has effect `..., args... → ..., string`: return the substituted string, expanding `&` and `\N` backrefs. Of these, `substr`, `index`, `tolower`, and `toupper`, plus scalar `length(s)`, are host-free; `length($0)`, `split`, `sub`, `gsub`, `match`, and `gensub` need the host (field/array/regex state). A listing of `substr("hello", 2, 3)`, host-free, producing "ell":

```

0 LoadConst 0 ; ["hello"] constants[0]
1 LoadInt 2 ; ["hello", 2]
2 LoadInt 3 ; ["hello", 2, 3]
3 AwkSubstr 3 ; ["ell"] 3 args, no host needed

```

The associative-array ops

`AwkArrayGet(u16)` has effect ..., `key` → ..., `value` (auto-vivifying to ""); `AwkArraySet(u16)` has effect ..., `value`, `key` → ...; `AwkArrayExists(u16)` has effect ..., `key` → ..., `bool` for (`k in arr`); `AwkArrayDelete(u16)` has effect ..., `key` → ... for `delete arr[k]`; `AwkArrayClear(u16)` clears the whole array; `AwkArrayLen(u16)` pushes the element count. The `u16` is the array's name-pool index. These route to the host because AWK arrays carry `SUBSEP` multi-key emulation and numeric-string keys.

The host-free math and bitwise builtins

These twenty-nine run natively without a host. The math group, each effect ..., `x` → ..., `result` (or two operands): `AwkInt` (truncate toward zero), `AwkSqrt`, `AwkSin`, `AwkCos`, `AwkExp`, `AwkLog`, `AwkAtan2`, plus `AwkIntdiv` (truncating quotient, `Undef` on zero), `AwkIntdiv0` (0 on zero), `AwkMkbool` (1/0 by truthiness). The bitwise group, operands truncated to integers: `AwkAnd(u8)`, `AwkOr(u8)`, `AwkXor(u8)` (variadic, ≥2 args), `AwkCompl`, `AwkLshift`, `AwkRshift`. The conversion and character group: `AwkStrtonum` (parse hex/octal/decimal), `AwkOrd` (first char to codepoint), `AwkChr` (codepoint to char). The time group: `AwkSystemtime`, `AwkStrftime(u8)`, `AwkMktime(u8)`, chrono-backed. And the PRNG: `AwkRand`, `AwkSrand(u8)`, against a VM-owned seed initialized to 1 — deterministic without a host.

The trapping arithmetic and the JIT variants

`AwkDiv` has effect ..., `a`, `b` → ..., `result`: float divide that raises a *fatal* error on a zero divisor (the gawk message), distinct from `Op::Div`'s quiet `Undef`. `AwkMod` is the analogous modulo. Because they conditionally trap, both are interpreter-only. The block-JIT-eligible variants carry byte-identical semantics through a guarded early-exit: `AwkDivJit`, `AwkModJit`, `AwkLshiftJit`, `AwkRshiftJit`, `AwkComplJit` (the shift/compl ones trapping on negative operands), and the warn-and-continue `AwkSqrtJit`, `AwkLogJit`. `AwkGetFieldNum(u16)` is the block-JIT field read, effect ... → ..., `float`, reaching the active record through a host-installed `extern "C"` hook and pushing the field as a float — the one statically-typed AWK source the native path reloads.

The signal that has no value

`AwkSignal(u8)` has no stack effect and no `Value` representation: it halts the chunk and stashes a control-flow code (`NEXT`, `NEXTFILE`, `EXIT`) the awkrs driver reads through `VM::awk_signal()` after `run` returns. It carries `next/nextfile/exit` — statements, not expressions — as VM state rather than a stack value, so a non-AWK front end that never emits it sees a `Halted` byte-identical to before. Sixty-one opcodes, one host trait, one PRNG seed, and one side channel: AWK's whole model, named in the instruction set and dispatched where its state lives.

Chapter 47 – A Counted Loop Through the Linear JIT

The compilation chapters of the fifth movement described the tiers; the four that follow walk them, taking one concrete program and lowering it through the linear JIT, the block JIT, the tracing JIT, and finally the AOT native path, showing the disassembly and the emitted intermediate code at each step. The program is the smallest that exercises the machinery without being trivial — an accumulating counted loop — and seeing the same loop four ways is the best way to understand what each tier does differently.

This chapter takes the *expression* the linear JIT is built for, because the linear tier compiles a straight line, not a loop. Consider the body of the loop in isolation, the expression `sum + i * 2`, with `sum` in slot 0 and `i` in slot 1. Its disassembly:

```
0  GetSlot 0      ; [sum]
1  GetSlot 1      ; [sum, i]
2  LoadInt 2     ; [sum, i, 2]
3  Mul           ; [sum, i*2]
4  Add           ; [sum + i*2]
```

This chunk is *linear-eligible*: `is_linear_eligible` walks it and finds only ops in the eligible set — the slot reads, the integer load, the arithmetic. There is no branch, no heap op, nothing outside the scalar core. So `try_run_linear` compiles it on first call.

What the linear JIT emits

The linear JIT simulates the operand stack at compile time as a stack of Cranelift SSA values, and walks the ops, appending one or two CLIF instructions per opcode. The simulation, op by op:

```
op 0  GetSlot 0      ; sim-stack: [v_sum]      v_sum = use_var(slot0)
op 1  GetSlot 1      ; sim-stack: [v_sum, v_i]  v_i   = use_var(slot1)
op 2  LoadInt 2     ; sim-stack: [v_sum, v_i, v2]  v2 = iconst.i64 2
op 3  Mul           ; sim-stack: [v_sum, v_mul] v_mul = imul v_i, v2
op 4  Add           ; sim-stack: [v_add]      v_add = iadd v_sum, v_mul
                        ; return v_add (boxed Int)
```

The compile-time stack mirrors the runtime stack exactly, but its entries are SSA value handles rather than *Values*, and each arithmetic opcode appends the corresponding CLIF instruction over the handles it pops. `Mul` over two integer handles emits `imul`; `Add` emits `iadd`. At the end one handle remains, and the linear JIT boxes it back into a `Value::Int` through `set_int_result`. The emitted function, in pseudo-CLIF:

```

function %expr(i64 vmPtr) -> i64 {
  block0(v_vm: i64):
    v_sum = use_var slot0      ; load sum from its input
    v_i   = use_var slot1      ; load i
    v2    = iconst.i64 2
    v_mul = imul v_i, v2
    v_add = iadd v_sum, v_mul   ; wrapping iadd, matching arith_int_fast
    ; box v_add as Value::Int into the VM result, return 0
    return v_add
}

```

There is no speculation here — the IR matches the bytecode operation for operation, because the ops are all monomorphic integer arithmetic with no type to assume. If either slot held a float, the eligible-op analysis would still admit the chunk but the lowering would promote: a non-integer operand makes `Add` emit `fadd` with an `fcvt_from_sint` on the integer side, exactly where the interpreter's `to_float` would run. The two operations with no single instruction — `Pow` and the float case of `Mod` — would become calls to the `powf` and `fmod` shims, the same Rust functions the interpreter calls.

Why the linear tier stops at the loop's edge

The linear JIT compiles this expression but *not* the loop around it, and the reason is the eligibility gate: a loop contains a backward `Jump`, and control flow is not in the linear-eligible set. The moment the chunk branches, `is_linear_eligible` returns false and the chunk is the block JIT's problem. This is the linear tier knowing its domain — straight-line scalar expressions, no branches, no heap — and declining everything else. Within that domain it is the one tier in the system that never speculates and never deopts, because with only monomorphic arithmetic there is no assumption that could be wrong. On the slot-based benchmark the payoff is twenty-nine times the interpreter on mixed arithmetic and about 1.8 times off hand-written Rust at `-03` — the gap a method-at-a-time JIT pays for compiling in microseconds. The cache lookup that decides whether this expression was already compiled is the chunk's `op_hash`, about twenty-four nanoseconds, so the second evaluation of the same expression skips compilation entirely.

Chapter 48 – The Same Loop Through the Block JIT

The linear tier compiled the loop *body* as an expression; the block JIT compiles the *whole loop*, branch and backedge included. Take the full counted accumulator `sum = 0; i = 0; while i < 10 { sum += i; i += 1 }`, with `sum` in slot 0 and `i` in slot 1, after the front end's peephole has run. Its disassembly, with the loop visible:

```
0 LoadInt      0 ; [0]
1 SetSlot      0 ; []      sum = 0
2 LoadInt      0 ; [0]
3 SetSlot      1 ; []      i = 0
4 AccumSumLoop 0, 1, 10 ; [] while i<10 { sum += i; i += 1 }
5 ...          ;
```

The loop has fused to a single `AccumSumLoop(0, 1, 10)` — sum slot, index slot, limit. This chunk is *block-eligible*: `is_block_eligible` finds only arithmetic, slot ops, and the fused loop, none of which escapes the block compiler's model. After the block warm-up threshold (default one invocation), `try_run_block` compiles the whole chunk to one native function.

What the block JIT emits

The block JIT lowers all mutable state through a single `*mut i64` slots pointer, and register-allocates the fused loop with Cranelift block parameters so the inner loop touches no memory. The emitted function for the `AccumSumLoop`, in pseudo-CLIF:

```
function %chunk(i64 slots) -> i64 {
  block0(v_slots: i64):
    ; sum = 0; i = 0 (initial stores, then enter the loop)
    v0 = iconst.i64 0
    jump loop(v0, v0) ; (sum=0, i=0) as block params

  loop(v_sum: i64, v_i: i64):
    v_cmp = icmp slt v_i, 10 ; i < 10 ?
    brif v_cmp, body, done

  body:
    v_sum2 = iadd v_sum, v_i ; sum += i (wrapping)
    v_i2 = iadd v_i, 1 ; i += 1
    jump loop(v_sum2, v_i2) ; back-edge carries the new values

  done:
    store v_sum -> slots[0] ; write sum back through the pointer
    store v_i -> slots[1] ; write i back
    return 0
}
```

The two loop variables, `sum` and `i`, are carried as block parameters on the `loop` block, so the inner iterations are pure register arithmetic — the slots pointer is read for the initial values and written once at `done`, and nothing in between touches memory. This is why the block JIT posts its largest speedups on tight numeric

loops: the README's million-iteration unfused sum drops from thirty milliseconds to three hundred and fifteen microseconds, ninety-five times, not because each iteration got a little faster but because the iterations stopped touching memory and stopped dispatching.

Float slots and the cache key

Had `sum` been a float — the AWK accumulator `sum = 0.0` — the block JIT would carry it as an `i64 Variable` holding the `f64`'s *bit pattern*, bitcasting `I64 → F64` on read and `F64 → I64` on write, with integer operands converted by `fcvt_from_sint` before float arithmetic. The slot's kind is passed in through `try_run_block_kinded`, and the kind vector is folded into the native-code cache key (the in-memory key and the on-disk `*.blk.fjit` blob name) so float-specialized code is never reused for an integer slot. The integer-only fused superinstructions — `AccumSumLoop` among them — bail to the interpreter on a float slot rather than miscompute, so the float accumulator would lower through the general kinded path, not the integer fusion. The whole compiled function is keyed by the chunk's `op_hash` plus the slot kinds, so a chunk seen before with the same kinds is loaded from the disk cache rather than recompiled — a cached block load about thirty-five microseconds against a hundred and fifty-two for cold codegen.

Chapter 49 – The Same Loop Through the Tracing JIT

The block JIT compiled the whole loop because the whole chunk was block-eligible. But suppose the loop body contained something the block tier cannot take whole — a call, an extension op, a polymorphic read — so `is_block_eligible` returns false. Then the chunk runs interpreted with the tracing JIT's recorder armed at backward branches, and the tracing tier compiles the *path* through the hot loop rather than the chunk. Take a loop with an internal branch, the `loop_with_branch` benchmark shape, `while i < n { if i > 0 { sum += i } ; i += 1 }`, with `i` in slot 0, `sum` in slot 1, `n` in slot 2. Its disassembly:

```
0  GetSlot      0      ; [i]          loop header
1  GetSlot      2      ; [i, n]
2  NumLt        ; [i<n]
3  JumpIfFalse  12      ; []          exit
4  GetSlot      0      ; [i]
5  LoadInt     0      ; [i, 0]
6  NumGt        ; [i>0]
7  JumpIfFalse  10      ; []          skip the add when i <= 0
8  GetSlot      1      ; [sum]       (then-branch)
9  AddAssignSlotVoid 1, 0 ; []         sum += i
10 PreIncSlotVoid 0      ; []         i += 1
11 Jump         0      ; []         → back to header
12 ...          ;          exit lands here
```

Recording the trace

The recorder arms at the backward `Jump 0` at index 11. After the trace threshold — fifty backward branches through the header — it begins capturing, on the next iteration, the exact straight-line sequence of opcodes the program executes, with the branch directions it actually took folded into the recorded line. Suppose at the recorded iteration `i > 0` is true, so the then-branch runs. The recorded trace is the linear path:

```
recorded ops:  GetSlot 0, GetSlot 2, NumLt, [guard: fell through, i<n true]
               GetSlot 0, LoadInt 0, NumGt, [guard: i>0 true -> then-branch]
               GetSlot 1, AddAssignSlotVoid 1,0,
               PreIncSlotVoid 0,
               [close: back-edge to header]
```

The two conditional jumps became *guards*: the trace was recorded under “`i < n` true” and “`i > 0` true,” and it carries a `brif` at each, checking that the runtime condition still matches the recorded direction. The `Cell` lattice specializes the arithmetic — every value on the recorded path is an integer, so the trace emits unboxed `iadd` and `icmp`, with `Cell::Const` folding any value known at record time. The emitted trace, in pseudo-CLIF:

```

trace(i64 slots):
    v_i = use_var slot0
    v_n = use_var slot2
    v_c1 = icmp slt v_i, v_n
    brif v_c1, cont1, exit_header           ; guard: i < n
cont1:
    v_c2 = icmp sgt v_i, 0
    brif v_c2, then, exit_branch           ; guard: i > 0 matched recording
then:
    v_sum = use_var slot1
    v_sum2 = iadd v_sum, v_i               ; sum += i
    def_var slot1, v_sum2
    v_i2 = iadd v_i, 1                     ; i += 1
    def_var slot0, v_i2
    jump trace_entry                       ; native loop continues

```

Guards, deopt, and the phases at work

Each guard that fails transfers to a side-exit that reconstructs VM state and hands a resume IP to the interpreter. The internal `i > 0` guard is the phase-3 caller-frame branch machinery: if a later iteration has `i <= 0`, the guard at `then` fails, the side-exit spills the slot variables, and the interpreter resumes at index 10 (the unrecorded direction, skipping the add). The entry guard at the header — the slot-type check — is the deopt for a type change: if a slot that was integer becomes a float, the trace refuses and the interpreter runs the iteration, and after five such mismatches the trace is blacklisted. Had the loop called a function, phase 2's inlining would resolve the `call` to the callee's bytecode IP and splice its body into the trace; had the call branched, phase 4 would materialize a synthetic `Frame` on deopt with its `return_ip` and copied slots so the interpreter resumes mid-callee. The steady-state payoff, traces primed: the hundred-thousand-iteration branchy loop runs a hundred and forty-eight times faster than the interpreter, within one percent of the block JIT's direct floor, because once the guards pass the native loop body is identical — the gap is only the per-call VM construction and slot copy, a constant that amortizes away as the loop runs longer.

Chapter 50 – A Chunk Through the AOT Native Path, Step by Step

The three JIT tiers compiled at runtime; the AOT native path compiles before the program runs, and this chapter traces a chunk through it from `analyze_native` to the emitted function, including a deopt. The chunk is a float accumulator with a guarded division — enough to exercise the kind lattice, the dual register lanes, and the divide-by-zero deopt at once. The source is `sum = 0.0; for i in 0..n { sum += i / d }`, with `sum` in slot 0 (float), `i` in slot 1 (integer), `d` in slot 2 (integer), `n` in slot 3. Its disassembly:

```
0  LoadFloat  0.0 ; [0.0]
1  SetSlot    0 ; []      sum = 0.0 (slot 0 kind: Float)
2  LoadInt    0 ; [0]
3  SetSlot    1 ; []      i = 0      (slot 1 kind: Int)
4  GetSlot    1 ; [i]     loop header
5  GetSlot    3 ; [i, n]
6  NumLt      ; [i<n]
7  JumpIfFalse 16 ; []     exit
8  GetSlot    1 ; [i]
9  GetSlot    2 ; [i, d]
10 Div        ; [i/d]     Float, or Undef if d == 0
11 GetSlot    0 ; [i/d, sum]
12 Add        ; [sum + i/d]
13 SetSlot    0 ; []      sum = ...
14 PreIncSlotVoid 1 ; []   i += 1
15 Jump       4 ; []      → header
16 ...
```

Step one: `analyze_native`

`analyze_native` performs an abstract interpretation over the operand stack, producing a `NativePlan` or `None`. It walks the chunk maintaining two pieces of state at each instruction: the abstract stack kinds and the set of definitely-assigned slots. It discovers leaders — index 4 (the `Jump` target), index 8 (after the `JumpIfFalse`), index 16 (the exit). It infers slot kinds: slot 0 is `Float` (set from `LoadFloat`), slots 1, 2, 3 are `Int`. At the join into the header it checks the abstract stack kinds match on every edge — they do, the stack is empty at the header on both the entry and the back-edge. It checks definite assignment: `sum`, `i` are assigned before the loop, `d` and `n` before they are read. And it notes the `Div` at index 10 as a *deopt point*, because division by zero yields `Undef`, which no register can hold. The plan it returns records the leaders, the per-slot kinds, the max stack depth, the deopt points, and the assigned set at each.

Step two: `build_entry_native`

`build_entry_native` emits one Cranelift block per leader and threads the operand stack and slots through `Variables`. Each stack position is backed by two parallel `Variables` — an `i64` lane and an `f64` lane — and the kind says which is live. Slot 0 (`Float`) is an `f64` register; slots 1–3 (`Int`) are `i64` registers. The `Add` at index 12 promotes: `i/d` is a `Float` and `sum` is a `Float`, so it emits `fadd` directly. Had one been an integer it would `fcvt_from_sint` first. The `Div` is the interesting block:

```

div_block:
    v_i = use_var slot1          ; i (i64)
    v_d = use_var slot2          ; d (i64)
    ; Div promotes to float; guard the zero divisor
    v_df = fcvt_from_sint v_d
    v_zero = fcmp eq v_df, 0.0
    brif v_zero, deopt_10, div_ok ; if d == 0, deopt
div_ok:
    v_if = fcvt_from_sint v_i
    v_q = fdiv v_if, v_df          ; i / d as Float
    ; ... push v_q (Float lane), continue to Add

```

The common path is a native `fdiv`; only a zero divisor takes `deopt_10`.

Step three: emit_deopt

`emit_deopt` at `deopt_10` reconstructs a VM state the interpreter can resume from, in order. It writes back the definitely-assigned slots — `sum` from its `f64` register through `store_slot_float`, `i` from its `i64` register through `store_slot_int` — skipping any unassigned slot so its register zero never stands in for `Undef`. It writes back globals the same way (none here). It spills the live operand stack — at index 10 the stack holds `i` and `d` (pushed at 8 and 9, not yet consumed), boxed bottom-first so the interpreter sees them in order. It sets `ip` to 10 with an `iconst`. And it calls `fusevm_aot_resume(vm, 10)`, which runs `VM::run()` from index 10 — re-executing the `Div` in the interpreter so it produces the exact `Undef` — and returns. Native code never re-enters after a deopt; the interpreter owns the remainder.

```

deopt_10:
    ; write assigned slots back
    call store_slot_float(vm, 0, v_sum) ; sum
    call store_slot_int(vm, 1, v_i)     ; i (slot 2,3 read-only, slot d,n assigned too)
    ; spill operand stack [i, d] bottom-first
    call push_int(vm, v_i)
    call push_int(vm, v_d)
    v_ip = iconst.i64 10
    v_r = call fusevm_aot_resume(vm, v_ip)
    return v_r

```

The whole path — analyze, build, deopt — is verified differentially: a fuzzer with a divisor of $(n \% 9) - 4$, so roughly one zero in nine, runs three hundred such chunks interpreted and native and asserts they agree, and a dedicated test sets a slot natively, forces a divide-by-zero, and checks the resumed interpreter reads the slot back correctly. The analysis also enforces, at the deopt point, that the may-assigned set equals the must-assigned set, so no conditionally-assigned slot's value is lost across the bail. Every register move, every kind choice, every writeback in this trace is checked against the interpreter that is the specification.

Chapter 51 – From Chunk to Native Object to Standalone Binary

The native path produces a Cranelift function; this chapter traces the last mile — turning that function into a relocatable object, linking it against the runtime, and running the standalone binary — using the crate's own example, whose chunk computes `6 * 7` and whose binary exits 42.

Step one: build the chunk and emit the object

The example builds the three-op chunk and calls `compile_object`:

```
let mut b = ChunkBuilder::new();
b.emit(Op::LoadInt(6), 1); // index 0
b.emit(Op::LoadInt(7), 1); // index 1
b.emit(Op::Mul, 1);        // index 2
let chunk = b.build();
aot::compile_object(&chunk, std::path::Path::new("/tmp/x.o")).expect("emit object");
```

`analyze_native` sees `LoadInt`, `LoadInt`, `Mul` — a straight-line integer computation ending in a numeric result — and returns a plan. `build_entry`, generic over the Cranelift `Module`, lowers it through an `ObjectModule`: `iconst 6`, `iconst 7`, `imul`, box 42 into the VM result. `compile_object` writes the finished function into the object and exports three symbols. The disassembly the compiler saw and the symbols the object exports:

chunk.ops:	exported from /tmp/x.o:	
0 LoadInt 6	fusevm_aot_entry	(the native fn)
1 LoadInt 7	fusevm_aot_chunk_blob	(bincode of the chunk)
2 Mul	fusevm_aot_chunk_len	(LE u64 length)

The chunk travels inside the object as data: `fusevm_aot_chunk_blob` is the `bincode` serialization of the `Chunk`, `fusevm_aot_chunk_len` its length as a little-endian `u64`, and `fusevm_aot_entry` the compiled native function, an `extern "C" fn(*mut VM) -> i64`.

Step two: link against the runtime

A front end's AOT `main` links the object against `libfusevm.a` (the `staticlib` crate type) and supplies one symbol, `fusevm_aot_register_builtins`, which installs its builtins and handlers. On macOS the link adds `-framework CoreFoundation`. The linked binary contains three things: the compiled chunk-driver from the object, the fusevm runtime from the static library, and the front end's builtins from the registration function.

Step three: run

At startup the binary calls `fusevm_aot_run_embedded`, which deserializes the embedded chunk blob back into a `Chunk`, constructs a `VM`, calls `fusevm_aot_register_builtins`, runs `fusevm_aot_entry` against the `VM`, and maps the `VMResult` to an exit code — an `Ok(Int(42))` exits with 42, an `Ok(Status(s))` with `s`, an `Error` to `stderr` and 1. The chunk is still *present* at runtime, not just compiled away, because the native

entry — in the threaded path and at the boundary — still calls back into `exec_op`, which needs the chunk's ops and constants to interpret the opcodes it runs. The result is a self-contained native executable: no interpreter process, no JIT warm-up, no toolchain on the target, and an exit code that is the program's computed value. A test confirms the whole pipeline by shelling out to `nm -g` to verify the three symbols are exported, and by JIT-compiling the same `build_entry` in process through `run_chunk_native` and checking its result — the in-process and shipped paths being byte-for-byte identical up to the one position-independence flag, because `build_entry` is generic over the `Module` and the only difference is whether Cranelift emits absolute or relative addresses.

Chapter 52 – The Five Frontends Against the Protocols

The host traits and the extension mechanism were described in the fourth movement; this chapter walks each of the five frontends driving them concretely, showing which opcodes each emits, which protocol it routes through, and the distribution of work that proves the machine is genuinely language-agnostic.

zshrs against the shell protocol

zshrs lowers shell source to the first-class shell opcodes and installs a `ShellHost` that does the effectful work. A pipeline `ls | wc -l` lowers to `PipelineBegin(2)`, an `Exec` for `ls`, a `PipelineStage`, an `Exec` for `wc -l`, a `PipelineEnd` — and the host's `pipeline_begin`, `pipeline_stage`, `pipeline_end`, and `exec` methods wire the pipes and spawn the processes. A glob `*.txt` lowers to a `Glob` op whose `host.glob` runs the match. A parameter expansion `${x:-y}` lowers to `ExpandParam(DEFAULT)` whose `host.expand_param` applies the modifier. Because most of what a shell does is already first-class shell opcodes, zshrs registers only about twenty *extension* ops for the genuinely zsh-specific behavior the core does not name, and routes its hundred-and-forty-odd builtins (`cd`, `setopt`, `zstyle`, ...) through `CallBuiltin`. The shell is the front end that uses the shell opcodes most, but it is not special-cased: a different front end could emit the same `PipelineBegin` and get the same plumbing.

awkrs against the AWK protocol

awkrs lowers AWK source to the sixty-one first-class AWK opcodes and installs an `AwkHost` that owns the record-and-field state. An AWK rule `$1 > 0 { sum += $1 }` lowers to: `LoadInt(1)`, `AwkFieldGet` (read `$1` through `host.field_get`), a comparison, a `JumpIfFalse` around the action, then the accumulation reading `$1` again and adding to a slot. The numeric-string duality — `$1` comparing numerically but printing as its text — is why `AwkFieldGet` routes to the host rather than producing a bare `Value`. awkrs registers about ninety-five extension ops for the AWK semantics the core leaves host-side, but a great deal runs host-free: a `substr` or `sqr` in an action compiles to `AwkSubstr` or `AwkSqr` and runs with no host call. The `next/nextfile/exit` statements lower to `AwkSignal`, read back through `VM::awk_signal()` after each per-record `run`, to drive awkrs's own record loop.

stryke, vimlrs, and elisprs against the extension protocol

The other three frontends lean on the extension mechanism in three different proportions, and the spread is the best illustration of the design's flexibility. `stryke`, the most ambitious, registers roughly four hundred and fifty extension ops for the largest standard library of the five; a `stryke` builtin call lowers to an `Extended(id, arg)` whose registered handler does the work, reaching the VM's stack through the take-out-and-restore dance. `vimlrs` takes the opposite road, routing roughly five hundred and ten builtins through the flat `CallBuiltin` dispatch table rather than the extension `match` — a vimscript library function is a `CallBuiltin(id, argc)`, the builtin table an indexed set of closures. `elisprs` registers just ten extension ops, because its real work is

in its own heap of cons cells reached through **Obj** handles: an elisp (`setcar x v`) lowers to an **Extended** op whose handler mutates the cons cell at the **Obj** handle's index in elisprs's heap, with fusevm carrying the handle and never dereferencing it. A schematic of an elisp cons operation:

```
0  GetSlot    0    ; [<obj:cell>]  x, an Obj handle
1  GetSlot    1    ; [<obj:cell>, v]
2  Extended   7, 0 ; []           handler: setcar(heap[cell], v)
```

Five frontends, five distributions across the same mechanisms — first-class opcodes, extension handlers, builtin tables, host traits, **Obj** handles — none colliding because each owns its own ID space. A shell mostly uses named opcodes, an AWK mostly uses first-class AWK ops and a host, a Lisp mostly uses opaque handles, and a scripting language mostly uses extension ops, and all four are equally at home on one machine. The protocols are general enough that the same compiled engine serves languages that share not a single ID, which is the language-agnosticism the whole architecture exists to demonstrate.

Chapter 53 — A Real Program, End to End

The opcode and compiler walks took the machine apart; this chapter and the seven after it put a few real programs back together and run them all the way through, from source to result, so the pieces are seen working in concert. The program here is the classic accumulator a benchmark would use, written in a generic scripting surface: `sum = 0; i = 1; while i <= 100 { sum = sum + i; i = i + 1 }; print sum`. It computes 5050. Follow it from source to a printed answer at every layer.

Source to chunk

A front end lexes, parses, and lowers. The two assignments before the loop, the loop with its test and body, and the print at the end become a flat op vector. With `sum` in slot 0 and `i` in slot 1, the unfused disassembly — before any peephole — is:

```
0 LoadInt      0      ; [0]
1 SetSlot      0      ; []      sum = 0
2 LoadInt      1      ; [1]
3 SetSlot      1      ; []      i = 1
4 GetSlot      1      ; [i]     loop header
5 LoadInt      100    ; [i, 100]
6 NumLe        ; [i<=100]
7 JumpIfFalse  17      ; []      exit
8 GetSlot      0      ; [sum]
9 GetSlot      1      ; [sum, i]
10 Add         ; [sum+i]
11 SetSlot     0      ; []      sum = sum + i
12 GetSlot     1      ; [i]
13 LoadInt     1      ; [i, 1]
14 Add         ; [i+1]
15 SetSlot     1      ; []      i = i + 1
16 Jump        4      ; []      → header
17 GetSlot     0      ; [sum]
18 PrintLn     1      ; []      print sum
```

The chunk's constant pool is empty (every literal fit inline as `LoadInt`), its name pool holds whatever names the source used, and its `op_hash` is computed at `build()` over these nineteen ops.

The peephole fuses the loop

A front end's loop peephole recognizes the counted-accumulator shape — same slot tested and incremented, a second slot accumulating it — and rewrites indices 4 through 16 into a single `AccumSumLoop`. But note: this loop accumulates `sum += i` and increments `i`, with the test `i <= 100`. The fused form depends on the front end's peephole supporting the `<=` bound; if it emits the half-open `AccumSumLoop` it adjusts the limit. The fused disassembly:

```

0 LoadInt      0 ; [0]
1 SetSlot      0 ; []      sum = 0
2 LoadInt      1 ; [1]
3 SetSlot      1 ; []      i = 1
4 AccumSumLoop 0, 1, 101 ; [] while i < 101 { sum += i; i += 1 }
5 GetSlot      0 ; [sum]
6 PrintLn      1 ; []      print sum

```

Seven ops where there were nineteen, and the loop is one dispatch.

Interpreter trace

On the first run, the interpreter executes the fused chunk. The dispatch loop reads index 0, advances `ip`, runs `LoadInt 0` (pushing 0), and so on. At index 4 the single `AccumSumLoop` handler runs the whole counted sum in a Rust loop — no per-iteration dispatch — leaving `sum = 5050` in slot 0. Index 5 pushes `sum`, index 6 prints it. `run` returns `VMResult::Halted` (the chunk ran off its end with the stack balanced), having printed `5050`. The interpreter never needed the compilers: a fused loop runs the million-iteration version in a hundred and forty-two nanoseconds, so the hundred- iteration version here is far below any compile threshold's worth of effort.

What the tiers would do

Were this loop hot enough — run many times, or with a higher iteration count — the tiers engage. The chunk is *block-eligible* (only arithmetic, slots, and the fused loop), so with the block warm-up threshold crossed, `try_run_block` compiles it to one native function carrying `sum` and `i` as Cranelift block parameters, the inner loop touching no memory, the result written back through the slots pointer. With the disk cache enabled, the second process to run this exact chunk loads the native code from `~/.cache/fusevm-jit` keyed by the `op_hash`, skipping codegen. And `compile_object` on the same chunk would AOT-lower it: `analyze_native` admits every op, `build_entry_native` emits the register loop, and the linked binary computes 5050 with no interpreter dispatch — though it would still carry the chunk as an embedded blob, because the `PrintLn` at the boundary spills to `exec_op`. One program, one chunk, and a spectrum of execution from the interpreter's Rust loop to a standalone native binary, every one computing the same 5050 because every one ultimately means what `exec_op` means.

Chapter 54 — The Tier Decision in VM::run, Traced

The tiered-execution chapter described the ladder; this chapter traces the decision concretely — what `VM::run` checks, in what order, to choose between the interpreter and the JIT tiers on a given invocation. The decision lives at the top of `run` and is governed by the thresholds and the eligibility caches, and following it for one chunk across several invocations shows the ladder being climbed.

The first invocation

Take a block-eligible chunk — the fused accumulator of the previous chapter — run for the first time with tracing enabled (`enable_tracing_jit()` set). `run` is entered with `ip == 0`. The first check is block eligibility: `run` consults the cached `block_eligible_cached` flag, which on the first run is unset, so it computes `is_block_eligible` once and caches the result — true for this chunk. With the chunk block-eligible and the block warm-up threshold at its default of one, `run` compiles the chunk on this first invocation through `try_run_block`, runs the native function, and returns. The trace recorder is never armed, because a block-eligible chunk short-circuits ahead of the tracing tier — the two tiers cover disjoint cases and never compete.

A non-block-eligible chunk

Now take a chunk the block JIT cannot take whole — one with an extension op or a host call in the loop. `run` computes `is_block_eligible`, caches false, and falls through to the interpreter *with the trace recorder armed at backward branches*. The interpreter runs the chunk normally, but at each backward `Jump` it bumps a per-header counter. On the invocation where a header's counter crosses the trace threshold (default fifty), the recorder captures the next iteration's straight-line path, the trace compiles, and subsequent iterations run the native trace until a guard fails. So for this chunk the decision is: block JIT declines, interpreter runs, recorder watches, and after fifty backward branches the hot loop goes native through the tracing tier.

The thresholds as the knobs

The two thresholds are read once per thread when the JIT is first touched, from `FUSEVM_JIT_BLOCK_THRESHOLD` and `FUSEVM_JIT_TRACE_THRESHOLD`, applied over the compiled defaults in `TraceJitConfig` (block 1, trace 50). Setting the block threshold to 0 makes `run` compile every block-eligible chunk on its first invocation — the most eager posture, free with the disk cache because the native code persists. Raising the thresholds makes `run` lazier, right for scripts that run once. The decision `run` makes is therefore tunable without recompiling: the same `run` code, the same chunk, and a different posture depending on the thresholds the thread read at startup. The block-eligibility cache (`block_eligible_cached`) ensures the eligibility test is computed once per chunk, not per invocation — a field read on every subsequent `run`, because a chunk's eligibility never changes, only whether it has warmed up. This is the ladder in operation: a few field reads and counter bumps at the top of `run` decide, per invocation, how hard to work on the code, and the same dispatch path serves the interpreter and every JIT tier so a value flows between them without translation.

Chapter 55 – Serialization: A Chunk Round-Tripped

A `Chunk` is `Serialize` and `Deserialize`, and a `Value` and an `Op` are too, and this chapter walks what survives a round trip and what does not — because the serialization boundary is where a chunk becomes a cached artifact, a network message, or the embedded blob of an AOT binary. The behavior is pinned by the test suite, so the walk is a walk through real assertions.

What a `Value` round-trip preserves

A `Value` serialized and restored is equal to the original for every variant the tests cover — `Undef`, `Bool`, `Int`, `Float`, `Str`, `Status` all survive a JSON round trip with equality intact. The one subtlety is NaN: `Value::Float(NAN)` does not round-trip *under* `PartialEq`, because NaN is unequal to itself, so the `serde` tests deliberately check only finite floats. The bytes survive; the equality check is what NaN defeats, which is IEEE semantics, not a serialization flaw.

What a `Chunk` round-trip preserves

A `Chunk` serialized to JSON and restored preserves its ops, its names, its lines, and its source — the test asserts each. The one field that does *not* survive is `op_hash`, which carries `#[serde(skip)]`: after a round trip it is zero, and the test asserts exactly that. This is correct and deliberate. The hash is a cache key derived from the ops and constants, cheap to recompute, and meaningless to carry across the serialization boundary — a deserialized chunk recomputes it when something needs it. A listing of the round trip the test performs:

```
let mut b = ChunkBuilder::new();
b.add_name("x");
b.add_constant(Value::Int(42));
b.emit(Op::LoadInt(42), 1);
b.set_source("test.fuse");
let chunk = b.build(); // op_hash computed
let s = serde_json::to_string(&chunk).unwrap();
let back: Chunk = serde_json::from_str(&s).unwrap();
// back.ops == chunk.ops, back.names == chunk.names, back.source == chunk.source
// back.op_hash == 0 (serde-skipped, recompute if needed)
```

Bincode and the AOT blob

The AOT path uses a different `serde` format for the same `Chunk`: `bincode`, a compact binary encoding, exported as the `fusevm_aot_chunk_blob` data symbol with its length in `fusevm_aot_chunk_len`. The choice of bincode over JSON is for size and speed — the embedded blob is binary, not text, and deserialization at the binary's startup is a byte-for-byte reconstruction of the `Vecs` rather than a parse. The same `serde` derives serve both formats: JSON for the test round trips and any human-readable caching a front end wants, bincode for the embedded blob. One set of derive macros, two wire formats, chosen per use — the single-definition discipline applied to serialization. And the same `#[serde(skip)]` on `op_hash` means the embedded chunk

arrives with a zero hash and the runtime recomputes it if the embedded program ever consults the JIT cache, which the AOT binary does not, so the zero never matters.

Chapter 56 – The Differential Fuzzers, Walked

The differential-correctness chapter described the discipline; this chapter walks one fuzzer concretely — how it generates a random chunk, how it compares the two executions, and what a failing seed gives you — because the fuzzers are the backbone of the compiler’s trustworthiness and seeing one run demystifies the whole strategy.

The generator

A fuzzer seeds a SplitMix64 generator with a fixed per-test constant, so its sequence of random chunks is identical on every run. SplitMix64 is a few inline lines: a `u64` state advanced by the golden-ratio increment `0x9E37_79B9_7F4A_7C15`, run through two multiply-xor-shift mixing rounds, yielding a well-distributed `u64` per call. The arithmetic fuzzer uses those `u64`s to build a random balanced RPN arithmetic program — a sequence of `LoadInt` with random values, random `Add/Sub/Mul`, and random forward and backward branches forming a small loop — emitting a `Chunk` whose ops it knows are all in the lowerable set. A schematic of one generated chunk:

```
0 LoadInt  -3 ; [-3]
1 SetSlot  0 ; []
2 GetSlot  0 ; [-3]      loop header
3 LoadInt  5 ; [-3, 5]
4 NumLt    ; [true]
5 JumpIfFalse 10 ; []
6 GetSlot  0 ; [-3]
7 Inc      ; [-2]
8 SetSlot  0 ; []
9 Jump     2 ; []        → header
10 GetSlot 0 ; [5]       result
```

The comparison

For each generated chunk the harness runs it two ways and asserts agreement:

```
// the differential core, in essence
let interp = { let mut vm = VM::new(chunk.clone()); vm.run() };
let native = run_chunk_native(&chunk, register); // compile + run in process
assert_results_match(interp, native);           // Ok==Ok (eq value), Halted==Halted,
↳ Error==Error
```

There is no expected-value table: the interpreter *is* the oracle, correct by definition, so the test passes when the compiler agrees with it. `run_chunk_native` JIT-compiles the same `build_entry` codegen the AOT object would ship and runs it on a fresh VM, so a codegen bug surfaces here rather than in a linked binary.

The categories and what each guards

The categories map the native path's risk surface, each with its count: four hundred integer-arithmetic- and-branch programs; four hundred bitwise-and-shift programs with operands in $-8..8$ to exercise modulo by zero and by negative one; four hundred float programs with promotion (asserting more than fifty actually lowered, so the test cannot pass by trivially falling back); three hundred `$?`-through-every-consumer programs; three hundred division programs with a divisor of $(n \% 9) - 4$ so about one in nine is zero; three hundred gcd/lcm programs over full-range integers; three hundred AWK-bitwise programs across the shift-mask boundary; two hundred `Obj`-slot string-accumulator programs. And the negative space: a chunk the analysis should decline — a mixed-kind slot, an unassigned read, a global aliased by a named array — is asserted to fall back *and* to still match the interpreter, and a partial-deopt fuzzer asserts that more than a hundred of four hundred programs actually deopted, turning “the deopt path is exercised” into a checked property. A failing case is a single seed and an iteration count, reconstructible exactly, so a miscompiled wrapping add or an inverted truthiness is a reproducible bug rather than a ghost.

Chapter 57 — The VMPool and the Frontend Bridge, Walked

The `VMPool` and the bridge pattern were described in the stack-and-frames and extension chapters; this chapter walks them concretely, because the way the five frontends actually drive the machine — not raw `emit`, but a cached, pooled, narrow-writeback bridge — is where the architecture meets real workloads.

The pool in use

A caller running many short-lived chunks recycles VMs through a pool:

```
let mut pool = VMPool::new();
for chunk in chunks {
    pool.with(chunk, |vm| { let _ = vm.run(); });
}
```

`pool.with` acquires a recycled VM, resets it onto the chunk via `VM::reset`, runs the closure, and releases the VM. `reset` preserves the `Vec` capacities of the stack, the frame slots, and the globals while clearing their contents, so a sequence of runs over similarly-shaped chunks reallocates nothing. The honest measurement: for a tiny three-op chunk the pool is *slower* than a fresh `VM::new` per call (about 163 ns against 130 ns), because `reset` does more bookkeeping than `new` skips; the pool wins for non-trivial chunk shapes — large name pools, many slots, a warmed tracing JIT — where the preserved capacity dominates.

The four-part bridge

All five frontends drive `fusevm::VM` through a bridge with the same four-part shape, and walking it shows where each cost is paid. First, a front-end-side *eligibility analysis* decides which subroutines or rule bodies can be lowered to fusevm ops at all — a function that does only arithmetic and control flow is a candidate; one that does language-specific magic the front end handles itself is not. Second, an *op-vector-to-Chunk translation* is cached behind a `OnceCell` or `HashMap` so the two-pass build runs once per source region, not once per call — the chunk for a hot function is built the first time it is called and reused thereafter. Third, a `VMPool` on the front end's `Runtime` recycles the slot, stack, and globals vectors across invocations via `VM::reset`. Fourth, a *narrow writeback* driven by a precomputed `Vec<u16>` of `Op::SetSlot` targets copies only the slots a region actually mutated back into the front end's own runtime — so a function that writes two locals copies two values back, not the whole frame. The on-disk JIT cache, keyed by `op_hash`, handles compiled-code persistence; these per-front-end caches handle the chunk-build and runtime-setup costs the disk cache cannot touch. strykelang adds a detail — `STK_VAL_LOAD_CONST` — to make `LoadConst`-bearing chunks disk-cache-safe by indexing constants rather than carrying per-process pointers. The bridge is the same separation one more time: the machine supplies the fast primitives, and each front end supplies the policy for when and how to reach them, with the caches sitting exactly where the machine's own caching does not reach.

Chapter 58 — Errors, Halts, and the AWK Signal Channel, Traced

`run` returns one of three things — `Ok`, `Error`, `Halted` — and this chapter traces how each arises and what a front-end driver does with it, because the three-way result is the contract every front end branches on, and the AWK signal channel is the example of growing it without breaking it.

The three results

`VMResult::Ok(Value)` arises when the program returns a value — a `ReturnValue` at top level, or falling off the end with a value the front end reads. `VMResult::Halted` arises when execution stops without a return value — a bare end-of-chunk with a balanced stack, or a control-flow opcode that halts the chunk. `VMResult::Error(String)` arises from a runtime fault — a fatal AWK division by zero, a host error surfaced up. A front-end driver branches on all three:

```
match vm.run() {
  VMResult::Ok(v)    => use_value(v),
  VMResult::Halted   => {},           // normal completion, no value
  VMResult::Error(e) => report_error(6e), // runtime fault
}
```

A fatal AWK division, traced

Take `x = 1 / 0` in an AWK action, which must raise a fatal error. The chunk has an `AwkDiv` at the division. The interpreter, executing `AwkDiv` with a zero divisor, does not produce `Undef` (that is `Op::Div`'s shell semantic); it constructs the gawk message "division by zero attempted" and returns `VMResult::Error(msg)`. The awkrs driver receives the `Error`, prints it, and exits with the error status — the same behavior gawk has. Had this been a *block-JIT-compiled* `AwkDivJit`, the native code would have guarded the divisor with `fcmp eq divisor, 0.0`, called the `fusevm_jit_awk_div_trap` libcall into a thread-local channel on the zero path, and returned a sentinel; the VM's block-dispatch path would then call `take_awk_div_trap()`, find the code set, and raise the identical fatal error — before writing slots back, so the partial native computation never becomes visible. The same fatal, two paths, byte-identical.

The signal channel, which is not a fourth result

The `next/nextfile/exit` statements could have been a fourth `VMResult` variant, but that would break all five front-end drivers. Instead `Op::AwkSignal(code)` halts the chunk and stashes the code in VM state, read through `VM::awk_signal()` after `run` returns `Halted`. The awkrs driver's per-record loop is:

```

loop_over_records(|record| {
  install_record(record);
  match per_record_vm.run() {                                // run the compiled rule chunk
    VMResult::Error(e) => fatal(e),
    _ => {}                                                    // Ok or Halted
  }
  match per_record_vm.awk_signal() {                          // then check the side channel
    Some(NEXT)      => continue_to_next_record(),
    Some(NEXTFILE)  => skip_to_next_file(),
    Some(EXIT)      => stop_everything(),
    None            => {}                                        // ordinary record completion
  }
});

```

A non-AWK front end never emits `AwkSignal`, so `awk_signal()` is always `None` for it and its `Halted` is byte-identical to before — the channel is VM state read through a new method, not a change to the shared result type, so the four front ends that do not need it are untouched. This is the side-channel pattern: a capability one front end needs, added as optional state, not as a breaking change to a contract four other front ends depend on.

Chapter 59 – The Disk Cache Lifecycle, Walked

The native-code disk cache was described in the fifth movement; this chapter walks its lifecycle concretely — a blob written, a blob loaded, a blob evicted — because the cache’s correctness rests on keying, atomicity, and conservatism that are easiest to see in operation.

A blob written

The first process to run a hot, block-eligible chunk compiles it through Cranelift and writes the result to `~/.cache/fusevm-jit`. The filename is tier-tagged and keyed: `.blk.` for a block-tier blob, the chunk’s `op_hash` plus the slot-kind vector for the key. The blob stores the native code plus a small relocation table — the libcall sites that must be re-patched on load, each keyed by a stable host-helper id like `H_SIN_F64` so they survive a different process’s helper addresses. The write publishes through a unique temp file and an atomic rename, so a reader racing the writer sees either the whole old state or the whole new one, never a torn blob. After the write, if the cache has grown past its cap (default 256 MiB), an oldest-first eviction runs opportunistically down to eighty percent of the cap — so most writes pay no scan cost, and only the ones that push the cache over the line trigger a prune.

A blob loaded

The second process to run the same chunk computes the same `op_hash` and slot kinds, finds the tagged blob, and loads it: mmap the code, re-patch the relocations against this process’s helper addresses, and flip the page to executable with the platform’s W^X handling — `pthread_jit_write_protect_np` plus icache invalidation on Apple Silicon, `mprotect` elsewhere. The loaded function is then called exactly as a freshly compiled one would be, skipping Cranelift codegen entirely — a cached block load about thirty-five microseconds against a hundred and fifty-two for cold codegen. The loader is conservative: if the blob carries a relocation it does not recognize as a known host-helper call, it declines the blob and falls back to in-memory compilation, so an untested target degrades to “no caching,” never to a miscompile. This is why the `AwkDivJit/AwkModJit` chunks, whose trap libcall is not a registered host-helper id, skip persistence entirely — the loader would not be able to re-patch them safely, so they are never written.

Transparent across the tiers

The cache covers all three JIT tiers, each with its own tag — `.lin.`, `.blk.`, `.trc.` — and its own key. The tracing tier additionally keys on the record-anchor IP and verifies a content hash over the recorded ops, IPs, slot types, and constants, so two divergent traces through one chunk never collide. The whole cache is *behavior-transparent*: it eliminates only Cranelift codegen time, leaving tier selection, warm-up thresholds, and results identical to an uncached run, so a cached and an uncached process compute the same answers by the same path. The controls — the cap, a forced prune, the current size, a full clear — are exposed through `JitCompiler` methods and `FUSEVM_JIT_CACHE_MAX_BYTES`, and the whole cache can be nuked with `rm -rf ~/.cache/fusevm-jit`, repopulating lazily on the next run. The lifecycle — write atomically,

load conservatively, evict opportunistically — is shaped by the same instinct as the rest of the machine: put the cost where it belongs, keep it off the path that runs most, and decline anything you cannot do with certainty.

Chapter 60 – A Catalogue of Worked Idioms

This closing reference chapter collects the small bytecode idioms that recur across front ends, each as a short disassembly, so a reader meeting an unfamiliar op sequence has a key. These are the shapes a front end's lowering produces over and over, and recognizing them is most of reading real fusevm bytecode.

Assignment and compound assignment

A plain assignment `x = e` evaluates `e` and stores: the expression's ops, then `SetSlot` (local) or `SetVar` (global). A compound assignment `x += e` to a local is a single fused op when `e` is another slot (`AddAssignSlotVoid`), or the general form otherwise:

```
; x += 1 (local x in slot 0), general form
0  GetSlot  0    ; [x]
1  LoadInt  1    ; [x, 1]
2  Add      ; [x+1]
3  SetSlot  0    ; []
; the peephole fuses ++x (void) to:
0  PreIncSlotVoid 0 ; []
```

Conditionals

An `if (c) { t }` is a test, a `JumpIfFalse` around the body, the body; an `if/else` adds a `Jump` over the else:

```
; if (c) { t } else { e }
0  ...c...      ; [c]
1  JumpIfFalse  4    ; []      skip then if false
2  ...t...      ;          then-branch
3  Jump         5    ; []      skip else
4  ...e...      ;          else-branch (JumpIfFalse lands here)
5  ...after...   ;          (Jump lands here)
```

Short-circuit boolean

`a && b` and `a || b` are the keep-jumps, leaving the deciding operand:

```
; a && b
0  ...a...      ; [a]
1  JumpIfFalseKeep  4 ; [a]    if a falsy, keep a, skip b
2  Pop          ; []        a truthy; discard it
3  ...b...      ; [b]        value becomes b
4  ...          ; [a or b]
```

A function call

`f(x, y)` pushes the args and calls; the callee returns a value or `Undef`:

```
0 GetSlot 0 ; [x]
1 GetSlot 1 ; [x, y]
2 Call 3, 2 ; [result]    call names[3] with 2 args
```

A shell pipeline and a glob

```
; ls *.txt | wc -l
0 LoadConst 0 ; ["*.txt"]
1 Glob ; [[matches]]    host expands the glob
2 PipelineBegin 2 ; []
3 ...exec ls + matches...
4 PipelineStage ; []
5 ...exec wc -l...
6 PipelineEnd ; [status]
```

An AWK field action

```
; { sum += $1 }    sum in slot 0
0 LoadInt 1 ; [1]
1 AwkFieldGet ; [$1]    host reads field 1
2 GetSlot 0 ; [$1, sum]
3 Add ; [sum + $1]
4 SetSlot 0 ; []
```

Each idiom is a shape a front end emits without thinking and the VM runs without surprise, and each appears, fused or unfused, in the disassemblies throughout this book. With the opcode walks of the reference movement and these idioms in hand, a real fusevm disassembly reads as plainly as the source it came from — which is the point of a companion: not to replace the reference, but to make the machine legible from the metal up.

Chapter 61 – A Recursive Call, Traced Through the Frame Stack

The functions section gave the calling convention five opcodes; this chapter traces a recursive function through them, watching the frame stack grow and shrink, because recursion is where the per-frame slot model and the `Frame`'s `return_ip/stack_base` earn their keep. The function is the recursive Fibonacci the benchmark uses — `fn fib(n) { if n < 2 { return n } return fib(n-1) + fib(n-2) }` — which at `fib(20)` makes 21,891 calls.

The function's chunk

The function body lowers to a chunk with its entry recorded in `sub_entries` under the name `fib`. With the parameter `n` arriving in slot 0, a schematic disassembly of the body:

```
(entry of fib, recorded in sub_entries as ("fib", 0))
0  GetSlot      0      ; [n]
1  LoadInt     2      ; [n, 2]
2  NumLt        ; [n<2]
3  JumpIfFalse  6      ; []      if n >= 2, fall through to the recursion
4  GetSlot      0      ; [n]      base case
5  ReturnValue   ; return n
6  GetSlot      0      ; [n]
7  LoadInt     1      ; [n, 1]
8  Sub          ; [n-1]
9  Call         fib, 1 ; [fib(n-1)]
10 GetSlot      0      ; [fib(n-1), n]
11 LoadInt     2      ; [fib(n-1), n, 2]
12 Sub          ; [fib(n-1), n-2]
13 Call         fib, 1 ; [fib(n-1), fib(n-2)]
14 Add          ; [fib(n-1) + fib(n-2)]
15 ReturnValue   ; return the sum
```

A call, frame by frame

Trace `fib(3)`. The top-level frame has `n = 3` in its slot 0; the dispatch loop reaches the `Call fib, 1` at index 9 (after computing `n - 1 = 2` on the stack). `Call` does three things. It resolves `fib` through `sub_entries` to entry index 0. It pushes a new `Frame` whose `return_ip` is 10 (the instruction after the call), whose `stack_base` is the current operand-stack height, and whose `slots` hold the one argument `2` bound to slot 0. And it sets `ip` to 0, the callee entry. The frame stack is now two deep:

```
frames (top last):
[ frame0: n=3, return_ip=<toplevel>, stack_base=0 ]
[ frame1: n=2, return_ip=10,      stack_base=1 ]   <- running fib(2)
ip = 0      (start of fib body, now executing fib(2))
```

`fib(2)` is not a base case, so it recurses again at index 9 with `n - 1 = 1`, pushing frame2 for `fib(1)`. `fib(1)` is a base case: index 3's `JumpIfFalse` does not jump (`1 < 2` is true), index 4 pushes `n = 1`, index

5's `ReturnValue` returns. `ReturnValue` pops frame2, restores the operand stack to frame2's `stack_base`, leaves the result 1 on the caller's stack, and sets `ip` to frame2's `return_ip` (10) — resuming `fib(2)` right after its first call, with `fib(1) = 1` on the stack. `fib(2)` then computes `n - 2 = 0`, calls `fib(0)` (another base case returning 0), adds, and returns 1. Each return pops one frame and restores the operand stack to the saved base.

Why per-frame slots are the whole trick

The reason this works without any special recursion machinery is that slots are *per-frame*: each `Call` pushes a fresh `slots` vector, so `fib(3)`'s `n` and `fib(2)`'s `n` and `fib(1)`'s `n` are three different storage locations, never aliasing. A recursive function gets fresh locals each call for free, which is exactly what recursion requires, and the slot-indexing model gives it without a heap or a closure. The `Frame`'s `return_ip` is where to resume and its `stack_base` is how far to truncate the operand stack on return, so a return leaves precisely the result and nothing else. This is also why the tracing JIT can *inline* a recursive call up to its depth cap (`MAX_INLINE_RECURSION` = 4): each inlined frame gets its own slot-variable scope, mirroring the per-frame slots, and a deopt inside an inlined callee materializes a synthetic `Frame` with the right `return_ip` and copied slots so the interpreter resumes mid-recursion with a correctly shaped stack. The recursion benchmark — 21,891 calls in 1.28 milliseconds, about sixty nanoseconds per call — measures exactly this machinery: a frame push, an argument bind, a return, a frame pop, repeated, which is the cost the fused loops bypass and the call- heavy code pays.

Chapter 62 — The ShellHost Trait, Method by Method

The host boundary was described in the fourth movement; this chapter walks the `ShellHost` trait method by method, because the trait's full surface — and the default each method chooses — is a precise map of where the shell-core boundary lies. Every method has a default, so a real host overrides only what it needs, and `DefaultHost` is literally `impl ShellHost for DefaultHost {}`.

Expansion methods

`glob(pattern, recursive) -> Vec<String>` expands a glob; the default uses the system `glob` crate, returning matched paths (empty on no match, the nullglob-style default the caller decides how to handle). `tilde_expand(s) -> String` expands `~` and `~user`; default identity. `brace_expand(s) -> Vec<String>` expands `{a,b}` and `{1..10}`; default a single-element vec of the input. `word_split(s) -> Vec<String>` splits by IFS; default whitespace splitting, collapsing runs, matching POSIX. `expand_param(name, modifier, args) -> Value` performs a parameter expansion, the modifier one of `op::param_mod::*`; default the empty string. `array_index(name, index) -> Value` indexes a shell array variable; default `Undef`. The defaults divide cleanly: `glob` and `word_split` have sensible language-agnostic answers, so they do real work; tilde rules, brace syntax, and parameter modifiers are genuinely the front end's, so they are inert rather than wrong.

Sub-execution and redirection

`cmd_subst(sub: &Chunk) -> String` runs a command-substitution body and captures stdout; default empty. `process_sub_in(sub) -> String` and `process_sub_out(sub) -> String` spawn a body and return a FIFO path; default empty. `redirect(fd, op, target)` applies a redirection, `op` from `op::redirect_op::*`; default no-op. `heredoc(content)` and `herestring(content)` supply stdin; default no-ops. `with_redirects_begin(count)` and `with_redirects_end()` bracket a scoped redirection saving and restoring fd state; default no-ops. These take `&Chunk` references from the parent's `sub_chunks` table — the isolation mechanism of Chapter 16 — so the host runs them on a fresh or pooled VM and reports the result.

Pipelines, subshells, traps

`pipeline_begin(n)`, `pipeline_stage()`, `pipeline_end()` -> `i32` bracket an N-stage pipeline; the default `pipeline_end` returns 0. `subshell_begin()` and `subshell_end() -> Option<i32>` save and restore subshell state, the `Option` letting the host update the parent's `$?` on a deferred exit. `trap_set(sig, handler: &Chunk)` installs a trap handler sub-chunk; `trap_check()` processes pending traps; both default no-ops.

Execution and matching

`exec(args)` -> `i32` spawns and waits; this is the one method with a *meaningful* default — `std::process::Command` with inherited stdio, returning the exit code or 127 on a spawn failure — because running a process is the same on any Unix. `exec_bg(args)` -> `i32` spawns detached with nulled stdio, returning a pid. `call_function(name, args)` -> `Option<i32>` calls a user-defined shell function, the `None` falling through to `exec`. `str_match(s, pat)` -> `bool` is a glob-pattern match; default exact equality. `regex_match(s, regex)` -> `bool` is the `=~` operator; default false. The trait is `Send`, so a host moves across threads with the VM, and the reason every method has a default beyond convenience — “so the trait can grow without breaking existing front ends” — is the same growth-without-breakage instinct as the AWK signal channel: a new shell op can be added with a default host method, and every existing front end keeps compiling.

Chapter 63 — The AwkHost Trait, Method by Method

The AWK host owns the record-and-field state the core `Value` cannot represent; this chapter walks its methods, because the signatures — especially the `AwkLvalue` targets and the `Option`-wrapped operands — are where AWK's numeric-string duality and assignable-location semantics enter the interface. As with the shell host, every method has a default, so `DefaultAwkHost` accepts them all and a non-AWK embedding runs AWK-containing chunks inertly but stack-balanced.

Fields and special variables

`field_get(i)` -> `Value` reads `$i` (out-of-range fields read as "" per POSIX); default "". `field_set(i, v)` assigns `$i`, extending the record with `OF5` when `i` is past `NF` and resplitting on `$0`; default no-op. `nf()` -> `i64` returns `NF`; default 0. `set_record(v)` replaces `$0` and resplits using the current `FS`; default no-op. `special_get(name)` -> `Value` and `special_set(name, v)` read and write `FS`, `OF5`, `ORS`, `RS`, `NR`, `FNR`, `SUBSEP`, `RSTART`, `RLENGTH`, `FILENAME`, `CONVFMT`, `OFMT`; some assignments have side effects (changing `FS` affects the next split, setting `NF` rebuilds `$0`).

Output and input

`print(args)` emits the args joined by `OF5` and terminated by `ORS` (with no args, `$0`); default no-op. `printf(fmt, args)` formats and emits with no trailing `ORS`. `sprintf(fmt, args)` -> `Value` formats and returns the string. `getline(source, operand, var_name)` -> `i64` is the `getline` family — `source` selects the form (the `Option`-wrapped `operand` is the file or command, `var_name` the target for the `var` forms) — returning 1, 0, or -1. The `Option` wrapping is the signature encoding which forms need which operands.

The string builtins and the lvalue problem

`length(s: Option<&Value>)` -> `i64` is `length` or `length(x)` — the `None` meaning `length($0)`. `substr(s, m, n: Option<i64>)` -> `Value` is the 1-based POSIX-clamped substring. `index(s, t)` -> `i64` is the 1-based position or 0. `split(s, arr_name, fs: Option<&Value>)` -> `i64` splits into a named array and returns the count. The substitution methods carry the signature that justifies the whole dedicated host: `sub(re, repl, target_ref: &AwkLvalue)` -> `i64` and `gsub(re, repl, target_ref: &AwkLvalue)` -> `i64` take an `AwkLvalue` — an *assignable location*, a field index, an array element, or a special variable — because `sub(/re/, "x", $2)` must write back into field two, which a value-receiving opcode could read but never update. `match_re(s, re)` -> `i64` sets `RSTART/RLENGTH` and returns `RSTART`. `gensub(re, repl, how, target: Option<&Value>)` -> `Value` returns the substituted string. `tolower(s)` and `toupper(s)` case-fold.

The pure builtins that need no host

A large group of `AwkHost` methods have *real* default implementations rather than inert ones, because they are pure on `Value` or depend only on VM-owned state — the twenty-nine host-free builtins. `int`, `sqrt`, `sin`, `cos`, `exp`, `log` compute their math; `and`, `or`, `xor`, `compl`, `lshift`, `rshift` do bitwise work with operands truncated to integers; `strtonum` parses hex/octal/decimal; `systemtime`, `strftime`, `mktime` are chrono-backed; `ord`, `chr`, `mkbool`, `intdiv`, `intdiv0` round out the set. These run even under `DefaultAwkHost`, which is why `substr("hello", 2, 3)` produces `"ell"` with no host registered. A registered host may still override them — for locale-aware casing, MPFR-precision math, or gawk's fatal-on-negative-bitwise behavior — but the native path is used only when no override is present. The split between the inert defaults (field and array methods, which need the host's runtime) and the real defaults (the pure builtins) is the map of which AWK semantics live host-side and which the core can compute on its own.

Chapter 64 – The Builtin ID Tables

The extension mechanism gave a front end three doors — `Extended`, `ExtendedWide`, and `CallBuiltin` — and this chapter walks the two ID tables the core ships ready-made: the shell builtin IDs in `shell_builtins.rs` and the reserved AWK op range in `awk_builtins.rs`. These are the concrete numeric contracts a front end emits against.

The 140 shell builtin IDs

`shell_builtins.rs` defines a hundred and forty `BUILTIN_*` constants, the IDs a `CallBuiltin(id, argc)` op carries, organized into deliberately spaced numeric bands by family so a new builtin slots into the right neighborhood without renumbering. The core utilities occupy the low numbers: `BUILTIN_CD` is 0, `BUILTIN_PWD` 1, `BUILTIN_ECHO` 2, `BUILTIN_PRINT` 3, `BUILTIN_PRINTF` 4, then `EXPORT`, `UNSET`, `SOURCE`, `EXIT`, `RETURN`, `TRUE`, `FALSE`, `TEST`, the colon and dot. The declaration builtins band at 20: `LOCAL` 20, `DECLARE` 21, `TYPESET` 22, `READONLY` 23, `INTEGER` 24, `FLOAT` 25. Input is at 30 (`READ`, `MAPFILE`). Loop control at 40 (`BREAK`, `CONTINUE`, `SHIFT`, `EVAL`, `EXEC`). History at 61 (`FC`, `R`). Aliases at 70 (`ALIAS`, `UNALIAS`). Option control at 80 (`SET`, `SETOPT`, `UNSETOPT`, `SHOPT`, `EMULATE`, `GETOPTS`). Function management at 90 (`AUTOLOAD`, `FUNCTIONS`, `UNFUNCTION`). Traps at 100. The directory stack at 110 (`PUSHD`, `POPD`, `DIRS`). Command resolution at 120 (`TYPE`, `WHENCE`, `WHERE`, `WHICH`, `HASH`, `REHASH`, `UNHASH`). Completion at 130 (`COMPGEN`, `COMPLETE`, `COMPDEF`, `COMPINIT`, and the rest). And the zsh-specific machinery at 140 and up (`ZSTYLE`, `ZMODLOAD`, `BINDKEY`, `ZLE`, `VARED`, `ZCOMPILE`, `ZFORMAT`, `ZPARSEOPTS`, `ZREGEXPARSE`). A front end installs each with `register_builtin(id, closure)`, and emits `CallBuiltin(BUILTIN_CD, 1)` to call `cd`:

```
// install and emit
vm.register_builtin(BUILTIN_CD, |_vm, _argc| Value::Status(0));
// ... in a chunk:
builder.emit(Op::CallBuiltin(BUILTIN_CD, 1), line);
```

The banding is the design substance: leaving gaps between families means a new completion builtin takes an unused number in the 130s without disturbing the 140s, so the table grows without churn.

The reserved AWK op range

`awk_builtins.rs` takes a different tack, reserving a *range* of `ExtendedWide` IDs for the AWK host: `AWK_OP_BASE` is 40,000 and `AWK_OP_END` is 41,000, a thousand-wide window. The VM routes any `ExtendedWide(id, payload)` whose `id` falls in `[40000, 41000)` to the registered `AwkHost`, and everything below it to the generic extension handler a front end installs with `set_extension_wide_handler`. The range was chosen high to leave the low extension-ID space free for generic front-end handlers. Within the range, the per-op IDs are laid out in bands too: `AWK_FIELD_GET` is `AWK_OP_BASE`, `AWK_FIELD_SET` base+1, `AWK_NF` base+2, `AWK_SET_RECORD` base+3; the special-variable ops at base+4 and +5; output at base+16 (`AWK_PRINT`, `AWK_PRINTF`, `AWK_SPRINTF`); input at base+24 (`AWK_GETLINE`); the string builtins at base+32 (`AWK_LENGTH`, `AWK_SUBSTR`, `AWK_INDEX`, `AWK_SPLIT`, `AWK_SUB`, `AWK_GSUB`, `AWK_MATCH`, `AWK_TOLOWER`, `AWK_TOUPPER`). The typed `Op::Awk*` variants of Chapter 46 map one-to-one onto these IDs and reach the identical `dispatch_awk` path, so a front end can emit a readable named variant or a raw `ExtendedWide(AWK_FIELD_GET, 0)` and both arrive at the same host method. The `is_awk_op(id)` helper is the range check the VM uses to decide which handler an `ExtendedWide` op goes to:

```
// $1 - push the field index, then read the field
let mut b = ChunkBuilder::new();
b.emit(Op::LoadInt(1), 1);
b.emit(Op::ExtendedWide(AWK_FIELD_GET, 0), 1); // routed to AwkHost::field_get
```

Two ID tables, two strategies — a banded flat table for the shell's builtins, a reserved range for the AWK host's ops — coexisting without a central registry because the ID space is partitioned: the shell's hundred and forty live in the low `CallBuiltin` space, the AWK host's in the 40,000s, the generic extension handlers below 40,000, and each front end owns its own slice.

Chapter 65 – The Benchmark Harnesses, Walked

The how-fast chapter reported the numbers; this chapter walks the harnesses that produce them, because how a benchmark is built is a claim about what it measures, and fusevm’s harnesses are built to measure honestly. The benches live under `benches/`, declared in `Cargo.toml`, each a criterion harness gated behind the feature it needs.

The seven harnesses

`Cargo.toml` declares seven `[[bench]]` targets. `vm_bench` measures interpreter and fused-op throughput — the classic-algorithm group, the dispatch floor, the fused-versus-unfused sum. `classic` measures the classic algorithms (iterative and recursive Fibonacci, Ackermann, nested loops). `jit_vs_interp` (gated on `jit`) compares the linear JIT against the interpreter and native Rust on slot-based inputs. `jit_crossover` (gated on `jit`) measures where the JIT overtakes the interpreter as iteration count grows. `jit_trace` (gated on `jit`) measures the tracing-JIT VM at steady state. `jit_disk_cache` (gated on `jit-disk-cache`) measures a cached block load against cold codegen. And `aot_native` (gated on `aot`) measures the AOT native path. Each is `harness = false`, meaning it uses criterion’s own runner rather than libtest’s, which is what gives the statistical machinery.

Why slot-based inputs

The single most important methodological choice, visible in the harness code, is that the JIT-versus- native comparisons use *slot-based inputs* — values read from slots rather than constants — so the compiler cannot fold the answer to a constant. A benchmark of `sum(1..1_000_000)` with a literal bound could be folded to the closed-form $n(n+1)/2$ by a sufficiently clever optimizer, measuring the folder rather than the loop; feeding the bound through a slot the optimizer cannot see forces the loop to actually iterate. The harness comment names this — “slot-based inputs prevent constant folding — honest apples-to-apples comparison” — and it is why the linear-JIT row reports 75 ns rather than a fictional sub-nanosecond. The comparisons are also *cached-JIT against interpret*, folding in the $O(1)$ op-hash cache lookup (about 24 ns), because that is the real “run a hot chunk” workload: a chunk that has run before is recognized and its native code reused.

Honesty in the numbers

The harnesses report losses as carefully as wins. The `VMPool` measurement publishes that the pool is *slower* than `VM::new` for a tiny three-op chunk (163 ns against 130 ns), because `reset` does more bookkeeping than `new` skips — a result that would be omitted from a marketing benchmark and is kept here. The tracing-JIT harness records the trace at a lowered threshold of 5 (against the production default of 50) so the cache is primed before measurement, and reports steady-state hot-path times, with the remaining 1.0–1.7x gap to the block JIT’s direct floor attributed explicitly to per-call VM construction and slot copy, not to native execution. The workflow for tracking regressions is criterion’s baseline mechanism — `--save-baseline before`, make a change, `--baseline before` — so a regression shows as a percentage delta with a confidence interval, distinguishable from jitter. The harnesses are the performance analogue of the differential fuzzers: the fuzzers

keep correctness from regressing by comparing against the interpreter, the saved baselines keep speed from regressing by comparing against the last known-good numbers, and both are built to report the truth rather than flatter the system.

Chapter 66 — The Contract Tests, Walked

Beyond the differential fuzzers, fusevm carries a family of *contract* tests — tests whose names begin `contract_` and whose job is to pin invariants the whole machine depends on, so a refactor that quietly breaks one fails loudly. Walking a few shows what the machine considers non-negotiable.

The stack-and-chunk contract

`contract_vm_stack_chunk_round4.rs` pins the invariants of the operand stack, the chunk, and their interaction — that a sequence of ops leaves the stack at the expected height, that a balanced program ends with its result and nothing else, that the chunk's tables (constants, names, sub-chunks) are addressed correctly by the ops that index them. These are the invariants a disassembly's stack comments assume: if `Add` did not leave the stack one shorter, every listing in the reference movement would be wrong, and the contract test is what guarantees it does. The value and chunk round-trip tests pin the serialization contract — that a `Chunk` survives *serde* with its ops, names, lines, and source intact and its `op_hash` deliberately skipped to zero — which is the contract the disk cache and the AOT blob depend on.

The fused-ops and JIT contract

`contract_fused_ops_and_jit.rs` pins that a fused superinstruction computes exactly what the unfused sequence it replaces would, and that the JIT's result matches the interpreter's. This is the contract behind every fusion: `AccumSumLoop(0, 1, n)` must leave the same `sum` in slot 0 that the seven-op unfused loop would, or the peephole that emits it would silently change results. The test runs both forms and asserts equality, the same differential discipline as the fuzzers applied to the specific fusion shapes. It also pins the integer-only fusions' *bail* behavior — that `AccumSumLoop` on a float slot falls to the interpreter rather than miscomputing — so the precondition the fusion declines under is a tested contract, not a hope.

The deopt and sub-chunk contract

`contract_deopt_constants_and_subchunks.rs` pins the harder invariants of the compilation boundary: that a deopt reconstructs VM state the interpreter can resume from, that constants survive the boundary correctly, and that sub-chunks (the `$(cmd)` bodies, trap handlers) are addressed and run correctly. The deopt contract is the one the AOT native path's correctness rests on — that after a divide-by-zero deopt, the resumed interpreter reads the slots and stack the native code wrote back, exactly as Chapter 50 traced. A dedicated test sets a slot to a known value natively, forces the deopt, and checks the resumed interpreter reads it back correctly; the contract test generalizes this to the full set of state a deopt must preserve. The contract tests are, in a sense, the executable form of this book: every invariant a chapter asserts — the stack balances, the fusion is equivalent, the deopt resumes cleanly, the chunk round-trips — is a `contract_` test that fails if the invariant breaks, so the prose and the test suite pin the same machine from two directions.

Chapter 67 – The Capability Matrix and Versioning

The book closes its reference walk where the source's own documentation begins — the capability matrix in `lib.rs` — because that matrix is the project's own honest statement of what is finished and what is frontier, and reading it is the right way to weigh everything this book has described.

The tracing-JIT phase matrix

`lib.rs` documents the tracing JIT as a matrix of capabilities against phases, and the matrix is the ground truth for what the tracing tier can and cannot do. Phase 1: loop bodies with integer slots and no calls. Phase 2: cross-call inlining of branchless callees. Phase 3: caller-frame `if/else` with side-exits. Phase 4: callee-frame branches with frame materialization on deopt. Phase 5 (and 5b): value-stack reconstruction on deopt, integer and float. Phase 6: side-exit deopt counter and auto-blacklist. Phase 7: persistent metadata export and import via `TraceMetadata`. Phase 8: bounded recursion inlining to depth 4. Phase 9: side-trace stitching from hot deopt sites. Each phase is a restriction the previous one's success made visible, lifted in turn, and the matrix lets a reader see exactly how far the tracing tier reaches — a loop with a branchy, recursive, float-bearing body up to the documented bounds, and beyond them a fall back to the interpreter.

The architecture diagram and the usage contract

`lib.rs` also draws the architecture: three frontends (the diagram names `stryke`, `awkrs`, `zshrs`) compiling to `fusevm::Op`, feeding `VM::run()` with the optional tracing JIT alongside. And it ships the canonical usage — build a chunk, make a VM, match the `VMResult` — that this book opened with, plus the public re-exports that Appendix F catalogues. The capability matrix and the usage example together are the contract the five frontends program against: the matrix says what the machine guarantees, the re-exports say what surface they may touch, and the version says how stable it all is.

Young, and saying so

The version is `0.14.5`, a pre-1.0 number that says, by convention, that interfaces may still shift and not everything is finished — and the project means it. The AOT native path's reach into dynamic-kind heap sources is documented as future work, not pretended done. A few constants carry doc comments that disagree with the code — the block-JIT warm-up threshold a comment calls 10 and the code sets to 1 — and this book has followed the code at each such point and flagged the disagreement. The capability matrix itself is dated by its phase numbers: a reader meeting `fusevm` at a later version may find phase 10 and beyond, dynamic-kind sources lowered, thresholds retuned. What does not change is the shape of the honesty: the matrix reports the frontier as a frontier, the fuzzers report the declines as carefully as the lowerings, the benchmarks publish the losses beside the wins, and the version number warns that the line is still moving. A companion book to a young machine can do no better than to describe it precisely and mark, as the source does, where precisely it is not yet done — which is what this reference movement has tried to do, opcode by opcode and tier by tier, all the way down to the metal.

Chapter 68 – The Coercion Tables in Full

The value chapter argued that the coercions are a contract five languages were built against; this chapter prints the contract in full, every variant against every coercion, because a reference reader wants the exact answer and the test suite pins exactly these cells. The four coercions are `to_int`, `to_float`, `as_str_cow` (the basis of `to_str`), and `is_truthy`, all from `value.rs`.

`to_int`, every variant

`to_int` produces an `i64`:

Variant	<code>to_int</code>
<code>Undef</code>	0
<code>Bool(b)</code>	1 if true, else 0
<code>Int(n)</code>	n
<code>Float(f)</code>	f truncated toward zero
<code>Str(s)</code>	parsed, or 0 if non-numeric
<code>Array(a)</code>	the length of a
<code>Hash</code>	0
<code>Status(c)</code>	c
<code>Ref</code>	0
<code>NativeFn</code>	0
<code>Obj</code>	0

The two cells that surprise are `Array` (its *length*, not its first element — Perl/AWK numeric coercion of an array) and `Float` (truncation toward zero in both directions, so `3.9` is 3 and `-3.9` is -3). The tests pin both, plus that a `Status` round-trips its exit code rather than collapsing to 0 or 1.

`to_float`, every variant

`to_float` produces an `f64`:

Variant	<code>to_float</code>
<code>Float(f)</code>	f
<code>Int(n)</code>	n as f64
<code>Bool(true)</code>	1.0
<code>Str(s)</code>	parsed, or 0.0 if non-numeric
<code>Status(c)</code>	c as f64
everything else (incl. <code>Bool(false)</code> , <code>Array</code> , <code>Undef</code>)	0.0

The quirk the tests pin: only `Bool(true)` produces 1.0, and `Bool(false)` drops through to the default 0.0 — the `Bool` arm of `to_float` is the `true` arm only.

as_str_cow, borrowed versus owned

`as_str_cow` produces a `Cow<str>`, and whether it borrows or allocates is the performance-relevant cell:

Variant	<code>as_str_cow</code>	Cow
<code>Str(s)</code>	the inner string	Borrowed
<code>Undef</code>	<code>" "</code>	Borrowed
<code>Bool(true) / Bool(false)</code>	<code>"1" / ""</code>	Borrowed
<code>Hash</code>	<code>"(hash)"</code>	Borrowed
<code>Ref</code>	<code>"(ref)"</code>	Borrowed
<code>Int(n)</code>	the formatted integer	Owned
<code>Float(f)</code>	the formatted float	Owned
<code>Status(c)</code>	the formatted code	Owned
<code>Array(a)</code>	elements joined by a single space	Owned
<code>NativeFn(id)</code>	<code>"(builtin:{id})"</code>	Owned
<code>Obj(id)</code>	<code>"(obj:{id})"</code>	Owned

The borrowed rows are the ones string comparison, concatenation, hash-key lookup, and I/O hit on the common case, kept off the allocator; the owned rows allocate because their text is computed. The tests pin the Perl-compatible `Bool` rendering (`"1"/""`), the single-space `Array` join, the bare-integer `Status` rendering (shell scripts `grep` for the exit code), and the byte-count `len` (so `"é"`, two UTF-8 bytes, has length 2).

is_truthy, every variant

`is_truthy` is the function every conditional jump consults:

Variant	truthy when
<code>Undef</code>	never (false)
<code>Bool(b)</code>	<code>b</code>
<code>Int(n)</code>	<code>n != 0</code>
<code>Float(f)</code>	<code>f != 0.0</code> (so NaN is truthy, since <code>NaN != 0.0</code>)
<code>Str(s)</code>	non-empty and not <code>"0"</code>
<code>Array(a)</code>	non-empty
<code>Hash(h)</code>	non-empty
<code>Status(c)</code>	<code>c == 0</code> (shell: zero exit is success is true)
<code>Ref / NativeFn / Obj</code>	always true

The two cells the whole machine turns on are `Str` (the string `"0"` is falsy, matching shell and AWK) and `Status` (truthy at zero, the inverse of an integer, the inversion the AOT native path's truthiness helper reproduces with `icmp eq, 0` where `Int` uses `icmp ne, 0`). Print these four tables and you have the semantic foundation every fusevm-hosted language stands on, pinned cell by cell so a single-branch refactor cannot silently move one.

Chapter 69 – Cross-Language Value Flow

The thesis of the book is one machine under many languages; this chapter makes that concrete at the level of a single value, tracing how a `Value` produced by one frontend flows to another without translation, because the shared value representation is the mechanism that makes five languages interoperate rather than five islands.

A string made once, used many ways

A `Str` is a `Value::Str(Arc<String>)`, and the `Arc` is what lets it cross boundaries cheaply. Suppose `zshrs` runs a command substitution and produces a string — `Value::str("hello")`, an `Arc<String>` — and hands it back through the host. That same `Value` can be pushed onto the operand stack, stored in a slot, placed in a global, used as a hash key, concatenated, compared — and every one of those operations sees the same `Arc<String>`, cloning it (a reference-count bump) rather than copying its bytes. If a `stryke` function receives that string as an argument, it is the same `Arc`; if it captures it in a closure, the capture is a pointer bump; if it stores it in an array shared with another part of the program, no bytes move. The string is allocated once, by whichever frontend made it, and flows through the machine and between frontends as a shared, reference-counted handle. A test pins this: cloning a `Value::str` shares the `Arc`, verified with `Arc::ptr_eq`. There is no marshalling layer between languages, because there is no boundary — the value is the same value, in the same representation, the whole way through.

An object made by a Lisp, carried by the machine

The harder case is a value with identity — an `elisp` cons cell — and the `Obj` handle is how it flows. When `elisprs` makes a cons cell, the cell lives in `elisprs`'s own heap, and the `Value` that represents it is an `Obj(u32)`, a handle into that heap. `fusevm` carries the `Obj` through its stack, slots, and globals without ever dereferencing it; two `Obj`s are equal exactly when their handles are equal, which is pointer identity. So a cons cell can be pushed, stored, passed to a function, returned, compared with `eq` — all on the machine's general value plumbing — and the machine never needs to understand what it points to. The handle flows; the heap stays `elisprs`'s. This is the value-flow extreme: even a value whose semantics the core cannot model (mutable, shared, identity-bearing structure) flows through the shared representation, because the representation reserved a variant for exactly “a handle to a frontend's private object.”

Why no translation is the whole point

The reason this matters is the reason the architecture exists. In a world of five separate interpreters, a value crossing from one language to another is a *translation* — serialize here, deserialize there, with the cost and the loss that implies. In `fusevm` there is no translation, because there is one `Value` and every language speaks it natively. A number is the same `Int(i64)` whether `awkr`s or `stryke` made it; a string is the same `Arc<String>`; an exit status is the same `Status(i32)` with the same truthiness; a private object is the same `Obj` handle. The day the machine learned to hold float slots in registers, every frontend's float accumulators got faster *because they were the same floats*. The shared value representation is not a convenience layered on top of

five languages; it is the substrate they are all built on, and cross-language value flow is free precisely because there is nothing between the languages to cross.

Chapter 10 — The Edge Cases the Tests Pin

A machine is defined as much by its corners as its common paths, and fusevm's test suite pins a specific set of arithmetic and value edges that every tier must honor identically. This chapter collects them, because a reference reader debugging a surprising result wants to know which corner they have hit.

Integer arithmetic wraps

Integer arithmetic is *wrapping*, not checked: `i64::MAX + 1` is `i64::MIN`, not a panic, because `arith_int_fast` passes `i64::wrapping_add` and its siblings. The compilers emit wrapping `iadd/isub/imul` to agree, and a fuzzer over random integer programs would catch any tier that diverged on overflow. Integer modulo by zero yields zero, not a trap — the interpreter's fallback returns zero, and the AOT native path guards the `srem` with a `select` so it produces zero rather than faulting. And `i64::MIN % -1`, which would overflow a hardware `srem`, is handled by the same guard, because the true quotient overflows but the remainder is zero.

Float edges follow IEEE

Float comparison follows IEEE: NaN is unequal to everything including itself, so `Value::Float(NAN) != Value::Float(NAN)`, and a test pins it. NaN is *truthy*, because `NaN != 0.0` is true, and the native truthiness helper uses an unordered `fcmp ne, 0.0` so it agrees. Signed zero is its own edge: `0.0` and `-0.0` are *equal* under `PartialEq` but have different bit patterns, so the `Hash` impl — which hashes floats by `to_bits` — hashes them differently, and a test pins that `LoadFloat(0.0)` and `LoadFloat(-0.0)` hash apart while comparing equal. This is the deliberate disagreement between equality and hashing the value chapter described, and the edge where it is visible.

Coercion edges

The coercion edges the previous chapter tabulated are pinned individually: an array's `to_int` is its length not its first element; a non-numeric string coerces to 0 / 0.0; `Bool(false)` to floats to 0.0 (the *true*-only arm); a `Status` round-trips its code; the string `"0"` is falsy; `len` is a byte count so `"é"` is 2. Each is a test name in `value.rs`, and each is a place a single-branch refactor could silently shift behavior — which is why they are pinned as a matrix rather than spot-checked.

The gcd/lcm and saturation edges

`GcdInt` returns 0 when both inputs are 0; `LcmInt` returns 0 when either is 0 and saturates at `i64::MAX` on overflow, detected with `umulhi`. `AbsInt` of `i64::MIN` wraps to `i64::MIN` (wrapping abs, built from `ineg` and a `select`). `TruncInt` saturates a float-to-int conversion, NaN to 0, infinities to the i64 bounds, matching Rust's `as i64`. The full-range gcd/lcm fuzzer draws operands across the `i64::MIN` edge and the overflow boundary specifically to exercise these, three hundred pairs checked against the interpreter. These edges are where a careless lowering would crash or diverge, and they are pinned because the native path's whole

claim is bit-for-bit agreement with the interpreter — a claim that is only as strong as its behavior on the inputs designed to break it.

Chapter 71 — The Op Enum's Encoding, in Detail

The opcode set is one Rust enum, and the *encoding* of that enum — how operands are sized, how the whole thing is kept small, how it is hashed — is the substrate every chapter of this book stands on. This chapter collects the encoding details from `op.rs`, because they are the rules a front end emits against and the constraints a maintainer adds opcodes under.

Operand sizes, and what they imply

Operands are sized for their job. A pool index is a `u16`, so a chunk may hold up to sixty-four thousand distinct names or constants — `GetVar(u16)`, `LoadConst(u16)`, `Call(u16, u8)` all index pools with two bytes. A small inline count is a `u8` — `Print(u8)`, `Exec(u8)`, `MakeArray(u16)` (note: the count of elements to pop is a `u16` here, large enough for big literals), `Call`'s argument count. A jump target is a `usize`, because an instruction pointer never needs to be small and the chunk could in principle be large. The three-field loop fusions pack carefully: `SlotIncltIntJumpBack(u16, i32, usize)` is a slot index, an `i32` limit, and a `usize` target — sized so the whole variant stays within the budget. The operand types are a small contract: a front end that needs more than sixty-four thousand names in one chunk would overflow a `u16`, which is why front ends scope their chunks (a function body is a chunk, not the whole program).

The twenty-four-byte budget

A test enforces `size_of::<Op>() <= 24` — three machine words. The reason is the dispatch loop: the op vector is a `Vec<Op>` streamed sequentially, and a smaller `Op` means more opcodes per cache line, fewer line fills, a hotter loop. The budget *forbids* an opcode from carrying a heap payload inline — no `LoadStr(String)`, because a `String` is three words and would blow the budget and bloat every chunk — so strings live in the constant pool and are reached by a `u16` index. It is why constants are pooled, why names are interned, why the largest opcodes use a `u16/i32/usize` triple rather than three `i64`s. The test is the enforcement: a maintainer who adds a variant carrying an inline `String` finds the test red, and the fix is to pool the payload and carry an index. The budget is a small type-level constraint that propagates outward into how the whole machine represents its program.

The Hash impl, payload class by payload class

`Op` has a hand-written `Hash` impl — not derived — because `op_hash` runs it over the op vector and the JIT and disk caches key on the result, so it must be total, stable, and collision-resistant across variants that share a payload value. The impl mixes in `std::mem::discriminant` first, then hashes the payload by class:

- Pool-index ops (`GetVar`, `SetVar`, `GetSlot`, the array/hash accessors, the slot super-ops) hash their `u16` index.
- Jump ops (`Jump`, the `JumpIf*` family) hash their `usize` target.
- The two-operand calls (`Call`, `CallBuiltin`, `CallFunction`) hash both name/id and argc.
- `LoadInt` hashes its `i64`; `LoadFloat` hashes `f.to_bits()`, so `0.0` and `-0.0` hash apart and a NaN hashes by its bits.

- The byte-payload ops (`Print`, `Exec`, `TestFile`, `ExpandParam`, the AWK ops with a `u8`) hash their byte.
- The three-field loop fusions hash all three fields.
- The nullary ops (`Add`, `Pop`, the comparisons, most of the math intrinsics) fall into a single catch-all that contributes only the discriminant.

The exhaustiveness is the point: every one of the two hundred and twenty-four variants has an arm, so there is no op whose identity the hash ignores. The tests pin the consequences — `Jump(7)` and `Call(7, 0)` hash apart (shared value 7, different discriminants); `GetVar(5)` and `SetVar(5)` hash apart; `LoadFloat(0.0)` and `LoadFloat(-0.0)` hash apart; `Call(10, 0)` and `Call(10, 2)` hash apart. A single sloppy arm here would manifest as the cache occasionally running the wrong native code for a chunk, which is why the impl is hand-written and exhaustively tested rather than derived. The `PartialEq` is derived, giving structural equality, and the `Serialize/Deserialize` are derived too, so an op survives a `serde` round trip — pinned by a test over a representative selection including `LoadInt`, `LoadFloat`, `Jump`, `GetSlot`, and the payloaded calls.

Chapter 72 — What the Machine Does Not Do

A machine is defined by its boundaries, and fusevm's boundaries are unusually sharp because so much that a language runtime normally contains has been deliberately pushed out. This chapter collects the refusals — the things fusevm is *not* — because each refusal is a design decision that shapes everything the machine *is*, and a reader who understands the boundaries understands why the core is small enough for one person to hold in mind.

It is not a language

fusevm ships no syntax, no surface, no parser. There is nothing to argue about regarding how a loop should read, because there is no loop syntax — only `Jump` and `JumpIfFalse` and the targets a front end computes. The closest the machine comes to a language is the opcode set, which is a contract, not a grammar. This refusal is what makes the machine reusable: a language is a set of opinions about syntax and semantics, and a reusable engine cannot hold those opinions, because five different languages hold five different ones. By refusing to be a language, fusevm becomes the thing five languages can share.

It is not a memory manager

fusevm has no garbage collector and no allocator of its own. Values that own heap data own it through Rust's ordinary `Arc`, `Box`, `Vec`, and `HashMap` — a `Str` is an `Arc<String>`, an `Array` a `Vec<Value>`, freed by Rust's ownership when the last reference drops. A frontend that needs a cyclic object graph — a Lisp with mutable shared cons cells — keeps that graph in *its own* heap behind an `Obj` handle, and manages its lifetime itself. The machine carries the handle and never owns the object. This refusal is why the core has no GC pauses, no allocator tuning, no memory-management surface at all: memory is Rust's job for the values fusevm owns, and the frontend's job for the objects it does not.

It is not a standard library

There are no string-formatting builtins, no file-system helpers, no networking, no collection algorithms beyond the few aggregate opcodes, in the core. The shell's hundred-and-forty builtins are registered by `zshrs`; `stryke`'s four hundred and fifty extension ops are `stryke`'s; `vimlrs`'s five hundred and ten builtins are `vimlrs`'s. The core provides the irreducible computational and structural primitives — arithmetic, comparison, the aggregate operations, control flow, the host seams — and every library function is a frontend's, reached through `CallBuiltin` or `Extended`. This refusal is what keeps the core small: a standard library is enormous and language-specific, and a shared engine cannot carry five of them, so it carries none and lets each frontend bring its own.

It is not a sandbox

The host trait decides what `exec` does, and a host that spawns real processes spawns real processes. The machine draws a clean line between the pure part (computation) and the effectful part (the host), but the line is a *trait boundary*, not a *security boundary* — a host that reaches the file system reaches the file system, and fusevm does not interpose. Sandboxing, if a frontend wants it, is the host's to implement: a host can run commands in a jail, refuse certain operations, or record rather than execute. The machine provides the seam; the policy is the host's. This refusal is honest about what the trait boundary is for — testability and reusability, not isolation — and a reader building a security-sensitive embedding should implement the policy in the host, not assume the machine provides it.

What the refusals buy

Each refusal pushes a concern out to where it belongs and keeps the engine small. No language means reusable; no GC means no pause and no tuning; no standard library means a core small enough to read in an afternoon; no sandbox means the trait boundary stays about purity and testability rather than trying to be a security model it is not. The report's own framing is that fusevm is “intentionally narrower than the others: it ships no parser, no GC, no stdlib — only the dispatch loop, the JIT bridge, and the extension hooks.” The compactness is the point. A VM you can read end-to-end is a VM whose compilers you can trust, whose semantics you can pin with `contract_` tests, and whose one definition of meaning — `exec_op` — fits in one function. The refusals are not gaps; they are the discipline that makes everything the machine does do well possible, and they are why the marginal cost of the sixth language is a frontend rather than a runtime.

Chapter 13 — The JIT Test Suite, Walked

The differential fuzzers and the contract tests pin the AOT native path; the JIT tiers carry their own test files, and walking them shows how each tier is verified — the linear, block, and tracing tiers each against the interpreter, plus the public API and the disk cache. These are the `jit_*.rs` files under `tests/`, and their structure mirrors the differential discipline applied tier by tier.

The per-tier correctness tests

`jit_linear.rs` verifies the linear JIT: it builds straight-line expression chunks, runs them through `try_run_linear`, and asserts the result matches the interpreter — that `40 + 2` compiles to `Int(42)`, that float promotion works, that the eligible-op set is enforced (a chunk with a branch returns `None`). `jit_block.rs` verifies the block JIT over whole CFGs: it builds loops and branches, runs them through `try_run_block`, and asserts the native function leaves the same slots the interpreter would, including the `SlotKind::Float` path where a slot's kind is passed through `try_run_block_kinded`. `jit_trace.rs` verifies the tracing JIT: it builds hot loops, runs them with `enable_tracing_jit()` past the threshold, and asserts the traced execution matches — exercising the phase-3 branch guards with a `loop_with_branch` shape and the phase-2 inlining with a call in the loop. Each test file is the same shape: build a chunk, run it interpreted and through the tier, assert agreement.

The fuzz and API tests

`jit_fuzz.rs` is the JIT analogue of the AOT fuzzers — a `SplitMix64`-seeded generator building random JIT-eligible chunks and checking the compiled result against the interpreter, so the tier's correctness is tested on combinations no hand-written test would enumerate. `jit_public_api.rs` pins the surface a frontend touches — `JitCompiler::new`, the eligibility predicates, the `try_run_*` entry points — so a refactor that changed a signature fails a test rather than a frontend. `jit_api_pool_deopt.rs` exercises the interaction of the JIT with the `VMPool` and the deopt path. And `jit_disk_cache.rs` (gated on the feature) verifies the cache is behavior-transparent: it runs a chunk with the cache cold and again with it warm, and asserts the results are identical and the second run loaded rather than recompiled — the transparency guarantee made executable.

What the suite guarantees together

The JIT tests and the AOT fuzzers together cover the whole compilation surface differentially: every tier, from the linear JIT's straight-line expressions to the AOT native path's register loops, is run two ways and checked against the interpreter that is the specification. The public-API tests pin the surface, the fuzz tests pin the combinations, the disk-cache tests pin the transparency, and the contract tests pin the invariants. There is no tier whose correctness rests on inspection alone; each is measured against the one implementation that is correct by definition. This is the executable form of the single-source-of-truth principle — five execution strategies, one definition of meaning, and a test for every strategy asserting it agrees with the one — and it is

why the compilation half of the machine could grow as aggressively as it did without accumulating risk: every addition was either provably correct against the interpreter or caught by a fuzzer as not.

Chapter 14 — Error Reporting and Diagnostics

A VM that only computed correct programs would not need diagnostics, but real programs have errors, and fusevm carries a small, deliberate diagnostic apparatus — the `lines` array, the `source` field, and the `Error` result — that this chapter walks, because how a machine reports a fault is part of its contract with the frontend driving it.

The lines array

A `Chunk` carries a `lines: Vec<u32>` parallel to its `ops` — one source line per instruction, recorded by `emit(op, line)`. This array does no work during execution; the dispatch loop never reads it. Its sole purpose is error reporting: when a runtime fault occurs at instruction `ip`, the frontend can map `ip` to `lines[ip]` to report the source line the failing opcode came from. This is why `emit` takes a line argument that every example in this book passes — `b.emit(Op::Add, 1)` records that this `Add` came from source line 1 — and why the line is never shown in a disassembly: it is metadata for diagnostics, not part of the program's behavior. A test pins that the lines array stays parallel to the ops and that `op_hash` ignores it, so two chunks differing only in line numbers share compiled code.

The source field

Alongside `lines`, the `Chunk` carries a `source: String` — the file name — set by `set_source` and used to qualify an error message with the file the fault occurred in. Between `source` and `lines[ip]`, a frontend has the file and line of any failing instruction, which is enough to produce a `file:line: message` diagnostic in the shape every programmer expects. Like the lines array, `source` does no work during execution and is part of the diagnostic apparatus, not the computation.

The Error result

The runtime channel for a fault is `VMResult::Error(String)`. When the interpreter hits a genuine runtime error — a fatal AWK division by zero, a host operation that failed irrecoverably — it constructs a message and returns `Error(msg)`, which `run` propagates to the frontend driver. The message is a `String`, human-readable, often the exact text a reference implementation produces (the gawk `"division by zero attempted"` for `AwkDiv`). The frontend driver branches on the result and decides what to do — print the message and exit, or recover — and a frontend that wants a `file:line:` prefix combines the `Error` message with the `source` and `lines[ip]` of the failing instruction. The three pieces — `lines` for the line, `source` for the file, `Error` for the message — are a minimal but complete diagnostic apparatus: enough to point a programmer at the failing line, kept entirely out of the execution path so it costs nothing when no error occurs. The design is the same as everywhere else: the common case (no error) pays nothing, and the rare case (an error) has exactly the metadata it needs and no more.

Chapter 75 – The Chunk’s Supporting Tables, in Use

The chunk chapter listed the `Chunk`’s nine fields; the executing three — ops, constants, names — have appeared throughout, but the supporting tables deserve a walk of their own, because each exists for a specific construct and seeing it used makes the chunk’s shape complete.

sub_entries: where functions begin

`sub_entries: Vec<(u16, usize)>` maps a name-pool index to the op index where a subroutine begins. When a front end compiles a function, it records the function’s entry with `add_sub_entry(name_idx, ip)`, and a `Call(name_idx, argc)` resolves the name through `find_sub` to that entry index. The recursive Fibonacci of Chapter 61 had its body’s entry recorded here; every `Call` is a `sub_entries` lookup followed by a frame push and a jump to the entry. `find_sub` returns the first match for a name, so a front end that registered two entries under one name gets the first — a detail a test pins.

block_ranges: where higher-order bodies live

`block_ranges: Vec<(usize, usize)>` records the `(start_ip, end_ip)` of the bodies of `map`, `grep`, `sort`, and `foreach`. A front end emits the body into the op vector like any other code, registers its range with `add_block_range` to get a `u16` index, and emits `MapBlock(index)` over the array. The VM runs the op range once per element. This is the labeled-span representation of Chapter 10 — a body of code as data without a closure object — and `block_ranges` is the table that holds the spans. A test pins that ranges get sequential indices and store their pairs faithfully.

sub_chunks: nested execution units

`sub_chunks: Vec<Chunk>` holds whole nested chunks — the bodies of `$(cmd)` command substitutions, `<(cmd)/>(cmd)` process substitutions, trap handlers, with-redirect blocks. Each is referenced by the `u16` payload of the op that spawns it: `CmdSubst(idx)`, `ProcessSubIn(idx)`, `TrapSet(idx)`. A front end builds the body with `add_sub_chunk(sub)` to get an index, and the host runs `&parent.sub_chunks[idx]` on a fresh or pooled VM, isolating it from the parent — the isolation mechanism of Chapter 16. A test pins that sub-chunks preserve their inner content and get sequential indices, and that a nested chunk’s ops survive the round trip.

lines and source: diagnostics

`lines` and `source`, walked in the previous chapter, complete the supporting tables: one source line per op for error reporting, the file name for qualification. Together with the executing three and the cached `op_hash`, these six supporting fields make the `Chunk` the single self-contained unit of execution — everything the VM, the host, and the diagnostics need to run a program and report on it, in one structure. A front end populates them through the `ChunkBuilder` methods that mirror each — `emit`, `add_constant`, `add_name`, `add_sub_`

`entry`, `add_block_range`, `add_sub_chunk`, `set_source` — and `build` seals it with the hash. The chunk is flat in its op vector *because* the genuinely nested and metadata-bearing parts were factored into these tables, which is what keeps the dispatch loop a tight scan over a vector and the compilers able to analyze a clean linear sequence.

Chapter 16 – The op Vector as Data: Layout and Dispatch

The book has treated the op vector as a sequence of instructions; this closing reference chapter treats it as *data* — a `Vec<Op>` with a memory layout and a cache behavior — because the machine's speed at the bottom rests on this vector being a dense, sequentially-scanned array, and understanding it as data explains the dispatch floor every benchmark is denominated in.

A dense array of small records

An `Op` is at most twenty-four bytes, enforced by a test, so a `Vec<Op>` is a contiguous array of twenty-four-byte (or smaller, padded to the enum's size) records. There is no indirection: the ops are inline in the vector, not boxed behind pointers, so walking the vector is a sequential scan over contiguous memory — exactly the access pattern a CPU's prefetcher is built for. This is why the opcode size budget matters: a fatter `Op` would mean fewer ops per cache line, more line fills per loop, a colder scan. The constant pool exists precisely to keep the op vector dense — a string lives in the pool and the op carries a two-byte index, so the op vector stays an array of small records rather than an array of fat ones with inline strings.

The dispatch loop as a scan

The interpreter's dispatch loop is a `while` over this vector: read `ops[ip]`, advance `ip`, match. The match compiles to a jump table over the enum discriminant — a single indexed branch, not a chain of comparisons — so dispatching one opcode is: an array load (the op), an increment (the ip), an indexed branch (the match), and the arm's work. For a `Nop` the arm's work is nothing, so a loop of `Nops` measures exactly the array-load-increment-branch cost: the 1.22-billion-operations-per-second dispatch floor, about eight hundred picoseconds per op. Every real opcode costs this floor plus its arm's work, which is why removing dispatch — fusing several ops into one, or compiling the loop away entirely — is where the speedups live: for a cheap op like `Add`, the dispatch is a large fraction of the total, and a fused loop or a compiled function pays the floor once or never instead of per iteration.

Why flat beats a tree

The alternative to a flat op vector is a tree — an AST the interpreter re-walks — and the op vector's advantages over a tree are exactly the layout advantages. A tree is pointer-chasing: each node is a heap allocation reached through a pointer, so walking it is a sequence of cache-missing indirections, and a loop that runs a million times walks the same subtree a million times paying the indirection each pass. A flat op vector is a sequential scan with no indirection, the instruction pointer a `usize` whose advance is an add and whose jump is an assignment. The front end pays the cost of flattening the tree into the vector *once*, at compile time — the back-patching of forward jumps, the slot-number assignment, the constant pooling — and the runtime never pays it again. This is the bytecode-VM bargain at its most fundamental: trade a one-time compile for a dense, sequentially-scanned instruction stream, and the per-iteration cost collapses from a tree walk to an array scan. Everything above it — the fused superinstructions, the three JIT tiers, the AOT native path — is an optimization on top of this bargain, removing dispatch from the hot scan or replacing the scan with native

code; but the bargain itself, the flat op vector as a dense array of small records, is the foundation the whole machine's speed is built on, and it is why a chunk is a `Vec<Op>` and not a tree.

Chapter 77 – Threaded and Native, the Same Chunk Both Ways

The AOT compiler chooses per chunk between the threaded path and the native path; this chapter compiles *one* chunk both ways and sets the results side by side, because the contrast is the clearest statement of what “lower what you can prove, fall back on the rest” costs and buys. The chunk is a small one that *could* go native — straight-line integer arithmetic — so we can see the same program as a threaded jump table and as register code.

The chunk

The chunk computes $(a + b) * 2$ with a, b in slots 0 and 1:

```
0  GetSlot  0      ; [a]
1  GetSlot  1      ; [a, b]
2  Add      ; [a+b]
3  LoadInt  2      ; [a+b, 2]
4  Mul      ; [(a+b)*2]
```

`analyze_native` admits every op — slot reads, integer arithmetic, an integer load — so the native path applies. But the threaded path *also* applies (it applies to everything), so we can compile both.

The threaded version

The threaded path emits one Cranelift block per opcode plus an entry, a dispatch, and a return block. Each op block calls `fusevm_aot_exec_op(vm, i)` — the same `exec_op` the interpreter uses — and returns the next index to a central `Switch`. In pseudo-CLIF:

```
entry:      jump dispatch(0)
dispatch(ip): switch ip -> [b0, b1, b2, b3, b4] (default -> ret)
b0:  next = exec_op(vm, 0); brif next<0, ret, dispatch(next)    ; GetSlot 0 via interpreter
b1:  next = exec_op(vm, 1); brif next<0, ret, dispatch(next)    ; GetSlot 1
b2:  next = exec_op(vm, 2); brif next<0, ret, dispatch(next)    ; Add
b3:  next = exec_op(vm, 3); brif next<0, ret, dispatch(next)    ; LoadInt 2
b4:  next = exec_op(vm, 4); brif next<0, ret, dispatch(next)    ; Mul
ret: finish(vm); return 0
```

Every opcode runs through the interpreter’s `exec_op`, against the boxed `vm.stack`. The native control flow has replaced the dispatch *loop*, but the work each op does is unchanged — `GetSlot` still reads the boxed frame, `Add` still boxes through `arith_int_fast`. The speedup over interpreting is the removed decode-and-dispatch overhead, modest, and the cost is a function call into the interpreter per op.

The native version

The native path lowers the same chunk to register code with no per-op call:

```

function %chunk(i64 slots) -> i64 {
  block0(v_slots: i64):
    v_a = use_var slot0
    v_b = use_var slot1
    v_ab = iadd v_a, v_b          ; Add, wrapping
    v2 = iconst.i64 2
    v_r = imul v_ab, v2          ; Mul
    ; box v_r as Value::Int into the VM result
    return v_r
}

```

No boxed stack, no `exec_op` calls — `a` and `b` are read into registers, added and multiplied with two native instructions, and the result boxed once at the end. The two compilations of the identical chunk are the whole AOT story in miniature: the threaded version is total and modestly faster than interpreting, running every op through the one definition of meaning; the native version is fast but applies only because `analyze_native` could *prove* every op lowerable. `build_entry` chooses the native version here because the analysis returned a plan; for a chunk with a string op or a host call it would choose the threaded version, which always works. Same chunk, two compilations, one chosen by what the analysis can prove — and the threaded version standing behind the native one as the fallback that makes the native one safe to attempt.

Chapter 78 – A Boundary Loop, Spilled and Continued

The inline/shim boundary lets a native loop reach a boxed opcode without leaving native execution; this chapter traces one concrete case — a hot numeric loop that prints its running total each iteration — through the spill, the call, and the continue, because the boundary is what turns “native loops only if pure” into “native loops with a print in them,” and seeing it work is seeing wholesale fallback avoided.

The loop

The loop is `i = 0; while i < n { sum += i; print sum; i += 1 }`, with `i` in slot 0, `sum` in slot 1, `n` in slot 2. Its disassembly:

```
0 LoadInt      0 ; [0]
1 SetSlot      0 ; []      i = 0
2 GetSlot      0 ; [i]      header
3 GetSlot      2 ; [i, n]
4 NumLt        ; [i<n]
5 JumpIfFalse  13 ; []      exit
6 GetSlot      1 ; [sum]
7 GetSlot      0 ; [sum, i]
8 Add          ; [sum+i]
9 SetSlot      1 ; []      sum += i
10 GetSlot     1 ; [sum]
11 Print        1 ; []      print sum  <-- a boxed sink op
12 PreIncSlotVoid 0 ; []      i += 1
13 Jump         2 ; []      ... actually back to 2 (header); exit target patched
~ elsewhere
```

The loop is almost entirely scalar — slot arithmetic and a comparison — except for the `Print` at index 11, which is a heap-touching opcode. Without boundary handling, that one `Print` would force `analyze_native` to decline the whole loop to the threaded path, and a hot numeric loop would run at threaded speed because it printed.

The spill, call, continue

`Print` is a *sink*: it consumes scalar values and produces nothing dynamic. So the native path handles it without leaving native execution. At index 11, the native code holds `sum` in a register. To run the `Print`, it:

```
print_site:
; spill the one operand from its register onto the boxed stack
call push_int(vm, v_sum)      ; box sum as Value::Int, push to vm.stack
; call the opcode through the interpreter shim
call fusevm_aot_exec_op(vm, 11) ; Print(1) pops one value and emits it
; vm.stack is now empty again; continue in registers
; ... fall through to i += 1, all in registers
```

The boxed `vm.stack` is empty before the spill and empty after the call — `Print` popped exactly the one value spilled — so nothing has to be reloaded into a register and no type guard is needed. The loop's arithmetic stays in registers across the print: `sum` and `i` are register-resident the whole loop, and only at the print does the native code briefly box one value, call the shim, and return to registers. A test, `native_loop_with_print_lower5`, confirms exactly this loop lowers rather than bailing.

Why this is the unlock

The contrast is stark. Wholesale fallback would run every iteration of this loop through the interpreter, because the loop contains a `Print`. Boundary handling runs the arithmetic natively and pays the spill-call-continue only at the print, a few instructions per iteration against the alternative of interpreting the whole loop. This is what makes “native loops with side effects” possible, and it generalizes: `PrintLn` and `SetStatus` are sinks handled the same way; `AwkGetFieldNum` and `GetStatus` are statically-typed sources reloaded with no guard; and the general box/unbox engine stages many heap ops with statically-known result kinds the same way. The boundary is the difference between a native compiler that handles only pure scalar loops and one that handles the loops real programs write — which print, set status, read fields — and it is built from one spill-call-continue mechanism applied wherever a scalar loop meets a boxed op whose effect on the value stack the analysis can account for.

Chapter 79 – Provisional Frontiers

This book has described a young machine, and honesty about a young machine means being precise about its frontier — what is finished, what is in progress, and what is named-but-not-yet. This chapter collects the provisional edges, clearly marked, so a reader weighing fusevm knows where the line currently is and which way it moves. Everything here is flagged in the source or the version as not-yet-done, and the book has noted each at the point it arose; this chapter gathers them.

The dynamic-kind sources

The largest open frontier is in the AOT native path: the *dynamic-kind source* opcode. The native path lowers sinks and statically-typed sources at the boundary, but a source whose result kind is not known until runtime — a general array read that might return an integer, a float, or a string; `ReadLine`; an arbitrary call — still falls back to the threaded path, because there is no single register lane to unbox an unknown kind into. This is *technical* future work, not a permanent limitation: the deopt machinery that already works for `Div` provides the mechanism, a runtime type-guard-then-deopt — assume a kind, check it at runtime, bail to the interpreter if wrong. The book has described the deopt as the mechanism that will eventually unblock this, and a later version of fusevm may well lower dynamic-kind sources behind a guard. As of `0.14.5` they fall back, and this chapter marks that as the active frontier rather than a finished edge.

The threshold comments

A smaller provisional edge is documentary: a few constants carry doc comments that disagree with the code. The block-JIT warm-up threshold is the notable one — a comment in places calls it 10, while `TraceJitConfig::default()` sets it to 1. This book has followed the code, not the comment, at each such point and flagged the disagreement, because the code is what runs. A reader checking the source against this book should expect the occasional stale comment and trust the code; the disagreements are the ordinary drift of a moving project, not errors in either the code or this account, but they are provisional in the sense that a future version will reconcile them.

What is finished, for contrast

It is worth stating, against the frontiers, what is *not* provisional, so the open edges are seen in proportion. The interpreter is finished and is the specification every tier is tested against. The value model, the two-hundred-and-twenty-four-opcode set, the host and AWK trait boundaries, and the extension mechanism are finished and heavily tested. The three JIT tiers — linear, block, tracing with its nine phases — are finished within their documented capability matrix. The AOT threaded path is finished and total. The AOT native path is finished for the scalar core, the kind lattice across all five kinds, the slot and global register residency, the math intrinsics, the trapping AWK ops, the deopt, and the sink-and-static- source boundary; it is a frontier only for the dynamic-kind sources. The disk cache is finished across all three tiers. Five frontends are in production against all of it. The frontier is a thin, well-marked line at the edge of a large finished machine — and the discipline that marks it (the fuzzers reporting declines, the benchmarks reporting losses, the version warning the line still moves) is the same discipline that built the finished part. A companion to a moving project can do no better than to describe the machine precisely and point, as the source does, exactly where it is still moving — which is here, at the dynamic-kind source, with the mechanism to close it already built and waiting.

Chapter 80 – The Classic Algorithms as Chunks

The benchmark table reported times for the classic algorithms; this chapter shows them as chunks, because the bytecode shape of each explains why it benchmarks as it does — why iterative Fibonacci is microseconds and recursive Fibonacci milliseconds, why the dispatch floor is what it is. Reading the chunk is reading the cost.

Iterative Fibonacci

`fib_iterative(n)` keeps two running values and a counter, and its loop is a counted accumulator the peephole fuses. With the two Fibonacci values in slots 0 and 1 and the counter in slot 2, the hot part is a fused loop — the loop body updates the pair and increments the counter, the whole thing one or a few dispatches per iteration. At `n = 35` it runs in 2.7 microseconds, because the loop is essentially a counted sum's cousin: register-friendly integer arithmetic over a slot-based counter, exactly the shape the fused superinstructions and the block JIT are tuned for. The chunk is small, the loop is tight, and the per-iteration cost is a handful of integer operations plus one fused dispatch — which is why iterative Fibonacci is among the fastest benchmarks.

Recursive Fibonacci

`fib_recursive(n)` is the chunk of Chapter 61 — a function that calls itself twice — and its cost is entirely different in kind. At `fib(20)` it makes 21,891 calls in 1.28 milliseconds, about sixty nanoseconds per call, and that sixty nanoseconds is *call overhead*: a `call` resolves the name through `sub_entries`, pushes a `Frame` with its `return_ip` and `stack_base` and a fresh `slots` vector, binds the argument, and jumps to the entry; a `ReturnValue` pops the frame, restores the stack, and resumes. The arithmetic in `fib` is trivial — one subtraction and one addition per call — so the benchmark measures the calling convention, not the math. This is why recursive Fibonacci is three orders of magnitude slower than iterative: the iterative version pays a fused dispatch per iteration, the recursive version pays a frame push-and-pop per call, and a frame push is far more than a fused dispatch. The contrast is the clearest single illustration of why the fused loops exist — they turn the recursive-style per-step overhead into the iterative-style register arithmetic wherever the shape allows.

Ackermann

`ackermann(3, 4)` makes 10,547 calls in 774 microseconds, and it is recursive Fibonacci's cost amplified: deep, irregular recursion with little arithmetic per call, so again the benchmark is dominated by the frame machinery. Ackermann is in the suite precisely because it stresses the call stack hard — deeply nested frames, each with its own slots — and it is where the per-frame slot model is exercised most. That it runs at all without special handling is the point: the calling convention is five opcodes and an ordinary call stack, and even Ackermann's pathological recursion is just frames pushed and popped, fresh slots each time.

The dispatch floor and the fused sum

Two benchmarks bracket the others as the unit and the headline. The dispatch floor — a million **Nops** in 819 microseconds, 1.22 Gops/sec — is the cost of pure dispatch, the array-load-increment-branch of Chapter 76, and it is the denominator every other number is measured in. The fused sum — **sum(1..1M)** as one **AccumSumLoop** in 142 nanoseconds against 31 milliseconds unfused — is the headline, the factor of more than two hundred thousand that is the entire fused-superinstruction argument reduced to two measurements. Read the classic algorithms as chunks and the benchmark table stops being a list of claims and becomes arithmetic: the iterative loops are fused register arithmetic over the dispatch floor, the recursive ones are frame machinery, and the gap between them is the gap between a fused dispatch and a frame push — which is why the fused loops are where the speed is and the calls are where it is not.

Chapter 81 – Allocation Discipline: Arc, In-Place, and the Pool

Speed at the bottom is not only about dispatch; it is about not allocating, and fusevm has a coherent allocation discipline that this chapter collects, because the three mechanisms that keep it off the allocator — the `Arc<String>`, the in-place aggregate mutation, and the `VMPool`'s capacity reuse — are each described in their own chapter but are one strategy seen three ways.

Strings allocate once

A `Str` is a `Value::Str(Arc<String>)`, and the `Arc` means a string is allocated once and shared thereafter by reference count. A clone is a pointer bump, not a byte copy. A string flowing across the stack, into a slot, between frontends, captured by a closure — every one of those is a reference-count increment, with the bytes allocated exactly once by whichever frontend made the string. And `as_str_cow` extends the discipline to *reading* a string: a `Str` borrows its inner buffer rather than allocating to read it, so string comparison, concatenation, hash-key lookup, and I/O run against a borrow on the common case. The string discipline is: allocate once at creation, share by reference count, read by borrow — and the only allocations a string-heavy program pays are the genuine creations, not the copies and reads.

Aggregates mutate in place

An `Array` is a `Vec<Value>` and a `Hash` a `HashMap<String, Value>`, and the aggregate opcodes mutate them *in place* in the globals vector — `ArrayPush`, `ArraySet`, `HashSet`, `HashDelete` reach the container where it lives and modify it, never materializing it on the operand stack, never cloning it. The read-only accessors borrow from the globals vector. The discipline turns the naive clone-modify-writeback — which makes a loop building an N-element array $O(N^2)$ in copies — into a genuine $O(N)$: each push is one element appended in place, no aggregate copied. This is why the `PushIntRangeLoop` fused superinstruction can build a million-element array in a tight loop: there is no per-element copy for it to eliminate, only the per-element dispatch, because the in-place model already eliminated the copy. The aggregate discipline is: mutate where it lives, borrow to read, never clone-and-write-back.

The pool reuses capacity

The third mechanism is the `VMPool`, which recycles `VM` instances so a caller running many chunks reuses the `Vec` capacities of the stack, the frame slots, and the globals across runs. `VM::reset` clears the contents but preserves the capacities, so a sequence of similarly-shaped chunks reallocates nothing — the stack that grew to a hundred entries on one run is reused at that capacity on the next. The pool's discipline is honest about its cost: for a tiny chunk it loses, because `reset` does more bookkeeping than `VM::new` skips; for a non-trivial chunk shape it wins, because the preserved capacity dominates. The frontend bridges put a pool on their `Runtime` precisely because they run many chunks with non-trivial shapes, and the capacity reuse is the allocation the pool exists to avoid.

One discipline, three mechanisms

Read together, the three are one strategy: do not allocate what you can share, mutate, or reuse. Share strings by reference count instead of copying them. Mutate aggregates in place instead of cloning them. Reuse VM capacities across runs instead of reallocating them. Each mechanism targets a different allocation a naive runtime would pay — the string copy, the aggregate writeback, the per-run vector allocation — and each replaces it with a cheaper operation: a reference bump, an in-place mutation, a capacity reuse. The discipline matters because allocation is the cost that does not show up in a dispatch count but dominates a real program's time: a loop that copies a growing array on every iteration is slow no matter how fast its dispatch. fusevm's speed at the bottom is dispatch *and* allocation — the flat op vector keeps dispatch cheap, and the `Arc`, the in-place mutation, and the pool keep allocation rare — and the two together are why a scripting language on this machine can post the numbers it does. The allocation discipline is the quiet half of the performance story, less visible than the JIT but no less load-bearing: a machine that allocated freely would be slow with the best JIT in the world, and fusevm's refusal to allocate what it can share, mutate, or reuse is what lets the JIT's speedups be the whole story rather than a partial recovery from allocator overhead.

The refcount is atomic, and that is the price of migration

One detail of the string discipline is worth drawing out, because it is where the allocation strategy meets the concurrency model: the `Arc` in `Value::Str(Arc<String>)` is *atomically* reference-counted, and the atomicity is not free. A non-atomic `Rc` would make each clone a plain increment of an ordinary integer; the `Arc`'s clone is an atomic increment, a memory operation the hardware must order against other cores' accesses, and on the hot path of a string-heavy loop those atomics are measurably more than plain increments. The machine pays that cost deliberately, because the atomic refcount is exactly what lets a string cross a thread boundary safely — an `Rc` string could not be shared with a worker thread without a data race on the count, while an `Arc` string can be handed to another VM on another core by the same reference-count bump that shares it within a thread. So the choice of `Arc` over `Rc` is the allocation discipline and the concurrency model deciding the same question together: the machine wants strings cheap to share, and it wants them shareable across threads, and `Arc` gives both at the cost of an atomic increment per clone. It is the same trade the concurrency chapter frames from the other side — a handful of atomic operations at the boundaries, in exchange for a hot path that never locks — seen here as an allocation decision: the string that allocates once and shares by reference count shares by an *atomic* count precisely so that the sharing can reach another core, and that atomic is the quiet, per-clone price the machine pays so that its cheapest operation, cloning a string, is also a thread-safe one.

Chapter 82 — Reading the Source: A Guided Tour

This book has grounded every claim in the eleven production files of `src/`; this closing chapter is a reading order for those files, because the best companion to a companion book is the source itself, and the eleven files have a natural sequence that mirrors the book's movements. A reader who wants to verify this account, or to extend the machine, should read them roughly in this order.

Start with the data: `value.rs`, `op.rs`, `chunk.rs`

Begin with `value.rs` — roughly six hundred and forty lines — because the `Value` enum is the substrate everything moves. Read the eleven variants, the four coercions (`to_int`, `to_float`, `as_str_cow`, `is_truthy`), and the hand-written `Hash` impl, then read the property tests at the bottom, which pin the coercion matrix cell by cell and are the executable form of Chapter 68. Then `op.rs` — about twelve hundred and fifty lines — the `Op` enum with its two hundred and twenty-four variants, the `file_test/redirect_op/param_mod` constant modules, the hand-written `Hash` impl by payload class, and the size-budget test. Read the doc comments on the opcodes; they are the source of every stack effect in the reference movement. Then `chunk.rs` — about four hundred and sixty lines — the `Chunk` struct's nine fields, the `ChunkBuilder` and its back-patching, and the op-hash and round-trip tests. With these three you have the data the machine moves, which is the first movement of this book.

Then the machine in motion: `vm.rs`

Next read `vm.rs` — about fifty-two hundred and seventy lines, the largest file after the compilers. This is the `VM`, the `Frame`, the `VMPool`, the dispatch loop, the `exec_op` handler with its arm per opcode, the fast-path helpers `arith_int_fast` and `cmp_int_fast`, and the AOT shim entry points. It is long because it is the entire interpreter plus the tier-decision logic plus the per-opcode semantics — the single source of truth every other tier is tested against. Read the dispatch loop and `exec_op` first; they are Chapters 13 and 14. The fast-path helpers are the specification the compilers match. This file is the third movement.

Then the embedding: `host.rs`, `awk_host.rs`, the builtins

Read `host.rs` — about four hundred and twenty lines — the `ShellHost` trait, `DefaultHost`, and the default-impl tests; the defaults are the map of the shell-core boundary, Chapter 62. Then `awk_host.rs` — about eight hundred lines — the `AwkHost` trait, `AwkLvalue`, `DefaultAwkHost`, and the chrono-backed time builtins, Chapter 63. Then the ID tables: `awk_builtins.rs` — about two hundred and seventy lines, the reserved AWK op range and the per-op IDs — and `shell_builtins.rs` — about a thousand and seventy lines, the hundred and forty banded shell builtin IDs. These four files are the fourth movement, the protocols a frontend drives.

Then the compilers: `jit.rs`, `aot.rs`

The two largest files are the compilers, and they are the fifth movement and the heart of the worked walkthroughs. `jit.rs` — about seventy-seven hundred and thirty lines — is the three JIT tiers, the `JitCompiler`, the `Cell` lattice, the trace recorder and its nine phases, the disk cache, and the libcall shims. `aot.rs` — about fifty-seven hundred and ninety lines — is the closed-world AOT compiler: `analyze_native`, the threaded and native `build_entry` paths, the `Kind` lattice, `emit_deopt`, `compile_object`, the shim roster, and the differential fuzzers. These two are more than half the production source between them, which is the proportion Chapter 35 and Appendix H noted: the data model is small and hard to design, the compilers are large and hard to write, and the line count reflects where the engineering hours went. Read `aot.rs`'s fuzzers last; they are the executable form of the differential-correctness chapter, and reading them is the surest way to believe the native path's claims.

And the root: `lib.rs`

Finally `lib.rs` — about seventy lines — the crate root: the module declarations, the public re-exports (Appendix F), the architecture diagram, and the tracing-JIT capability matrix (Chapter 67). It is short, and it is the right last read because it is the table of contents to everything else and the project's own honest statement of what is finished and what is frontier. Read in this order — data, machine, embedding, compilers, root — the eleven files retrace the five movements of this book, and the book is, in the end, an expansion of exactly what those files say, with every opcode name, type, constant, and behavior traceable back to the line that defines it. The source is the ground truth; this book is the path through it; and a reader who has both has the machine whole, from the `Value` that moves to the native binary that ships, all twenty-three thousand seven hundred and eighty-one lines of it, one machine under five languages.

Chapter 83 – The Extension Handler Protocol, in Code

The extension mechanism was surveyed across the frontends in Chapter 52 and its ID tables walked in Chapter 64; this chapter walks the *protocol* itself in code — how a handler is installed, how the VM dispatches to it, and how the borrow dance lets it reach back into the machine — because the protocol is the seam every non-core capability flows through, and seeing it in operation completes the embedding picture.

Installing the three kinds of handler

A frontend installs three kinds of handler on a VM, each for a different door into the extension space. The narrow extension handler takes a small inline operand; the wide one takes a `usize` payload; the builtin table is a flat dispatch by ID. The installation:

```
let mut vm = VM::new(chunk);

// narrow Extended(id, arg) - id: u16, arg: u8
vm.set_extension_handler(Box::new(|vm, id, arg| {
    match id {
        0 => { /* language op 0: read vm.stack, push a result */ }
        1 => { /* language op 1 */ }
        _ => {}
    }
}));

// wide ExtendedWide(id, payload) - id: u16, payload: usize
vm.set_extension_wide_handler(Box::new(|vm, id, payload| {
    // ids in [40000, 41000) are routed to the AwkHost instead, not here
    match id { /* ... */ _ => {} }
}));

// flat builtin table - CallBuiltin(id, argc)
vm.register_builtin(BUILTIN_CD, |vm, argc| { /* ... */ Value::Status(0) });
```

Each handler is a closure the VM holds. The narrow and wide handlers are `match`-on-id dispatchers, suited to a frontend like `stryke` with hundreds of distinct ops; the builtin table is an indexed set of closures, suited to a frontend like `vimlrs` with five hundred-odd flat library functions. A frontend picks the door that fits its vocabulary's shape, and none collide because each owns its ID space — and because the wide handler's 40,000-range is reserved for the AWK host, so an AWK frontend's wide ops route to `AwkHost` while a generic frontend's low-id wide ops route to its own handler.

Dispatch and the borrow dance

When the VM's dispatch loop reaches an `Extended(id, arg)`, it must call the installed handler — and the handler needs to reach back into the VM (read the stack, push a result, consult globals) while the VM is the thing that owns it. Rust will not lend the handler a mutable reference to a VM the handler is borrowed from, so the VM performs a take-out-and-restore dance: it moves the handler out of itself into a local (leaving a

placeholder), calls the handler passing `&mut self` to the now-handler-less VM, and moves the handler back in afterward. In schematic form:

```
exec Extended(id, arg):
  let handler = self.extension_handler.take(); // move out, leaving None
  if let Some(h) = handler {
    h(self, id, arg); // handler runs with &mut self
    self.extension_handler = Some(h); // restore
  }
```

The same dance governs the wide handler, the builtin table, the shell host, and the AWK host — every reach from effectful or language-specific code back into the pure machine. It is invisible from the handler's side: the handler simply receives an `&mut VM` and does its work. The one constraint it imposes is that a handler cannot re-entrantly trigger another call that would try to take the already-taken handler — which is why a handler does its work directly rather than recursing through the same dispatch door.

Obj handles across the seam

The protocol's most powerful use is carrying a frontend's private objects, and the `Obj` handle is how. An elisp `(setcar x v)` lowers to an `Extended` op whose handler mutates the cons cell at the handle's index in elisprs's heap:

```
0 GetSlot 0 ; [<obj:cell>] x, an Obj(handle) into elisprs's heap
1 GetSlot 1 ; [<obj:cell>, v]
2 Extended 7, 0 ; [] handler pops v and the Obj, does heap[cell].car = v
```

The handler pops the `Obj` and the value from `vm.stack`, reaches into elisprs's own heap at the handle's index, and mutates the cons cell in place — all through the `&mut VM` the borrow dance lent it, with fusevm never understanding what a cons cell is. The `Obj` flowed onto the stack through the general value plumbing, the handler reached it through the extension protocol, and the mutation happened in the frontend's heap. This is the protocol at its fullest: a capability the core cannot model (mutable identity-bearing structure), reached through one handler closure, against one opaque handle, with the borrow dance making the reach type-safe. Three doors — narrow, wide, builtin — one borrow dance, and the `Obj` handle for private state: that is the entire extension protocol, and it is enough to layer a Lisp, a shell, an AWK, a vimscript, and a systems scripting language on one machine that knows none of them.

Chapter 84 – The Single Thread and the Shape of Concurrency

The host-boundary chapter noted, almost in passing, that both host traits are `Send`, and that the bound is what lets a configured VM migrate to a worker thread. That one sentence deserves a whole chapter, because the concurrency model it implies is the exact opposite of the one most virtual machines reach for, and the shape of it — one thread per VM, migration but not sharing, immutability across the seam — is a design decision as load-bearing as the flat chunk or the indexed slots.

One VM, one thread, at a time

The first thing to understand is that a `VM` is a single-threaded object, and deliberately so. It owns a stack, a frame stack, a globals vector, an instruction pointer, and a boxed host, and nothing in that list is guarded by a lock, an atomic, or a channel. `exec_op` reads and writes those fields directly, with no synchronization, because it assumes it is the only thing touching them — and it is, because a `VM` is never shared between threads while it runs. This is not a limitation the machine apologizes for; it is the source of the tight dispatch loop. A stack machine whose every push had to take a lock, or whose every slot write had to be an atomic store, would pay a synchronization tax on every opcode, and the dispatch floor of Chapter 76 — the array-load-increment-branch that runs at over a gigahertz of opcodes per second — exists precisely because there is no such tax. Single-threaded execution is what makes the interpreter fast, and the JIT and the AOT native path inherit that assumption wholesale: the compiled code for a chunk holds locals in machine registers with no memory barriers at all, because the registers belong to one thread and no other thread can observe them.

Send, not Sync: the deliberate line

The concurrency the machine *does* offer is migration, not sharing, and the distinction is the whole model. Rust draws two lines around a type: `Send` means the type can be *moved* to another thread, and `Sync` means a *shared reference* to it can cross a thread boundary — that many threads may hold `&T` at once. The host traits require `Send`, and the VM built on them is `Send`; neither is `Sync`, and that is not an oversight. A `Send`-but-not-`Sync` VM is one you can hand to a worker thread whole — the stack, the frames, the globals, the host, all of it moving together — but one you cannot hand to two threads to poke at concurrently. The move transfers ownership: after a front end moves a VM onto a worker, the spawning thread no longer has it, so there is exactly one owner at any instant, and the single-threaded assumption inside `exec_op` is never violated. The absence of `Sync` is the machine refusing to promise something it would have to lock to keep — and by refusing it at the type level, it makes the promise it *does* keep (one owner, no synchronization) enforceable by the compiler rather than by discipline. A front end that tried to share one VM across two threads would fail to compile, which is the machine surfacing a concurrency bug at the trait boundary rather than as a data race at three in the morning.

How a front end gets parallelism

If a VM is single-threaded, a front end that wants to use many cores runs many VMs — one per thread, each owning its own stack and globals, each executing an independent chunk or an independent copy of the same

chunk. A shell backgrounding a pipeline stage, an AWK front end fanning a large input across worker threads, a scripting language evaluating independent tasks on a thread pool: each of these is not one VM shared but N VMs distributed, and the `Send` bound is exactly what makes the distribution type-safe. The `VMPool` of Chapter 81 composes with this cleanly — a pool per thread, each recycling the VMs its thread runs, so a work-stealing executor that pins a pool to each worker pays no cross-thread allocation and no lock, only the ordinary cost of moving a fresh chunk onto a thread that already has a warmed pool. The model is share-nothing at the VM level, which is the concurrency model that scales without contention because there is nothing to contend over: two workers running two VMs touch no common mutable state, so they never wait on each other.

What crosses the seam, and what it costs

Share-nothing at the VM level does not mean copy-everything at the value level, and the piece that makes the model cheap is the one already in the value representation: the `Arc`. A `Value::Str` is an `Arc<String>`, and `Arc` is *atomically* reference-counted, which means a string can be shared across threads by reference-count bump rather than by byte copy — the same discipline that makes a string cheap to clone within a thread makes it cheap to hand between threads. A chunk, likewise, is immutable once built, so many VMs on many threads can execute the same chunk through a shared `Arc` with no coordination: read-only shared data needs no lock, and a compiled chunk is read-only data. The one place the atomic refcount is not free is exactly there — an `Arc` clone is an atomic increment, marginally more expensive than the non-atomic increment an `Rc` would use — and that cost is the price of the model: the machine pays a handful of atomic operations at the boundaries where strings and chunks cross threads, in exchange for never paying a lock inside the dispatch loop. It is the right trade for a scripting engine, where the hot path is arithmetic and dispatch within one thread and the cross-thread events are comparatively rare, and it is why the value model chose `Arc` over `Rc` in the first place: the atomic refcount is the tax that buys the migration.

The model, stated whole

Put the pieces together and the concurrency model is a single coherent choice: one thread per VM, no synchronization inside execution, migration between threads by ownership transfer under the `Send` bound, sharing of immutable data across threads by atomic reference count, and parallelism by running many VMs rather than sharing one. It is the model that keeps the interpreter's dispatch loop a tight array-load-increment-branch, keeps the JIT's register code barrier-free, and keeps the AOT native path's locals in registers with no atomics — and it is the model that scales, because share-nothing has no contention to bottleneck on. The machine's concurrency story is, like the rest of it, an argument about where cost should be paid: not on every opcode as a synchronization tax, but at the boundaries, as a handful of atomic refcount bumps and one ownership move — the cost paid rarely, at the seam, so the hot path can stay as fast single-threaded as a stack machine can be, and a front end can still put five cores to work by putting five VMs on them.

Chapter 85 – Lowering Control Flow: How a Front End Builds Loops

The opcode chapters described the control-flow instructions — the jumps, the conditional branches, the call and return — as things the machine executes. This chapter turns them around and asks how a front end *emits* them: how an `if`, a `while`, a `for`, a `break`, and a `continue` in some source language become the flat jump-and-branch shape the machine runs, and how the fused loops the whole book has celebrated arise not from the machine but from the ordinary bytecode a front end lowers to. The machine never sees a loop; it sees a backward jump. The loop is a shape in the op vector, and this chapter is how a front end draws that shape.

Jumps are indices, and that is the whole trick

fusevm's jumps target an absolute index into the op vector: `Op::Jump(target)`, `Op::JumpIfFalse(target)`, and their kin carry a position, not a relative offset, and control simply sets `ip` to it. This is the representational choice that makes control-flow lowering tractable, because a front end walking a syntax tree does not know, when it emits a forward jump, where the jump will land — the target is code it has not generated yet. So it emits the jump with a placeholder target and remembers the position of the jump; when it later reaches the landing point, it *back-patches* the placeholder to the now-known index. The `ChunkBuilder` supports exactly this: emit a branch with a dummy target, keep the emitted instruction's index, and patch it once the destination is settled. Every structured construct in every one of the five front ends is, underneath, a small dance of emit-placeholder, generate-body, patch-placeholder — and the machine, running the result, sees only a vector of ops with settled indices and never knows a tree was walked to produce them.

The if, and the diamond

The simplest structured construct is the `if`, and its lowering is the canonical use of a forward patch. An `if cond then A else B` lowers to: emit the code for `cond` (leaving a boolean on the stack); emit a `JumpIfFalse` with a placeholder, remembering it; emit the code for `A`; emit an unconditional `Jump` with a placeholder, remembering it; patch the `JumpIfFalse` to *here*, the start of `B`; emit the code for `B`; patch the `Jump` to *here*, the join point past `B`. The result is the classic diamond — a test that falls through to the then-branch or jumps to the else-branch, with the then-branch jumping over the else to the join — expressed as two forward jumps, both back-patched. An `if` with no `else` is simpler still: one `JumpIfFalse` past the then-branch, patched to the join. There is no `if` opcode; there is a boolean, a conditional branch, and two patched indices, and every conditional in every front end is that.

The while, the for, and the backward jump

A loop is the diamond with one edge run backward. A `while cond { body }` lowers to: remember *here* as the loop top; emit `cond`; emit a `JumpIfFalse` with a placeholder for the loop exit; emit `body`; emit an unconditional `Jump` back to the remembered loop top; patch the `JumpIfFalse` to *here*, past the loop. The backward jump is not back-patched — its target, the loop top, was known before the body was emitted — and it is the backward-ness of that jump that makes the shape a loop rather than a straight line. A `for`-style counted loop is the same shape with an initializer emitted before the loop top and an increment emitted at

the loop's bottom just before the backward jump: init, top, test-and-exit, body, increment, jump-to-top. This init/top/test/body/increment/back structure is the exact shape the peephole recognizes and fuses — which is the point of the next section.

Where the fused loops come from

The whole book has celebrated the fused superinstructions — the `AccumSumLoop` that sums a range in nanoseconds, the counted-increment loops that collapse a slot read, an increment, a slot write, and a conditional branch into one dispatch. It is worth being precise about where they come from, because they are not something a front end emits directly. A front end lowers a counted loop the naive way — the init/top/test/body/increment/back shape of the previous section, several opcodes per iteration — and a *peephole fusion pass* over the resulting op vector recognizes that shape and rewrites the run of opcodes into a single fused superinstruction. The front end does not know the fused ops exist; it emits the ordinary loop, and the fusion pass turns the ordinary loop into a fused one wherever the shape matches exactly. This is why the fused loops benefit all five front ends without any front end doing fusion-aware work: fusion is a transformation on the emitted bytecode, downstream of every front end and upstream of every tier, so a front end inherits it for free by lowering loops the obvious way. The corollary is that a front end which lowers a loop into an unusual shape — one the peephole does not recognize — simply does not get the fusion, and pays the per-iteration dispatch instead; the fusion is an optimization the machine applies when it can prove the shape, and declines, safely, when it cannot, which is the same prove-or-decline conservatism that runs through the compilers.

Break, continue, and the patch list

The last pieces are `break` and `continue`, and they are the reason a front end keeps not one remembered jump but a *list* of them. A `break` is a forward jump to the loop exit — but the exit's index is not known until the loop is fully emitted, so each `break` emits a placeholder jump and adds its position to a list of pending exit-patches for the current loop; when the loop finishes and the exit index is known, every jump on that list is patched to it at once. A `continue` is the mirror image: a jump to the loop's increment-or-test point, which for a `while` is the loop top and is already known, so a `continue` often needs no back-patch at all, while for a counted loop it targets the increment and may. Nested loops keep a stack of these patch lists, one frame per enclosing loop, so a `break` in an inner loop patches to the inner exit and a labeled break — in the front ends that have labels — patches to the named loop's list. The whole of structured control flow, then, is jumps and patch lists: a conditional branch for the test, a backward jump for the loopback, a list of forward jumps for the breaks, all resolved to absolute indices by the time the chunk is built. The machine runs a flat vector of settled jumps and has no concept of a loop, a break, or a scope — those are shapes the front end drew and the fusion pass sometimes recognized, and the flatness that results is exactly what lets the dispatch loop be a tight array walk and the compilers treat the op vector as a control-flow graph to be analyzed. Structured source in, flat jumps out, and the machine none the wiser: that is control-flow lowering, and it is the same for all five languages because the target is one machine that speaks only in indices.

Chapter 86 – The Numeric Tower and Its Consequences

Scattered through the opcode chapters are the facts of fusevm's arithmetic: integers wrap, floats are IEEE, a string can be read as a number, a comparison coerces. This chapter gathers them into one account, because fusevm's numeric model is a coherent set of decisions with consequences that reach from the coercion tables up into what the native compiler can lower, and seeing them together explains behaviors that, met one at a time, look arbitrary.

Two number types, and the wrap

fusevm has exactly two numeric variants: `Int(i64)`, a 64-bit signed integer, and `Float(f64)`, a 64-bit IEEE double. There is no bignum, no rational, no decimal — the tower is two rungs, chosen because two machine-native types are what a stack machine can hold in a register and hand to the native compiler without boxing. The consequence that surprises people first is that integer arithmetic *wraps*: `Int::MAX` plus one is `Int::MIN`, silently, because the arithmetic opcodes use Rust's `wrapping_add` and its siblings rather than the checked or panicking variants. This is a deliberate choice with a clear rationale. Wrapping arithmetic is total — it has a defined result for every input, so an `Add` can never trap on overflow — and totality is what lets the arithmetic opcodes be branch-free and the native compiler lower them to a single machine `add` with no overflow check. A checked arithmetic would have to branch on every operation to test for overflow, or trap, and either would cost the tight integer loops the fusion pass works so hard to build. The machine chooses the C-like, two's-complement wrap — the same behavior the hardware gives for free — and inherits the hardware's speed, at the cost of the mathematician's infinity. A front end that needs overflow detection layers it above the machine; the machine itself wraps, always, and says so.

The IEEE float, and the truthiness of NaN

The float rung is IEEE 754 double precision, and it carries IEEE's semantics faithfully, including the uncomfortable ones. `Float` equality is IEEE equality, which means `NaN` is not equal to itself — a fact the value model pins in a test and the hash implementation respects by hashing floats *by bit pattern* so that a `NaN` used as a hash key can be found again even though it compares unequal to itself. The consequence with the longest reach is the truthiness rule: a `Float` is *truthy* unless it is *exactly* zero, and since `NaN` is not zero, **`NaN` is *truthy***. A condition that evaluates to `NaN` — a `0.0/0.0`, a `sqrt` of a negative — takes the true branch, which is the behavior a front end's user will eventually trip over and which the machine documents rather than papers over. Positive and negative zero are both falsy, because both are exactly zero; every other float, including the infinities, is *truthy*. These are not fusevm inventions — they are IEEE's rules, carried up through the value model unmodified — and carrying them faithfully rather than "fixing" them is the machine's usual conservatism: it gives the front end the hardware's float, warts and all, so that a language which wants different float semantics can build them, and a language which wants the hardware's floats gets them at the hardware's speed.

Promotion: when an int meets a float

Arithmetic between the two rungs promotes, and the rule is the expected one made precise. When an `Add`, a `Sub`, a `Mul`, or a `Div` meets an `Int` and a `Float`, the `Int` is coerced to `Float` via `to_float` and the operation proceeds in floating point, yielding a `Float`. Int-and-int stays integer (and wraps); float-and-anything goes float. The fast paths the interpreter keeps — `arith_int_fast` and its comparison cousin — are the int-and-int case specialized, the common one, so that two integers add without touching the general coercion machinery, and the general path handles the promotion when a float is present. This matters to the native compiler because promotion is a *kind* transition, and the kind lattice of Chapter 28 is exactly the machinery that tracks it: an expression the compiler can prove is int-and-int lowers to integer registers and machine integer ops; one that might promote must be lowered in the float lane or guarded. The promotion rule is simple to state and consequential to compile, and it is one of the places the value model's coercions and the compiler's kind analysis are the same fact seen from two sides.

The number that is also a string

The rung that makes fusevm a scripting-language engine rather than a systems-language one is the string-number duality, and it lives in the coercions. `to_int` and `to_float` are defined on every value, not just the numbers: a `Str` parses as a number by reading its leading numeric prefix — `"42"` is 42, `"3.14"` is 3.14, `"12abc"` is 12, `"abc"` is 0 — which is the AWK and shell semantics where a string used in arithmetic *is* a number. This is why an AWK field, which arrives as a string, can be summed without an explicit conversion, and why a shell variable holding `"5"` can be compared numerically. The duality runs the other way too: `as_str_cow` gives every value a string form, so a number used where a string is wanted stringifies (an `Int` to its decimal, a `Float` to its shortest round-tripping form). The consequence a user meets is that comparison must *choose* an interpretation — is `"10" < "9"` a string comparison (true, lexicographically) or a numeric one (false)? — and fusevm resolves it by having distinct opcodes: the numeric comparisons (`NumLt` and `kin`) coerce both sides through `to_float` and compare as numbers, while the string comparisons (`StrLt` and `kin`) compare as bytes. The front end chooses which opcode to emit based on its language's rules — AWK's numeric-string heuristics, the shell's context — and the machine faithfully executes the chosen comparison. The duality is not ambiguity in the machine; it is two clean operations, and the front end's job is to pick the right one for its language's semantics.

Consequences, gathered

Read together, the numeric decisions form a consistent picture and a consistent set of consequences. Two machine-native rungs mean every number fits a register and the native compiler can hold it there. Wrapping integers mean arithmetic is total and branch-free and fuses into tight loops, at the cost of silent overflow. IEEE floats mean the hardware's exact semantics, including a truthy `NaN` and a bit-hashed float key, carried up faithfully rather than sanded down. Promotion means int-and-float goes float, which the fast paths specialize away for the common int-and-int case and the kind lattice tracks for the compiler. And the string-number duality means a string is a number when arithmetic wants one, a number is a string when concatenation wants one, and comparison is two opcodes so the front end can choose. None of these is arbitrary; each is the cheapest faithful choice for a machine that must be fast enough to compile and general enough for five scripting languages to share, and each has a consequence a user will eventually meet — the overflow that wrapped, the `NaN` that was truthy, the `"10"` that compared less than `"9"`. The machine documents them all rather than hiding them, because the alternative to a documented sharp edge is an undocumented one, and a machine five languages share cannot afford surprises it has not written down.

Chapter 87 – The Division of Labor: fusevm’s Analysis and Cranelift’s Allocator

The native chapters described locals held in registers, kinds proven, floats in float lanes, and deopt when a proof fails. It is worth a chapter on the *seam* between fusevm and Cranelift in all of this, because the native path is a collaboration between two compilers with a clean division of labor, and understanding which one does what dispels the most common confusion about the AOT path: fusevm does not allocate registers, and Cranelift does not understand fusevm. Each does exactly its half, and the half each does is the half it is good at.

Two compilers, one lowering

Cranelift is a code generator: hand it a function in its own SSA intermediate representation — a graph of typed values, each defined once, flowing through blocks — and it will allocate machine registers, schedule instructions, and emit native code for the target architecture. What Cranelift will *not* do is decide what that SSA should be; it lowers the IR it is given and asks no questions about where the IR came from. fusevm’s `build_entry` is the thing that decides: it walks the chunk’s opcodes and *emits* Cranelift IR for them, one opcode at a time, translating each stack operation into the SSA values and instructions that compute the same result. The division is therefore clean and total. fusevm decides *what to compute* — which opcode becomes which IR nodes, which slot lives in which SSA value, which kind each value carries — and Cranelift decides *how to compute it on this machine* — which SSA value lands in which hardware register, when a value spills to the stack, how the instructions schedule. fusevm is the front end to Cranelift’s back end, and the bytecode-to-SSA translation in `build_entry` is the entire contract between them.

Slots become SSA values, and Cranelift takes it from there

The register residency the book has celebrated is, seen from this seam, a translation choice fusevm makes and Cranelift honors. When fusevm’s analysis proves a slot holds a single kind across a region — an integer accumulator, a float counter — `build_entry` represents that slot not as a load-from-memory and store-to-memory pair around every use, but as a Cranelift SSA value threaded through the block: a slot read becomes a use of the SSA value, a slot write becomes a new SSA definition, and the slot never touches memory at all within the compiled region. That is the whole of “locals in registers” — fusevm emits the slot as an SSA value rather than as memory traffic — and the actual assignment of that SSA value to, say, `x14` on AArch64 is Cranelift’s register allocator’s decision, made by Cranelift’s own liveness analysis and allocation, entirely beneath fusevm’s notice. fusevm’s job was to *decide the slot could be a value*; Cranelift’s job was to *put the value in a register*. When Cranelift runs out of registers and must spill, it spills — and fusevm neither knows nor cares which values spilled, because a spill is a correctness-neutral performance detail in Cranelift’s domain, not fusevm’s.

Kinds are fusevm's, types are Cranelift's

The kind lattice of Chapter 28 and Cranelift's type system look similar and are not the same thing, and the difference is exactly the division of labor. Cranelift's IR is typed — an SSA value is an `i64` or an `f64` or a pointer — but those types describe *machine representation*, the width and lane of a register. fusevm's `Kind` describes *value semantics* — this is an integer, this is a float, this is a heap pointer to a reference-counted string — and it is fusevm's analysis, not Cranelift's, that proves them. The translation maps one to the other: a fusevm `Kind::Int` becomes a Cranelift `i64`, a `Kind::Float` an `f64`, a heap kind a pointer. But the *proving* — the analysis that establishes a slot is `Kind::Int` throughout, so it may be lowered as an `i64` rather than as a boxed `Value` — is `analyze_native`'s work, done on fusevm's side before a single Cranelift instruction is emitted. Cranelift is told the type; fusevm figured out that the type was safe to assert. This is why the kind lattice lives in `aot.rs` and not in Cranelift: it is a value-semantics analysis specific to fusevm's value model, and Cranelift, which knows nothing of `Value`, could not perform it. fusevm proves the kind, hands Cranelift the type, and Cranelift allocates the register — three steps, two compilers, one clean handoff at each seam.

Deopt: handing the registers back

The place the division of labor is most delicate is deopt, because a deopt must take a computation that is mid-flight in Cranelift-allocated registers and hand its state back to the interpreter, which knows only the stack and the slots vector in memory. fusevm's `emit_deopt` is what bridges this: at a point where a proof might fail — a division that could produce a non-integer, a source whose kind is not statically known — it emits a runtime guard, and on the failing path it emits the code to *write the live register state back to memory* in the layout the interpreter expects, then transfer control to the interpreter at the right instruction pointer. The SSA values that were living in registers get stored to the slots vector and the operand stack; the interpreter, resuming, finds the world exactly as it would have built it, and continues in the threaded path. The subtlety is that fusevm must know, at the guard point, which SSA values correspond to which interpreter-visible locations — which register is which slot — so that the writeback reconstructs a coherent interpreter state. This is fusevm reaching into the boundary between the two compilers' domains: it cannot ask Cranelift which physical register a value is in (that is Cranelift's private decision), but it can ask Cranelift to *store* a given SSA value to a given memory location, and that store — SSA value to slot memory — is exactly the writeback. fusevm names the values and the destinations; Cranelift emits the stores from whatever registers it chose; and the interpreter finds its state in memory where it expects it. Deopt is the two compilers cooperating to spill a speculation, and it works because each stays in its lane: fusevm decides *what* to write back and *where*, Cranelift decides the register the write comes *from*.

Why the seam is where it is

The division of labor could have been drawn elsewhere — fusevm could have generated machine code directly, as the fused superinstructions effectively do for their tiny fixed shapes, or leaned entirely on an external tool. It is drawn at the SSA seam for a reason that recurs through the whole architecture: put the hard, general, portable work where it is written once and shared, and keep for yourself only the work that is specific to your value model. Register allocation, instruction scheduling, and target-specific code generation are hard, general, and portable — the same for any language targeting AArch64 or x86-64 — so fusevm delegates them to Cranelift, which does them once for every language that uses it. Value-semantics analysis, kind proving, and the writeback layout are specific to fusevm's `Value` and its interpreter's memory model — no external tool could do them without understanding fusevm — so fusevm keeps them. The seam falls exactly on the line between “general code generation” and “fusevm-specific analysis,” and that line is the same economy the whole machine is built on: do the shared, hard, general thing once (here, by reusing Cranelift), and write only the thing that is yours (here, the kind lattice, the deopt, the bytecode-to-SSA lowering). fusevm is to Cranelift what the five front ends are to fusevm — a specific front end to a general, reused, expensive-to-build back end — and the AOT native path is that pattern applied one level down, which is why it feels of a piece with everything above it.

Chapter 88 — fusevm Among the Reusable Machines

fusevm is not the first machine built to be shared, and it is worth placing it among its predecessors, because the comparison sharpens what fusevm decides differently. The idea of a reusable runtime — build the expensive engine once, target it from many languages — is old and has several famous embodiments, each of which made a set of choices, and fusevm’s choices are legible against theirs. This chapter is the one that steps out of the source tree to ask where fusevm sits in the landscape of machines that many languages share.

The bytecode-plus-JIT lineage

The most familiar reusable machine is the managed bytecode VM with a JIT — the pattern of the Java Virtual Machine and the Common Language Runtime. These are enormously successful and enormously *large*: a bytecode format, a just-in-time compiler, a garbage collector, a class loader, a threading model, a vast standard library, all in one runtime that many languages target. fusevm shares the front half of that pattern — a compact bytecode, a tiered JIT — and deliberately declines the back half. It ships no garbage collector, no class system, no standard library; the value model is eleven variants with reference-counted strings, not a managed heap with a tracing collector. The consequence is a machine that is orders of magnitude smaller — twenty-odd thousand lines against millions — and readable in an afternoon, at the cost of the things the big runtimes provide: fusevm’s front ends bring their own object models (through the `Obj` handle), their own libraries (through the builtin tables), and their own memory management above the reference-counted core. Where the JVM says “target me and inherit a managed universe,” fusevm says “target me and inherit an engine, and bring your own universe.” It is a smaller promise, kept more cheaply, and the smallness is the point.

The AOT-IR lineage

At the other pole is the ahead-of-time compiler infrastructure — LLVM the canonical example — which many languages target not to run on but to *compile through*: emit LLVM IR, and inherit a world-class optimizer and native code generator for every architecture. fusevm shares LLVM’s ambition of turning a shared IR into native code, and its AOT native path is a compiler in exactly this spirit. But LLVM is not a runtime — it is a compiler library; a language built on LLVM still needs its own runtime, its own interpreter for the code it cannot compile ahead of time, its own dynamic dispatch. fusevm is *both*: it is an interpreter and a JIT and an AOT compiler, so a front end targeting it gets the run-it-now path and the compile-it-ahead path from one engine, with the interpreter as the fallback the AOT path deopts into. Where LLVM gives you a compiler and leaves the runtime to you, fusevm gives you the runtime and the compiler as one thing, joined at the deopt seam — which is the join LLVM-based languages must build themselves and fusevm’s front ends inherit. fusevm’s Cranelift dependency is, in effect, a small LLVM used the way LLVM is meant to be used; what fusevm adds on top is the runtime LLVM deliberately omits.

The meta-tracing and partial-evaluation lineage

The most intellectually ambitious reusable machines are the ones that try to derive a fast implementation from a slow one automatically — the meta-tracing of RPython, which produces a tracing JIT for any interpreter written in it (PyPy being the famous instance), and the partial evaluation of Truffle on the GraalVM, which specializes a self-describing interpreter into fast code. These are more automatic than fusevm: they aim to take an interpreter and *generate* the compiler, so a language implementer writes only the interpreter and the machine derives the JIT. fusevm does not do this; its JIT and AOT compiler are hand-written, not derived, and a new opcode that wants to be fast in the native path must be taught to `build_entry` by hand. What fusevm gets for declining the automation is control and comprehensibility: the compiler is code you can read, its capabilities are a matrix the project publishes, and its correctness is pinned by differential fuzzers against the interpreter rather than trusted to a meta-level derivation. The meta-tracing systems make a grander promise — the compiler for free — and pay for it in the difficulty of understanding what the derived compiler did and why it did or did not speed a given program up. fusevm makes a smaller, more legible promise: the compiler is written, once, for one opcode set, and shared by every front end that lowers to that opcode set — automation at the level of the *bytecode*, not the *interpreter*. The five front ends share a compiler not because a meta-system derived five compilers but because there is one compiler and one bytecode, and that is a humbler and more inspectable form of the same economy.

The single-language JIT, generalized

There is also the lineage fusevm most resembles in its *techniques* while differing in its *scope*: the high-performance single-language tracing JIT, of which LuaJIT is the exemplar — a tracing compiler of extraordinary quality built for exactly one language. fusevm’s tracing tier, with its guards and its blacklisting and its deopt, is recognizably in this tradition, and borrows its shape: record a hot path, speculate on the types seen, guard the speculations, and bail when a guard fails. The difference is scope. LuaJIT’s tracing compiler knows it is compiling Lua and can bake Lua’s semantics into its speculations; fusevm’s tracing compiler knows only the opcode set, and its speculations are on the *kinds* flowing through opcodes, not on any language’s types — so the same tracing tier serves a shell, an AWK, and a Lisp, none of which it knows anything about. fusevm takes the single-language JIT’s techniques and lifts them to the bytecode level, trading the last increment of language-specific speed for the ability to serve five languages with one tracer. It is LuaJIT’s method applied to a language-agnostic bytecode, which is a different bet: less peak specialization, far more reuse.

Where fusevm sits

Set against these four lineages, fusevm’s position is a specific and unusual combination rather than a new technique. It has the bytecode-plus-tiered-JIT shape of the managed VMs but not their managed heap, garbage collector, or standard library — it is that pattern stripped to the engine. It has the AOT-to-native ambition of the compiler infrastructures but is a runtime as well as a compiler, joined at the deopt seam the compiler-only systems leave to their users. It shares a compiler across many languages like the meta-tracing and partial-evaluation systems, but by hand-writing one compiler for one bytecode rather than deriving many from many interpreters — a more legible economy. And it uses the tracing-JIT techniques of the great single-language engines but at the bytecode level, serving five languages where those serve one. The distinctive fusevm combination is: no garbage collector, a tiny readable core, an interpreter *and* a JIT *and* an AOT-to-standalone-binary compiler in one engine, shared by five real languages, with correctness pinned by differential fuzzing against the interpreter. No single predecessor makes all of those choices at once, and the combination is the machine’s identity — not a breakthrough in compiler technique, but a particular and coherent set of trades, aimed squarely at the goal the whole book returns to: that a new language should cost a front end, not a runtime, and that the runtime it does not have to write should be as small, as fast, and as trustworthy as one engine, carefully built, can be.

Chapter 89 – The Long Life of a Chunk

The book has followed a chunk through many single transitions — cold to warm, threaded to native, memory to disk, object to binary — each in its own chapter. This chapter follows *one* chunk through its whole life, from the moment a front end finishes building it to the moment it runs as native code in a shipped executable, because seeing the stages in sequence reveals the invariant that holds across all of them and is the quiet guarantee the whole architecture rests on: at every stage of a chunk's life, running it produces the same observable result.

Birth: the front end builds it

A chunk begins as the output of a front end's lowering. The lexer and parser have done their work, the control-flow lowering of Chapter 85 has drawn its jumps and patched them, the constants have been interned into the pool, and the `ChunkBuilder` has produced a `Chunk`: a flat op vector, a constant pool, a name table, a `sub_entries` map of where functions begin, a line table for diagnostics. The peephole fusion pass has run, rewriting recognizable loop shapes into fused superinstructions. At this moment the chunk is complete, immutable, and inert — a value that fully describes a program and has not yet run. Its op-hash, a content hash over the executing fields, is computable now and is the identity that will key its compiled forms in the cache. The chunk is `Clone`, so a front end can hold it, copy it, hand it to a pool; it is `Send`, so it can move to a worker thread; and it is immutable, so many threads can share it. Everything that follows is a way of *running* this fixed artifact, and none of it changes the artifact.

Youth: the interpreter runs it cold

The first time the chunk runs, it runs cold, in the interpreter — `VM::run` walking the op vector through `exec_op`, one dispatch per opcode, the array-load-increment-branch of the dispatch floor. This is the slowest way the chunk will ever run and also the *reference* way: the interpreter is the specification, and whatever result it produces is, by definition, the correct result. Every later, faster form of the chunk will be measured against this one. A chunk that runs once and is never seen again lives its whole life here, in the interpreter, and that is fine — the interpreter is fast enough that a cold chunk is not a problem, and the machinery of tiering exists precisely so that only the chunks that run *often* pay the cost of being compiled. The interpreter is the chunk's childhood home, and the home it can always return to.

Adulthood: the tiers compile it

A chunk that runs hot climbs the tier ladder. The counters in the VM notice a chunk, or a loop within it, executing often, and the JIT engages — the linear tier for a straight-line hot region, the block tier for a chunk that is a clean control-flow graph, the tracing tier for a hot path through irregular control flow. The compiled native code is cached in memory keyed by the op-hash, so the second hot invocation reuses the first's compilation. If the block tier compiled it and the chunk is disk-cacheable — its shape uses only relocations the cache format understands — the native code is also written to the disk cache, so a *later run of the whole program* finds it already compiled and skips even the JIT warm-up. Through all of this the chunk itself, the inert artifact from its birth, is unchanged; what changes is that faster executable forms of it accumulate beside it, in memory and on disk, each keyed by the op-hash that proves it is a compilation of *this* chunk and no other. The tiering is the chunk growing faster with use, and the op-hash is what keeps the faster forms honestly attached to the source they were compiled from.

Legacy: the AOT compiler ships it

The chunk's final form is the one it takes when a program is compiled closed-world, ahead of time, to a standalone binary. Here the AOT compiler runs over the whole program's chunks, `analyze_native` proving what it can prove, `build_entry` lowering the provable parts to native registers through Cranelift and the threaded path handling the rest, `compile_object` emitting a native object file, and the linker producing an executable that carries no interpreter loop for the compiled regions — the chunk has become machine code in a file that runs with no fusevm process interpreting it, only the native code and the small runtime the AOT path links in for the parts that deopt. This is the chunk at the end of its life, as far from the flat op vector of its birth as it will travel: registers instead of a stack, native branches instead of dispatched jumps, libcalls instead of interpreted intrinsics. And yet — this is the invariant — it computes the same result.

The invariant that holds across all of it

The thread through every stage is the one the differential-correctness chapter established and this chapter now sees end to end: the chunk produces the same observable result whether it runs cold in the interpreter, warm in any of the three JIT tiers, cached from disk, or compiled ahead of time to a standalone binary. The interpreter defines the result; every other form is measured against it, by the differential fuzzers that feed the same chunk to the interpreter and the compiler and demand identical output, by the guards that deopt a speculation back to the interpreter the instant it might diverge, by the cache loader that refuses any relocation it cannot prove it understands. A chunk's life is a progression of faster forms, and the guarantee that makes the progression safe is that faster never means different — that the two-hundred-thousand-fold speedup of the fused sum, the register residency of the native path, the disk-cached warm start, are all speed changes and never semantic ones. This is why a front end can lower to fusevm and stop thinking about performance: it emits one chunk, and the machine runs that chunk as fast as it safely can on this invocation, on this machine, at this tier, and the front end's user sees the same answer every time, only sooner. The long life of a chunk is many forms and one meaning, and the many forms are the performance story while the one meaning is the correctness story — and the whole architecture is the arrangement that lets them be true at once, so that a chunk can grow from an interpreted op vector into a shipped native binary without ever, at any stage, computing a different program than the one its front end lowered.

Chapter 90 — Why One Machine

Step back from the registers and the deopt and the kind lattice, and the architecture's payoff is not any one of them — it is *coherence*, and coherence is what makes a family of five languages tractable rather than a maintenance catastrophe.

One value representation underlies all five front ends, so a string is the same string whether `elisprs` or `stryke` or `zshrs` made it, and a value flows between languages without translation — even a front end's private objects ride through on the identity-comparable `Obj` handle. One opcode set means a performance improvement to the machine makes every language faster at once: the day the native path learned to hold float slots in registers, every AWK accumulator in every embedding got faster, with no AWK-specific work, and the day the block JIT learned `SlotKind::Float` the same chunks became disk-cacheable across runs. One host boundary means the editor or the OS is wired into the *machine*, once, through a trait whose every method has a default — rather than into five runtimes. And one compiler — four, counting the three JIT tiers and the AOT path, but built from one lowering vocabulary and one generic `build_entry` — means the expensive, hard, easy-to-get-wrong work of generating correct fast native code is done a single time and inherited five ways.

The cost of the fifth language, in this architecture, is a lexer and a lowering to opcodes — not a fifth interpreter, not a fifth garbage collector, not a fifth JIT. The evidence is in how differently the five front ends use the machine: `stryke`'s four hundred and fifty extended ops, `vimlrs`'s five hundred and ten builtin IDs, `awkrs`'s ninety-five, `zshrs`'s twenty, `elisprs`'s ten — five wildly different vocabularies, one engine, no forks. That is the sentence the whole machine exists to make true, and every decision in this book — the small flat chunk, the indexed locals, the host trait, the single `exec_op`, the analysis that lowers what it can prove and falls back on the rest — is in service of it. `fusevm` is one machine, and being one machine is the point.

The shape of the proof

It is worth ending the argument where the project ends its own: in the numbers that show the coherence is real rather than aspirational. Twenty-three thousand seven hundred and eighty-one lines of production Rust, in eleven files, implement the whole machine — the value model, the chunk, the two hundred and twenty-four opcodes, the interpreter, the three JIT tiers, the AOT compiler. Those lines are shadowed by more than a hundred and twenty thousand lines of tests, over five times their weight, including the seventy-six hundred-odd `#[test]` functions and the differential fuzzers that pin the compiler against the interpreter. And on top of that one engine sit five live front ends — a shell, an AWK, a systems scripting language, a `vimscript`, an `elisp` — each contributing only a lexer, a lowering, and a set of builtins, each inheriting the interpreter, the JIT, and the AOT compiler for free. The ratio that matters is not any single benchmark but this one: one machine, five languages, and the marginal cost of the sixth is a front end. A project that claimed coherence without the test weight would be claiming faith; a project that claimed five front ends without the shared engine would be claiming five maintenance burdens wearing a trench coat. `fusevm`'s claim is backed by the structure of its own source tree — the engine small enough to read in an afternoon, the tests heavy enough to trust, the front ends numerous enough to prove the machine is genuinely language-agnostic — and that structure is the real argument for one machine, made not in prose but in the shape of the code.

A closing on conservatism

If a single habit of mind runs through every chapter, it is conservatism in the precise, load-bearing sense: at every point where the machine could either prove something or guess it, it proves it or declines. The

interpreter is the single source of truth, and every faster path is measured against it rather than trusted on its own. The native AOT path lowers only what `analyze_native` can prove and falls back on everything else. The fused superinstructions bail to the interpreter the instant a slot is the wrong kind. The disk-cache loader refuses any relocation it does not recognize. The tracing JIT guards its speculations and deopts when they fail, blacklisting a trace that keeps being wrong. None of these is a performance feature; all of them are *correctness* features that happen to enable performance features, because a fast path you can trust is one you can build aggressively, and a fast path you cannot trust is one you must build timidly or not at all. The whole machine is fast because it is careful — because the threaded path always has the native path's back, the interpreter always has the JIT's back, and the differential fuzzers always have the compiler's back. That is the deepest sense in which fusevm is one machine: not just one value model and one opcode set and one host boundary, but one standard of correctness, applied from the interpreter up through the AOT register allocator, so that five languages can share not only an engine but a guarantee.

The one sentence

If the whole book had to collapse to one sentence, it would be the one the architecture exists to make true: a new language should cost a front end, not a runtime. Everything in these pages is an elaboration of that sentence or a demonstration that it holds. The small flat chunk, the indexed locals, the eleven-variant value model — these make the machine cheap to target. The single `exec_op`, the differential fuzzers, the threaded-path fallback — these make the machine trustworthy to share. The fused superinstructions, the three JIT tiers, the AOT native path with its kind lattice and its deopt — these make the machine fast enough that a language built on it does not pay for the convenience in speed. And the host traits, the extension mechanism, and the `Obj` handle — these make the machine general enough that a shell and an AWK and a Lisp can all live on it without resembling each other. Take any one decision in the book and trace it back, and it lands on that sentence: the chunk is flat so the loop is tight so cold code is fast so a front end need not write its own interpreter; the kind lattice is precise so the compiler can lower aggressively so a front end inherits native speed it did not build. fusevm is not a fast VM that happens to be reusable, nor a reusable VM that happens to be fast; it is one machine designed so that fastness and reusability are the same property, because both come from doing the hard, shared work once and well. That is the whole of it. A new language costs a lexer and a lowering, and inherits an interpreter, a JIT, an AOT compiler, and a guarantee — and being the one machine that makes that true is, from the first chapter to the last, the entire point.

Appendix A – The Value Variants

The eleven kinds a `Value` can take, from `value.rs`, with the runtime meaning each carries:

- `Undef` — absence of a value; the `#[default]`; falsy; `to_int` is 0; stringifies to `"`.
- `Bool(bool)` — a boolean from a conditional, `[[ ]]`, or a comparison; stringifies Perl-style (`"1" / ""`).
- `Int(i64)` — a 64-bit signed integer; arithmetic wraps.
- `Float(f64)` — a 64-bit float; equal by IEEE (`NaN ≠ NaN`), hashed by bits; truthy unless exactly zero, so `NaN` is truthy.
- `Str(Arc<String>)` — a reference-counted string; cheap to clone; `len()` is the byte count; borrowed by `as_str_cow`.
- `Array(Vec<Value>)` — an ordered list; truthy when non-empty; `to_int` is its length; mutated in place in globals.
- `Hash(HashMap<String, Value>)` — an associative array; keys are strings; mutated in place.
- `Status(i32)` — an exit code; truthy when `zero` (shell success); `to_int` is the code.
- `Ref(Box<Value>)` — a reference for pass-by-reference and nesting; always truthy.
- `NativeFn(u16)` — a handle to a builtin for dispatch.
- `Obj(u32)` — an opaque, identity-comparable handle into a front end's own object heap (an elisp cons cell, a symbol); carried but not owned by fusevm.

Appendix B – The Opcode Sections

The two hundred and twenty-four opcodes of `op.rs`, by section, with counts. Each section is a capability a front end may rely on.

- **Constants & stack** (12) — `Nop`; `LoadInt`, `LoadFloat`, `LoadConst`, `LoadTrue`, `LoadFalse`, `LoadUndef`; `Pop`, `Dup`, `Dup2`, `Swap`, `Rot`.
- **Variables & slots** (7) — `GetVar`, `SetVar`, `DeclareVar`; `GetSlot`, `SetSlot`; `SlotArrayGet`, `SlotArraySet`.
- **Arrays & hashes** (20) — `Get/Set/DeclareArray`, `ArrayGet/Set/Push/Pop/Shift/ Len`, `MakeArray`; `Get/Set/DeclareHash`, `HashGet/Set/Delete/Exists/Keys/ Values`, `MakeHash`.
- **Arithmetic** (9) — `Add`, `Sub`, `Mul`, `Div`, `Mod`, `Pow`, `Negate`, `Inc`, `Dec`.
- **String** (3) — `Concat`, `StringRepeat`, `StringLen`.
- **Comparison** (14) — numeric `NumEq/Ne/Lt/Gt/Le/Ge`, `Spaceship`; string `StrEq/Ne/Lt/Gt/Le/Ge`, `StrCmp`.
- **Logical / bitwise** (9) — `LogNot/And/Or`, `BitAnd/Or/Xor/Not`, `Shl`, `Shr`.
- **Control flow** (5) — `Jump`, `JumpIfTrue/False`, `JumpIfTrue/FalseKeep`.
- **Functions & scope** (5) — `Call`, `Return`, `ReturnValue`, `PushFrame`, `PopFrame`.
- **I/O** (3) — `Print`, `PrintLn`, `ReadLine`.
- **Collections** (2) — `Range`, `RangeStep`.
- **Higher-order** (5) — `MapBlock`, `GrepBlock`, `SortBlock`, `SortDefault`, `ForEachBlock`.
- **Math intrinsics** — the always-float family `SqrtFloat`, `SinFloat...Log10Float`, `AbsFloat`, `Atan2Float`, the rounding quartet, and the integer `AbsInt`, `TruncInt`, `GcdInt`, `LcmInt`, `TimeInt`, `PowFloat` (counted among the arithmetic-adjacent ops).
- **Fused superinstructions** (11) — `PreIncSlot`, `PreIncSlotVoid`, `PreDecSlot`, `PostIncSlot`, `PostDecSlot`, `AddAssignSlotVoid`, `SlotLtIntJumpIfFalse`, `SlotIncLtIntJumpBack`, `AccumSumLoop`, `ConcatConstLoop`, `PushIntRangeLoop`.
- **Builtins** (1) — `CallBuiltin(id, argc)`, fronting 140 builtin IDs in `shell_builtins.rs`.
- **Shell** (29) — `Exec`, `ExecBg`, `Pipeline*`, `Redirect`, `HereDoc`, `HereString`, `CmdSubst`, `Subshell*`, `ProcessSub*`, `Glob`, `GlobRecursive`, `TestFile`, `SetStatus`, `GetStatus`, `TrapSet/Check`, `ExpandParam`, `WordSplit`, `BraceExpand`, `TildeExpand`, `CallFunction`, `StrMatch`, `RegexMatch`, `WithRedirects*`.
- **AWK** (61) — the `Awk*` family: fields and records, special vars, `print/ printf/sprintf`, `getline`, the string builtins, associative arrays, the math and bitwise builtins, the JIT-eligible arithmetic variants, and `AwkSignal`.
- **Extension** (2) — `Extended(u16, u8)`, `ExtendedWide(u16, usize)`, against `Obj` handles.

Three opcodes carry a `u8` that is itself a small enumeration, defined as named constant modules in `op.rs`. `TestFile(u8)` (`op::file_test`): `IS_FILE` 0, `IS_DIR` 1, `IS_READABLE` 2, `IS_WRITABLE` 3, `IS_EXECUTABLE` 4, `EXISTS` 5, `IS_NONEMPTY` 6, `IS_SYMLINK` 7, `IS_SOCKET` 8, `IS_FIFO` 9, `IS_BLOCK_DEV` 10, `IS_CHAR_DEV` 11. `Redirect(fd, op)` (`op::redirect_op`): `WRITE` 0 (>), `APPEND` 1 (>>), `READ` 2 (<), `READ_WRITE` 3 (<>), `CLOBBER` 4 (>|), `DUP_READ` 5 (<6), `DUP_WRITE` 6 (>6), `WRITE_BOTH` 7 (6>), `APPEND_BOTH` 8 (6>>). `ExpandParam(u8)` (`op::param_mod`): `DEFAULT` 0 (`${v:-x}`), `ASSIGN` 1 (`${v:=x}`), `ERROR` 2 (`${v:?x}`), `ALTERNATE` 3 (`${v:+x}`), `LENGTH` 4 (`${#v}`), `STRIP_SHORT` 5 (`${v#p}`), `STRIP_LONG` 6 (`${v##p}`), `RSTRIP_SHORT` 7 (`${v%p}`), `RSTRIP_LONG` 8 (`${v%*p}`), `SUBST_FIRST` 9 (`${v/p/r}`), `SUBST_ALL` 10 (`${v//p/r}`), `UPPER` 11 (`${v^^}`), `LOWER` 12 (`${v,,}`), `UPPER_FIRST` 13 (`${v^}`), `LOWER_FIRST` 14 (`${v,}`), `INDIRECT` 15 (`${!v}`), `KEYS` 16 (`${!a[@]}`), `SLICE` 17 (`${v:o:l}`).

The sixty-one AWK ops, by group: fields and records (`AwkFieldGet`, `AwkFieldSet`, `AwkNf`, `AwkSetRecord`); special variables (`AwkSpecialGet`, `AwkSpecialSet`); output (`AwkPrint`, `AwkPrintf`,

AwkSprintf); input (AwkGetline); string builtins (AwkLength, AwkSubstr, AwkIndex, AwkSplit, AwkSub, AwkGsub, AwkMatch, AwkToLower, AwkToUpper, AwkGensub); associative arrays (AwkArrayGet/Set/Exists/Delete/Clear/Len); math (AwkInt, AwkSqrt, AwkSin, AwkCos, AwkExp, AwkLog, AwkAtan2, AwkDiv, AwkMod, AwkIntdiv, AwkIntdiv0, AwkMkbool); bitwise (AwkAnd, AwkOr, AwkXor, AwkCompl, AwkLshift, AwkRshift); conversion and characters (AwkStrtonum, AwkOrd, AwkChr); time (AwkSystemtime, AwkStrftime, AwkMktime); PRNG (AwkRand, AwkSrand); the JIT-eligible scalar variants (AwkDivJit, AwkModJit, AwkSqrtJit, AwkLogJit, AwkLshiftJit, AwkRshiftJit, AwkComplJit, AwkGetFieldNum); and the control-flow signal (AwkSignal).

Appendix C — The Native AOT Op Set

What the closed-world AOT compiler's *native* path lowers to registers today, and what falls back to the always-correct threaded path. A chunk lowers natively only if `analyze_native` admits every reachable opcode.

Lowered to native registers:

- All integer and float arithmetic — `Add`, `Sub`, `Mul`, `Negate`, `Inc`, `Dec`, `Mod` (integer guarded `srem`, float `fmod` libcall), `Pow`/`PowFloat` (`powf` libcall), and `Div` (native `fdiv`; divide-by-zero deopts).
- All numeric comparisons and `Spaceship`, with int/float promotion and NaN-correct `fcmp`.
- Bitwise and shifts; the full boolean/logical set; `LogNot`/`LogAnd`/`LogOr`.
- The math intrinsics — `Abs`/`Sqrt`/`Floor`/`Ceil`/`Trunc`/`Round`, the thirteen transcendentals through one `unary_math` dispatcher, `Atan2Float`, `AbsInt`, `TruncInt` (saturating), `GcdInt`/`LcmInt` (internal native loops).
- Stack shuffles (`Dup`/`Dup2`/`Swap`/`Rot`/`Pop`).
- All control flow, including the value-keeping `JumpIf*Keep`.
- Slots and globals — `Int`, `Bool`, `Float`, `Status`, and `Obj`, register-resident, definite-assignment guarded — and every slot super-op including `AccumSumLoop`.
- The awk JIT scalar ops (`AwkDivJit`/`AwkModJit`, `AwkLshift`/`Rshift`/`ComplJit`, `AwkSqrt`/`LogJit`), via a native error-return or warn branch on bad operands.
- The inline/shim **boundary**: sinks (`Print`/`PrintLn`/`SetStatus`), statically-typed sources (`AwkGetFieldNum` → `Float`, `GetStatus` → `Status`), and a general box/unbox staging for many heap ops with statically-known result kinds.

Still falls back to threaded:

- Truly dynamic-kind sources (general array/hash reads, `ReadLine`, calls) — pending a runtime type-guard-then-deopt on top of the existing deopt machinery.
- Any chunk `analyze_native` declines — a mixed-kind slot, an unassigned-slot read, a non-numeric/non-`Obj` final result, a global aliased by a named array, or one where every reachable op is a deopt point.

Appendix D — The JIT Tier Reference

The three runtime tiers and their triggers, coverage, and speculation, from [lib.rs](#), the README, and [jit.rs](#). The AOT compiler is a fourth, compile-time, path sharing the same Cranelift backend.

- **Linear** — trigger: [is_linear_eligible](#) plus first call. Coverage: straight-line expression chunks; returns a [Value](#) (int or float). Speculation: none; the IR matches the bytecode exactly.
- **Block** — trigger: [is_block_eligible](#) plus the block warm-up threshold (default 1 invocation; the code constant, not the stale “10” in some comments). Coverage: the whole-chunk CFG — loops, branches, fused backedges — with state flowing through a [*mut i64](#) slots pointer. Speculation: none; slot ops assume [i64](#) unless a [SlotKind](#) vector says [Float](#), folded into the cache key.
- **Tracing** — trigger: [enable_tracing_jit\(\)](#) plus the trace threshold (default 50 backward branches through a loop header). Coverage: the hot path through anything, recorded and type-specialized; nine phases of inlining, branches, frame materialization, value-stack reconstruction, side-exit blacklisting ([MAX_SIDE_EXITS](#) = 50), bounded recursion ([MAX_INLINE_RECURSION](#) = 4), and side-trace stitching ([MAX_TRACE_CHAIN](#) = 4); recorded trace capped at 256 ops. Speculation: a slot-type entry guard plus per-branch [brif](#) guards; a deopt to the interpreter on any guard miss, blacklisted after five entry-guard mismatches.

Tunables: [FUSEVM_JIT_BLOCK_THRESHOLD](#) and [FUSEVM_JIT_TRACE_THRESHOLD](#) per process; [TraceJitConfig](#) and [JitCompiler::set_config](#) per thread. Disk cache: [jit-disk-cache](#) feature, [~/.cache/fusevm-jit](#), [FUSEVM_JIT_CACHE_DIR](#) and [FUSEVM_JIT_CACHE_MAX_BYTES](#) (default cap 256 MiB, oldest-first eviction).

Appendix E — Glossary

- **Chunk** — the compiled form of a program: ops, a constant pool, a name table, the supporting tables (lines, sub-entries, block-ranges, sub-chunks, source), and a cached `op_hash`. The only executable unit.
- **Slot** — an indexed local variable in a call frame; reached by number, not name; the storage the compilers lift into registers.
- **Op / opcode** — one instruction in a chunk's `ops` vector; a variant of `enum Op`, required to fit in 24 bytes.
- **op_hash** — a `DefaultHasher` over a chunk's ops and constants (not lines or names), computed at build time; the O(1) JIT cache key; `#[serde(skip)]`.
- **exec_op** — the interpreter's single per-opcode handler; the one definition of what each opcode means, reached by every execution tier including the AOT threaded path through `fusevm_aot_exec_op`.
- **Fused superinstruction** — one of eleven opcodes that collapse a common multi-opcode loop idiom into a single dispatch (`AccumSumLoop`, `ConcatConstLoop`, `SlotIncltIntJumpBack`, ...).
- **Tiered execution** — interpreter → linear JIT → block JIT → tracing JIT, climbed as code proves hot; plus the ahead-of-time AOT compiler. The tracing tier is the one wired into `VM::run`.
- **Cranelift** — the Rust compiler backend (version 0.130) all tiers emit through; fast to compile, modest optimizer, no C++ dependency.
- **Variable** — a Cranelift frontend SSA variable, written with `def_var` and read with `use_var`; how every fusevm compiler threads the stack and locals through loops without hand-building ϕ -nodes.
- **Trace** — a recorded straight-line path through a hot loop, the tracing JIT's unit of compilation; capped at 256 ops.
- **Guard** — a runtime check in compiled code that its type or branch-direction assumptions still hold; failure triggers a deopt.
- **Threaded path** — the AOT strategy (`build_entry_threaded`) that compiles native control flow as a `Switch`-dispatched per-op jump table but runs each opcode through `exec_op`; total, modestly faster than interpreting.
- **Native path** — the AOT strategy (`build_entry_native`) that lowers opcodes to real instructions with values in registers; fast, but only for chunks `analyze_native` can prove lowerable.
- **Kind** — the native path's compile-time type for a register value: `Int`, `Bool`, `Float`, `Status`, or `Obj`. `Int/Bool/Status/Obj` ride an `i64` lane, `Float` an `f64` lane.
- **Definite assignment** — the forward dataflow (set-intersection at joins) proving a slot is written on every path before it is read, so a register's zero never stands in for `Undef`.
- **Boundary** — the spill/reload mechanism letting native code reach a boxed opcode (a sink like `Print`, or a statically-typed source like `AwkGetFieldNum`) without leaving native execution.
- **Deopt** — a one-way exit from native code back to the interpreter, with VM state reconstructed (assigned slots/globals written back, operand stack spilled) so `fusevm_aot_resume` can drive `run` from the deopt IP.
- **Host** — the embedding's implementation of effectful operations (the `ShellHost` and `AwkHost` traits, with `DefaultHost/DefaultAwkHost`), reached through a trait whose every method has a default so one machine serves many embeddings.
- **Obj handle** — a `u32` index into a front end's own object heap, carried through fusevm's stack and slots and identity-compared, so a Lisp's cons cells run on a machine that knows nothing about them.
- **Frame** — a call-stack entry holding a `return_ip`, a `stack_base`, and a `slots` vector; per-call, so recursion gets fresh locals; materialized synthetically by the tracing JIT on a deopt inside an inlined callee.
- **VMResult** — the three-way return of `run`: `Ok(Value)`, `Error(String)`, or `Halted`; the contract every front-end driver branches on. `AwkSignal` halts and stashes a code rather than adding a fourth variant.
- **VMPool** — a recycler of `VM` instances that preserves vector capacities across runs via `VM::reset`; wins

- for non-trivial chunk shapes, loses for tiny chunks.
- **Cell** — the tracing JIT's abstract-value lattice: `Const(i64)`, `ConstF(f64)`, `Dyn`, `DynF`; tracks type (which lane) and value (const-foldable when known).
- **SlotKind** — the block JIT's per-slot type, `Int` or `Float`; folded into the native-code cache key so float-specialized code is never reused for an integer slot.
- **NativePlan** — the output of `analyze_native`: leaders, entry kinds, max depth, per-slot kinds, deopt points, and definite-assignment sets; codegen consults it at every op.
- **Definite assignment vs. may-assignment** — the must-write dataflow (intersection at joins) that guards `GetSlot` against `Undef`; at deopt points the may-write set (union) must equal it, so no conditionally-assigned slot is lost.
- **GLOBAL_TAG** — the `0x1_0000` bit OR'd into a global's index in the unified definite-assignment set, keeping global indices disjoint from slot indices.
- **NATIVE_SLOT_LIMIT** — the 4096 cap on slot and global indices the native path will lower; a chunk reaching beyond it is declined to the threaded path.
- **Sink / source** — a sink (`Print`, `SetStatus`) consumes scalars and produces nothing dynamic, so it spills and continues with no reload; a source produces a value the native code consumes, lowerable when its result kind is static (`AwkGetFieldNum` → `Float`, `GetStatus` → `Status`).
- **Libcall** — a call from native code to a small `extern "C"` shim that runs the interpreter's own Rust function (`powf`, `fmod`, the `unary_math` dispatcher), used where no single machine instruction exists; results match the interpreter bit-for-bit.
- **op-hash** — the `DefaultHasher` over a chunk's ops and constants (not lines or names), the O(1) JIT and disk-cache key; floats hashed by bits; `#[serde(skip)]`.
- **Numeric-string duality** — AWK's strnum semantics (a field compares numerically but prints as its original text), unrepresentable in `Value`, kept host-side in `awkrs` and reached through `AwkHost`.

Appendix F — The Public API

The surface a front end consumes, as re-exported from `lib.rs`. Everything else is an internal detail subject to change between versions.

- `Value` (from `value`) — the eleven-variant value enum, with constructors `int/float/str/bool/array/hash/status` and coercions `to_int/to_float/to_str/as_str/cow/is_truthy/len/is_empty`.
- `Op` (from `op`) — the 224-variant instruction enum, plus the constant modules `op::file_test`, `op::redirect_op`, and `op::param_mod`.
- `Chunk`, `ChunkBuilder` (from `chunk`) — the compiled unit and its incremental builder (`emit`, `add_constant`, `add_name`, `current_pos`, `patch_jump`, `add_sub_entry`, `add_block_range`, `add_sub_chunk`, `set_source`, `build`).
- `VM`, `VMResult`, `VMPool`, `Frame` (from `vm`) — the machine, its three-way result (`Ok/Error/Halted`), the recycling pool (`acquire/release/with`), and the call frame the tracing JIT materializes on deopt.
- `ShellHost`, `DefaultHost` (from `host`) — the shell host trait and its all-defaults implementation; installed with `VM::set_shell_host`.
- `AwkHost`, `AwkLvalue`, `DefaultAwkHost` (from `awk_host`) — the AWK host trait, its assignable-location type, and its all-defaults implementation; installed with `VM::set_awk_host`.
- **The JIT surface** (from `jit`, behind the `jit` feature) — `JitCompiler` and its `try_run_linear/try_run_block/try_run_block_kinded` entry points and eligibility predicates; `SlotKind`; `TraceJitConfig`; `JitExtension`; `NativeCode`; `BlockNum`; the deopt types `DeoptInfo` and `DeoptFrame`; the trace persistence types `TraceMetadata` and `TraceLookup`; and `set_awk_field_num_hook`.
- **The extension surface** (on `VM`) — `set_extension_handler`, `set_extension_wide_handler`, `register_builtin`, and the AWK signal channel `awk_signal`.
- **The AOT surface** (from `aot`, behind the `aot` feature) — `compile_object`, `run_chunk_native`, `fusevm_aot_run_embedded`, and the exported symbols `fusevm_aot_entry`, `fusevm_aot_chunk_blob`, `fusevm_aot_chunk_len`.

Appendix G — Features, Crate Types, and Dependencies

The build-time knobs that decide how much of fusevm a front end pays for.

- `default = []` — no features; the interpreter, the value model, the full opcode set, the host traits, and the extension mechanism, with no Cranelift dependency at all. A front end that wants only a fast interpreter takes this.
- `jit` — pulls in `cranelift-jit`, `cranelift-codegen`, `cranelift-frontend`, `cranelift-native`, `cranelift-module`; enables the three-tier JIT.
- `jit-disk-cache` — implies `jit` and adds `libc`; persists compiled native code to `~/.cache/fusevm-jit` across process restarts, on by default once enabled.
- `aot` — implies `jit` and adds `cranelift-object`, `bincode`, and `libc`; enables the closed-world AOT compiler that lowers a whole chunk to a relocatable object.

The crate ships two crate types: `lib` (the ordinary rlib, for front ends that depend on fusevm as a crate) and `staticlib` (producing `libfusevm.a`, so an AOT-compiled object can link against the runtime into a standalone binary). The non-optional dependencies are few — `serde` (with `derive` and `rc` for `Value` and `Chunk` serialization), `tracing`, `glob` (the default shell-host glob), and `chrono` (the AWK time builtins) — and everything heavy is gated behind a feature, which is how a front end that wants only an interpreter compiles without ever building Cranelift.

Appendix H — The Source Tree

The eleven production files of `src/`, with what each owns, for the reader who wants to read the machine itself after reading about it.

- `value.rs` (~640 lines) — the `Value` enum, its constructors and coercions, the hand-written `Hash` impl, and the coercion-matrix property tests.
- `op.rs` (~1,250 lines) — the 224-variant `Op` enum, the `file_test/redirect_op/param_mod` constant modules, the hand-written `Hash` impl, and the size-budget test.
- `chunk.rs` (~460 lines) — the `Chunk` struct, the `ChunkBuilder`, and the back-patching and op-hash tests.
- `vm.rs` (~5,270 lines) — the `VM`, the `Frame`, the `VMPool`, the dispatch loop, the `exec_op` handler, the fast-path helpers, and the AOT shim entry points.
- `host.rs` (~420 lines) — the `ShellHost` trait, `DefaultHost`, and the default-impl tests that pin the inert behavior.
- `awk_host.rs` (~800 lines) — the `AwkHost` trait, `AwkLvalue`, `DefaultAwkHost`, and the chrono-backed time builtins.
- `awk_builtins.rs` (~270 lines) — the reserved AWK op-ID range (`AWK_OP_BASE` 40,000), the per-builtin ID constants, the getline-source and signal-code enumerations.
- `shell_builtins.rs` (~1,070 lines) — the 140 shell builtin ID constants, organized into spaced numeric bands by family.
- `jit.rs` (~7,730 lines) — the three JIT tiers, the `JitCompiler`, the `Cell` lattice, the trace recorder and its nine phases, the disk cache, and the libcall shims.
- `aot.rs` (~5,790 lines) — the closed-world AOT compiler: `analyze_native`, the threaded and native `build_entry` paths, the `Kind` lattice, `emit_deopt`, `compile_object`, the shim roster, and the differential fuzzers.
- `lib.rs` (~70 lines) — the crate root: the module declarations, the public re-exports, and the capability-matrix documentation.

The proportions tell the story on their own: the two compilers, `jit.rs` and `aot.rs`, are more than half the production source between them, and the interpreter in `vm.rs` is most of the rest. The data model — `value.rs`, `op.rs`, `chunk.rs` — that the first movement spends three chapters on is, in lines, the smallest part of the machine, because the data model is the part that is hard to *design* and easy to *write*, while the compilers are the part that is hard to write and is where the engineering hours, and the test weight, mostly went.

Appendix I – The Disassembly Notation

The reference movement (Chapters 35–60) prints bytecode in a consistent disassembly notation; this appendix codifies it for quick reference.

- **Columns.** A disassembly line is `idx op operand ; stack after`. The `idx` is the instruction's position in the chunk's `ops` vector — the `usize` the `ip` walks, and the target a `Jump(idx)` names. The `op` is the opcode name. The `operand` is the inline payload (an `i64` for `LoadInt`, a `u16` pool index for `LoadConst/GetVar`, a `usize` target for `Jump`, a `u8` count for `Print`). The comment after `;` shows the operand stack *after* the instruction executes, written `[bottom, ..., top]`.
- **Pool annotations.** A `u16` operand indexing a pool is shown with the resolved value in a comment: `LoadConst 0 ; constants[0] = "hello", GetVar 3 ; names[3] = "count"`. The index is the truth the VM runs; the resolved value is a reading courtesy.
- **Backward jumps.** A jump whose target index is less than its own index is a loop bottom, drawn with an arrow: `Jump 2 ; → back to header`. Forward jumps (exits, `if`-skips) are shown with their target and noted as patched: `JumpIfFalse 9 ; forward exit (patched)`.
- **Stack effect shorthand.** In the op-by-op walks, an opcode's effect is written *before* → *after* over abstract stack symbols: `Add` is `..., a, b → ..., result`; `Pop` is `..., a → ...`; `Dup` is `..., a → ..., a, a`; `ArrayPush(a)` is `..., value → ...` (it pushes nothing, mutating the named array in place). The `...` is the unchanged lower stack.
- **Pseudo-CLIF.** The compilation walks (Chapters 47–51, 77–78) show emitted code in a pseudo-CLIF that abbreviates Cranelift's IR: `iconst.i64 n` (an integer constant), `iadd/isub/imul/srem` (wrapping integer arithmetic), `fadd/fsub/fmul/fdiv/fcmp` (float), `icmp` (integer compare), `brif` (conditional branch), `jump` (unconditional), `switch` (the threaded path's jump table), `use_var/def_var` (read/write a frontend `Variable`), `fcvt_from_sint/fcvt_to_sint_sat` (int/float conversions), and `call <shim>` (a libcall or a runtime shim like `fusevm_aot_exec_op`). Block parameters are shown in parentheses on the block label, as Cranelift writes them.
- **Frame diagrams.** The recursion trace (Chapter 61) shows the frame stack top-last, each frame as `[ frameN: <locals>, return_ip=..., stack_base=... ]`, with the active `ip` noted below.

With this notation a reader can read any disassembly in the book — and any real fusevm bytecode dump — as a running account of the operand stack evolving under a sequence of small, fixed-size instructions, which is all a stack-machine program ever is.

As a final worked example combining every element of the notation, here is `count = count + 1; print count` — a global read-modify-write and a print — with `count` at name-pool index 0, disassembled in full:

idx	op	operand	; stack after
0	GetVar	0	; [<count>] read names[0] = "count"
1	LoadInt	1	; [<count>, 1]
2	Add		; [<count>+1] wrapping integer add
3	SetVar	0	; [] write back to global "count"
4	GetVar	0	; [<count>] read it again for the print
5	PrintLn	1	; [] emit one value with a newline

Six instructions, each a tag and a small operand; the stack column read top to bottom is the program executing; the pool annotations resolve the indices; and nothing on any line is anything but the operand stack changing under one fixed-size instruction. That is the whole of reading fusevm bytecode, and with the opcode walks and

the worked idioms of the reference movement behind it, a real disassembly reads as plainly as the source it came from.